Digital Messaging Platform: Architecture and Workflows

Enterprise omnichannel messaging and notification orchestration (EMAIL, SMS, PUSH): a reference implementation

Version	v1.0
Date	2026-10-18
Author	Reference implementation team, authored with Claude Code

Independent reference implementation. All data is synthetic and no real provider is called. Validation status of every claim is stated explicitly; anything not executed (Docker, Kubernetes, Cloud Foundry, k6, RabbitMQ, Redis, real PostgreSQL, CI servers) is labelled NOT executed. Illustrative numbers are marked illustrative.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Contents

How to Read This Document 10
- Reading paths 10
- How each chapter is organised 10
- Evidence legend: what was executed 11
- Conventions 12
- What is in the box 12

1. Business Problem and Requirements 13
- 1.1 The problem in plain words 13
- 1.2 Goals and non-goals 13
- 1.3 Stakeholders and personas 14
- 1.4 Business scenarios that drive the design 14
- 1.5 Constraints that shaped the build 15
- Reference material 15
- Product requirements document (example) 15
- Feature specification 17

2. Functional and Non-Functional Requirements 19
- 2.1 Functional requirements 19
- 2.2 Non-functional requirements 20
- 2.3 Bounded everything: the specified limits 20
- 2.4 Out-of-scope but considered 21
- Reference material 21
- Acceptance criteria 21
- Test strategy 23

3. Overall System Architecture 25
- 3.1 Architectural style 25
- 3.2 Components and responsibilities 27
- 3.3 Runtime topology 28
- 3.4 Key design properties 29
- 3.5 Technology choices at a glance 29
- Reference material 30
- Architecture overview 30
- Delivery semantics explainer 32
- High-level design 33

4. Angular and Backend Architecture 36
- 4.1 The Angular console 36
- 4.2 Backend layering 43
- 4.3 Configuration model 45
- 4.4 Error handling philosophy 45
- Reference material 46
- Low-level design (design intent) 46

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

API contract guide 48

5. Notification Submission Flow 50
5.1 What the caller sees 50
5.2 The submission transaction 50
5.3 Implemented behaviour: policy rejection returns 202 52
5.4 Idempotency: four layers 53
5.5 Validation rules 54
5.6 Scheduled and deferred messages 54
5.7 Edge cases 54
5.8 Tests that prove it 55
Reference material 55
Submission flow (design document) 55
ADR-0004: Idempotency 56

6. Message Lifecycle and State Machine 58
6.1 Why an explicit state machine 58
6.2 The state machine 58
6.3 Transition matrix 60
6.4 Multi-step outcomes use legal single steps 60
6.5 Receipts and the monotonic rank rule 61
6.6 Cancel versus dispatch 62
6.7 Compare-and-set ownership 62
6.8 The lifecycle as an operator sees it 62
6.9 Tests that prove it 63
Reference material 64
ADR-0003: Message lifecycle state machine 64
Delivery semantics (failure windows) 66

7. Queue-Based Dispatch 67
7.1 Why a queue at all 67
7.2 The transactional outbox 67
7.3 The MessageBus port 69
7.4 The dispatch consumer 70
7.5 Backpressure and graceful shutdown 72
7.6 Tests that prove it 73
Reference material 73
Dispatch and retry flow (design document) 73
ADR-0001: RabbitMQ versus Kafka 74
ADR-0009: Outbox pattern 75
Spec 0001: Priority aging 77

8. Provider Adapter Architecture 81
8.1 The idea 81
8.2 The port 81
8.3 Resilience around every call 83
8.4 The simulators 84

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8.5 Worked example: adding a provider 84
8.6 Tests that prove it 85
Reference material 85
ADR-0002: Provider abstraction 85
Spec 0002: New provider adapter 86

9. Email, SMS, and Push Workflows 90
9.1 One pipeline, three personalities 90
9.2 Email workflow 91
9.3 SMS workflow 91
9.4 Push workflow 92
9.5 Fallback between channels 92
9.6 What the console shows 92
9.7 Tests that prove it 92
Reference material 93
Callback normalisation: vendor payloads and mapping 93

10. Template Management 95
10.1 Why templates are governed 95
10.2 Model 95
10.3 Approval flow 97
10.4 The safe renderer 98
10.5 Pinning (ADR-0007) 99
10.6 Tests that prove it 99
Reference material 99
Template approval flow (design document) 99
ADR-0007: Template versioning and pinning 100

11. Customer Preferences and Suppression 102
11.1 Two different concepts 102
11.2 The data 102
11.3 The policy chain 103
11.4 Managing preferences 104
11.5 Tests that prove it 104
Reference material 105
ADR-0012: Consent and suppression policy 105

12. Delivery Callbacks and Status Reconciliation 107
12.1 Why callbacks are hard 107
12.2 The callback pipeline as implemented 107
12.3 Ambiguous outcomes and reconciliation 111
12.4 Tests that prove it 114
Reference material 114
Callback normalisation (current, implemented) 114
Reconciliation flow (design document) 116
ADR-0010: Ambiguous outcomes and reconciliation 117

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13. Retry and Dead-Letter Flows 119
13.1 Principles 119
13.2 What is retried 119
13.3 Backoff with jitter 119
13.4 The retry loop 121
13.5 Dead-letter queue 122
13.6 Retry storms 123
13.7 Tests that prove it 123
Reference material 123
ADR-0006: Retry and dead-letter policy 123
Runbook: DLQ growth 125
Runbook: Retry storm 126
14. Bulk Messaging with Spring Batch 128
14.1 When bulk is the right tool 128
14.2 Job design 129
14.3 Counters, skips and duplicates 130
14.4 Authorization 131
14.5 Job states 131
14.6 Restart and the exactly-one guarantee 132
14.7 Throughput and fairness 133
14.8 Tests that prove it 133
Reference material 134
Bulk job flow (design document) 134
Bulk test plan 134
Runbook: Bulk job stuck 139
Runbook: Bulk job restart 140
15. Database ER Diagram 143
15.1 Design stance 143
15.2 Entity-relationship diagrams 143
15.3 Table catalogue 148
15.4 Access patterns and indexes 149
15.5 Transactions and the connection pool 149
15.6 Other stores 150
15.7 Retention and legal hold 150
Reference material 150
Data model 150
Query plans and indexes 153
Transactions and connection pooling 155
MongoDB usage 157
Retention and archival 159
Schema evolution 160
Oracle mapping 162
ADR-0005: Database choice 165

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

16. Security and Audit Architecture 167
16.1 Security principles 167
16.2 Authentication 168
16.3 Authorization 168
16.4 Data protection 169
16.5 Audit architecture 170
16.6 Threats and where they are addressed 170
16.7 Honest boundary 171
Reference material 171
Threat model 171
OWASP API Top 10 (2023) analysis 173
Service-to-service authentication 175
Secrets management 177
Data minimisation and retention 178
Production versus reference 180
ADR-0011: Multi-tenancy and authorization 181

17. Cloud Foundry Deployment 183
17.1 Why Cloud Foundry is a first-class target 183
17.2 The manifest 184
17.3 Operating model 185
17.4 Environments and profiles 185
17.5 Cloud Foundry versus Kubernetes in one table 186
Reference material 186
Cloud Foundry guide 186
ADR-0008: Cloud Foundry versus Kubernetes 189

18. Docker and Kubernetes Architecture 192
18.1 Container images 192
18.2 Docker Compose stack 192
18.3 Kubernetes architecture 193
18.4 Environments 194
Reference material 195
Deployment guide (Docker Compose, environments, checklists) 195
Kubernetes guide 197
Deployment views 198

19. CI/CD Pipeline 200
19.1 One pipeline, three syntaxes 200
19.2 Stages and gates 200
19.3 Artifact versioning and immutability 201
19.4 Secrets in the pipeline 201
19.5 What can be reproduced locally 202
19.6 Known gaps recorded by the pipeline author 202
19.7 Release readiness and rollback 202
Reference material 202

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

CI/CD documentation 202
Release readiness checklist 204
Rollback plan 206
Pull request checklist 207

20. Observability and Performance 209
20.1 Three signals 209
20.2 Metrics catalogue 210
20.3 SLIs, SLOs and alerts 211
20.4 Performance engineering approach 212
20.5 Load tests and what was and was not run 214
Reference material 214
Observability 214
SLIs, SLOs and alerting 217
Performance test plan 220
Performance investigation guide 221
Runbook: High latency and resource saturation 224
Runbook: Queue backlog 225

21. AI-Assisted Development Workflow 227
21.1 What "AI-assisted" means here 227
21.2 The invariants an assistant must not break 228
21.3 Spec-driven loop (idea to production in 11 steps) 228
21.4 How the agents were organised for this repository 229
21.5 What AI assistance is good and bad at here 229
21.6 Prompts and skills 229
Reference material 230
AI engineering workspace 230
Spec-driven workflow 232
Context engineering 234
Multi-agent workflow 236
Debugging workflows 238
Context map 240

22. Failure and Recovery Scenarios 242
22.1 Designing for failure 242
22.2 Scenario summary 244
22.3 How the scenarios are driven 245
22.4 Operator playbooks 245
Reference material 246
Reliability scenarios (full analysis) 246
Troubleshooting scenarios 251

23. Architecture Mind Map 253
23.1 The whole platform on one page 253
23.2 From concern to mechanism to evidence 254
23.3 One message, end to end (the narrative to remember) 254

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

23.4 Ten sentences to say in an interview 255

24. API Examples 256
24.1 Conventions 257
24.2 Endpoint catalogue 257
24.3 Error codes 258
24.4 Submit a message 259
24.5 Duplicate submission 260
24.6 Policy rejection (implemented as 202, not 4xx) 260
24.7 Malformed request 261
24.8 Unsupported channel 261
24.9 Unauthorized and forbidden 261
24.10 Validation error 262
24.11 Cancel 262
24.12 Status and detail 263
24.13 Templates 264
24.14 Preferences 264
24.15 Callbacks 265
24.16 Bulk jobs 265
24.17 Backpressure and rate limiting 266
24.18 Reproducing these locally 266

25. Technical-Leadership Artifacts 267
25.1 What a technical lead produces besides code 267
25.2 ADR summary 267
25.3 Decisions that implementation changed 268
25.4 Sprint plan 269
25.5 Risk register highlights 270
25.6 Review examples 270
Reference material 271
ADR index and process 271
Sprint task breakdown 272
Dependency and risk register 273
Code review scenarios 275
Mentoring examples 277
AI code review: worked examples 279
AI review example 1: Incorrect state transition 279
AI review example 2: Missing idempotency 280
AI review example 3: Race condition 282
AI review example 4: Unsafe retry 283
AI review example 5: N+1 queries 284
AI review example 6: Cross-tenant authorization 285
AI review example 7: Unbounded concurrency 286
AI review example 8: Resource leak 287
AI review example 9: Insufficient tests 288
AI review example 10: Sensitive data in logs 289

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Post-incident review template 291

26. Limitations and Future Enhancements 293
26.1 What is simulated 293
26.2 What was not executed 293
26.3 Known gaps and technical debt (recorded in the repository) 294
26.4 Not production-ready: what production would add 295
26.5 Future enhancements 295
26.6 How to keep this document honest 296

Appendix A. Traceability 297

Appendix B. Implemented Deviations from the Contract 299
Deviations record (docs/design/DEVIATIONS.md) 299

Appendix C. Glossary 305
Glossary 305

Appendix D. Operational Runbooks 310
Runbook index 310
Runbook: Provider outage 311
Runbook: Provider rate limit 312
Runbook: Consumer crash and ambiguous outcomes 313
Runbook: Callback signature failures 315
Runbook: Database outage 316
Runbook: Redis outage 317
Runbook: High API latency 318
Data retention and replay policy 319

Appendix E. End-to-End Run Output (Executed) 321

Appendix F. Validation Report and Measurements 327
Validation report 327
Performance measurements (executed, local) 329

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

How to Read This Document

This is **PDF 1** of two companion documents for the Digital Messaging Platform reference implementation. It explains the architecture and the workflows: how a notification request travels from an API call to a delivered (or honestly unresolved) message, which design decisions shape that journey, and how the platform is built, deployed, observed and operated. PDF 2 is the interview preparation guide.

The platform is an **independent reference implementation** of an enterprise omnichannel messaging and notification orchestration platform for EMAIL, SMS and PUSH. All data is synthetic, no real provider is ever called, and provider vendors are replaced by simulators. It is inspired by the kind of messaging role found in large financial-services firms; it does not reproduce any employer's internal systems.

Reading paths

If you are	Start with	Then read
An engineer joining the project	Chapters 3, 5, 6, 7	8 to 14, then 22
A solution architect	Chapters 1 to 3, 6	15 to 20, 26
An SRE or operator	Chapters 7, 12, 13, 20	22, appendix runbooks
A security reviewer	Chapters 11, 12, 16	26, appendix B
A technical lead or manager	Chapters 1, 2, 25, 26	21, 22
Preparing for an interview	Chapters 3, 5, 6, 22	23 (mind map) and PDF 2

How each chapter is organised

Every chapter opens with a plain-language explanation, then the exact mechanism with real class and package names, then diagrams, tables of states, codes or configuration, and the tests that prove the behaviour. Many chapters end with **Reference material**: the repository's own design documents are included in full so that this PDF is self-contained. Their text is reproduced as written in the repository, with headings demoted and Mermaid diagrams replaced by the native figures of this PDF.

> **Which document wins**
>
> Where an included design document and `docs/design/DEVIATIONS.md` disagree, **the deviations record describes the implemented behaviour**. Examples: a policy rejection at submission returns HTTP 202 with `status=REJECTED` (not a 4xx); a replayed callback returns 200 `replayed` (not 409); the consumer checks policy, provider and circuit state before the claim. The chapters of this PDF always describe the implemented behaviour, and Appendix B reproduces the deviations record.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Evidence legend: what was executed

This document never claims that something ran unless a source document records it.

Area	Status	Evidence
Backend on the `local` profile (single JVM, H2 in PostgreSQL mode, in-memory bus, in-memory Redis and Mongo substitutes)	Executed	Maven unit and integration tests in `backend/src/test`; see `docs/validation/VALIDATION_REPORT.md` for the recorded results
Angular console unit tests and production build	Executed per validation report	`docs/validation/VALIDATION_REPORT.md`
Real PostgreSQL 16, RabbitMQ, Redis, MongoDB servers	**NOT executed**	Testcontainers tests (`-Pcontainers`) compile but need Docker
Docker image build, Docker Compose	**NOT executed**	No Docker on the authoring machine; files parsed by `scripts/validate-infra.sh`
Kubernetes manifests, Helm chart	**NOT executed**	Parsed only; no cluster, `kubectl` or `helm`
Cloud Foundry manifest and scripts	**NOT executed**	Parsed by `cloud-foundry/validate-manifest.mjs` only
GitHub Actions, Jenkins, Bamboo pipelines	**NOT executed**	YAML parsed; Jenkinsfile Groovy not linted
k6 load scripts	**NOT executed**	k6 not installed; no throughput figure from k6 exists
Prometheus rules, Grafana dashboards, OTel collector	**NOT executed**	Syntax and structure parsed; PromQL not checked by `promtool`

Validation evidence that exists

Evidence	What it records
`docs/validation/e2e-run-output.txt` (reproduced in Appendix E)	A real HTTP end-to-end run of the packaged jar (`backend/target/messaging-platform.jar`, profile `local`, H2, in-memory bus, in-memory Mongo, simulators, Windows 11, JDK 21) driven by `node tests/end-to-end/e2e.mjs`: **254 checks passed, 0 failed, 0 skipped in 267.9 s**, across 14 sections (authentication, channels, idempotency, error contract, tenant isolation, consent, templates, fault modes, callbacks, cancel and audit, bulk, ops and metrics, API rate limit, log redaction). The file notes that an earlier run of the same script had 9 failing checks, all caused by the test itself (a seeded suppressed recipient, a wrong audit action name, counter expectations, a 503 from backpressure while a 20,000-row backlog drained, TCP connect timeouts on a 400-way parallel burst); the test was corrected, not weakened, and re-run on a freshly started backend
`docs/validation/screenshots/`	Console screenshots from a UI check against the running local backend; a selection appears in Chapter 4
`docs/validation/VALIDATION_REPORT.md`	The consolidated report generated from real outputs at the end of the project; this PDF quotes only what is present in `docs/validation/` when it was built

Numbers in this document come in three kinds, always labelled: **specified** (taken from configuration or contract, for example a 256 KB body limit), **computed** (derived arithmetically from a formula, for example backoff ceilings), and **illustrative** (assumed for capacity thinking, never measured). Whatever a validation run actually produced is quoted only from `docs/validation/`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Conventions

- Paths are relative to the repository root (`schwab-digital-messaging-platform`). Backend paths abbreviate `backend/src/main/java/com/example/messaging` as the package `com.example.messaging`.
- Figure captions end with an **Implementation:** reference naming the file or package the figure documents.
- Status names (`DELIVERY_UNKNOWN`), error codes (`IDEMPOTENCY_KEY_REUSED`) and configuration keys (`messaging.dispatch.max-attempts`) are verbatim from the source.
- The traceability appendix (Appendix A) maps requirements to code and tests; the glossary (Appendix C) defines the vocabulary.

What is in the box

Part	Chapters	Question it answers
A. Problem and shape	1 to 4	What are we building and how is it structured?
B. Core workflows	5 to 14	How does a message move, fail, retry and reconcile?
C. Data and trust	15, 16	Where does state live and who may touch it?
D. Run it	17 to 20	How is it deployed, built, observed and measured?
E. Engineering practice	21 to 25	How was it built, how does it fail safely, how is it led?
F. Honesty	26	What is simulated, not executed or missing?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. Business Problem and Requirements

1.1 The problem in plain words

Imagine a large investment firm with dozens of applications: the brokerage website sends trade confirmations, the security team sends login alerts, the marketing team sends newsletters, the onboarding flow sends welcome emails. If every application talks to the email, SMS and push vendors directly, five things go wrong, and they go wrong in exactly the places where regulators, customers and auditors are watching.

Symptom	Why it hurts	What the platform does about it
Each team re-implements consent and opt-out	A marketing text to an opted-out customer is a compliance incident	One ordered policy chain, applied at accept time and again at dispatch time (Chapter 11)
Naive retries create duplicate messages	A customer receives the same trade confirmation three times	Four layers of idempotency (Chapter 5) and no blind retry after an ambiguous result (Chapter 12)
Nobody can answer "was it delivered?"	Support cannot help; auditors cannot trace	An honest lifecycle with distinct accepted, provider-accepted, delivered and unknown states (Chapter 6)
Vendor outage stops the business	Password-reset codes do not arrive	Circuit breakers, backoff with jitter, dead-letter queue, operator replay (Chapter 13)
Vendor lock-in	Switching an SMS vendor means touching every application	Provider ports and adapters (Chapter 8)

A kid-friendly analogy: think of the platform as a **post office with a very careful clerk**. You hand over a letter and get a receipt immediately (HTTP 202). The clerk writes the letter into a ledger first (the database), and only then puts it in the outgoing tray (the queue), so a power cut can never lose a letter that you hold a receipt for. The clerk checks that the addressee has not asked for no mail (consent), waits until morning for non-urgent letters (quiet hours), hands letters to a courier (provider), and never sends the same letter twice just because the courier said "I am not sure if I delivered it": the clerk phones the courier and asks first (reconciliation).

1.2 Goals and non-goals

Goals	Non-goals
Accept messages fast (202) and never lose an accepted message	Real vendor integrations
Honest lifecycle: accepted versus sent-to-provider versus delivered versus unknown	Exactly-once across the provider boundary (impossible)
Policy enforcement (consent, suppression, quiet hours) at accept and at dispatch	Marketing campaign authoring UI
Operable: metrics, runbooks, replay, reconciliation	A claim of production compliance (see `docs/security/prod-vs-reference.md`)
Provider-agnostic and tenant-isolated	Multi-region active/active

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1.3 Stakeholders and personas

Persona	Needs	Role claim
Application developer (service client)	Simple, idempotent API; clear, stable error codes	`ROLE_SENDER`
Template author and approver	Safe preview, versioning, four-eyes approval	`ROLE_TEMPLATE_AUTHOR`, `ROLE_TEMPLATE_APPROVER`
Operations engineer	Queue and dead-letter visibility, replay, provider controls	`ROLE_OPERATOR`
Tenant administrator	Preferences, suppressions, audit trail	`ROLE_TENANT_ADMIN`
Auditor	Read-only audit and delivery events	`ROLE_AUDITOR`
Platform administrator	Provider configuration, consumer pause	`ROLE_PLATFORM_ADMIN`

The seven roles are the constants of `com.example.messaging.security.Roles` (`backend/src/main/java/com/example/messaging/security/Roles.java`):

JAVA

```java
public static final String PLATFORM_ADMIN = "PLATFORM_ADMIN";
public static final String TENANT_ADMIN = "TENANT_ADMIN";
public static final String TEMPLATE_AUTHOR = "TEMPLATE_AUTHOR";
public static final String TEMPLATE_APPROVER = "TEMPLATE_APPROVER";
public static final String OPERATOR = "OPERATOR";
public static final String SENDER = "SENDER";
public static final String AUDITOR = "AUDITOR";
```

1.4 Business scenarios that drive the design

Scenario	Category	Priority	Channel	Design consequence
Trade confirmation after an order fills	TRANSACTIONAL	NORMAL	EMAIL with SMS fallback	Idempotent submit, template pinning, fallback evaluated through consent
One-time login code	SECURITY_ALERT	CRITICAL	SMS or PUSH	Ignores quiet hours; prioritised queue
Account service notice	SERVICE	NORMAL or LOW	Any	Deferred during quiet hours at LOW or NORMAL
Monthly market commentary newsletter	MARKETING	LOW	EMAIL	Needs explicit opt-in; suppressed by opt-out and quiet hours; typically a bulk job
Fraud alert while the SMS vendor is down	SECURITY_ALERT	CRITICAL	SMS	Circuit breaker, retry, fallback channel, DLQ and replay
Late or duplicate vendor delivery receipt	any	n/a	any	HMAC, replay guard, monotonic rank rule

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1.5 Constraints that shaped the build

- The authoring machine has no Docker, Kubernetes, Cloud Foundry or k6, so those assets are statically validated only and labelled NOT executed everywhere in this document.
- The code must run with zero infrastructure: a single JVM with an in-memory bus, an H2 database in PostgreSQL mode, and in-memory substitutes for Redis and MongoDB. The same code uses real adapters under the `prod-like` profile.
- SQL must be portable (no PostgreSQL-only features) so that the Flyway migrations run on H2 and PostgreSQL and map to Oracle (`docs/database/oracle-mapping.md`).

Reference material

Product requirements document (example)

Source document: `docs/leadership/prd.md`

Product: Omnichannel Messaging and Notification Orchestration Platform (reference implementation, synthetic data). Document status: example PRD written to show structure; metrics are illustrative.

1. Problem

Business applications (account alerts, trade confirmations, onboarding, marketing) each integrate directly with email/SMS/push vendors. Result: inconsistent consent handling, duplicate sends on retries, no unified delivery status, vendor lock-in, and slow incident response ("did the customer get it?").

2. Goals

#	Goal	Measure (illustrative)
G1	One API to send EMAIL/SMS/PUSH with a consistent lifecycle	100% of pilot apps integrated via `POST /api/v1/messages`
G2	Never lose an accepted message; never silently duplicate	0 accepted-lost; duplicate rate < 0.01%
G3	Enforce consent, suppression and quiet hours centrally	0 marketing sends to opted-out recipients (audited)
G4	Honest delivery status incl. "unknown"	status answer within 1 call (`/messages/{id}/status`)
G5	Operable at scale: backlog, DLQ, replay, reconcile	MTTR for provider outage < 30 min with runbook
G6	Provider-agnostic: add a vendor in days	`AcmeSmsProvider` example without orchestration change
G7	Governed templates	100% of production templates four-eyes approved and versioned

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Non-goals

Real vendor integration, campaign authoring/segmentation, A/B testing, in-app inbox, claims of regulatory compliance.

4. Personas

Persona	Needs	Role
Application developer (service client)	simple, idempotent API, clear errors	`ROLE_SENDER`
Template author / approver	safe preview, versioning, approval	`ROLE_TEMPLATE_AUTHOR` / `ROLE_TEMPLATE_APPROVER`
Operations engineer	queue/DLQ visibility, replay, provider controls	`ROLE_OPERATOR`
Tenant admin	preferences, suppressions, audit	`ROLE_TENANT_ADMIN`
Auditor	read-only audit, delivery events	`ROLE_AUDITOR`
Platform admin	provider config, consumer pause	`ROLE_PLATFORM_ADMIN`

5. User stories (selected)

1. As a developer I submit a message with an idempotency key so that retries never double-send.
2. As a developer I query status and see whether it was accepted by the provider vs delivered.
3. As a marketer's service I send a bulk CSV job with a rate cap that restarts safely.
4. As an approver I publish a template version only if I am not its author.
5. As an operator I see DLQ items with reason and replay them after fixing the cause.
6. As an auditor I see who changed preferences/templates and when.
7. As a compliance owner I want marketing sends suppressed for opted-out recipients even on fallback channels.

6. Functional requirements

FR1 Submit/query/cancel messages (CONTRACT section 5). FR2 Template lifecycle with pinning (ADR-0007). FR3 Policy enforcement (ADR-0012). FR4 Provider routing, retry, fallback (ADR-0002/0006). FR5 Delivery event ingestion and normalization. FR6 Reconciliation of unknown outcomes (ADR-0010). FR7 Bulk jobs. FR8 Ops console (Angular). FR9 Audit trail. FR10 Metrics/tracing/logs.

7. Non-functional requirements

Availability and latency SLOs (illustrative, `docs/operations/slis-slos.md`); security (tenant isolation, HMAC callbacks, no content at rest); scalability (horizontal consumers); portability (CF and Kubernetes); testability (H2 profile, simulators).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8. Success metrics

Delivery-status coverage, time-to-detect provider issues, DLQ age, duplicate reports, approval cycle time, onboarding time per app.

9. Risks and open questions

See `risk-register.md`. Open: per-tenant quotas, cost attribution, multi-region.

10. Release plan

Sprint plan in `sprint-task-breakdown.md`; phased rollout: pilot tenant, canary, GA (design-level).

Feature specification

Source document: `docs/leadership/feature-specification.md`

Spec-level document for the core feature; the spec-driven template is `specs/spec-template.md`; full examples in `specs/0001-priority-aging.md`, `specs/0002-new-provider-adapter.md`.

Summary

Authenticated service clients submit a templated message for a recipient and channel. The platform accepts it durably (202), enforces policy, dispatches asynchronously through a provider, tracks delivery and exposes status.

Actors

SENDER client, TENANT_ADMIN, OPERATOR, provider (callbacks).

Behavior

1. **Accept**: validate DTO; claim idempotency; resolve recipient (tenant-scoped); evaluate policy; resolve and pin template version; validate variables; persist message (VALIDATED) + outbox + audit; respond 202 with `Location: /api/v1/messages/{id}`.
2. **Schedule**: `schedule.sendAt` future keeps VALIDATED until due; quiet hours may defer.
3. **Dispatch**: consumer re-evaluates policy, renders, rate-limits, calls provider, records attempt, applies outcome (see `docs/architecture/dispatch-retry-flow.md`).
4. **Status**: `GET /messages/{id}/status` returns lightweight status; `/history` returns merged timeline.
5. **Cancel**: allowed in VALIDATED, QUEUED, RETRY_PENDING; else 409.
6. **Fallback**: optional `fallbackChannels`, each independently policy-checked.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Inputs and validation

Field	Rule
`recipientId` xor `recipient.externalRef`	exactly one; must exist in tenant
`channel`	EMAIL/SMS/PUSH; recipient must have a valid address for it
`category`, `priority`	enums; category must match template category
`template.code`, `locale`, `version?`	PUBLISHED version resolved and pinned
`variables`	all required present; no unknown; length limits; string values only (see source)
`schedule.sendAt`	future, <= max horizon (config); `expiresAt` > `sendAt`
body size	<= 256 KB
header `Idempotency-Key`	required, 1-128 chars [A-Za-z0-9_-:.]

Outputs

`MessageAcceptedResponse {id,status,channel,correlationId,scheduledAt,links}`; errors via `ApiError` codes in CONTRACT section 5.

Edge cases

Case	Behavior
Duplicate request	original response + `Idempotent-Replay: true`
Same key, different body	422 `IDEMPOTENCY_KEY_REUSED`
Recipient opted out (marketing)	202 with status REJECTED, `statusReason=OPTED_OUT`
Hard-suppressed address (tenant scoped)	202 with status REJECTED, `statusReason=SUPPRESSED`
Template RETIRED after accept	still renders (pinned)
Provider timeout	`DELIVERY_UNKNOWN` then reconciliation
Expired before dispatch	not sent; terminal per source
Backpressure	503 + `Retry-After`

Telemetry

`messaging.messages.accepted`, `messaging.transitions`, `messaging.dispatch.delay`, `messaging.provider.calls`.

Security

Tenant from JWT; masked responses; no content at rest; audit all transitions.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. Functional and Non-Functional Requirements

2.1 Functional requirements

The functional requirements are derived from the build contract (`docs/design/CONTRACT.md`) and the acceptance criteria (`docs/leadership/acceptance-criteria.md`). The right-hand column names the code that implements each one.

ID	Requirement	Implemented by
FR-1	Accept a message for EMAIL, SMS or PUSH through `POST /api/v1/messages` and answer 202 without calling a provider	`web/MessageController`, `service/NotificationService`
FR-2	Treat `Idempotency-Key` plus tenant as an idempotency scope; identical replays return the original response	`service/IdempotencyService`, table `idempotency_record`
FR-3	Enforce consent, suppression, category rules and quiet hours	`service/PolicyService`
FR-4	Render templates with a pinned, approved version; reject missing variables	`template/TemplateRenderer`, `template/VariableValidator`, `service/TemplateService`
FR-5	Dispatch asynchronously with priorities, retries, backoff, jitter and a dead-letter queue	`scheduler/OutboxPublisher`, `messaging/DispatchConsumer`, `service/RetryPolicy`
FR-6	Route through a provider abstraction with circuit breaker, timeout and rate limit	`provider/ChannelProvider`, `provider/ResilientProviderInvoker`, `provider/ProviderRegistry`
FR-7	Receive signed provider callbacks, deduplicate, order them and apply them monotonically	`callback/*`
FR-8	Resolve ambiguous outcomes by callback or provider lookup, never by blind retry	`scheduler/StuckMessageSweeper`, `service/ReconciliationService`
FR-9	Allow cancel (only before processing), operator requeue and dead-letter replay or discard	`service/NotificationService`, `service/DefaultDeadLetterOperations`, `service/ReplayService`
FR-10	Provide bulk sends from CSV with skip, restart and throttling	`batch/*`
FR-11	Manage templates with versions, locales, preview and four-eyes approval	`web/TemplateController`, `service/TemplateService`
FR-12	Manage recipient preferences with optimistic locking, and suppressions	`service/PreferenceService`, `web/RecipientController`
FR-13	Provide operations views: queues, summary, providers, failed messages, audit and delivery events	`web/OpsController`, `web/ProviderController`, `web/EventController`
FR-14	Record an append-only audit trail for every state change and sensitive action	`service/AuditService`, table `audit_event`
FR-15	Serve a role-aware Angular console	`frontend/messaging-console/src/app`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2.2 Non-functional requirements

ID	Quality	Requirement (specified unless marked)	Tactic
NFR-1	Durability	No accepted message is lost (SLO target 99.99 percent, illustrative)	Transactional outbox, DB as source of truth (ADR-0009)
NFR-2	Correctness	Duplicates produce at most one business effect	Four-layer idempotency (ADR-0004)
NFR-3	Latency	Submit p95 under 300 ms (illustrative target)	No provider call on the request thread, short transactions
NFR-4	Availability	API 99.9 percent monthly (illustrative)	Stateless instances, circuit breakers, graceful shutdown
NFR-5	Scalability	Competing consumers, priority queues, streaming bulk	`MessageBus` port, chunked Spring Batch
NFR-6	Security	Tenant isolation; no PII in logs or on the bus; HMAC callbacks	ADR-0011, `common/Redaction`, `callback/SignatureVerifier`
NFR-7	Operability	Metrics, 19 alerts, the runbooks, replay, reconciliation	`metrics/MessagingMetrics`, `observability/`, `docs/operations/runbooks/`
NFR-8	Extensibility	A new provider needs no orchestration change	`NewProviderWithoutOrchestrationChangeTest`
NFR-9	Portability	Same SQL on H2, PostgreSQL; Oracle mapping documented	Flyway portable SQL rule
NFR-10	Auditability	Reconstruct any message history	`GET /messages/{id}/history`, pinned template versions
NFR-11	Resource bounds	Everything bounded: threads, prefetch, queue depth, request size, retries, bulk concurrency	See table 2.3

2.3 Bounded everything: the specified limits

The platform makes unbounded growth impossible by giving every resource a limit. These are the shipped defaults in `backend/src/main/resources/application.yml`.

Resource	Key	Default
Dispatch worker threads	`messaging.dispatch.concurrency`	8
Consumer prefetch	`messaging.dispatch.prefetch`	16
Max delivery attempts	`messaging.dispatch.max-attempts`	5
Backoff base and cap	`backoff-base-ms`, `backoff-cap-ms`	1000 ms, 60000 ms
Poison threshold	`poison-threshold`	5 deliveries
Delivery receipt timeout	`receipt-timeout`	15 minutes
Reconcile after	`reconcile-after`	5 minutes

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Resource	Key	Default
Lock TTL for PROCESSING	`lock-ttl`	2 minutes
Backpressure	`messaging.backpressure.max-queue-depth`	5000
API rate limit	`messaging.api.rate-per-sec`, `burst`	50 per second, burst 100
Request body	`messaging.api.max-body-bytes`	262144 (256 KB)
Bulk concurrency, chunk, skip limit, rows	`messaging.bulk.*`	4, 500, 100, 100000
Retention	`messaging.retention.*`	messages 90 d, audit 365 d, events 30 d
Callback tolerance	`messaging.callback.tolerance`	5 minutes
Callback replay window	`messaging.pipeline.replay-window`	24 hours
Circuit breaker	`messaging.pipeline.resilience.*`	50 percent, window 20, min 10 calls, 5 s open, 3 probes

2.4 Out-of-scope but considered

Real vendor contracts, deliverability reputation management, campaign segmentation, in-app inbox and multi-region active/active are explicitly out of scope. Chapter 26 lists them with the future enhancement that would address each.

Reference material

Acceptance criteria

Source document: `docs/leadership/acceptance-criteria.md`

Given/When/Then, by area. IDs are referenced by tests and sprint tasks. Automated tests should carry the AC id in a display name or comment.

Submission

- **AC-1** Given a valid token (SENDER, tenant A) and a unique `Idempotency-Key`, when `POST /api/v1/messages` is called with a valid body, then the response is 202 with `Location`, status `VALIDATED` or `QUEUED` is observable via `GET /messages/{id}/status`, and one `outbox_event` and one `audit_event` exist.
- **AC-2** Given an already-completed key with identical body, when repeated, then 200 + `Idempotent-Replay: true` and the same id; no new rows.
- **AC-3** Given the same key with a different body, then 422 `IDEMPOTENCY_KEY_REUSED`.
- **AC-4** Given 20 concurrent identical requests, then exactly one message exists.
- **AC-5** Given a body > 256 KB then 413 `PAYLOAD_TOO_LARGE`.
- **AC-6** Given missing required template variable then 400 `TEMPLATE_VARIABLE_MISSING` and nothing persisted.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Security and tenancy

- **AC-10** Given tenant B token, when requesting tenant A's message by id, then 404 for GET, cancel, history, status.
- **AC-11** Given no token / expired token then 401; given SENDER calling operator endpoints then 403.
- **AC-12** Given an unsigned or wrongly signed callback then 401 and no state change; given a stale timestamp (> 5 min) then 401.
- **AC-13** Given a replayed callback (same `X-Event-Id`) then no second state change.
- **AC-14** `MessageDetail` never contains rendered body or unmasked address.

Policy

- **AC-20** Given marketing to opted-out recipient then REJECTED `OPTED_OUT`; fallback channels also respect consent.
- **AC-21** Given hard-suppressed address then every category REJECTED `SUPPRESSED`.
- **AC-22** Given marketing inside quiet hours then deferred (VALIDATED with moved `scheduled_at`); SECURITY_ALERT is sent.

Lifecycle and reliability

- **AC-30** Illegal transitions are impossible via API (409) and not reachable by events (logged+audited).
- **AC-31** Given provider TRANSIENT_FAILURE then RETRY_PENDING with `next_attempt_at` within [0.5, 1.0] x min(cap, base x 2^(n-1)) of now.
- **AC-32** Given attempts exhausted without fallback then DEAD_LETTERED(RETRIES_EXHAUSTED); operator replay re-evaluates policy.
- **AC-33** Given TIMEOUT then DELIVERY_UNKNOWN with exactly one provider call; no automatic resend for EMAIL/SMS until lookup says not received.
- **AC-34** Given DELIVERED callback before SENT callback then final status DELIVERED, SENT recorded `applied=false`.
- **AC-35** Given provider acceptance without callback then after `receipt-timeout` status DELIVERY_UNKNOWN (never DELIVERED).
- **AC-36** Given a poison envelope then DLQ POISON_MESSAGE within `poison-threshold` deliveries.
- **AC-37** Given a template retired after accept then the message still renders with the pinned version.

Templates

- **AC-40** Approver equal to author is refused (403/409); different approver publishes; previous version retired.
- **AC-41** Editing a PUBLISHED version is refused.
- **AC-42** Preview never sends and escapes HTML.

Bulk

- **AC-50** CSV with duplicates and invalid rows: job COMPLETED_WITH_ERRORS, failures listed, one message per valid unique row.
- **AC-51** Restart after interruption produces no duplicates.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **AC-52** Missing `messages:bulk` scope then 403.

Operations

- **AC-60** `/api/v1/ops/queues` shows depth per priority, DLQ depth, retry-pending, oldest age.
- **AC-61** Metrics in CONTRACT section 8 appear on `/actuator/prometheus`.
- **AC-62** Backpressure over `max-queue-depth` yields 503 + `Retry-After`.

Non-functional (illustrative targets, measured only when executed)

- **AC-70** POST /messages p95 < 150 ms on local harness at agreed load (record only from executed runs).
- **AC-71** Graceful shutdown completes in-flight within 30 s and loses no message.

Test strategy

Source document: `docs/leadership/test-strategy.md`

Principles

1. Test behavior at the lowest level that can prove it; prove distributed-failure behavior at integration level with deterministic fault injection.
2. Deterministic: injected `Clock` and `RandomGenerator`; no sleeps (use awaiting helpers with timeouts); unique H2 db per context.
3. Never claim a result without executing: results live only in `docs/validation/VALIDATION_REPORT.md`.

Pyramid

Layer	Scope	Tools	Examples
Unit	pure logic	JUnit 5, AssertJ, Mockito sparingly	`MessageStateMachineTest`, `RetryPolicyTest`, `TemplateRendererTest`, `SignatureVerifierTest`, `PolicyServiceTest`
Slice	web/security/repository	`@WebMvcTest`, `@DataJpaTest`	role matrix, validation, ApiError shape
Integration	full Spring context, H2, in-memory bus	`AbstractIntegrationTest`	`NotificationFlowIT`, `FailureRecoveryIT`, `CallbackIT`, `BulkJobIT`
Container	real PG/Rabbit/Redis/Mongo	Testcontainers, `-Pcontainers`, `disabledWithoutDocker=true`	`*ContainerIT` (N/E here)
Contract	OpenAPI vs implementation	springdoc path assertion test	`OpenApiContractTest`
Frontend unit	services, interceptors, guards, components	Karma + Jasmine	per CONTRACT 10

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Layer	Scope	Tools	Examples
API e2e	HTTP against running app	node `fetch` scripts in `tests/end-to-end`, `.http` files in `tests/api-http`	happy path, idempotency, callbacks
Security	authz matrix, forged callbacks	`tests/security` scripts	cross-tenant 404
Failure recovery	scripted faults	`tests/failure-recovery` + `FailureRecoveryIT`	10 scenarios
Performance	load	k6 (N/E), JVM harness (executable)	`LoadHarnessIT`

Must-have test matrices

- State machine: all (from,to) pairs vs table.
- Policy: category x channel x consent x suppression x quiet hours x priority.
- Authorization: endpoints x roles x tenant (own/foreign).
- Provider outcomes: each `ProviderOutcome` => expected status, attempt, metric, audit.
- Callback: event type x current status ordering (rank rule), duplicates, conflict.
- Idempotency: sequential/concurrent/different body/expiry.

Concurrency testing

`ExecutorService` + `CountDownLatch` start gates; repeat N times; assert uniqueness invariants (one row), CAS losers acked. Property: run key races at least 20 iterations in CI (cost small).

Failure injection

`FaultInjector` modes `{NONE,TRANSIENT,PERMANENT,RATE_LIMIT,TIMEOUT,AMBIGUOUS,ALTERNATING}` with probability; datasource/coordination wrappers throwing; test hook for simulated crash points.

Test data

Synthetic seed (2 tenants, 60 recipients, 8 templates). Builders over fixtures; no real data.

Coverage and quality gates (targets)

Line coverage target (illustrative) >= 80% on `domain`, `service`, `callback`, `messaging`; mutation testing (PIT) optional on `domain` and `RetryPolicy` (N/E). Flaky test policy: quarantine within one day, fix within sprint.

Definition of "insufficient tests" (review smell)

Only happy path; mocks everywhere and no integration; assertions on logs instead of state; no concurrent test for a race fix; sleeping.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Overall System Architecture

3.1 Architectural style

The platform is a **modular monolith**: one Spring Boot 3.5 application (`backend/`, Maven module `messaging-platform`, Java 21) whose packages are the modules. Every instance runs the API, the dispatch consumers and the schedulers; instances are identical and stateless, so scaling is "run more of them". The style was chosen over microservices because the central correctness trick of the platform, writing the message state and the outbox row in **one database transaction**, is trivial inside one deployable and painful across services.

Three design moves keep the monolith honest:

1. **Ports and adapters** for everything outside the JVM. `MessageBus`, `CoordinationService`, `ChannelProvider` and `ProviderDiagnosticsStore` are interfaces with an in-memory implementation (profiles `local`, `test`) and a real one (`prod-like`). The same orchestration code runs in both.
2. **A pure state machine.** `com.example.messaging.domain.MessageStateMachine` is a static, table-driven class with no I/O, so the whole lifecycle can be tested exhaustively (`MessageStateMachineTest`).
3. **Compare-and-set ownership.** Whoever wins an `UPDATE ... WHERE status = :expected` owns the next step. This one idea resolves races between cancel and dispatch, between duplicate envelopes, and between late callbacks and reconciliation.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

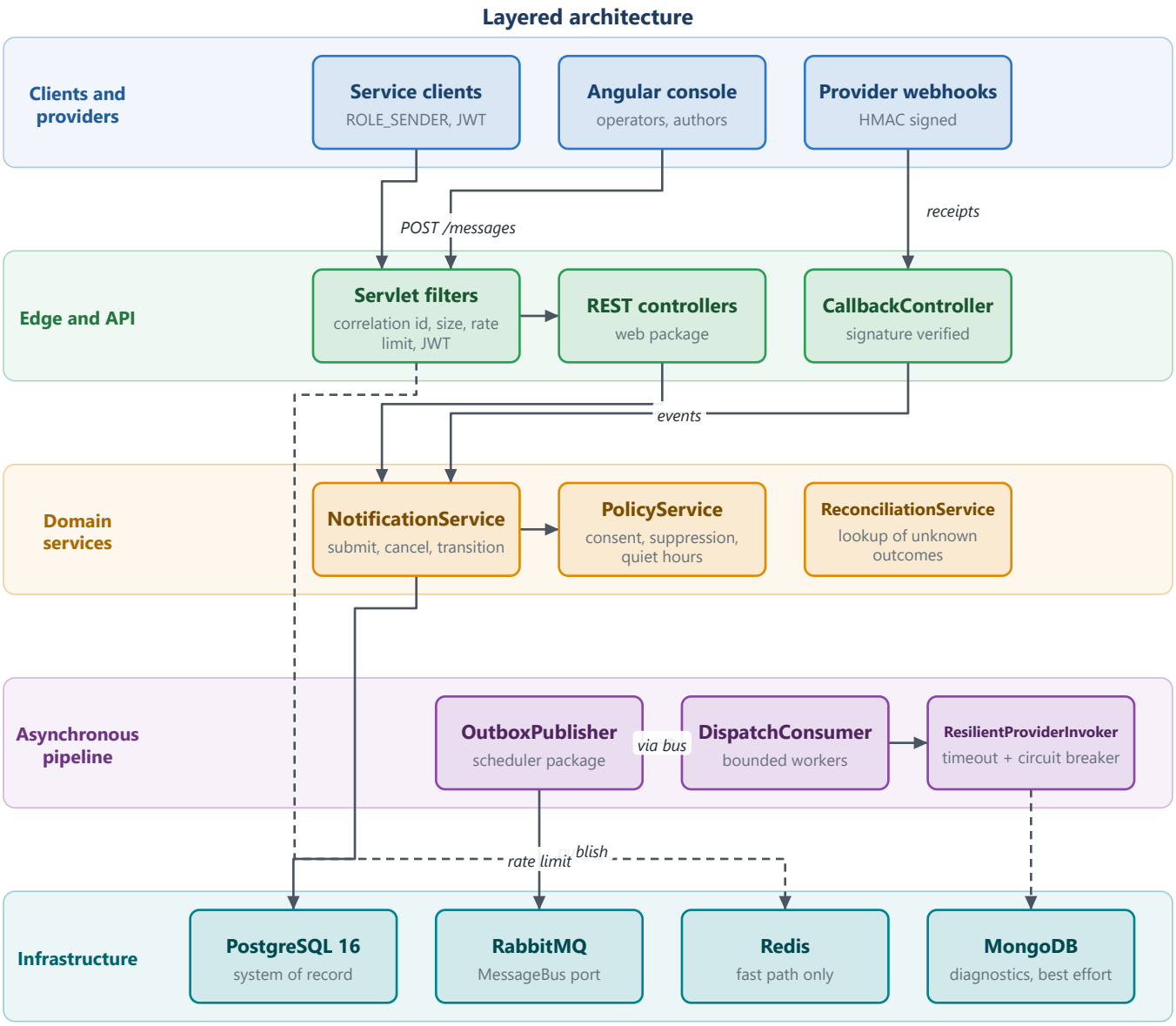


Figure 1. Layered view of the platform: clients and providers on the outside, a stateless Spring Boot modular monolith in the middle, and four data services below it. Implementation: backend/src/main/java/com/example/messaging/ (packages web, security, service, messaging, provider, callback), infrastructure/

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3.2 Components and responsibilities

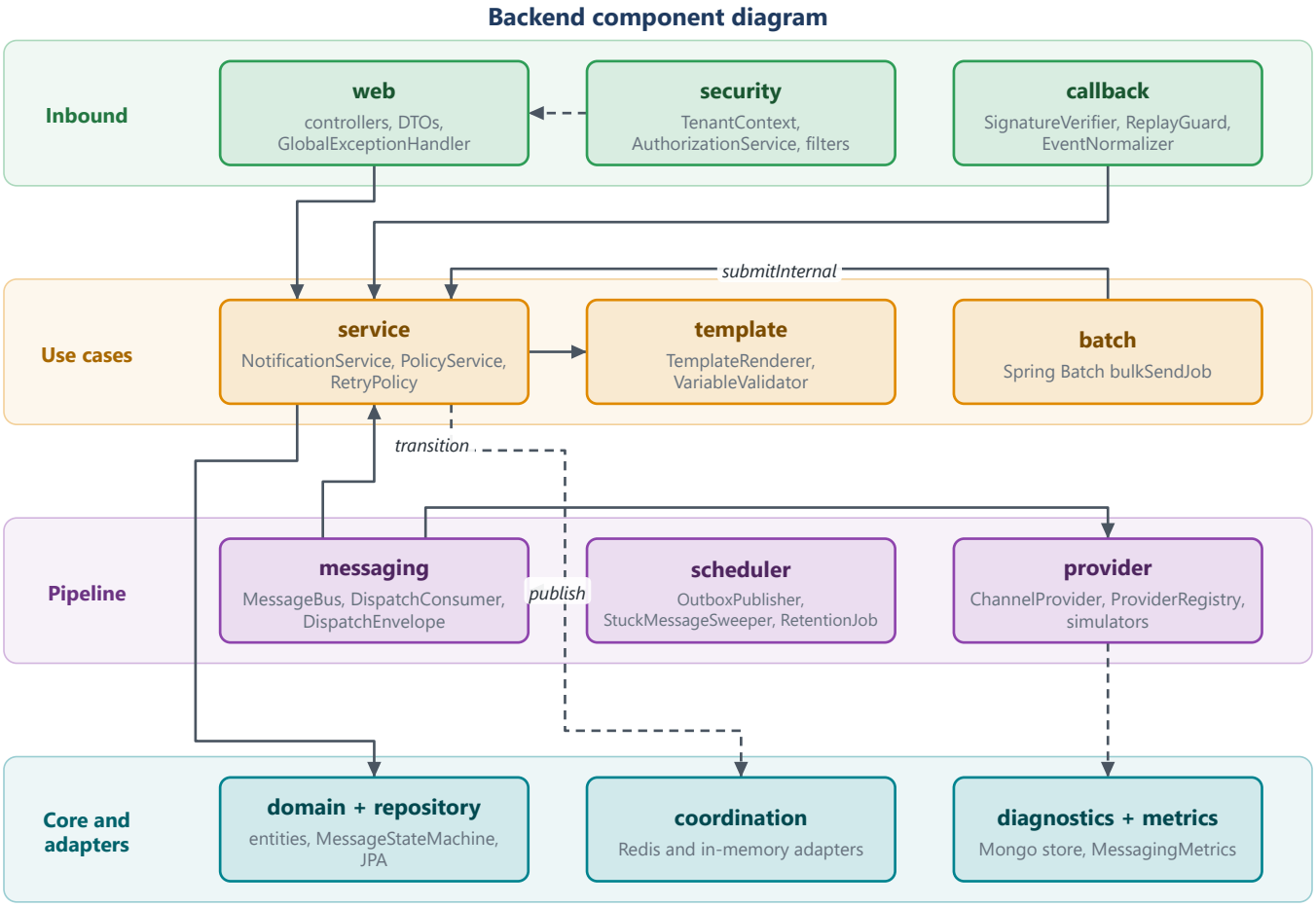


Figure 2. Backend packages grouped by responsibility. Arrows show the main compile-time dependencies; ports (MessageBus, CoordinationService, ChannelProvider) isolate the infrastructure. Implementation: backend/src/main/java/com/example/messaging/

Package	Responsibility	Notable classes
`web`	HTTP controllers, DTO records, error mapping	`MessageController`, `GlobalExceptionHandler`, `TemplateController`, `BulkJobController`
`security`	AuthN and AuthZ, tenant context, rate limit, body size	`TenantContext`, `AuthorizationService`, `RateLimitFilter`, `RequestSizeFilter`, `DevTokenController`
`service`	Use cases and policy	`NotificationService`, `PolicyService`, `RetryPolicy`, `ReconciliationService`, `IdempotencyService`, `AuditService`, `TemplateService`, `PreferenceService`
`domain`, `repository`	Entities, state machine, Spring Data JPA	`MessageStateMachine`, `MessageRequest`, `MessageRequestRepository`
`messaging`	Bus port, consumer, backpressure, Rabbit adapter	`MessageBus`, `DispatchConsumer`, `InMemoryMessageBus`, `RabbitMessageBus`, `BackpressureGuard`
`provider`	Channel ports, simulators, resilience	`ChannelProvider`, `ProviderRegistry`, `ResilientProviderInvoker`, `FaultInjector`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Package	Responsibility	Notable classes
`callback`	Webhook authentication and normalisation	`SignatureVerifier`, `ReplayGuard`, `EventNormalizer`, `DeliveryEventProcessor`
`batch`	Bulk send with Spring Batch	`BulkJobConfig`, `RecipientRowProcessor`, `BulkMessageWriter`, `BulkJobService`
`scheduler`	Outbox, poller, sweeper, retention	`OutboxPublisher`, `ScheduledDispatchPoller`, `StuckMessageSweeper`, `RetentionJob`
`coordination`	Redis ports and fallbacks	`CoordinationService`, `RedisCoordination`, `ResilientCoordination`
`diagnostics`, `metrics`	Mongo diagnostics, Micrometer meters	`ProviderDiagnosticsStore`, `MessagingMetrics`
`template`	Safe renderer	`TemplateRenderer`, `VariableValidator`

Package notes from the deviations record

`ProviderOutcome` lives in package `provider`, not `domain`. The `coordination` package was added for the Redis ports. `ProviderAdminService` lives in `provider` because it knows the simulators; `service`, `messaging`, `scheduler`, `callback` and `domain` never name a concrete provider (checked by `NewProviderWithoutOrchestrationChangeTest`).

3.3 Runtime topology

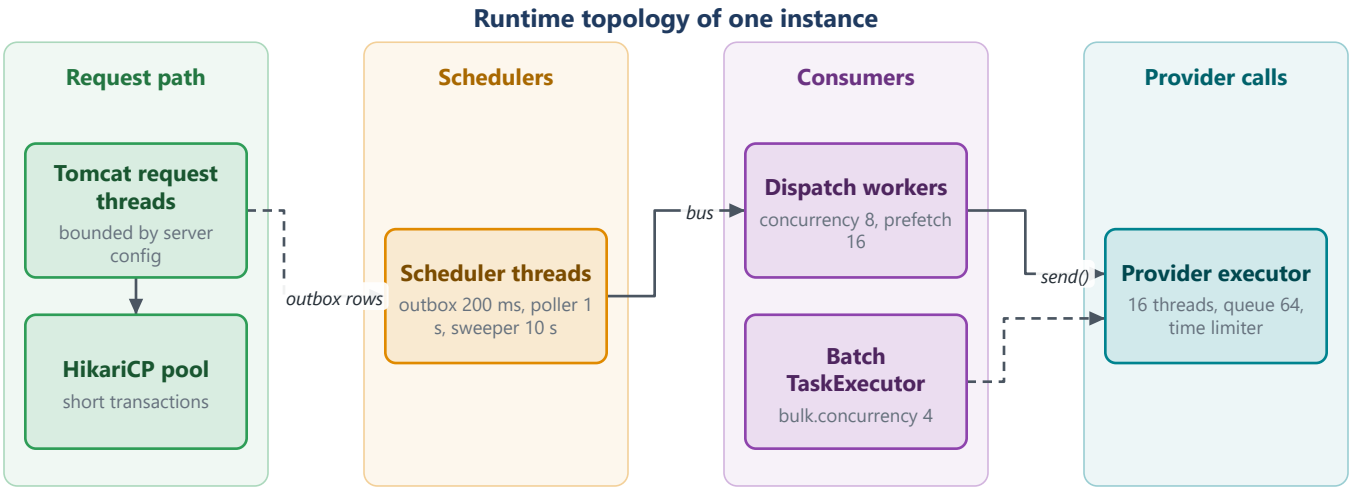


Figure 3. Thread pools of one instance. Provider calls run on a separate bounded executor so a hanging provider cannot starve the consumer workers or the Tomcat threads. Implementation: backend/src/main/java/com/example/messaging/config/AsyncConfig.java, backend/src/main/java/com/example/messaging/config/PipelineProperties.java, backend/src/main/resources/application.yml

The provider call executor is separate from the consumer workers on purpose: a provider that hangs for ten seconds ties up executor threads (and trips the circuit breaker), not the workers that hold database transactions and not the Tomcat threads that serve the API.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3.4 Key design properties

#	Property	Mechanism	ADR
1	Persist first, publish after	State, outbox and audit committed together; a scheduled publisher moves rows to the bus	0009
2	At-least-once transport, idempotent processing	Four idempotency layers	0004
3	Explicit state machine with CAS ownership	`MessageStateMachine`, CAS repository updates	0003
4	No content or PII on the bus	`DispatchEnvelope` carries identifiers only; variables are stored, rendered text is not	0007
5	Ambiguity is a state	`DELIVERY_UNKNOWN` plus reconciliation	0010
6	Everything bounded	Table 2.3: threads, prefetch, queue depth, body size, retries, bulk concurrency	0006
7	Optional-degradation dependencies	`ResilientCoordination` falls back to the database; Mongo writes are best effort	0004, 0005
8	Ports and adapters	One JVM for development; real brokers in production	0001, 0002

The envelope that travels on the bus is deliberately tiny (`com.example.messaging.messaging.DispatchEnvelope`):

JAVA

```
public record DispatchEnvelope(
        String dispatchId,      // messageId:attemptNo, the consumer dedupe key
        String messageId, String tenantId, Channel channel, Priority priority,
        int attemptNo, String correlationId, Instant enqueuedAt) { ... }
```

3.5 Technology choices at a glance

Concern	Choice	Alternative considered	Why
Runtime	Java 21, Spring Boot 3.5.16	Quarkus, Micronaut	Team familiarity, ecosystem
Primary database	PostgreSQL 16; H2 2.x in `MODE=PostgreSQL` for local and test	MySQL, Oracle, NoSQL	ACID plus portable SQL (ADR-0005)
Diagnostics store	MongoDB, raw redacted callback payloads only	Store in PostgreSQL	Schema-free vendor payloads, TTL; never on the critical path
Cache and coordination	Redis (fast path, token bucket) with in-memory and database fallback	Hazelcast	Lua token bucket, widely available
Asynchronous transport	RabbitMQ behind the `MessageBus` port	Kafka	Priorities, per-message TTL, dead-letter exchange, push consumers (ADR-0001)
Batch	Spring Batch 5, JDBC repository	Custom loop	Chunking, restart, skip

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Concern	Choice	Alternative considered	Why
Security	Spring Security 6 resource server, JWT RS256	Opaque tokens	Stateless, tenant claim
Resilience	Resilience4j circuit breaker plus time limiter	Hystrix (retired)	Maintained, composable
Observability	Micrometer, Prometheus, OpenTelemetry	Vendor APM	Open standards
Frontend	Angular 20, standalone components, Signals, Angular Material 20	React	Enterprise familiarity

Reference material

Architecture overview

Source document: `docs/architecture/architecture-overview.md`

Independent reference implementation of an enterprise omnichannel messaging and notification orchestration platform (EMAIL, SMS, PUSH). All data is synthetic; no real provider is called; simulators stand in for vendors. Binding detail: `docs/design/CONTRACT.md`.

Goals and non-goals

Goals	Non-goals
Accept messages fast (202) and never lose an accepted message	Real vendor integrations
Honest lifecycle: accepted vs sent-to-provider vs delivered vs unknown	Exactly-once across the provider boundary (impossible)
Policy enforcement (consent, suppression, quiet hours) at accept and dispatch	Marketing campaign authoring UI
Operable: metrics, runbooks, replay, reconciliation	Claim of production compliance (see `docs/security/prod-vs-reference.md`)
Provider-agnostic, tenant-isolated	Multi-region active/active

Logical architecture

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Key design properties

1. **Persist first, publish after** (ADR-0009): the DB is the source of truth; the broker is a delivery mechanism.
2. **At-least-once transport, idempotent processing** (ADR-0004): effectively-once business effect.
3. **Explicit state machine with CAS ownership** (ADR-0003).
4. **No content or PII on the bus**: `DispatchEnvelope` carries ids only. Variables are stored, rendered body is not; rendering happens at dispatch with the pinned template version (ADR-0007).
5. **Ambiguity is a state** (ADR-0010): `DELIVERY_UNKNOWN` plus reconciliation.
6. **Bounded everything**: consumer concurrency, prefetch, queue depth backpressure, request size, rate limits, bulk concurrency, retries.
7. **Redis and Mongo are optional-degradation dependencies**: `ResilientCoordination` falls back; diagnostics are best-effort.
8. **Ports and adapters** for bus, coordination, providers, diagnostics so local/test run in one JVM.

Component responsibilities

Package	Responsibility	Notable classes
`web`	HTTP, DTOs, error mapping	`GlobalExceptionHandler`
`security`	AuthN/Z, tenant, rate limit, body size	`TenantContext`, `AuthorizationService`, `RateLimitFilter`, `RequestSizeFilter`
`service`	Use cases and policy	`NotificationService`, `PolicyService`, `DispatchService`, `RetryPolicy`, `ReconciliationService`, `IdempotencyService`, `AuditService`
`domain`, `repository`	Entities, state machine, JPA	`MessageStateMachine`
`messaging`	Bus port, consumer, DLQ, replay	`MessageBus`, `DispatchConsumer`, `DeadLetterService`, `ReplayService`
`provider`	Channel ports, simulators, resilience	`ProviderRegistry`, `ResilientProviderInvoker`, `FaultInjector`
`callback`	Webhook auth and normalization	`SignatureVerifier`, `ReplayGuard`, `EventNormalizer`
`batch`	Bulk send	`BulkJobConfig`, `RecipientRowProcessor`
`scheduler`	Outbox, poller, sweeper, retention	`OutboxPublisher`, `StuckMessageSweeper`
`metrics`	All custom meters	`MessagingMetrics`

Runtime topology (threads and pools)

- Tomcat request threads (API); bounded by server config.
- `DispatchConsumer` workers: `messaging.dispatch.concurrency` (default 8).
- Provider call executor (time limiter): bounded `ThreadPoolExecutor`; separate from consumer workers to avoid starvation by hanging providers.
- Scheduler threads: outbox, poller, sweeper, retention.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Batch `TaskExecutor`: `messaging.bulk.concurrency` (default 4).
- HikariCP pool sizing: see `docs/database/transactions-and-connection-pooling.md`.

Related documents

Flows: `submission-flow.md`, `dispatch-retry-flow.md`, `callback-flow.md`, `reconciliation-flow.md`, `bulk-job-flow.md`, `template-approval-flow.md`. Data: `er-diagram.md`. Ops views: `deployment-views.md`. Semantics: `delivery-semantics.md`. Failure analysis: `reliability-scenarios.md`.

Delivery semantics explainer

Source document: `docs/architecture/delivery-semantics.md`

Three guarantees, plainly

Term	Meaning	Where it applies here
At-most-once	Never duplicated, may be lost	Not used. A "fire and forget" publish before DB commit would be this (rejected, ADR-0009)
At-least-once	Never lost (once persisted), may be duplicated	The transport: outbox to broker, broker to consumer, provider callbacks. Redelivery and replay are expected
Exactly-once	Each effect happens once	Impossible end to end across a network boundary to a third party; we do not claim it
Effectively-once (business)	At-least-once delivery + idempotent processing = one visible effect	What we implement: dedupe at API, consumer, callback, provider ref (ADR-0004)

Three different facts about a message

Fact	Status that records it	Evidence	What it does NOT mean
Platform accepted it	`ACCEPTED/VALIDATED` (202)	committed row + outbox	not sent
Provider accepted it	`SENT_TO_PROVIDER`	`ProviderOutcome.ACCEPTED` + `provider_message_id`	not delivered; vendor may still bounce or drop
Delivered	`DELIVERED`	provider-originated receipt callback or reconciliation lookup	for email, delivered to mailbox server is not "read"; OPENED/CLICKED are separate engagement events
Unknown	`DELIVERY_UNKNOWN`	timeout/ambiguous result, crash, or missing receipt	not failed, not delivered: it is unresolved

Rule: provider acceptance never implies delivery; absence of a callback is never delivery; a timeout is never failure.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Where duplicates can still happen (honest list)

1. Provider ignores `idempotencyRef` and we retry after "not received" that was wrong (ADR-0010 residual risk).
2. Operator replays a message that was actually delivered (replay UI shows attempt history and warns for `DELIVERY_UNKNOWN`; replay is audited).
3. Client reuses a different `Idempotency-Key` for the same logical send (client bug).
4. Idempotency key expiry (TTL) then re-use.

Ordering

No global ordering guarantee. Priority queues reorder by priority; retries reorder in time. Per-recipient ordering is not guaranteed; clients that need ordering must serialize submissions. Callbacks may arrive out of order; the rank rule and `occurred_at` handle this (CONTRACT 3.4).

Failure windows and their handling

Crash/failure point	Result	Handling
After DB commit, before publish	Message sits VALIDATED with outbox NEW	Publisher retries (no loss)
After publish, before outbox SENT	Duplicate envelope	Consumer dedupe
After consumer CAS, before provider call	Message PROCESSING, no attempt effect	Sweeper to DELIVERY_UNKNOWN is conservative; attempt row STARTED without provider id lets reconciliation conclude "not received"
After provider accept, before persist	Provider has it, we do not know	DELIVERY_UNKNOWN then lookup/callback
After persist, before ack	Redelivery	CAS/processed_message dedupe
Provider callback lost	Receipt never arrives	`receipt-timeout` to DELIVERY_UNKNOWN to lookup

See `reliability-scenarios.md` and `docs/architecture/reconciliation-flow.md`.

High-level design

Source document: `docs/leadership/hld.md`

Complements `docs/architecture/architecture-overview.md` (diagrams) with decisions, quality attributes and trade-offs.

1. Context

Producers (apps via REST), consumers of status (apps, console), external providers (email/SMS/push vendors, simulated), infra (PostgreSQL, RabbitMQ, Redis, MongoDB), observability stack.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. Architectural style

Modular monolith (single Spring Boot module `messaging-platform`, packages per responsibility) deployed as N identical instances running API, consumers and schedulers. Chosen over microservices because the reference needs transactional simplicity (state + outbox in one DB transaction) and a single-JVM local run; module boundaries (packages, ports) keep a later split possible (consumer service, callback service, batch worker).

3. Key flows

Submission, dispatch/retry, callbacks, reconciliation, bulk, template approval: see architecture folder.

4. Quality attributes and tactics

Attribute	Tactics	ADR
Durability	outbox, DB source of truth	0009
Correctness under duplicates	four-layer idempotency	0004
Availability	stateless instances, circuit breakers, fallback coordination, graceful shutdown	0006, 0008
Latency	async accept, no provider call in request, short transactions	0001
Scalability	competing consumers, priority queues, partition-free DB design, bulk streaming	0001
Security	JWT, tenant scoping, HMAC callbacks, minimization	0011
Operability	metrics, runbooks, replay, reconciliation	0010
Extensibility	ports/adapters	0002
Auditability	append-only audit, pinned template versions	0003, 0007

5. Data

PostgreSQL authoritative; Redis ephemeral; Mongo diagnostics best-effort. See `docs/database/`.

6. Capacity thinking (illustrative)

Assume 500 msg/s sustained peak, 50% bursts to 2,000 msg/s via bulk. Consumers: per-message latency ~300 ms provider call => 8 workers per instance ~26 msg/s; 20 instances ~ 500 msg/s (bounded by provider quota, not by us). DB: ~6 short writes per message => ~3,000 writes/s peak; fits a single tuned PostgreSQL primary but retention and partitioning become important. These are back-of-envelope numbers, not measurements.

7. Failure modes

Documented in `docs/architecture/reliability-scenarios.md` with proving tests.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8. Deployment

Kubernetes primary, CF equivalent (ADR-0008). Profiles local/test/prod-like.

9. Alternatives and evolution

Kafka event stream sink from outbox for analytics; CDC outbox; RLS or schema-per-tenant; split consumers; multi-region active/passive.

10. Risks

`risk-register.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Angular and Backend Architecture

4.1 The Angular console

The console (`frontend/messaging-console`) is an Angular 20 application built from **standalone components**, **Signals** and **lazy routes**, with Angular Material 20 for components. It is the operator and author interface; service clients use the REST API directly. Its role is to make the workflows of this document tangible: submit a message, watch its lifecycle timeline, approve a template, inspect a dead letter, replay it.

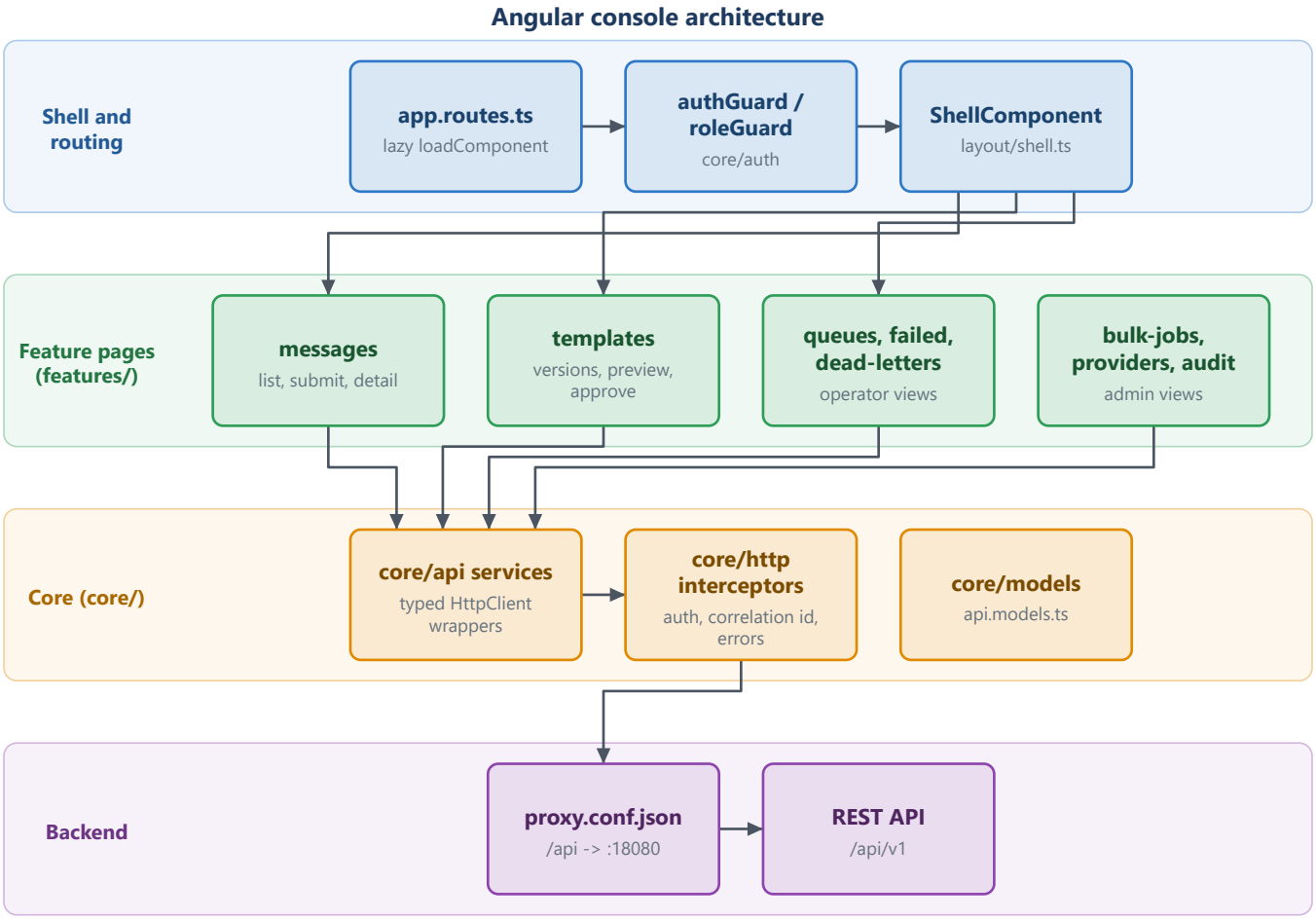


Figure 4. Angular 20 console: lazy standalone routes, role-aware guards (UX only), interceptors and a thin API layer. The API enforces authorization; guards only hide screens. Implementation: frontend/messaging-console/src/app/app.routes.ts, frontend/messaging-console/src/app/core/

Routes

Every feature page is lazy loaded (`loadComponent`) from `frontend/messaging-console/src/app/app.routes.ts`. Role guards use `data.roles` taken from the `ACCESS` matrix in `core/auth/access.ts`, which mirrors the endpoint role table of the contract. The guards are **UX only**: the backend answers 403 or 404 whatever the UI shows.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Route	Page	Roles that see it
`/login`	Dev-token persona picker	everyone (local profile only)
`/dashboard`	Counts, success rate, p95 dispatch delay	any authenticated
`/messages`, `/messages/:id`	List with filters; detail with lifecycle timeline	any authenticated
`/messages/new`	Reactive submit form, idempotency key generated	SENDER, TENANT_ADMIN, PLATFORM_ADMIN
`/templates`, `/templates/:id`	Versions, locale tabs, preview, submit and approve	any authenticated (actions by role)
`/preferences`	Recipient search and edit with optimistic-lock conflict UX	OPERATOR, TENANT_ADMIN, PLATFORM_ADMIN, SENDER (edit: not SENDER)
`/bulk-jobs`, `/bulk-jobs/:id`	Jobs with progress polling	SENDER, TENANT_ADMIN, PLATFORM_ADMIN
`/providers`	Health, circuit state, fault injection when allowed	OPERATOR, TENANT_ADMIN, PLATFORM_ADMIN
`/queues`	Depth per priority, retry and DLQ counts	OPERATOR, PLATFORM_ADMIN
`/failed`, `/dead-letters`	Investigation lists, replay, discard	OPERATOR, TENANT_ADMIN, PLATFORM_ADMIN
`/delivery-events`, `/audit`	Search	OPERATOR, AUDITOR, TENANT_ADMIN, PLATFORM_ADMIN / AUDITOR, TENANT_ADMIN, PLATFORM_ADMIN
`/admin`	Pause consumers, provider config	PLATFORM_ADMIN

Cross-cutting frontend pieces

Concern	Where	Behaviour
Authentication	`core/auth/auth.service.ts`, `guards.ts`	Dev JWT held in memory and `sessionStorage`; guards redirect to `/login` or `/forbidden`
HTTP	`core/http/interceptors.ts`	Adds `Authorization`, a correlation id; maps errors; drives the loading indicator
Error text	`core/http/error-mapper.ts`	Maps the backend `ApiError.code` to an actionable sentence
Form validation	`core/forms/validators.ts`	Per-channel validators (E.164 phone, subject rules) mirroring server rules
Shared widgets	`shared/`	`data-table`, `pager`, `status-chip`, `timeline`, `masked-value`, `confirm-dialog`, `error-banner`, `charts`
Tests	`*.spec.ts`	Karma and Jasmine unit tests for services, interceptors, guards and key components

Known frontend deviation

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

> The deviations record states that the history screen's `HistoryEntry` model expects a flat shape while `GET /messages/{id}/history` returns the specified `TimelineEntry {at, kind, type, summary, detail: object}`. The mapping change is recorded as still needed in the frontend; see Appendix B.

Console screenshots

The following screenshots were recorded from the running console by a UI check against the local backend and are stored in `docs/validation/screenshots/` (see `docs/validation/VALIDATION_REPORT.md` for how they were captured). All data shown is synthetic.

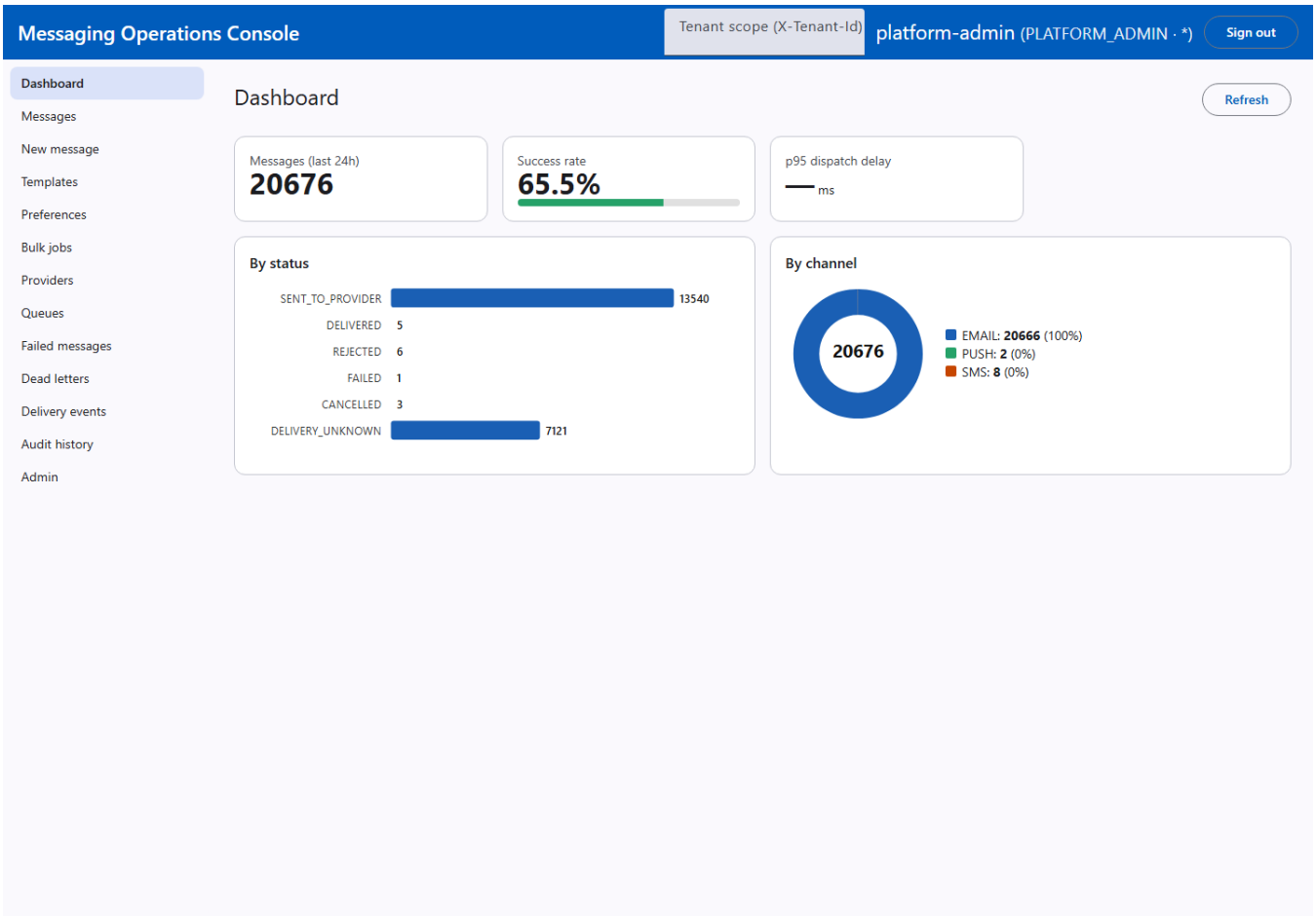


Figure 5. Dashboard: counts by status and channel, success rate and p95 dispatch delay for the platform-admin persona. Source: docs/validation/screenshots/02-dashboard.png

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

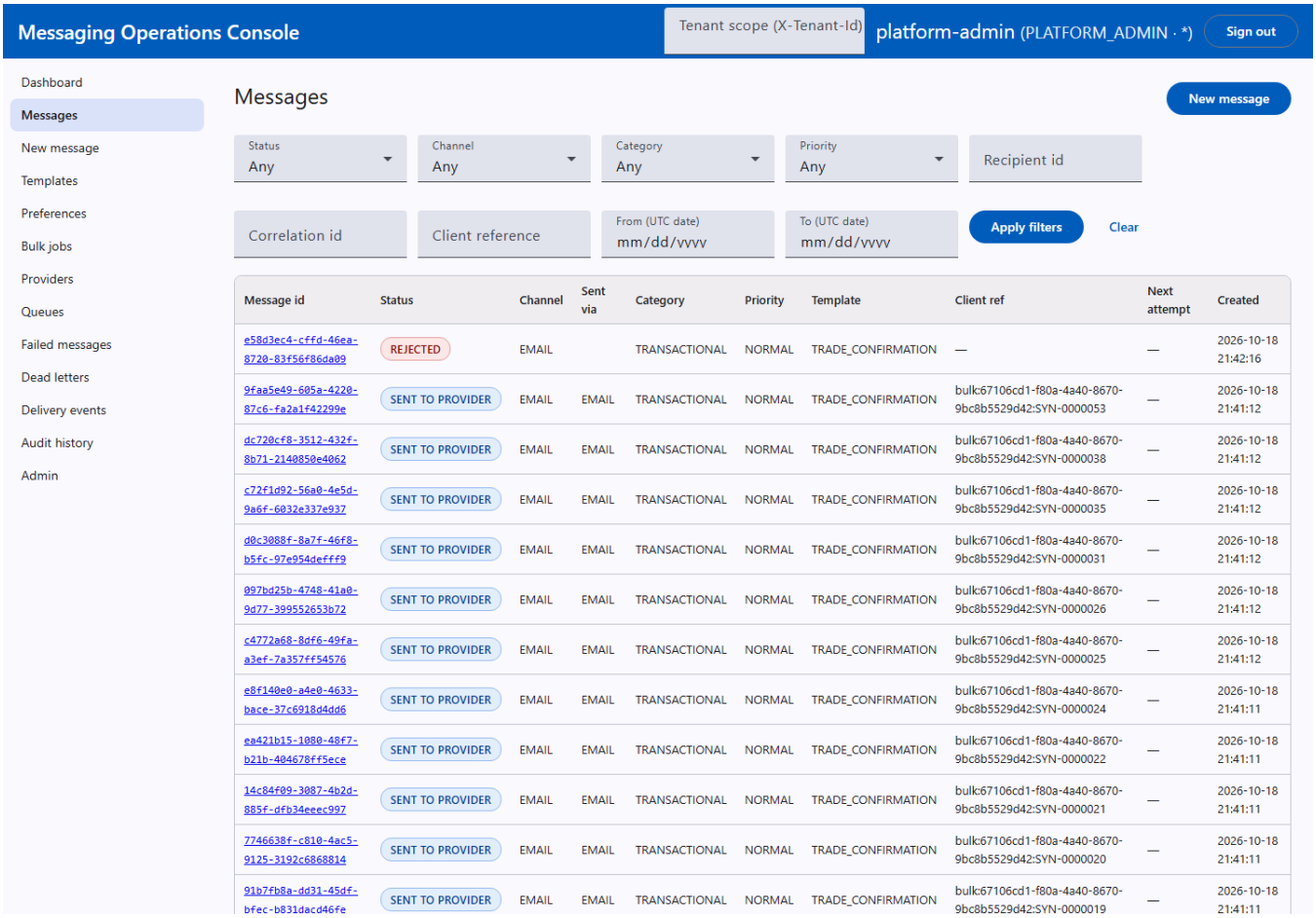


Figure 6. Messages list with filters and the PageResponse paging. Source: docs/validation/screenshots/03-messages.png

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

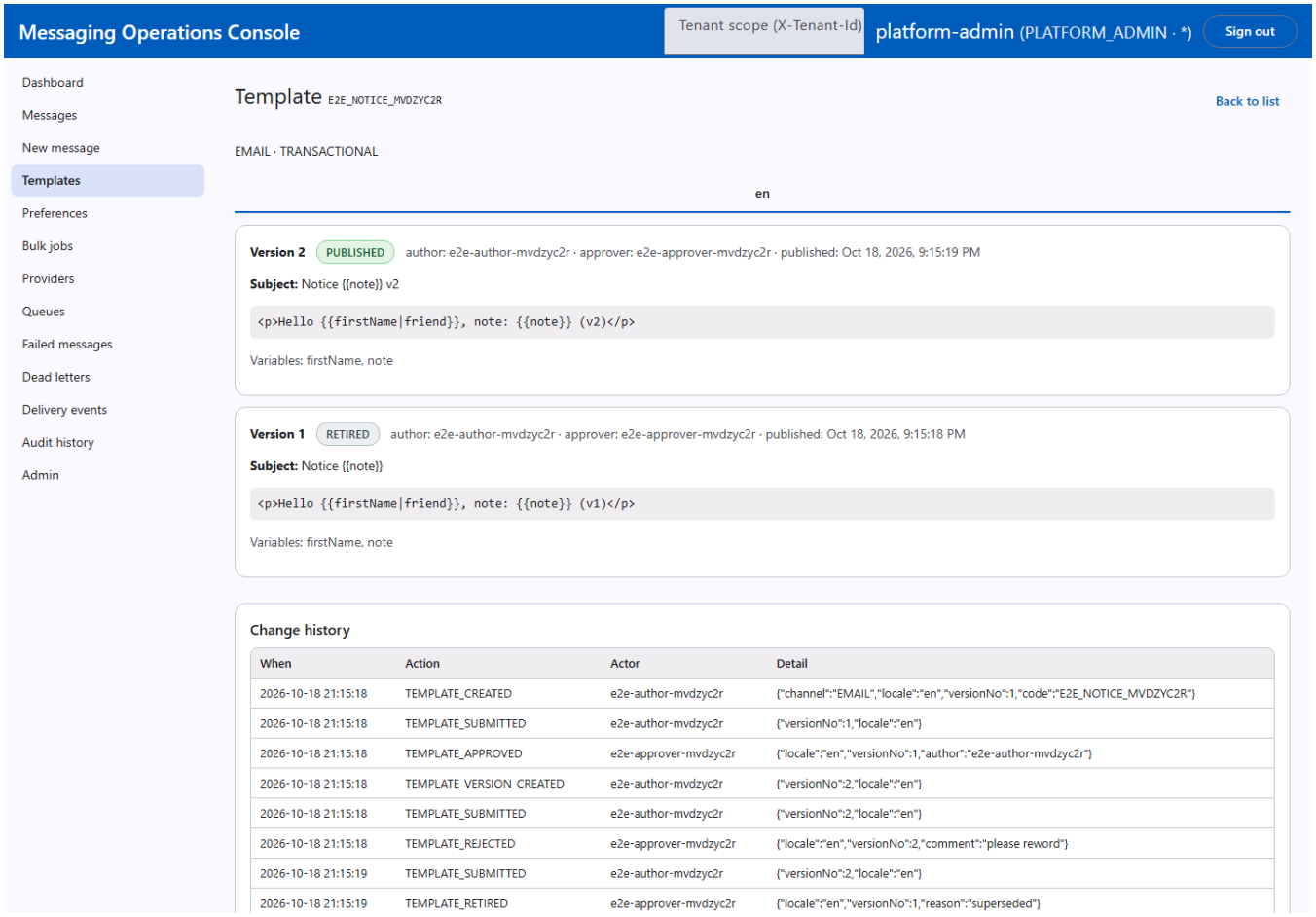


Figure 7. Template detail: versions, locale tabs, preview and approval actions. Source: docs/validation/screenshots/07-template-detail.png

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

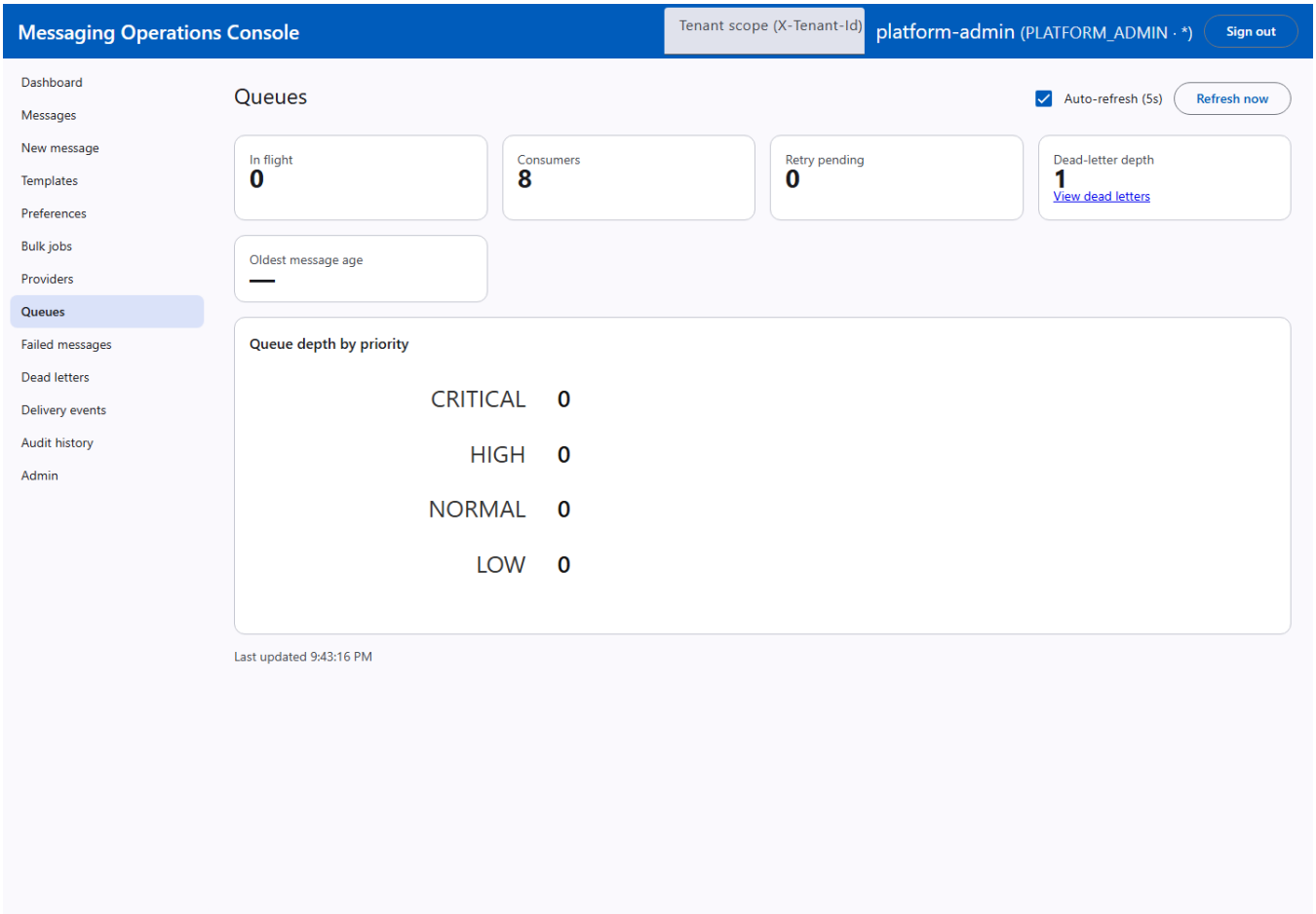


Figure 8. Queues view: depth per priority, in-flight, consumers, DLQ and retry-pending counts. Source: docs/validation/screenshots/08-queues.png

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

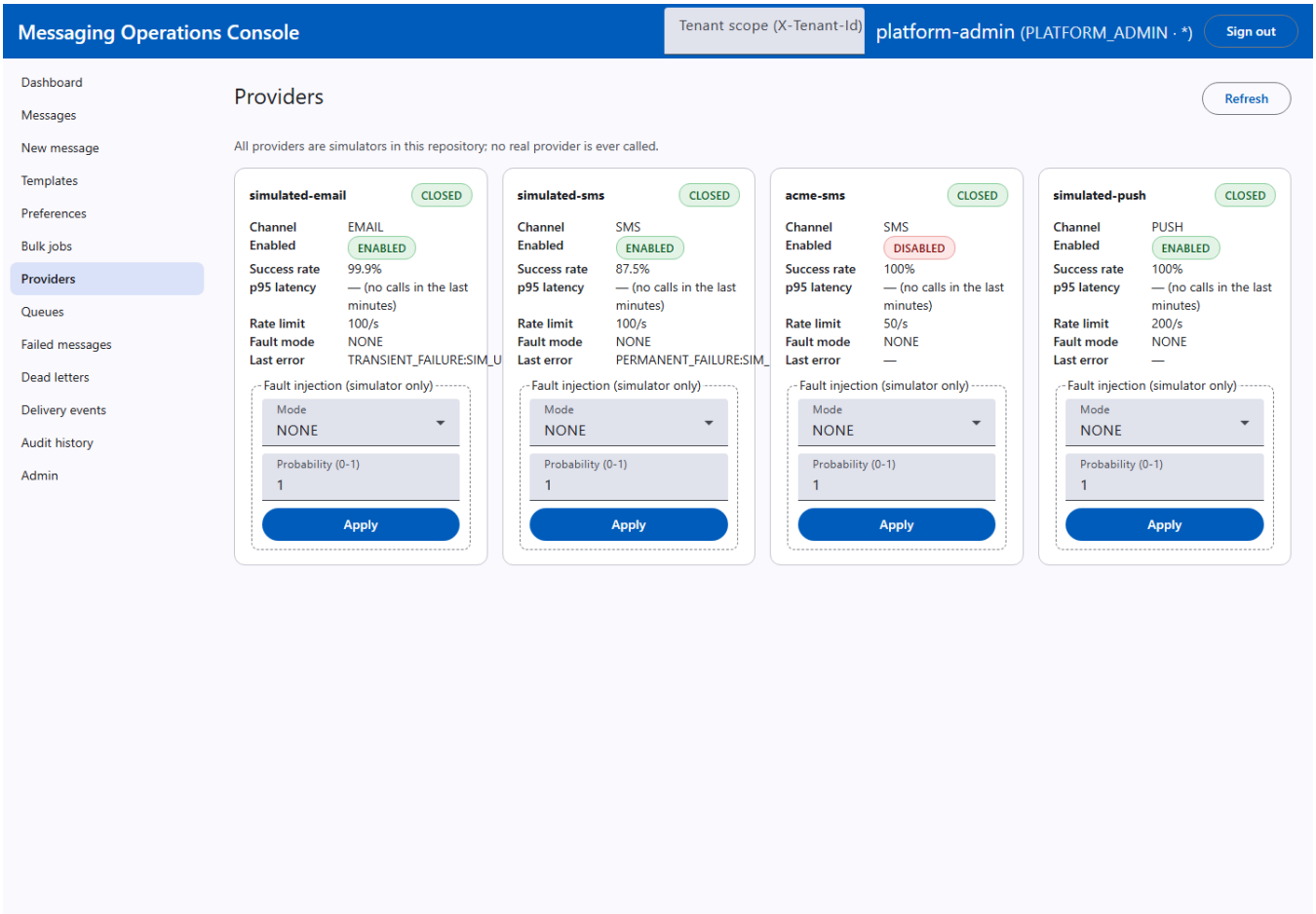


Figure 9. Providers view: health, circuit state, rate limit and simulator fault mode. Source: docs/validation/screenshots/09-providers.png

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

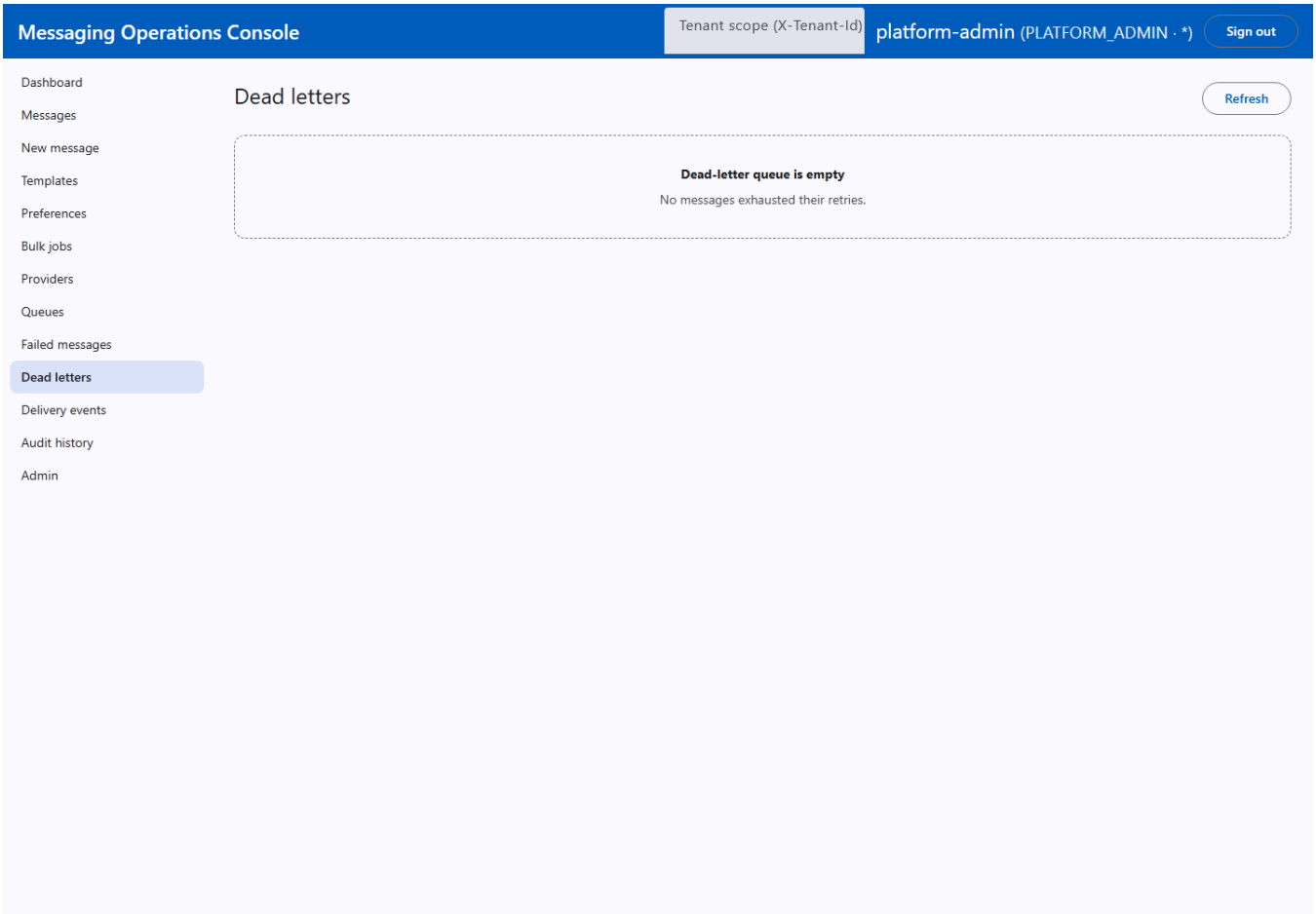


Figure 10. Dead letters view with replay and discard actions for operators. Source: docs/validation/screenshots/11-dead-letters.png

4.2 Backend layering

A request passes through a fixed chain. Filters are ordered so that cheap rejections happen first and so that every log line carries a correlation id.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

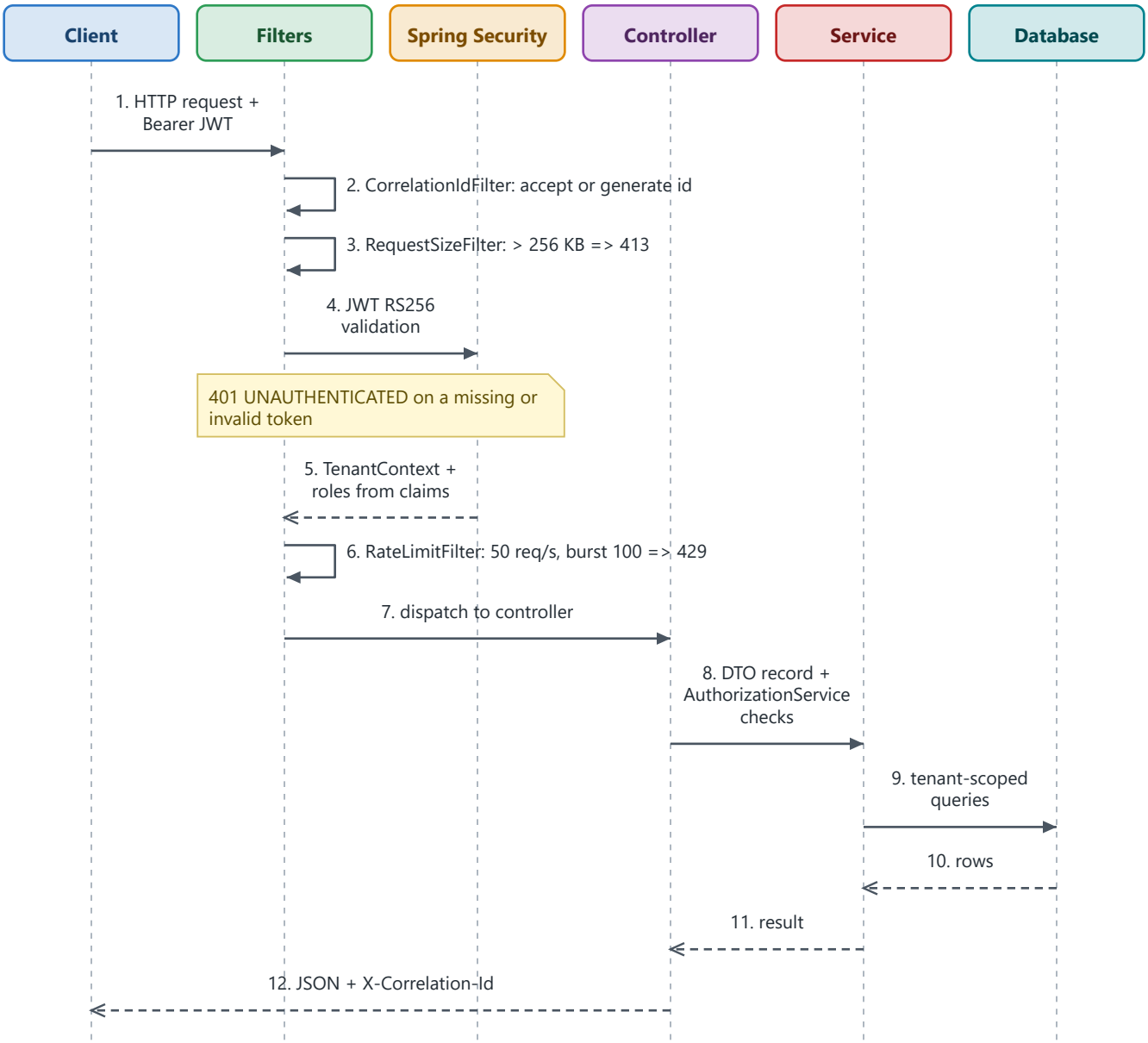


Figure 11. Request handling chain for an authenticated API call: filters first, then Spring Security, controller, service, repository. Implementation: backend/src/main/java/com/example/messaging/common/CorrelationIdFilter.java, backend/src/main/java/com/example/messaging/security/RequestSizeFilter.java, backend/src/main/java/com/example/messaging/security/RateLimitFilter.java, backend/src/main/java/com/example/messaging/config/SecurityConfig.java

Controllers and DTOs

Controllers in `com.example.messaging.web` are thin: they bind a DTO record, check coarse authorization, call one service method and map the result. All DTOs are Java records under `web/dto` (`MessageDtos`, `TemplateDtos`, `RecipientDtos`, `BulkJobDtos`, `OpsDtos`, `AuditDtos`). Error mapping is centralised in `GlobalExceptionHandler`, which converts exceptions to the stable `ApiError` payload (Chapter 24 lists the codes).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Services

`NotificationService` is the heart: submission, query, cancel, requeue, replay, discard, and the single `transition(...)` method through which **every** status change passes. `transition` takes the expected `from` status, so it is a compare-and-set and returns whether this caller won. No other class writes the status column.

```java
public boolean transition(String messageId, MessageStatus from, MessageStatus to,
        String reason, String actor) { ... }
```

Domain and persistence

Entities extend `PersistableEntity`. Optimistic locking (`@Version`) protects `recipient`, `recipient_preference`, `template_version`, `message_request` and `bulk_job`. Repositories avoid N+1 queries with fetch joins, entity graphs and projections (`MessageStatusView`, `StatusCount`, `ChannelCount`); `RepositoryQueryCountIT` guards this.

JPA runs with `ddl-auto: validate` (Flyway owns the schema), `open-in-view: false`, and `hibernate.jdbc.time_zone=UTC`.

4.3 Configuration model

Configuration is typed. `messaging.*` keys bind to records in `config/MessagingProperties`, `config/PipelineProperties`, `config/SimulatorProperties`, `batch/BulkProperties` and `scheduler/RetentionProperties`. There is **no default profile**: the application must be started with `--spring.profiles.active=local`, `test` or `prod-like`, so that a forgotten profile cannot enable the dev token issuer (a secure-by-default decision recorded in the deviations).

Profile	Database	Bus	Coordination	Mongo	Security
`local`	H2 (PostgreSQL mode)	`InMemoryMessageBus`	in-memory	in-memory server	`POST /dev/token` enabled
`test`	H2, unique per context	in-memory	in-memory	in-memory	deterministic, pipeline auto-start off unless the test class turns it on
`prod-like` (group `rabbit`)	PostgreSQL	`RabbitMessageBus`	Redis	MongoDB	JWT `issuer-uri`, no dev endpoints

4.4 Error handling philosophy

Errors are part of the API. The rules, implemented in `GlobalExceptionHandler`:

1. Every error body is an `ApiError {timestamp, status, error, code, message, path, correlationId, details[]}`.
2. `code` is stable and machine-readable; `message` is for people and never contains a stack trace (`server.error.include-*` are `never`).
3. Database connection failures answer **503** with `Retry-After: 5` (fail closed; clients retry with the same idempotency key) instead of a 500.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Resources owned by another tenant answer **404**, not 403, so existence is not disclosed.
5. Unique-constraint conflicts map to the additive code `DUPLICATE_RESOURCE` (409), not to a misleading optimistic-lock code.

Reference material

Low-level design (design intent)

Source document: `docs/leadership/lld.md`

This document records design intent. The implemented consumer differs in order: cancel, expiry, policy, provider selection, circuit state and the token bucket are checked before the claim, and the claim is one transaction (see Chapter 7 and DEVIATIONS "Backend stage 2", item 1).

Class-level view for the critical paths. Names follow CONTRACT section 2; method signatures are design intent: confirm in source before relying on them.

1. Domain

- `MessageStatus` enum, `MessageStateMachine`: `static boolean canTransition(from,to)`, `MessageStatus require(from,to)` throws `IllegalStateTransitionException`. Table-driven `EnumMap<MessageStatus, EnumSet<MessageStatus>>`.
- `MessageRequest` entity (`@Version`): setters for status are package-private; transitions only through `MessageTransitions` (service) using CAS repository method `int casStatus(id, from, to, reason, now)`.

2. Submission (`NotificationService.submit`)

```
@Transactional
submit(principal, idemKey, request):
  claim = idempotencyService.claim(tenant, idemKey, hash(request))  // returns REPLAY / NEW / CONFLICT /
    IN_PROGRESS
  recipient = recipientRepo.findByIdAndTenantId(...)  // 404 if absent
  decision = policyService.evaluate(...)
  tplVersion = templateService.resolvePublished(tenant, code, locale, version)
  variableValidator.validate(tplVersion, variables)
  msg = MessageRequest.accepted(...) ; transition(ACCEPTED -> VALIDATED)
  outboxRepo.save(OutboxEvent.dispatch(msg)) ; auditService.record(...)
  idempotencyService.complete(claim, response)
```

3. Outbox publisher

```
@Scheduled(fixedDelay=200)
publishBatch():
  ids = outboxRepo.claimBatch(limit, now)           // CAS NEW -> CLAIMED (or equivalent, see source)
  for each: bus.publish(envelope) ; casStatus(message, VALIDATED, QUEUED) ; outbox SENT
  on failure: release claim (leave NEW) + metric; no exception escapes the scheduler thread
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Dispatch (`DispatchConsumer.onMessage` -> `DispatchService.process`)

```
onMessage(env):
  if (!processedMessage.tryInsert(env.dispatchId)) { ack; return; }
  if (!repo.casStatus(env.messageId, QUEUED, PROCESSING)) { ack; return; }
  try { dispatchService.process(env); ack }
  catch (PoisonException e)    { dlq(POISON_MESSAGE); ack }
  catch (RuntimeException e)   { failureCounter.increment(); nackRequeueOrDlq(env) }
process(env):
  msg = load ; policy re-check ; rendered = renderer.render(pinned) ;
  if (!limiter.tryAcquire(provider)) { bus.requeueWithDelay(env, shortDelay); revertToQueued; return; }
  attempt = attemptRepo.start(msg, attemptNo)               // STARTED, committed
  result = invoker.send(provider, request, idempotencyRef)  // timeout + breaker, NO transaction
  finish(attempt, result) ; apply(result)                   // new short transaction
```

5. Retry policy

```
delayMs(n) = min(capMs, baseMs * 2^(n-1));  nextAt = now + delayMs(n) * (0.5 + 0.5*random.nextDouble())
```

`RetryPolicy` receives `Clock` and `RandomGenerator` for deterministic tests.

6. Callback processing

`CallbackController` reads raw bytes (`@RequestBody byte[]`), verifies signature, then normalizer, then `DeliveryEventProcessor.apply(event)`:

```
@Transactional apply(e):
  if (!eventRepo.insertIfAbsent(e)) return DUPLICATE
  msg = locate(e)
  decision = rank(e.type) vs current status/timestamp
  if applicable: repo.casStatus(...) else markIgnored(reason)
  audit
```

7. Reconciliation and sweeper

`StuckMessageSweeper.sweep()` (`@Scheduled`): PROCESSING with `locked_until < now` and SENT_TO_PROVIDER older than `receipt-timeout` => DELIVERY_UNKNOWN via CAS. `ReconciliationService.reconcile()` handles DELIVERY_UNKNOWN older than `reconcile-after`.

8. Provider invocation

`ResilientProviderInvoker.send` wraps `CircuitBreaker` + `TimeLimiter` on `providerExecutor` (bounded `ThreadPoolExecutor`, `AbortPolicy` mapped to RATE_LIMITED/TRANSIENT). Exceptions map to `ProviderOutcome` (timeouts => TIMEOUT; connection refused before send => TRANSIENT_FAILURE; unknown/after-send errors => AMBIGUOUS).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

9. Concurrency table

Thread pool	Size	Bound/Queue	Rejection
Tomcat	server config	accept queue	503
Dispatch workers	`dispatch.concurrency`	prefetch	backpressure
Provider executor	>= workers	small queue	map to retry
Scheduler	small fixed	none	skip tick
Batch	`bulk.concurrency`	chunk	job slows

10. Error mapping

`GlobalExceptionHandler`: `ApiException` subclasses to `ApiError` + status; `IllegalStateTransitionException` => 409; optimistic lock => 409 `CONFLICT_OPTIMISTIC_LOCK`; unexpected => 500 `INTERNAL_ERROR` with correlation id only.

API contract guide

Source document: `docs/leadership/api-contract-guide.md`

Authoritative spec: `backend/src/main/resources/openapi/messaging-api-v1.yaml` (copy at `docs/api/openapi.yaml`). Endpoint list and roles: CONTRACT section 5. This guide explains conventions and the change policy.

Conventions

Topic	Rule
Base path	`/api/v1`; `API-Version: 1` response header
Media type	`application/json`; errors as `ApiError`
Auth	Bearer JWT; callbacks use HMAC
IDs	UUID strings; opaque to clients
Timestamps	ISO-8601 UTC (`Z`)
Enums	UPPER_SNAKE; unknown values rejected `MALFORMED_REQUEST`; clients must tolerate NEW enum values in responses
Pagination	`page` (0-based), `size` (<= 200), `sort=field,dir`; `PageResponse`
Correlation	send `X-Correlation-Id` (<= 64 chars, `[A-Za-z0-9-_.]`) or receive generated; echoed
Idempotency	`Idempotency-Key` required on `POST /messages`; unique per logical send
Rate limiting	`X-RateLimit-*`; 429 + `Retry-After`
Async semantics	202 means accepted, not sent. Poll status or use callbacks (future: webhooks to clients not provided)
Not-found policy	foreign-tenant resources return 404

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Error handling for clients

HTTP	code	Client action
400	`VALIDATION_FAILED`, `MALFORMED_REQUEST`, `TEMPLATE_VARIABLE_MISSING`, `UNSUPPORTED_CHANNEL`	fix request; do not retry
401	`UNAUTHENTICATED`	refresh token
403	`FORBIDDEN`	needs role/scope
404	`NOT_FOUND`, `TEMPLATE_NOT_FOUND`, `RECIPIENT_NOT_FOUND`	check ids
409	`IDEMPOTENCY_IN_PROGRESS`, `ILLEGAL_STATE_TRANSITION`, `CONFLICT_OPTIMISTIC_LOCK`, `DUPLICATE_RESOURCE` (unique-constraint conflict, e.g. duplicate template code)	wait/refetch/change the identifier
413	`PAYLOAD_TOO_LARGE`	reduce
422	`IDEMPOTENCY_KEY_REUSED`	new key

Policy rejections are **not** errors: `POST /messages` answers 202 with `status=REJECTED` and `statusReason` (the enum value `POLICY_SUPPRESSED` is unused). An idempotent replay of a submission answers 200 with `Idempotent-Replay: true`; a replayed provider callback answers 200 `{"result":"replayed"}` (`REPLAYED_EVENT` is unused). | 429 | `RATE_LIMITED` | backoff using `Retry-After` | | 5xx / 503 | `INTERNAL_ERROR`, `SERVICE_UNAVAILABLE` | retry with same Idempotency-Key, exponential backoff + jitter |

Safe retry matrix

Method	Retry on timeout?
`POST /messages`	yes, same Idempotency-Key
`POST /messages/{id}/cancel`	yes (idempotent: 200 or 409)
`PUT preferences`	after refetch (optimistic lock)
bulk job `POST`	no auto-retry; check `GET /bulk-jobs` first

Compatibility and change policy

Non-breaking: new optional request fields, new response fields, new endpoints, new enum values in responses (documented), new error codes. Breaking: removing/renaming fields, changing types/semantics, tightening validation, changing status codes, changing default behavior. Breaking changes require `/api/v2`, 6-month deprecation (`Deprecation`, `Sunset` headers; illustrative), ADR, consumer notification. Contract tests: committed OpenAPI vs springdoc path assertion test; diff check in review (`ai-engineering/standards/api-design.md`).

Webhook (provider) contract

Not a client API: `POST /api/v1/callbacks/{providerId}`; see `docs/architecture/callback-flow.md`.

Examples

Ready-to-run HTTP files live in `tests/api-http` (see source). Example request: CONTRACT section 5.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

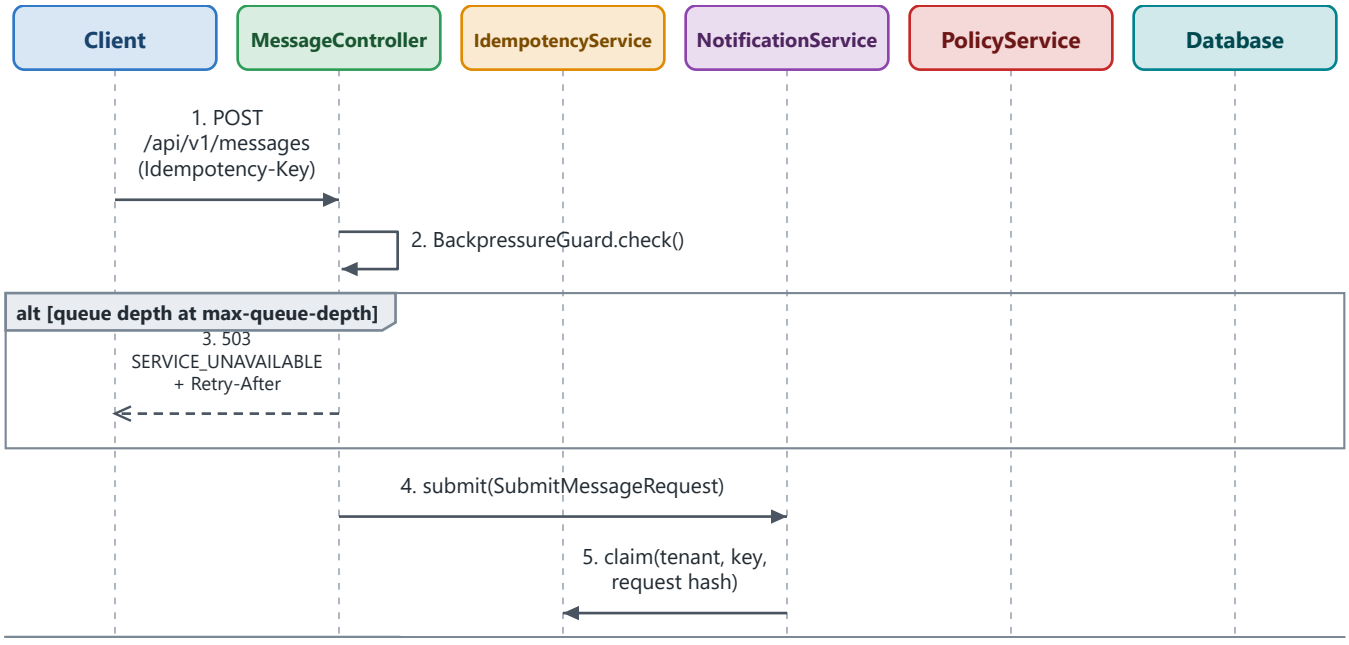
5. Notification Submission Flow

5.1 What the caller sees

`POST /api/v1/messages` is the front door. It is open to `SENDER`, `TENANT_ADMIN` and `PLATFORM_ADMIN` (`@PreAuthorize` on `MessageController.submit`) and it answers **202 Accepted** as soon as the message is safely persisted. The request thread never talks to a provider: delivery happens later, asynchronously, and the caller learns the outcome by polling `GET /api/v1/messages/{id}/status` or reading the lifecycle timeline.

Aspect	Rule	Source
Required header	`Idempotency-Key` (1 to 128 characters)	`MessageController`
Body limit	256 KB, else 413 `PAYLOAD_TOO_LARGE`	`RequestSizeFilter`
Rate limit	50 req/s per client, burst 100, else 429 with `Retry-After`	`RateLimitFilter`
Tenant	Always taken from the JWT `tenant` claim, never from the body	`TenantContext`
Success	202 with `Location: /api/v1/messages/{id}`	`MessageController`
Replay	200 with `Idempotent-Replay: true` and the original body	`MessageController`
Backpressure	503 `SERVICE_UNAVAILABLE` with `Retry-After` when queue depth plus due outbox rows reach `messaging.backpressure.max-queue-depth` (5000)	`BackpressureGuard`
Database down	503 with `Retry-After: 5` (fail closed)	`GlobalExceptionHandler`

5.2 The submission transaction



Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

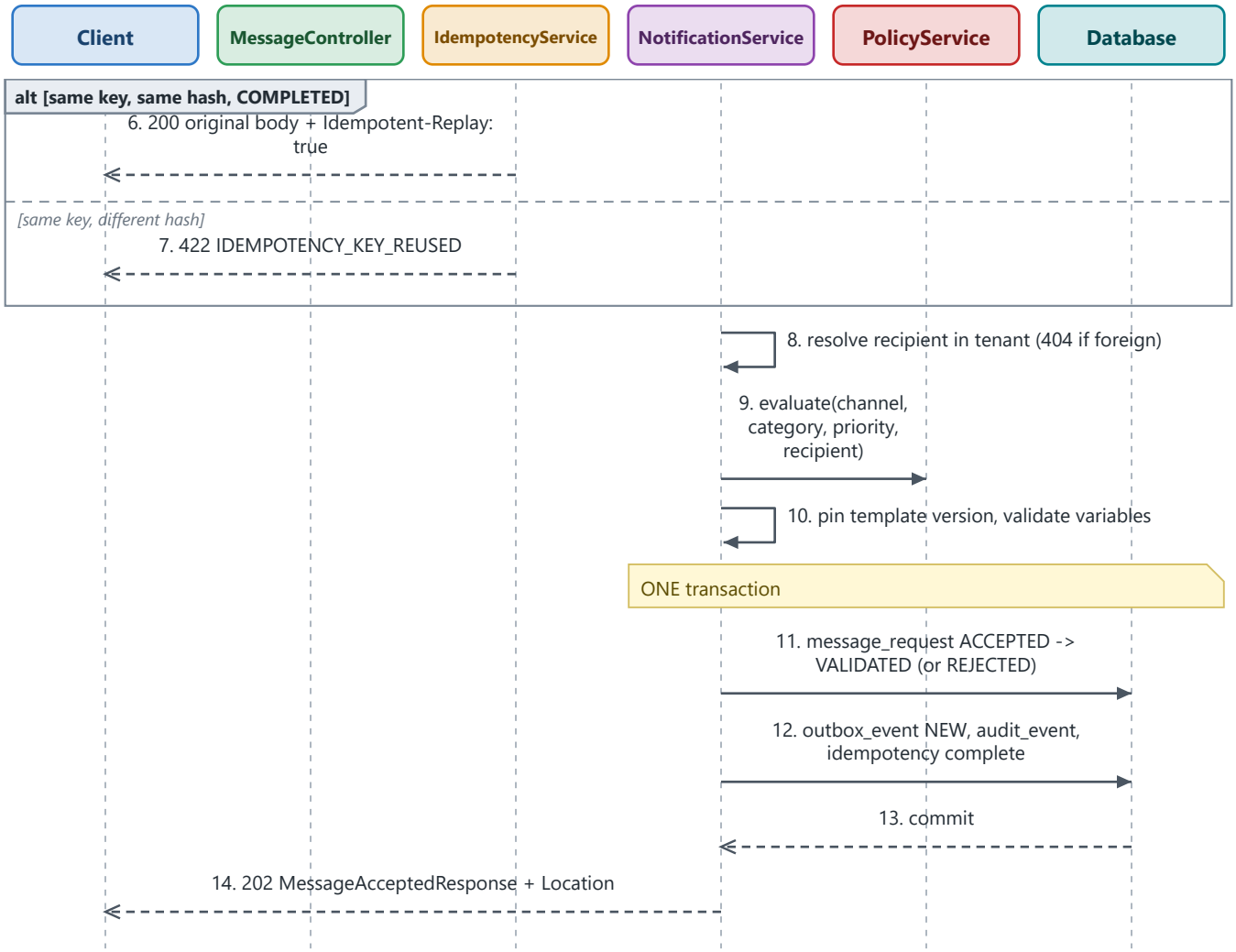


Figure 12. Notification submission: one database transaction writes the message, the outbox row, the audit event and the idempotency response; the provider is never called on the request thread. Implementation: backend/src/main/java/com/example/messaging/web/MessageController.java, backend/src/main/java/com/example/messaging/service/NotificationService.java, backend/src/main/java/com/example/messaging/service/IdempotencyService.java

The sequence has a deliberate shape. Everything that can **reject cheaply** happens first (size, rate limit, authentication, backpressure, idempotency claim). Everything that **changes state** happens in one transaction at the end. If anything in that transaction fails, the whole transaction rolls back, including the idempotency claim, so the client's retry is a clean retry.

JAVA

```
// MessageController.submit (excerpt)
backpressure.check(); // 503 + Retry-After when the dispatch backlog is full
NotificationService.SubmitResult result = notifications.submit(request, idempotencyKey);
if (result.replay()) {
    return ResponseEntity.ok().header("Idempotent-Replay", "true")
            .location(URI.create("/api/v1/messages/" + body.id())).body(body);
}
return ResponseEntity.status(HttpStatus.ACCEPTED)
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
        .location(URI.create("/api/v1/messages/" + body.id())).body(body);
```

State written by one submission

Row	Table	Content
Message	`message_request`	`ACCEPTED` then `VALIDATED` (or `REJECTED`), template pinned (code, version, locale), variables stored, rendered text **not** stored
Outbox	`outbox_event`	`NEW`, `available_at` = now, or the scheduled time
Audit	`audit_event`	`MESSAGE_ACCEPTED` (or `MESSAGE_REJECTED`)
Idempotency	`idempotency_record`	request hash and the full response, status `COMPLETED`

5.3 Implemented behaviour: policy rejection returns 202

The build contract originally allowed `POLICY_SUPPRESSED` as a 4xx. The implementation decided otherwise (DEVIATIONS, "Backend foundation", item 2): policy rejections at submission (opt-out, suppression, deleted recipient, no address, expiry before a deferred send) are **persisted** as `REJECTED` messages and answered **202** with `status=REJECTED` and a `statusReason`. This keeps the idempotent replay and the audit trail consistent: the same request with the same key returns the same answer, and the rejection is visible in the history.

JAVA

```
// NotificationService (excerpt): policy is evaluated for the time the message would actually be sent
PolicyDecision decision = policy.evaluate(recipient, req.channel(), req.category(), req.priority(), sendAt);
MessageStatus status = MessageStatus.VALIDATED;
...
if (decision.isRejected()) { status = MessageStatus.REJECTED; reason = decision.reason().name(); }
else if (decision.isDeferred()) { scheduledAt = decision.until(); reason = "DEFERRED_QUIET_HOURS"; ... }
MessageStateMachine.assertTransition(MessageStatus.ACCEPTED, status);
```

The error code `POLICY_SUPPRESSED` therefore exists in the `ErrorCode` enum but is not returned by submission.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5.4 Idempotency: four layers

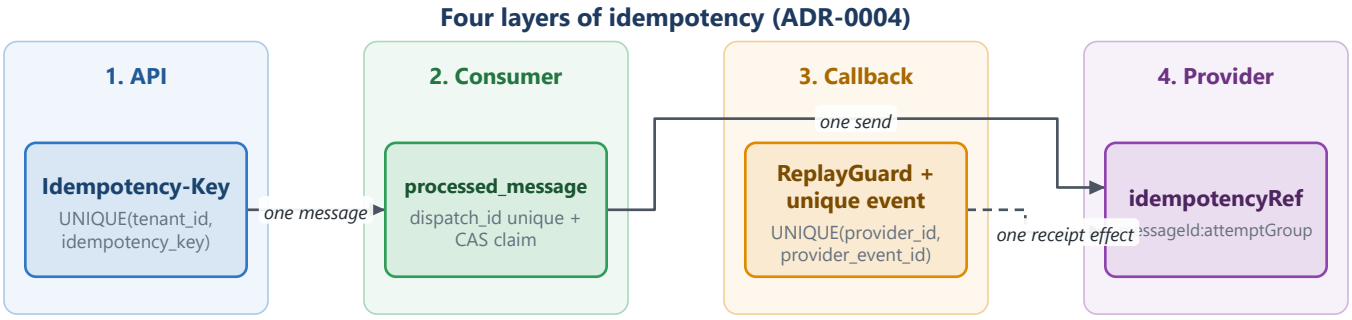


Figure 13. Duplicates are neutralised at four independent layers so that at-least-once transport yields one business effect. Implementation: backend/src/main/java/com/example/messaging/service/IdempotencyService.java, backend/src/main/java/com/example/messaging/messaging/DispatchConsumer.java, backend/src/main/java/com/example/messaging/callback/ReplayGuard.java, backend/src/main/java/com/example/messaging/provider/ProviderRequest.java

Layer	Key	Guarantee	Where
API	`(tenant_id, idempotency_key)` unique	One message per client intent; same key and same hash replays the original response	`IdempotencyService`, table `idempotency_record`
Consumer	`processed_message.dispatch_id` unique plus CAS status claim	One dispatch per (message, attempt) even when the broker redelivers	`DispatchConsumer`
Callback	`replay_guard` plus `UNIQUE(provider_id, provider_event_id)`	One effect per vendor event	`ReplayGuard`, `DeliveryEventProcessor`
Provider	`idempotencyRef = messageId:attemptGroup`	A compliant provider deduplicates a retried send	`ProviderRequest`

The API layer has three outcomes for a key that was seen before:

Situation	Response	Error code
Same key, same request hash, completed	200 + `Idempotent-Replay: true`, original body	none
Same key, different request hash	422	`IDEMPOTENCY_KEY_REUSED`
Same key, the first request still in flight	409	`IDEMPOTENCY_IN_PROGRESS`

An in-flight claim lives for `messaging.idempotency.claim-ttl` (default 30 s); a completed record for `record-ttl` (default 24 h). Redis is a fast path in front of the table; if Redis is down, `ResilientCoordination` falls back and the database unique key still enforces correctness (scenario 7 in Chapter 22).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5.5 Validation rules

Field	Rule
`recipientId` xor `recipient.externalRef`	Exactly one; the recipient must exist in the caller's tenant (a foreign recipient is a 422 `RECIPIENT_NOT_FOUND`)
`channel`	`EMAIL`, `SMS` or `PUSH`; anything else is 400 `UNSUPPORTED_CHANNEL`; the recipient must have a valid address for it
`category`, `priority`	Enums; the category must match the template's category
`template.code`, `locale`, `version`	A `PUBLISHED` version is resolved and pinned; `version: null` means the latest published at accept time
`variables`	All required variables present, no unknown ones, string values with length limits (at most 50 properties)
`schedule.sendAt`, `expiresAt`	`sendAt` at most 366 days ahead (a past or missing value means now); `expiresAt` after `sendAt`
`fallbackChannels`	Each evaluated through the full policy chain

5.6 Scheduled and deferred messages

`SCHEDULED` is **not** a status. A message with a future `sendAt`, or one deferred by quiet hours, is `VALIDATED` with `scheduled_at` set, and its outbox row carries `available_at` equal to the due time. The publisher selects `status=NEW AND available_at <= now`, so the row simply waits in the outbox. A message whose `expiresAt` precedes its deferred send time is `REJECTED` with `EXPIRES_BEFORE_DEFERRED_SEND`.

5.7 Edge cases

Case	Behaviour
Duplicate request	Original response with `Idempotent-Replay: true`
20 concurrent identical requests	Exactly one message (AC-4, scenario 2)
Template retired after accept	Still renders: the version is pinned (ADR-0007)
Provider timeout later	`DELIVERY_UNKNOWN`, then reconciliation
Backpressure	503 and `Retry-After`; the idempotency claim is rolled back, so the retry with the same key is fresh
Marketing to an opted-out recipient	202 with `status=REJECTED`, `statusReason=OPTED_OUT`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5.8 Tests that prove it

Test	Proves
`NotificationApiIT`	Submit, status, list, cancel and the error matrix over HTTP
`IdempotencyServiceTest`	Claim, replay, conflict and in-progress outcomes
`PolicyServiceTest`	The ordered policy chain, table driven
`MessageLifecycleIT`	Transitions end to end through the service layer
`FailureRecoveryIT#scenario02_*`	Sequential and concurrent duplicates yield one row
`BackpressureIT`	503 with `Retry-After`, nothing persisted for the refused request
`OpenApiContractIT`	The served API matches the committed OpenAPI spec

Reference material

Submission flow (design document)

Source document: `docs/architecture/submission-flow.md`

Where this document says policy suppression is either POLICY_SUPPRESSED or accepted-then-REJECTED, the implemented behaviour is accepted-then-REJECTED with HTTP 202.

`POST /api/v1/messages` (roles SENDER, TENANT_ADMIN, PLATFORM_ADMIN). Returns 202 quickly; no provider call is made on the request thread.

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter.

Notes

- **Backpressure**: before accepting, the API checks queue depth against `messaging.backpressure.max-queue-depth`; if over, 503 `SERVICE_UNAVAILABLE` + `Retry-After`. The idempotency claim is rolled back so the client may retry with the same key.
- **Rejections at accept**: validation failure is 400 `VALIDATION_FAILED`; unknown template 404-class `TEMPLATE_NOT_FOUND`; missing variable `TEMPLATE_VARIABLE_MISSING`; policy rejections (opt-out, suppression, deleted recipient, no address, expiry before a deferred send) are persisted as `REJECTED` and answered **202** with `status=REJECTED` and `statusReason` (`PolicyReason`), not a 4xx; `POLICY_SUPPRESSED` is not returned by the implemented API. Unique-constraint conflicts (duplicate template code+channel or locale version) answer 409 `DUPLICATE_RESOURCE`. A database outage answers 503 `SERVICE_UNAVAILABLE` + `Retry-After: 5`.
- **Scheduled**: `schedule.sendAt` in the future keeps status VALIDATED and defers outbox publication until due; quiet-hours deferral works the same way.
- **State written**: `ACCEPTED` then `VALIDATED` within the same transaction; `QUEUED` is set later by `OutboxPublisher` (ADR-0009).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Failure of the transaction** rolls back everything, including the idempotency claim, so a client retry is a clean retry.
- Related: ADR-0004, ADR-0007, ADR-0009, ADR-0012.

ADR-0004: Idempotency

Source document: `docs/adr/0004-idempotency.md`

Status: Accepted (2026-10-18)

Context

Duplicates arise from client retries on timeout, broker redelivery (at-least-once), consumer crashes, duplicate/out-of-order provider callbacks, and operator double-clicks. A customer must not receive the same security alert twice because a load balancer retried.

Decision

Four independent layers; none relies on another being perfect.

Layer	Key	Mechanism	Outcome on duplicate
1. API submission	`Idempotency-Key` header (required on `POST /messages`) + tenant	`IdempotencyService` claims `idempotency_record` (UNIQUE(tenant_id, idem_key)), status `IN_PROGRESS` then `COMPLETED` with stored response; Redis fast-path in front, DB is the source of truth	Same hash: HTTP 200 + `Idempotent-Replay: true` and original body. Different hash: 422 `IDEMPOTENCY_KEY_REUSED`. In flight: 409 `IDEMPOTENCY_IN_PROGRESS`
2. Consumer	`dispatchId` (one per message, attemptNo)	`processed_message` UNIQUE(dispatch_id) insert + CAS `QUEUED to PROCESSING`	Loser acks and skips
3. Callback	`(provider_id, provider_event_id)`	`delivery_event` UNIQUE; plus `replay_guard(event_id)` by `X-Event-Id`	Same `X-Event-Id` inside the replay window: **200** `{"result":"replayed"}`, no state change. Same `(provider_id, provider_event_id)` under another transport id: 202 `duplicate`, no second row (the unique constraint prevents one). Either way the provider stops retrying (`docs/architecture/callback-flow.md`)
4. Provider call	`idempotencyRef = messageId:attemptGroup`	Passed to provider; compliant providers dedupe; also used by `lookup()`	Provider returns same result or `lookup` shows prior acceptance

Details:

- Request hash = SHA-256 over canonical JSON of the semantic body (excluding `metadata` ordering noise); see `IdempotencyService` source.
- Bulk jobs derive deterministic keys `bulk-<jobId>-<externalRef>` so restart cannot double-submit.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Idempotency records expire (`expires_at`, retention job); key reuse after expiry creates a new message by design (document to clients: keys must be unique per logical send for at least the TTL).
- The claim and the `message_request` insert are in one transaction, so a crash cannot leave a completed record without a message.

Alternatives considered

Option	Why not
Redis-only idempotency	Redis loss would permit duplicates; Redis is a fast path only (`ResilientCoordination` falls back to DB)
Server-generated dedupe on payload content	Legitimate repeats (two identical OTPs) would be dropped
Exactly-once broker semantics	Not possible across the provider boundary (see delivery-semantics doc)
Dedupe only at consumer	Does not stop duplicate `message_request` rows and duplicate audit trails

Consequences

- Positive: safe client retries, safe broker redelivery, safe webhook retries, restartable batch.
- Negative: extra write per submission; clients must generate keys.
- Residual risk: a provider that ignores `idempotencyRef` can still duplicate after an AMBIGUOUS outcome; hence ADR-0010 (no blind resend for EMAIL/SMS).

Validation

- `FailureRecoveryIT#scenario02_duplicateSubmission_sameKey_returnsOriginal_noSecondMessage`, `#scenario03_duplicateAndOutOfOrderCallbacks_applyMonotonically`.
- Concurrency test: N parallel identical submissions yield exactly one `message_request` (see source `service` tests).
- Metric: `messaging.idempotency.hits`; callback duplicates counted in `messaging.callbacks{result="duplicate"}`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Message Lifecycle and State Machine

6.1 Why an explicit state machine

A notification is not "sent" or "not sent". Between the API call and the customer's inbox there are at least four different facts, and conflating them is the root of most messaging incidents.

Fact	Status that records it	What it does NOT mean
The platform accepted the request	`ACCEPTED`, `VALIDATED`	Not sent
The **provider** accepted it	`SENT_TO_PROVIDER`	Not delivered; the vendor can still bounce or drop it
It was **delivered**	`DELIVERED`	For email, delivered to the mailbox server, not "read"
We **do not know**	`DELIVERY_UNKNOWN`	Not failed and not delivered: unresolved

Three rules follow, and every part of the code obeys them: provider acceptance never implies delivery; the absence of a callback is never delivery; a timeout is never failure.

An analogy for children: a parcel has "*the shop packed it*", "*the courier picked it up*", "*it arrived at the door*" and "*the courier lost track of it*". Four different sentences, and a good tracking page never mixes them up.

6.2 The state machine

The machine is a static `EnumMap<MessageStatus, Set<MessageStatus>>` in `com.example.messaging.domain.MessageStateMachine`:

JAVA

```
TABLE.put(ACCEPTED, EnumSet.of(VALIDATED, REJECTED));
TABLE.put(VALIDATED, EnumSet.of(QUEUED, CANCELLED, REJECTED));
TABLE.put(QUEUED, EnumSet.of(PROCESSING, CANCELLED));
TABLE.put(PROCESSING, EnumSet.of(SENT_TO_PROVIDER, RETRY_PENDING, FAILED, DELIVERY_UNKNOWN, REJECTED));
TABLE.put(RETRY_PENDING, EnumSet.of(QUEUED, DEAD_LETTERED, CANCELLED));
TABLE.put(SENT_TO_PROVIDER, EnumSet.of(DELIVERED, FAILED, DELIVERY_UNKNOWN));
TABLE.put(DELIVERY_UNKNOWN, EnumSet.of(DELIVERED, FAILED));
TABLE.put(FAILED, EnumSet.of(DEAD_LETTERED, QUEUED));
TABLE.put(DEAD_LETTERED, EnumSet.of(QUEUED));
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

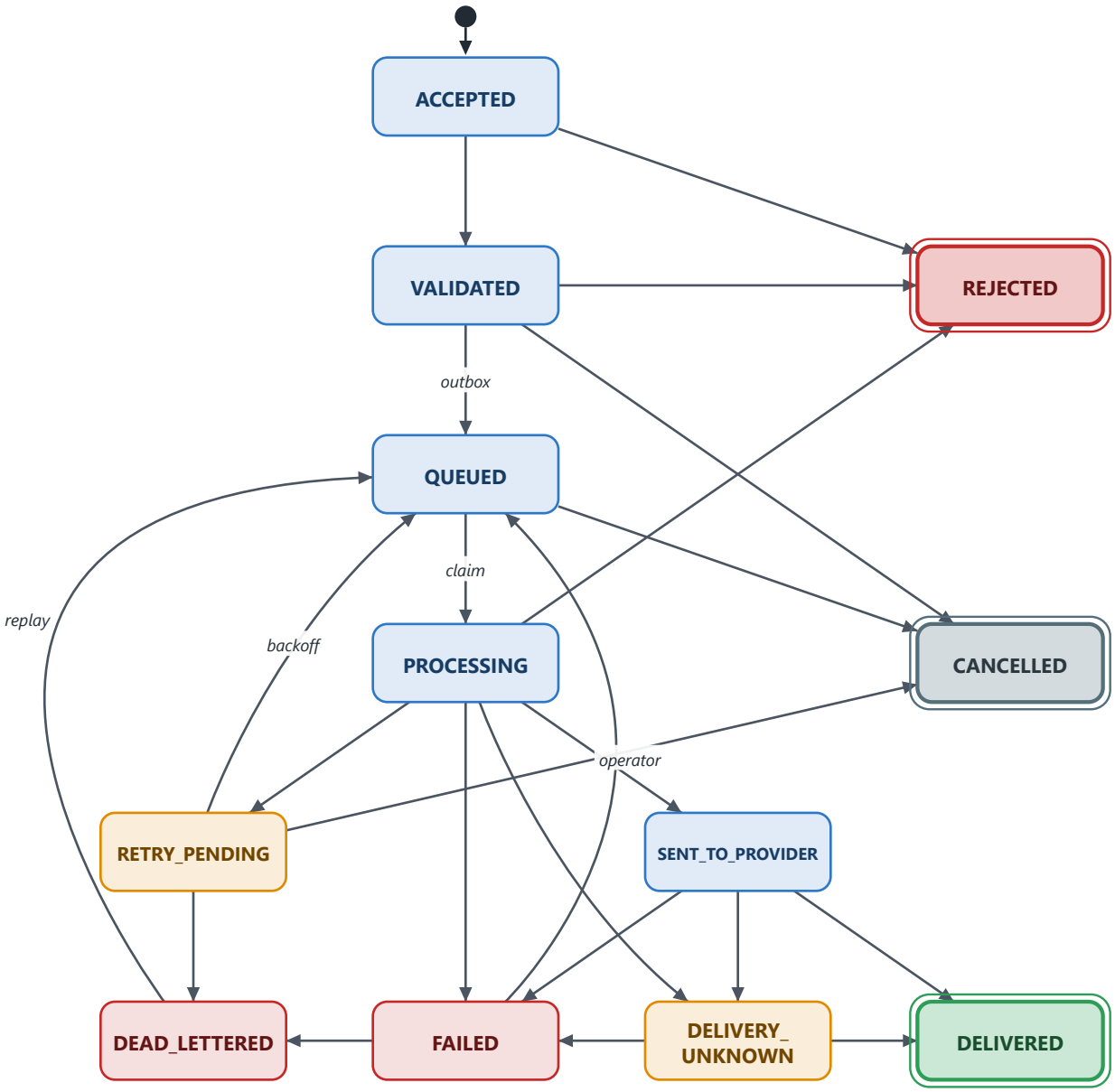


Figure 14. Message lifecycle state machine: the exact transition table of MessageStateMachine (12 statuses). SCHEDULED is not a status: scheduled messages are VALIDATED with a future scheduled_at. Implementation: backend/src/main/java/com/example/messaging/domain/MessageStateMachine.java, backend/src/main/java/com/example/messaging/domain/MessageStatus.java

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6.3 Transition matrix

From	Allowed to	Actor	Trigger
ACCEPTED	VALIDATED, REJECTED	Submission service	Synchronous, inside the submit transaction
VALIDATED	QUEUED, CANCELLED, REJECTED	`OutboxPublisher`, API cancel, dispatch-time policy	Outbox row due; cancel call; policy decided at dispatch
QUEUED	PROCESSING, CANCELLED	`DispatchConsumer`, API cancel	Claim; cancel call
PROCESSING	SENT_TO_PROVIDER, RETRY_PENDING, FAILED, DELIVERY_UNKNOWN, REJECTED	`DispatchConsumer`	Provider outcome
RETRY_PENDING	QUEUED, DEAD_LETTERED, CANCELLED	Retry scheduler, operator	Backoff elapsed; attempts exhausted
SENT_TO_PROVIDER	DELIVERED, FAILED, DELIVERY_UNKNOWN	Callback processor, sweeper	Receipt; no receipt within `receipt-timeout`
DELIVERY_UNKNOWN	DELIVERED, FAILED	Callback processor, `ReconciliationService`	Late callback or lookup evidence
FAILED	DEAD_LETTERED, QUEUED	Pipeline; operator requeue only for QUEUED	Fallback exhausted; operator action
DEAD_LETTERED	QUEUED	Operator replay only	`POST /dead-letters/{id}/replay`
DELIVERED, REJECTED, CANCELLED	none	terminal	

Cancellation is permitted only from `VALIDATED`, `QUEUED` and `RETRY_PENDING` (`MessageStateMachine.canCancel`). `FAILED` is terminal unless an operator requeues; `DEAD_LETTERED` is terminal unless an operator replays (`isOperatorOnly`).

6.4 Multi-step outcomes use legal single steps

Real outcomes sometimes need two moves, and the table has no shortcut for them (there is no `QUEUED -> REJECTED`, for example). The implementation never bypasses the table; it chains individually legal transitions in one transaction (DEVIATIONS, stage 2, item 4):

Outcome	Chain
Fallback after `PERMANENT_FAILURE`	`PROCESSING -> FAILED -> QUEUED`
Fallback or dead letter after exhausted retries	`PROCESSING -> RETRY_PENDING -> QUEUED` or `-> DEAD_LETTERED`
Reconciliation "not received"	`DELIVERY_UNKNOWN -> FAILED -> QUEUED`
Mirroring a poison message	`QUEUED -> PROCESSING -> FAILED -> DEAD_LETTERED`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Outcome	Chain
Expiry or dispatch-time rejection	claim first (`QUEUED -> PROCESSING`), then `PROCESSING -> FAILED(EXPIRED)` or `-> REJECTED`

Quiet hours met at dispatch time defer through the **outbox** (a new outbox row with `available_at` at the end of the window; the message stays `QUEUED`); there is deliberately no `QUEUED -> VALIDATED` transition.

6.5 Receipts and the monotonic rank rule

Provider receipts can arrive in any order and more than once. `MessageStateMachine.resolveReceipt(current, type)` decides what a receipt does:

JAVA

```
public static ReceiptResolution resolveReceipt(MessageStatus current, DeliveryEventType type) {
    if (type.rank() < 3) return ReceiptResolution.IGNORE_LOWER_RANK;
    if (current == DELIVERED) {
        return type == DeliveryEventType.DELIVERED
                ? ReceiptResolution.IGNORE_LOWER_RANK : ReceiptResolution.CONFLICT;
    }
    boolean receivable = current == PROCESSING || current == SENT_TO_PROVIDER || current == DELIVERY_UNKNOWN;
    ...
}
```

Receipt	Rank	Effect
`PROVIDER_ACCEPTED`, `SENT`, `DEFERRED`	1	Never downgrades; recorded, ignored once the outcome is known
`DELIVERED`	3	Applies from `PROCESSING`, `SENT_TO_PROVIDER`, `DELIVERY_UNKNOWN`
`BOUNCED`, `FAILED`, `REJECTED`	3	To `FAILED` (a hard bounce also adds a suppression)
`OPENED`, `CLICKED`, `UNSUBSCRIBED`	engagement	Recorded; unsubscribe clears marketing opt-in
Opposite terminal after the first		`CONFLICT`: the first terminal wins, audit `DELIVERY_EVENT_CONFLICT`

`canTransitionViaReceipt` additionally permits `PROCESSING -> DELIVERED` and `PROCESSING -> FAILED`: a receipt may overtake the consumer's own persistence of `SENT_TO_PROVIDER`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6.6 Cancel versus dispatch

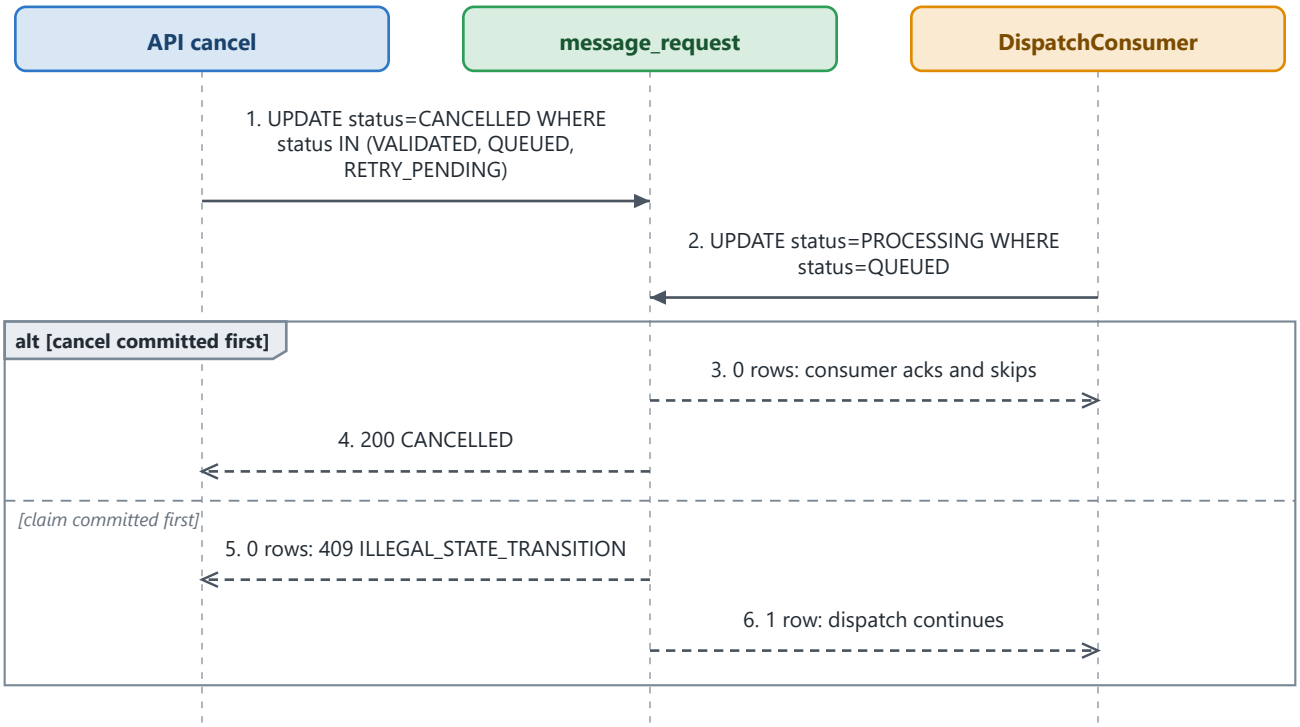


Figure 15. Cancel versus dispatch: both actors use a compare-and-set on the status column, so exactly one wins and the loser backs off without side effects. Implementation: backend/src/main/java/com/example/messaging/service/NotificationService.java, backend/src/main/java/com/example/messaging/messaging/DispatchConsumer.java, backend/src/test/java/com/example/messaging/pipeline/CancelRaceIT.java

6.7 Compare-and-set ownership

The lifecycle's integrity does not rely on locks held across calls. Each transition is a single conditional update, which the database serialises:

```sql
UPDATE message_request SET status = :to, ... WHERE id = :id AND status = :from
```

`NotificationService.transition(messageId, from, to, reason, actor)` returns `true` only for the caller whose update matched a row. Callers that lose back off: the consumer acknowledges and skips, the API answers 409 `ILLEGAL_STATE_TRANSITION`, the publisher moves on.

6.8 The lifecycle as an operator sees it

The message detail page of the console renders the merged history (`GET /api/v1/messages/{id}/history`) as a lifecycle timeline next to the attempt list. The screenshot below was recorded from the running local backend (synthetic data; the page never shows message content, and the address is masked): the message was accepted, published by the outbox (`VALIDATED -> QUEUED`), claimed by the dispatch consumer (`QUEUED -> PROCESSING`),

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

attempted once through `simulated-email` (outcome `ACCEPTED`) and left in `SENT_TO_PROVIDER`: provider acceptance, not delivery, with no delivery events yet.

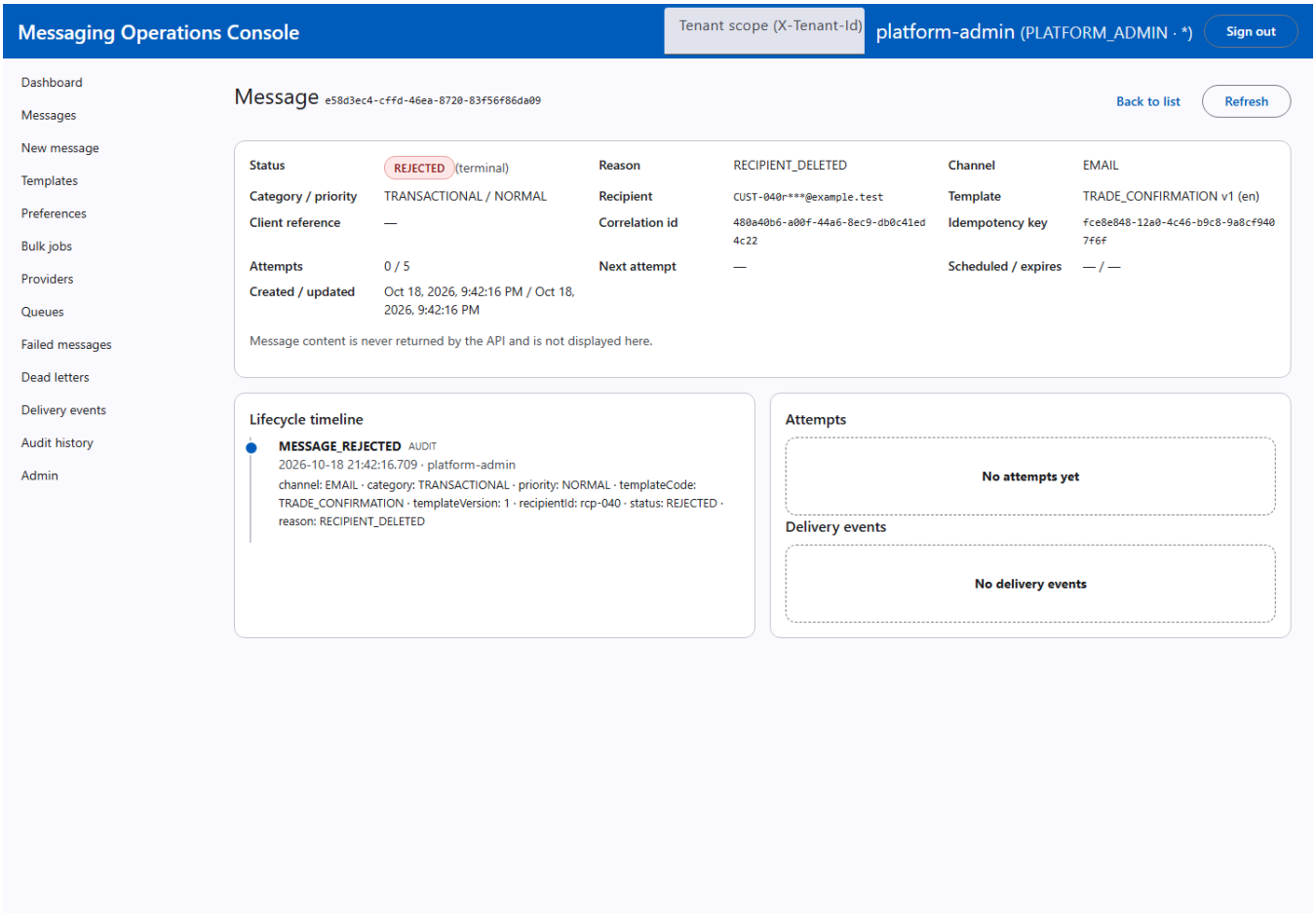


Figure 16. Message detail with lifecycle timeline and attempts. Source: docs/validation/screenshots/05-message-detail.png

6.9 Tests that prove it

Test	Proves
`MessageStateMachineTest`	The full 12 by 12 transition matrix, cancellable and requeueable sets, receipt resolution
`CancelRaceIT`	Exactly one of cancel and dispatch wins
`ConcurrencyIT`	Concurrent claims of one message produce one owner
`MessageLifecycleIT`	Legal transitions through the service, history timeline contents
`FailureRecoveryIT#scenario03_*`	Duplicate and out-of-order callbacks apply monotonically

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Reference material

ADR-0003: Message lifecycle state machine

Source document: `docs/adr/0003-message-lifecycle-state-machine.md`

Status: Accepted (2026-10-18)

Context

A message crosses an API thread, a broker, a consumer thread, an external provider and an asynchronous callback. Without an explicit state model, concurrent actors (consumer, sweeper, callback processor, operator) corrupt each other's state, and "sent" gets confused with "delivered".

Decision

Model the lifecycle as an explicit finite state machine (`domain.MessageStateMachine`, pure Java) over `MessageStatus`. Every transition goes through it; illegal transitions throw `IllegalStateTransitionException` (HTTP 409 `ILLEGAL_STATE_TRANSITION` for API-driven actions; logged + audited for event-driven ones). `SCHEDULED` is **not** a status: scheduled messages are `VALIDATED` with a future `scheduled_at`.

Transition table (authoritative: CONTRACT 3.1)

From	Allowed to	Owner / actor
ACCEPTED	VALIDATED, REJECTED	submission service (synchronous)
VALIDATED	QUEUED, CANCELLED, REJECTED	outbox publisher / poller; API cancel (a dispatch-time policy rejection is applied from PROCESSING after the claim, see `DEVIATIONS.md`)
QUEUED	PROCESSING, CANCELLED	dispatch consumer; API cancel
PROCESSING	SENT_TO_PROVIDER, RETRY_PENDING, FAILED, DELIVERY_UNKNOWN, REJECTED	dispatch consumer
RETRY_PENDING	QUEUED, DEAD_LETTERED, CANCELLED	retry scheduler; operator
SENT_TO_PROVIDER	DELIVERED, FAILED, DELIVERY_UNKNOWN	callback processor; sweeper
DELIVERY_UNKNOWN	DELIVERED, FAILED	callback processor; `ReconciliationService`
FAILED	DEAD_LETTERED, QUEUED (operator requeue)	operator
DEAD_LETTERED	QUEUED (operator replay)	operator
DELIVERED, REJECTED, CANCELLED	none (terminal)	

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Terminal classes: **hard terminal** = DELIVERED, REJECTED, CANCELLED. **Soft terminal** = FAILED (until operator requeue), DEAD_LETTERED (until operator replay). `terminal` in API responses is true for hard terminals and soft terminals.

Ownership and concurrency

- Exactly one actor owns each state (table column 3). Cross-actor races are resolved by **compare-and-set** updates (`UPDATE ... SET status=:to WHERE id=:id AND status=:from`), plus `version` optimistic locking on `message_request`. A CAS loser acks and skips (consumer) or returns 409 (API).
- `DELIVERED` receipts are ranked by the monotonic rule in CONTRACT 3.4: a lower-rank event never overwrites a higher one; `DELIVERED` vs later `BOUNCED` keeps the first terminal and audits `DELIVERY_EVENT_CONFLICT`.
- Provider acceptance (`SENT_TO_PROVIDER`) never implies delivery.
- Cancellation allowed only from VALIDATED, QUEUED, RETRY_PENDING.
- Every transition emits `messaging.transitions{from,to}` and an `audit_event`.

Alternatives considered

Option	Why not
Free-form status strings updated ad hoc	Illegal jumps (e.g. DELIVERED to QUEUED) become possible
Spring Statemachine library	Heavy; the table is small and benefits from being a plain testable enum map
Event sourcing	Strong audit but high complexity; we keep `audit_event` + `message_attempt` + `delivery_event` as an append-only history instead
Pessimistic row locks	Throughput cost, deadlock risk across consumer/sweeper/callback

Consequences

- Positive: exhaustive transition tests; operators and the UI render one timeline; failure modes are named states (`DELIVERY_UNKNOWN`).
- Negative: any new state needs an ADR, table update, UI badge, metrics and test matrix update.
- Risk: code paths that update `status` directly (bypassing the machine). Guard: a repository-level test greps/asserts no direct status setter outside the machine (see `ai-engineering/ai-code-review/01-incorrect-state-transition.md`).

Validation

- `MessageStateMachineTest` should enumerate the full `MessageStatus x MessageStatus` matrix against the table above (see source).
- Failure scenarios: `docs/architecture/reliability-scenarios.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Delivery semantics (failure windows)

Source document: `docs/architecture/delivery-semantics.md`

Where duplicates can still happen (honest list)

1. Provider ignores `idempotencyRef` and we retry after "not received" that was wrong (ADR-0010 residual risk).
2. Operator replays a message that was actually delivered (replay UI shows attempt history and warns for `DELIVERY_UNKNOWN`; replay is audited).
3. Client reuses a different `Idempotency-Key` for the same logical send (client bug).
4. Idempotency key expiry (TTL) then re-use.

Ordering

No global ordering guarantee. Priority queues reorder by priority; retries reorder in time. Per-recipient ordering is not guaranteed; clients that need ordering must serialize submissions. Callbacks may arrive out of order; the rank rule and `occurred_at` handle this (CONTRACT 3.4).

Failure windows and their handling

Crash/failure point	Result	Handling
After DB commit, before publish	Message sits VALIDATED with outbox NEW	Publisher retries (no loss)
After publish, before outbox SENT	Duplicate envelope	Consumer dedupe
After consumer CAS, before provider call	Message PROCESSING, no attempt effect	Sweeper to DELIVERY_UNKNOWN is conservative; attempt row STARTED without provider id lets reconciliation conclude "not received"
After provider accept, before persist	Provider has it, we do not know	DELIVERY_UNKNOWN then lookup/callback
After persist, before ack	Redelivery	CAS/processed_message dedupe
Provider callback lost	Receipt never arrives	`receipt-timeout` to DELIVERY_UNKNOWN to lookup

See `reliability-scenarios.md` and `docs/architecture/reconciliation-flow.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. Queue-Based Dispatch

7.1 Why a queue at all

A provider call takes hundreds of milliseconds and sometimes fails. If the API made that call, request latency would be the provider's latency, an outage would be an API outage, and a traffic spike would hit the vendor unthrottled. A queue turns the spike into a backlog the platform drains at the pace the providers allow. The cost is that delivery is asynchronous and at-least-once, which the rest of the design handles with the outbox, idempotent consumers and the state machine.

The kid-friendly version: the clerk puts letters in an outgoing tray. The couriers take them one at a time, urgent ones first. If the tray overflows, the clerk politely tells new customers to come back in a moment (HTTP 503 with `Retry-After`) instead of letting letters fall on the floor.

7.2 The transactional outbox

The dangerous gap in naive designs is the moment between "committed to the database" and "published to the broker". A crash there loses the message; publishing first and committing second can publish a message that never existed. The platform closes the gap with the **transactional outbox pattern** (ADR-0009): the submit transaction writes an `outbox_event` row, and a scheduled `OutboxPublisher` moves rows to the bus.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

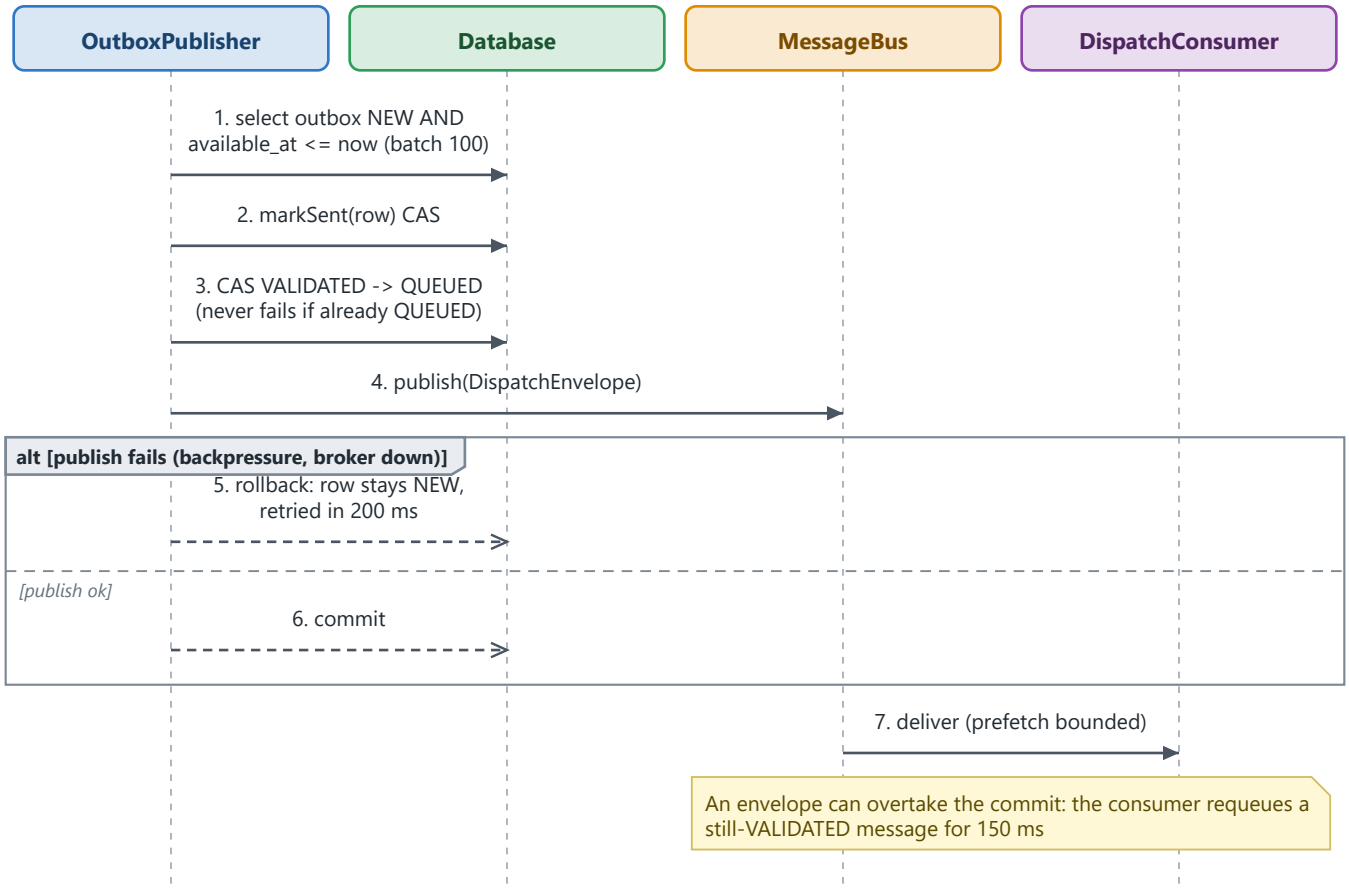


Figure 17. Transactional outbox: the publisher claims NEW rows, moves VALIDATED to QUEUED and publishes inside one transaction; a failed publish rolls everything back, which gives at-least-once publication. Implementation: backend/src/main/java/com/example/messaging/scheduler/OutboxPublisher.java, backend/src/main/java/com/example/messaging/messaging/OutboxWriter.java, backend/src/main/java/com/example/messaging/messaging/BackpressureGuard.java

Implemented details that matter (DEVIATIONS, stage 1 and 2):

- The publisher selects `status=NEW AND available_at <= now`, in batches of 100, every 200 ms (`messaging.pipeline.scheduler.outbox-interval-ms`, `outbox-batch`).
- It **claims** the row (`markSent` compare-and-set), moves `VALIDATED` to `QUEUED` (never failing if the message is already `QUEUED`, which happens for requeue and replay) and publishes, all in one transaction. A failed publish (backpressure, broker down) rolls everything back, so the row stays `NEW` and is retried: **at-least-once** publication.
- Cancelled messages are skipped by the publisher (`status` not `VALIDATED` or `QUEUED`).
- An envelope can overtake the commit of its own `QUEUED` transition. The consumer then sees a message still `VALIDATED` and requeues it for 150 ms (`validated-requeue-delay`), at most 40 deliveries, then dead-letters it.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7.3 The MessageBus port

```java
public interface MessageBus {
    void publish(DispatchEnvelope envelope);                        // BusRejectedException on backpressure
    void publishDelayed(DispatchEnvelope envelope, Duration delay);// retry backoff, rate-limit requeue
    void subscribe(DispatchHandler handler);
    QueueStats stats();
    void pause(); void resume(); boolean isPaused();
    void start(); void stop(); boolean isRunning();
}
```

The handler returns a `BusDecision` that drives **ack**, **requeue with delay** or **dead-letter**. Two adapters implement the port with the same semantics:

Adapter	Profile	Properties
`InMemoryMessageBus`	`local`, `test`	Bounded priority queue (capacity 5000), worker threads, requeue-with-delay, in-memory DLQ; unit tested by `InMemoryMessageBusTest`, `GracefulShutdownTest`
`RabbitMessageBus`	`rabbit` (group of `prod-like`)	Priority queue (`x-max-priority` 9), TTL delay tiers, dead-letter exchange, manual ack, prefetch 16; topology in `infrastructure/rabbitmq/definitions.json` and `RabbitTopology`

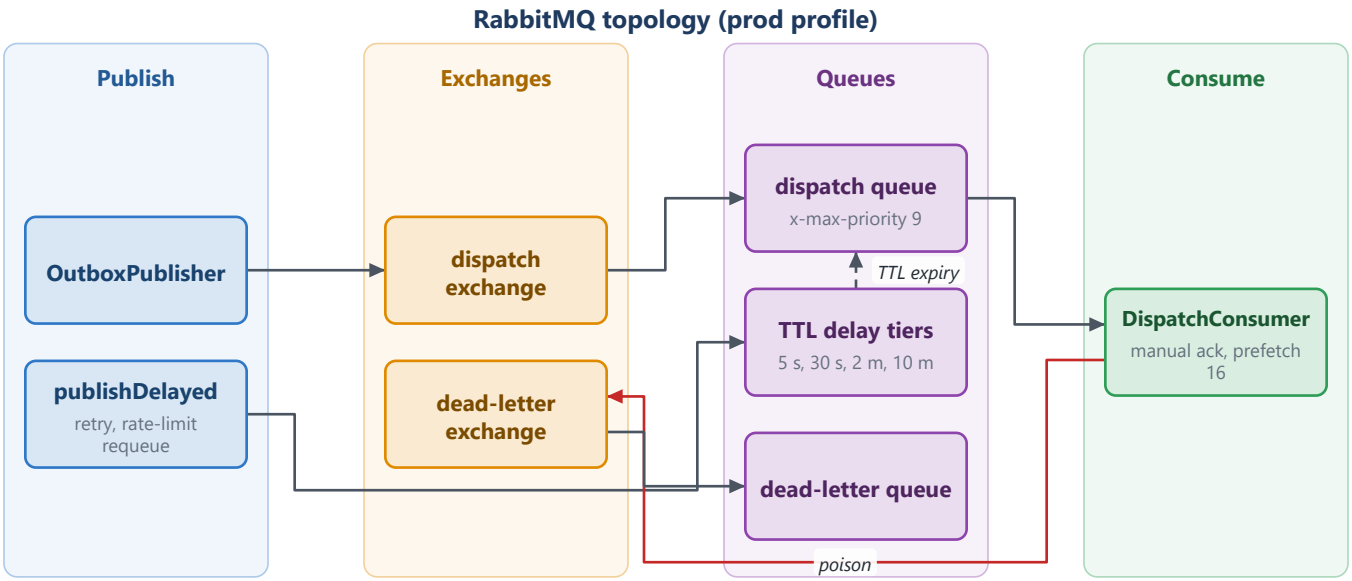


Figure 18. Priority dispatch queue with TTL retry tiers and dead-letter exchange. Rabbit code is NOT executed in this repository: only the pure parts and the topology versus definitions.json are tested. Implementation: infrastructure/rabbitmq/definitions.json, backend/src/main/java/com/example/messaging/messaging/RabbitTopology.java, backend/src/main/java/com/example/messaging/messaging/RabbitMessageBus.java

> **Rabbit code was not executed**
>
> There is no broker on the authoring machine. Only the pure parts (`RabbitBusSupportTest`) and the topology versus `definitions.json` (`RabbitTopologyTest`) run in the default build. `RabbitMessageBusContainerIT` compiles under `-Pcontainers` but was not run. With Rabbit, only the total ready count of the dispatch queue is available, so `GET`

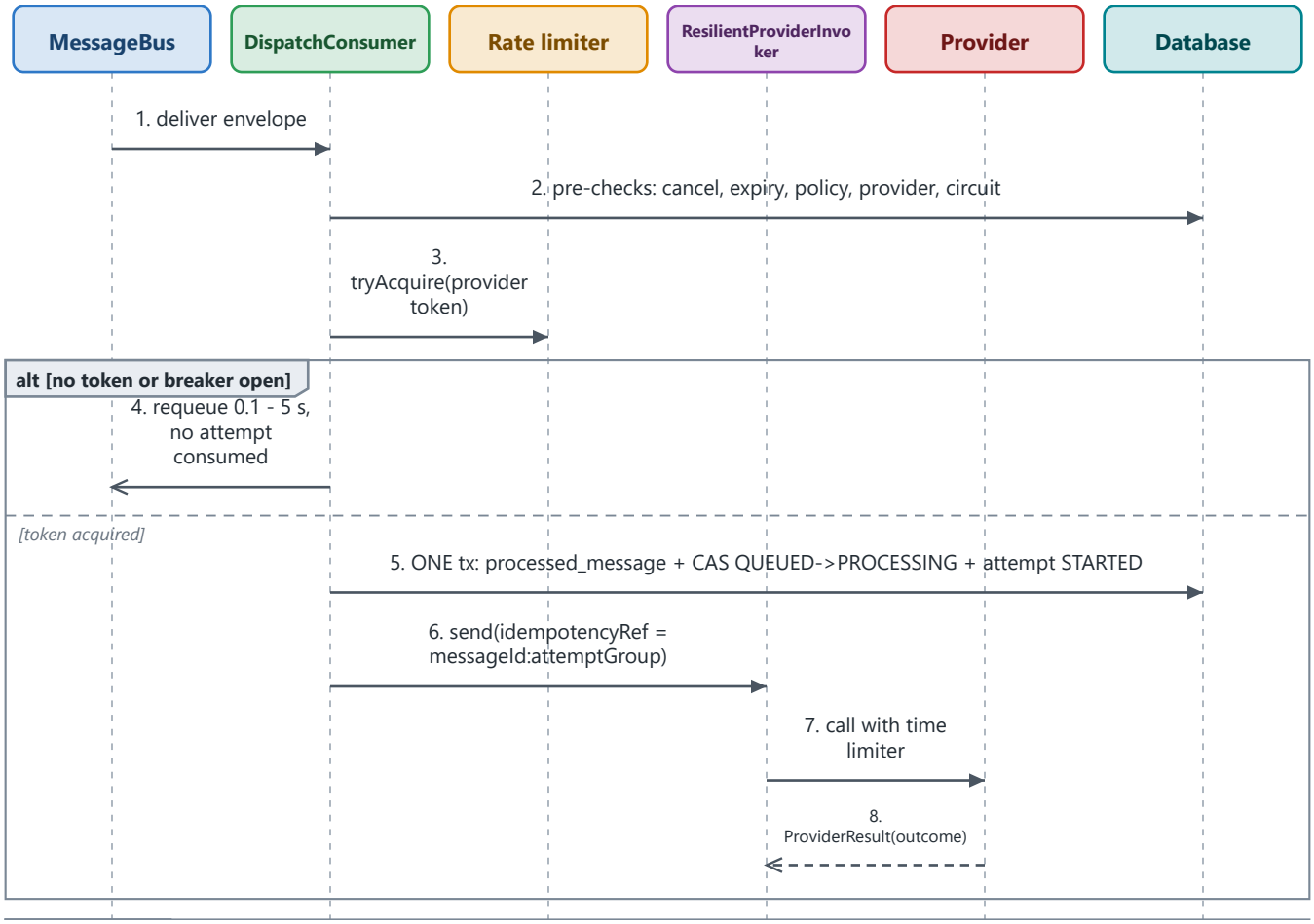
Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

> `/ops/queues` reports it under `NORMAL`; delays are rounded up to the next TTL tier (5 s, 30 s, 2 m, 10 m) and longer delays finish with an `x-not-before` header.

Priorities

`Priority{CRITICAL(9), HIGH(6), NORMAL(3), LOW(1)}`. Higher weights are delivered first. Bulk jobs default to lower priorities so they cannot starve urgent traffic; aging (spec `specs/0001-priority-aging.md`) is the planned starvation protection for `LOW`.

7.4 The dispatch consumer



Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

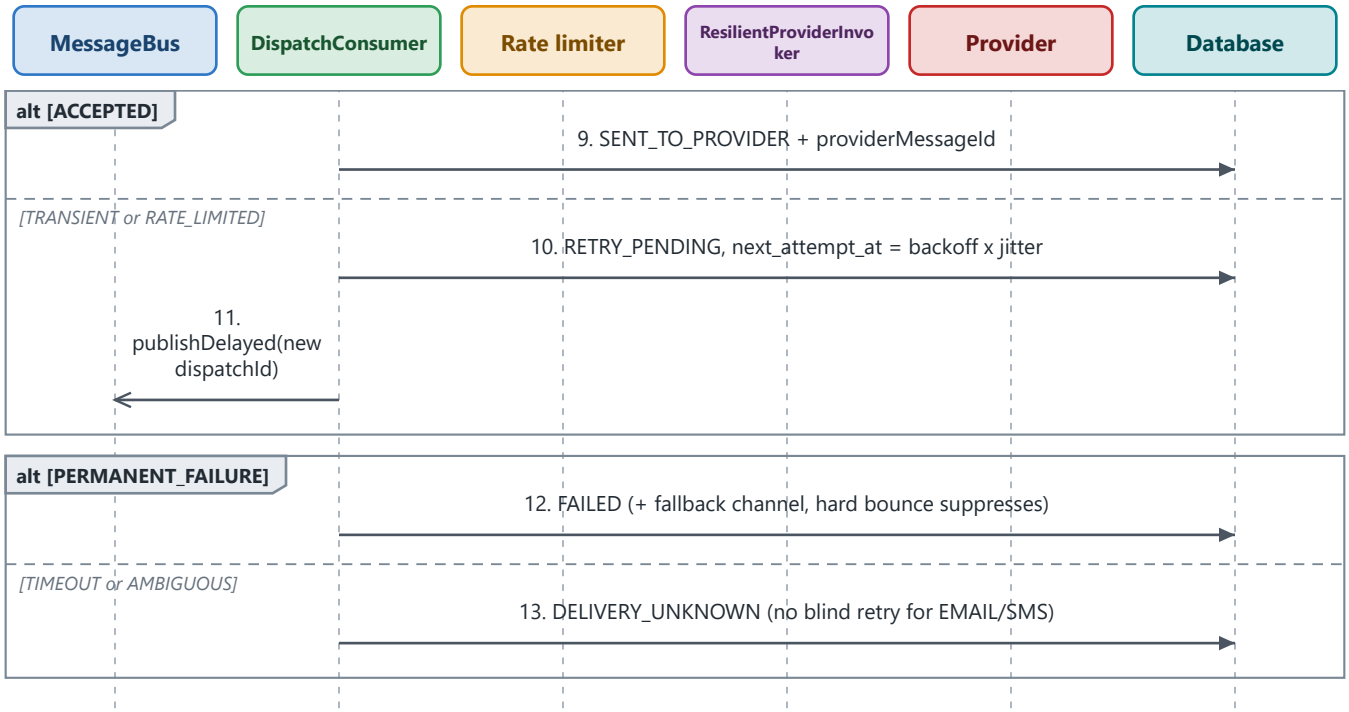


Figure 19. Dispatch with retry: pre-claim checks, one claim transaction, provider call under a circuit breaker, then an outcome-specific transition. Implementation: backend/src/main/java/com/example/messaging/messaging/DispatchConsumer.java, backend/src/main/java/com/example/messaging/provider/ResilientProviderInvoker.java, backend/src/main/java/com/example/messaging/service/RetryPolicy.java

Order of operations as implemented

The design documents describe the consumer as "claim, then check". The implementation reverses the first half, for a concrete reason (DEVIATIONS, stage 2, item 1): a requeue because of the rate limiter reuses the same `dispatchId`, so it must not be blocked by the dedupe table and must not consume an attempt.

1. **Pre-claim checks** (no state change): cancel or status, expiry, policy, provider selection, circuit state, provider token bucket. Failing the token bucket or an open breaker requeues the delivery for 0.1 to 5 seconds without consuming an attempt.
2. **The claim** is one transaction: unique `processed_message` row, `QUEUED -> PROCESSING` compare-and-set, `locked_until = now + lock-ttl`, `attempt_count` increment, and the `message_attempt` row inserted as `STARTED`. Losing the race means acknowledge and skip.
3. **The provider call** runs outside any database transaction, under `ResilientProviderInvoker`.
4. **The outcome** is applied in a new short transaction, with the attempt row finished.

The circuit-breaker permit is **reserved before the claim** (`ResilientProviderInvoker.acquire`) and released again if the message is not claimed, is rate limited or fails to render. A recovering provider therefore never burns the attempt budget of messages that merely met a refusing breaker.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Outcome mapping

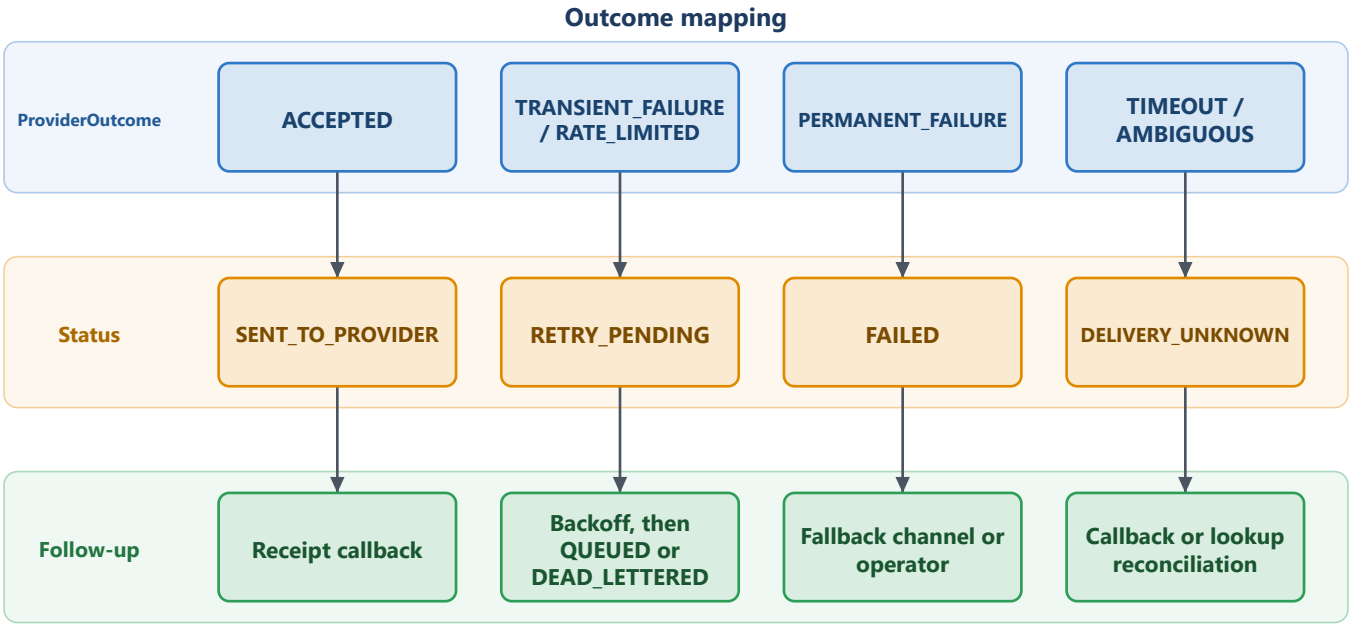


Figure 20. How each ProviderOutcome maps to a lifecycle transition (CONTRACT 6, ADR-0006, ADR-0010). Implementation: backend/src/main/java/com/example/messaging/provider/ProviderOutcome.java, backend/src/main/java/com/example/messaging/messaging/DispatchConsumer.java

Outcome	Result	Notes
`ACCEPTED`	`SENT_TO_PROVIDER`, provider message id stored	Receipt callback finishes the story
`TRANSIENT_FAILURE`, `RATE_LIMITED`	`RETRY_PENDING`, `next_attempt_at = now + min(cap, base x 2^(n-1)) x jitter`; exhausted attempts go to fallback or `DEAD_LETTERED(RETRIES_EXHAUSTED)`	Chapter 13
`PERMANENT_FAILURE`	`FAILED`, fallback if allowed; a hard bounce adds a suppression	
`TIMEOUT`, `AMBIGUOUS`	EMAIL and SMS: `DELIVERY_UNKNOWN`; PUSH: retried like a transient failure (the provider deduplicates on the idempotency reference)	Chapter 12
Poison envelope	Dead-letter with `POISON_MESSAGE` after `poison-threshold` (5) deliveries; unreadable bodies immediately	Chapter 13

7.5 Backpressure and graceful shutdown

- **Backpressure.** `BackpressureGuard.check()` runs at the top of `MessageController.submit`. When the bus depth plus due outbox rows reach `messaging.backpressure.max-queue-depth` (5000) the API answers 503 `SERVICE_UNAVAILABLE` with `Retry-After`; the outbox publisher also sees `BusRejectedException` and keeps rows `NEW`.
- **Bounded consumers.** `concurrency` 8 workers per instance, `prefetch` 16 (Rabbit prefetch, or in-memory queue handoff).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Graceful shutdown.** `SmartLifecycle` stops intake, waits up to `shutdown-timeout` (30 s) for in-flight work; unacknowledged deliveries return to the queue and the next consumer dedupes through `processed_message` and the CAS. A hard kill leaves a `PROCESSING` row that `StuckMessageSweeper` converts to `DELIVERY_UNKNOWN` (Chapter 12).
- **Pause and resume.** `POST /api/v1/ops/consumers/pause|resume` (PLATFORM_ADMIN) stops handing deliveries to workers while publishing continues; the backpressure and backlog-recovery tests use this.

7.6 Tests that prove it

Test	Proves
`InMemoryMessageBusTest`	Priority order, ack, requeue with delay, DLQ, bounded capacity
`GracefulShutdownTest`	In-flight work finishes, unacked deliveries return
`DispatchFlowIT`, `ChannelDispatchIT`	Message flows to `SENT_TO_PROVIDER` for each channel
`IdempotentConsumerIT`	Duplicate envelopes cause one provider call
`ConcurrencyIT`	Racing consumers cause exactly one provider call
`BackpressureIT`	503 plus `Retry-After`, no loss of accepted messages
`RabbitTopologyTest`	Declared topology equals `definitions.json`

Reference material

Dispatch and retry flow (design document)

Source document: `docs/architecture/dispatch-retry-flow.md`

Implemented order differs in the details listed in section 7.4 and in DEVIATIONS (pre-claim checks, claim as one transaction, outbox-based quiet-hours deferral).

> *Implemented order (see `docs/design/DEVIATIONS.md`, stage 2 "Behaviour that differs", items 1-6). The first plan checked dedupe and the status CAS first; the code checks cheap refusals first and claims in one transaction.*

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter.

Notes: quiet hours met at dispatch time defer through a **new outbox row** with `available_at` = end of the window (the message stays QUEUED; there is no QUEUED to VALIDATED transition). Expiry and dispatch-time policy rejection claim first and then move PROCESSING to FAILED(`EXPIRED`) / REJECTED, because the state table has no QUEUED to REJECTED/FAILED. Multi-step outcomes are consecutive, individually legal transitions in one transaction (fallback after PERMANENT_FAILURE: PROCESSING to FAILED to QUEUED; after exhausted retries: PROCESSING to RETRY_PENDING to QUEUED or DEAD_LETTERED). Operator requeue/replay is a CAS FAILED|DEAD_LETTERED to QUEUED plus an outbox event, with a fresh attempt budget (`max_attempts = attempt_count + messaging.dispatch.max-attempts`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Outcome mapping

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter.

Retry scheduling

1. `RetryPolicy` computes `next_attempt_at = now + min(cap, base*2^(n-1)) * U(0.5,1.0)`; message becomes `RETRY_PENDING`, counter `messaging.retry.scheduled{channel}`.
2. Re-enqueue is done by bus delayed redelivery (Rabbit TTL+DLX) or, if the bus cannot delay, `ScheduledDispatchPoller` picking `status=RETRY_PENDING AND next_attempt_at <= now`; both end in a CAS `RETRY_PENDING to QUEUED` and a new envelope with a new `dispatchId`.
3. Exhausted attempts: fallback channel if the request allows it and policy approves on that channel; else `DEAD_LETTERED(RETRIES_EXHAUSTED)`.
4. Poison envelope: `DEAD_LETTERED(POISON_MESSAGE)` immediately, with metric `messaging.deadletter.total{reason="POISON_MESSAGE"}`.

Shutdown

`SmartLifecycle` stops intake, waits up to 30 s for in-flight, unacked deliveries return to the queue; next consumer dedupes via `processed_message`/CAS. A message left PROCESSING by a hard kill is handled by `StuckMessageSweeper` (ADR-0010).

See ADR-0003, ADR-0006, ADR-0010 and `delivery-semantics.md`.

ADR-0001: RabbitMQ versus Kafka

Source document: `docs/adr/0001-asynchronous-messaging-rabbitmq-vs-kafka.md`

Status: Accepted (2026-10-18)

Context

`POST /api/v1/messages` must return `202 Accepted` quickly and decouple the caller from slow, failure-prone providers. Dispatch needs: four priority levels (`CRITICAL(9)`, `HIGH(6)`, `NORMAL(3)`, `LOW(1)`), per-message delay for retry backoff, dead-lettering, bounded consumer concurrency, and simple operations. Volumes are modelled as bursty (campaign bulk jobs next to latency-sensitive security alerts). Messages are small envelopes (`DispatchEnvelope`: ids only, no content, no PII).

Decision

Use **RabbitMQ** as the production transport behind the `MessageBus` port (`messaging` package). Two implementations: `RabbitMessageBus` (profile `rabbit`, topology in `RabbitTopologyConfig` and `infrastructure/rabbitmq/definitions.json`) and `InMemoryMessageBus` (profiles `local`, `test`) with identical ack/nack/requeue-with-delay/DLQ semantics. Business code never imports RabbitMQ types.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Why RabbitMQ for this workload:

- Native priority queues (`x-max-priority=9`) so a SECURITY_ALERT is not stuck behind a marketing bulk job.
- Dead-letter exchanges and per-queue/per-message TTL give delayed retry and DLQ without a separate scheduler.
- Push-based consumers with `prefetch` give natural bounded concurrency and backpressure.
- Work-queue semantics (competing consumers, ack/nack) match "dispatch one message once".

Alternatives considered

Option	Pros	Cons	Verdict
Kafka	Replayable log, huge fan-out, stream analytics, strong ordering per partition	No per-message priority, no native delayed retry (needs retry topics), consumer-group rebalancing complexity, head-of-line blocking per partition	Better for event streaming of delivery events; not for this dispatch queue
Amazon SQS / cloud queue	Managed	Vendor coupling, priority via multiple queues, no local parity	Possible adapter behind `MessageBus`
DB-as-queue only (poller)	No broker	Polling latency, DB contention at scale	Used only for the outbox and scheduled sends, not the hot path
ActiveMQ/Artemis	JMS standard	Less ops tooling familiarity	Not chosen

When to revisit: if consumers need to replay history by offset, or if delivery-event analytics become a first-class stream, introduce Kafka as a *second* sink fed from the outbox, not as a replacement for the dispatch queue.

Consequences

- Positive: priority handling and delayed retry are broker features; simple topology; local runs need no broker.
- Negative: no historical replay (replay is DB-driven via `ReplayService` from `DEAD_LETTERED`/`FAILED`); the broker is a stateful dependency to operate (quorum queues recommended in prod, see `infrastructure/rabbitmq`).
- Risks: broker-DB divergence (mitigated by ADR-0009 outbox and ADR-0004 consumer dedupe); priority queues degrade with very large backlogs (monitor `messaging.queue.depth{priority}`).
- Delivery semantics: at-least-once transport; effectively-once business processing (see `docs/architecture/delivery-semantics.md`).

Validation

- `InMemoryMessageBus` and `RabbitMessageBus` share a contract test (see source `messaging` test package). Rabbit-backed tests are `*ContainerIT` in profile `-Pcontainers` and are **not executed** in this environment.
- Metrics: `messaging.queue.depth{priority}`, `messaging.queue.inflight`, `messaging.queue.dlq.depth`.

ADR-0009: Outbox pattern

Source document: `docs/adr/0009-outbox-pattern.md`

Status: Accepted (2026-10-18)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Context

Accepting a message means (a) persisting `message_request` and (b) publishing to the broker. Doing both without a shared transaction ("dual write") yields two failure modes: DB commit then broker publish fails = **lost message** (client got 202, nothing sent); broker publish then DB commit fails = **ghost message** (dispatch for a message that does not exist).

Decision

Use the **transactional outbox**:

1. In one DB transaction `NotificationService` writes `message_request` (VALIDATED), `outbox_event` (`status=NEW`), `idempotency_record` completion and an `audit_event`.
2. `scheduler.OutboxPublisher` runs every 200 ms, selects `status=NEW AND available_at <= now` (scheduled and quiet-hours-deferred messages wait in the outbox until due), claims each row with a `markSent` compare-and-set, moves the message `VALIDATED to QUEUED` (CAS, tolerating an already QUEUED message for requeue/replay events) and publishes the `DispatchEnvelope` to `MessageBus`, all inside **one transaction**: a failed publish rolls the claim back, giving at-least-once delivery (`docs/design/DEVIATIONS.md`, stage 2 item 3).
3. Publication is **at-least-once**: a crash between publish and marking `SENT` re-publishes; the consumer dedupes by `dispatchId` (ADR-0004), so duplicates are harmless.
4. Scheduled sends: `ScheduledDispatchPoller` enqueues messages whose `scheduled_at <= now` using the same outbox path.
5. Multi-instance safety uses DB row claim (CAS), not ShedLock; no leader election.
6. If the outbox lags (broker down), API backpressure (`messaging.backpressure.max-queue-depth`) returns 503 + `Retry-After` rather than growing unboundedly.

Ghost-message case cannot occur (publish happens after commit). Lost-message case cannot occur (row persisted with the request).

Alternatives considered

Option	Why not
Publish inside transaction (best-effort)	Dual write
Two-phase commit/XA across DB and Rabbit	Operationally fragile, limited support
Change Data Capture (Debezium) on outbox	Excellent at scale; adds infrastructure; documented as evolution path
Publish-then-persist	Ghost messages
Skip broker, poll `message_request` only	Higher latency, DB load; used only as the fallback/reconciler

Consequences

- Positive: no lost/ghost dispatch; replay is a DB operation.
- Negative: up to ~200 ms added latency (+ DB polling load); outbox table needs retention (`RetentionJob` purges `SENT` rows).
- Risk: poller stall = silent backlog. Alert on oldest `NEW` outbox age and `messaging.dispatch.delay` p95 (see slis-slos). Monitor index on `outbox_event(status, created_at)` (see source migration).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Validation

- Tests: crash-between-publish-and-mark replays once with a single provider call (consumer dedupe); `FailureRecoveryIT#scenario06_databaseOutage_submissionFailsClosed_thenRecovers`.
- Sequence: `docs/architecture/submission-flow.md`.

Spec 0001: Priority aging

Source document: `specs/0001-priority-aging.md`

- Status: **Worked example, not implemented in code.** A grep of `backend/src` for aging / effective-priority logic finds nothing; there is no `messaging.dispatch.aging.*` property, no test and no metric for it. Decision record: `docs/adr/0013-priority-aging.md` (Proposed). Use it to practise spec-driven development, not as evidence of a built feature.
- Owner (role): Architecture
- Date: 2026-10-18
- Related: ADR-0001 (RabbitMQ priority queues), ADR-0003, ADR-0004, ADR-0006; `docs/architecture/dispatch-retry-flow.md`; worked in `ai-engineering/spec-driven-workflow.md`

1. Context and problem

Priorities `CRITICAL(9)`, `HIGH(6)`, `NORMAL(3)`, `LOW(1)` map to RabbitMQ `x-max-priority=9`. A sustained stream of higher-priority messages can starve LOW (and under heavy HIGH load, NORMAL). LOW includes real service notices and bulk jobs that must eventually go out. Strict priority gives no upper bound on LOW waiting time. Observed risk class (design analysis, not a measured incident): a long HIGH burst during a bulk run.

2. Goals and non-goals

Goals: bounded waiting for every priority; CRITICAL latency protected; deterministic, observable, reversible behavior. Non-goals: per-tenant priorities, preempting in-flight work, changing consent/policy order, changing retry semantics.

3. Requirements

- R1 (MUST) A QUEUED or RETRY_PENDING-eligible message whose age exceeds `aging.thresholds.<priority>` has its *effective priority* raised one level per threshold elapsed.
- R2 (MUST) Effective priority never exceeds `aging.max-effective` (default HIGH); CRITICAL is only reachable by requested priority.
- R3 (MUST) Requested `priority` is immutable; reporting uses requested priority; dispatch ordering uses effective priority.
- R4 (MUST) Promotion is idempotent: at most one promotion per message per level; double execution of the job must not double-send.
- R5 (MUST) Feature flag `messaging.dispatch.aging.enabled` default `false`; off means byte-identical behavior.
- R6 (SHOULD) Promotion audited (`PRIORITY_PROMOTED`, ids only) and counted.
- R7 (MUST) Works with `RabbitMessageBus` and `InMemoryMessageBus`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Acceptance criteria

- AC-1 Given aging enabled and a LOW message queued longer than `lowToNormalAfter`, when the job runs, then effective priority becomes NORMAL, requested stays LOW.
- AC-2 Given a message already at `max-effective`, then no further promotion.
- AC-3 Given the job runs twice concurrently, then exactly one promotion record and at most one dispatch of the message.
- AC-4 Given flag off, then no promotions and no new envelopes.
- AC-5 Given a sustained HIGH flood at 90% of consumer capacity, then a LOW message is dispatched within `lowToNormalAfter + normalToHighAfter + wait at HIGH` (bounded; asserted in IT with fake clock).
- AC-6 Given a CRITICAL burst, then CRITICAL dispatch delay p95 is within 10% of aging-off baseline in the local harness (measure only if executed).
- AC-7 Given an aged message is later cancelled before dispatch, then it is CANCELLED and never sent.
- AC-8 Given the stale lower-priority envelope is delivered after promotion, then it is acked and skipped (no second provider call).

5. Design

- Migration (expand-only, portable): `ALTER TABLE message_request ADD COLUMN effective_priority VARCHAR(16)` (nullable; null means requested), `ADD COLUMN priority_promoted_at TIMESTAMP`. Index decision: age scan uses `(status, created_at)`-prefix; evaluate against existing `(tenant_id,status,created_at)` and `(status,next_attempt_at)`; add `ix_msg_status_created (status, created_at)` only if the plan shows scanning (document in query-plans doc). Oracle DDL updated.
- `AgingPolicy` (pure): `Optional<Priority> promote(Priority requested, Priority effective, Duration age, AgingConfig cfg)`.
- `PriorityAgingJob` (`scheduler`): every `aging.interval` (default 10 s), selects up to `aging.batch` (default 200) QUEUED messages ordered by `created_at` where age > smallest threshold. For each: CAS `UPDATE message_request SET effective_priority=:new, priority_promoted_at=:now WHERE id=:id AND status='QUEUED' AND COALESCE(effective_priority,priority)=:old`. If 1 row updated: publish a new `DispatchEnvelope` (new `dispatchId`, same `attemptNo`, higher broker priority) via `MessageBus`; audit; metric.
- Stale envelope handling: both envelopes target the same message. The first consumer CAS `QUEUED->PROCESSING` wins; the second finds status != QUEUED (or `processed_message` conflict) and acks/skips (existing consumer logic, invariant 3). One `dispatchId` per (message, attemptNo) rule: to keep it, the promoted envelope reuses the *original* `dispatchId` plus an `envelopeRevision` field, and `processed_message` dedupe keys on `dispatchId` only (the second copy is skipped regardless of which copy arrives first). **Decision needed at design review**: reusing dispatchId (preferred; no contract change beyond an additive envelope field) vs new dispatchId (requires CAS-only dedupe). Spec adopts reuse.
- Config (`messaging.dispatch.aging.*`): `enabled`, `interval`, `batch`, `max-effective`, `thresholds.low-to-normal` (e.g., PT5M), `thresholds.normal-to-high` (PT15M); illustrative defaults.
- Alternatives: (a) weighted fair dequeue / separate consumer pools with reserved share (simple, robust, wastes capacity, no broker change; recommended complement: reserve 1 of 8 workers for LOW/NORMAL); (b) broker plugin priority tweaks (not supported in-place); (c) DB-driven dispatch ordering (higher DB load). Chosen: promotion job; reserved-worker option documented as fallback.
- ADR: propose ADR-0013 "Priority aging" before implementation.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Failure modes

Failure	Outcome
Job crashes after CAS before publish	message promoted in DB, envelope not published; original lower-priority envelope still exists so message is not lost; next tick republishes only if still QUEUED and `priority_promoted_at` older than `republish-after`; duplicates skipped by consumer
Job runs on two instances	CAS makes second a no-op per message
Broker down	job publish fails; metric; retry next tick; no state change beyond DB promotion (flag `needs_republish` derivable from `priority_promoted_at`)
Message consumed between select and CAS	CAS affects 0 rows; skip
Clock skew between instances	thresholds in minutes; tolerate; use DB `created_at` and app `Clock`

7. Security and privacy

No new inputs. Prevents a client from exploiting starvation, but must not let a client raise effective priority beyond `max-effective` (requested priority validation unchanged). Audit has ids only. No sensitive-change independent review required beyond normal, except reviewer must verify invariant 3 (double dispatch) given the duplicate envelope design.

8. Performance and capacity

Age scan bounded by `FETCH FIRST :batch ROWS ONLY` every 10 s; expected negligible vs dispatch load; extra envelopes bounded by promotions. Measurement plan: in-JVM harness scenario with HIGH flood; record numbers only from executed runs.

9. Observability

New metric `messaging.priority.promotions{from,to}` (propose adding to CONTRACT section 8 via DEVIATIONS entry), gauge of oldest queued age per effective priority (extends `messaging.queue.depth{priority}` semantics: document), log at DEBUG per batch, INFO summary. Alert: LOW oldest age > 2x `lowToNormalAfter` with aging enabled. Runbook `queue-backlog.md` to mention aging.

10. Test plan

- `AgingPolicyTest#promotesAfterThreshold_onlyOneLevelPerThreshold`, `#neverExceedsCeiling`, `#nullEffectiveTreatedAsRequested`.
- `PriorityAgingIT#lowMessageUnderHighFlood_isDispatchedWithinBound` (fake clock, FaultInjector NONE, consumer concurrency 1).
- `PriorityAgingIT#concurrentJobRuns_promoteOnce_dispatchOnce`.
- `PriorityAgingIT#staleEnvelope_isSkippedAfterPromotion_noDoubleSend`.
- `PriorityAgingIT#cancelledAfterPromotion_isNeverSent`.
- `PriorityAgingIT#flagOff_noPromotionsNoEnvelopes`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- `LoadHarness` scenario `criticalDelayNotRegressed` (executed locally only).
- Migration test: Flyway on H2 applies; columns nullable.

11. Task decomposition

Task	Role	Depends	DoD
T1 migration + entity fields + Oracle DDL	Backend		integration context boots
T2 `AgingPolicy` + config + unit tests	Backend		tests green
T3 `PriorityAgingJob` with CAS + republish	Backend	T1,T2	IT concurrent test
T4 envelope revision field + consumer skip verification	Backend	T3	stale envelope test
T5 metrics, audit, docs/runbook	Backend/Doc	T3	metric asserted
T6 harness scenario	Performance	T3	executed result recorded or "not measured"
T7 ADR-0013 and DEVIATIONS entry	Architecture		review gate A

12. Rollout and rollback

Ship with flag off. Staging: enable with synthetic flood and harness. Canary: enable on one instance (job is CAS-safe). Success: LOW wait bounded, CRITICAL delay unchanged, duplicate provider calls = 0 (compare attempts per dispatchId). Rollback: set flag false; columns remain unused (expand-only), previous app version ignores them.

13. Documentation updates

`docs/architecture/dispatch-retry-flow.md`, `docs/operations/runbooks/queue-backlog.md`, `docs/operations/slis-slos.md` (queue age by priority), ADR-0013, CONTRACT/DEVIATIONS, `ai-engineering/context/lifecycle.md` (note: effective priority).

14. Open questions

1. Reuse `dispatchId` vs new (owner: architecture; decide at gate A).
2. Does RETRY_PENDING age from `created_at` or from `next_attempt_at`? Proposal: from the moment it re-enters QUEUED.
3. Per-tenant fairness is out of scope: revisit if a single tenant dominates.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8. Provider Adapter Architecture

8.1 The idea

Email, SMS and push vendors all do roughly the same thing and fail in roughly the same ways, but their APIs, payloads, status words and quirks differ. The platform draws a boundary: **orchestration** (retries, policy, state, audit) knows only a small interface and a normalised outcome; **adapters** translate to and from each vendor. Replacing a vendor, or adding one, must not require touching the orchestration code. That is not a hope: `NewProviderWithoutOrchestrationChangeTest` proves it with a vendor that exists only inside the test.

8.2 The port

JAVA

```
public interface ChannelProvider {
    String providerId();                // matches provider_config.provider_id and the callback path segment
    Channel channel();
    ProviderResult send(ProviderRequest request);   // never throws for provider-side problems
    Optional<ProviderStatus> lookup(String idempotencyRef);  // reconciliation; empty = callbacks only
}
```

Implementations return a `ProviderResult` carrying one of six normalised outcomes:

JAVA

```
public enum ProviderOutcome {
    ACCEPTED, TRANSIENT_FAILURE, PERMANENT_FAILURE, RATE_LIMITED, TIMEOUT, AMBIGUOUS;
    public boolean isRetryable() { return this == TRANSIENT_FAILURE || this == RATE_LIMITED; }
    public boolean isUncertain() { return this == TIMEOUT || this == AMBIGUOUS; }
}
```

`isRetryable` and `isUncertain` are the two questions the rest of the system asks. Uncertain outcomes are **never** blindly retried for EMAIL and SMS.

Channel-specific sub-interfaces (`EmailProvider`, `SmsProvider`, `PushNotificationProvider`) let a registry look up providers by channel.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

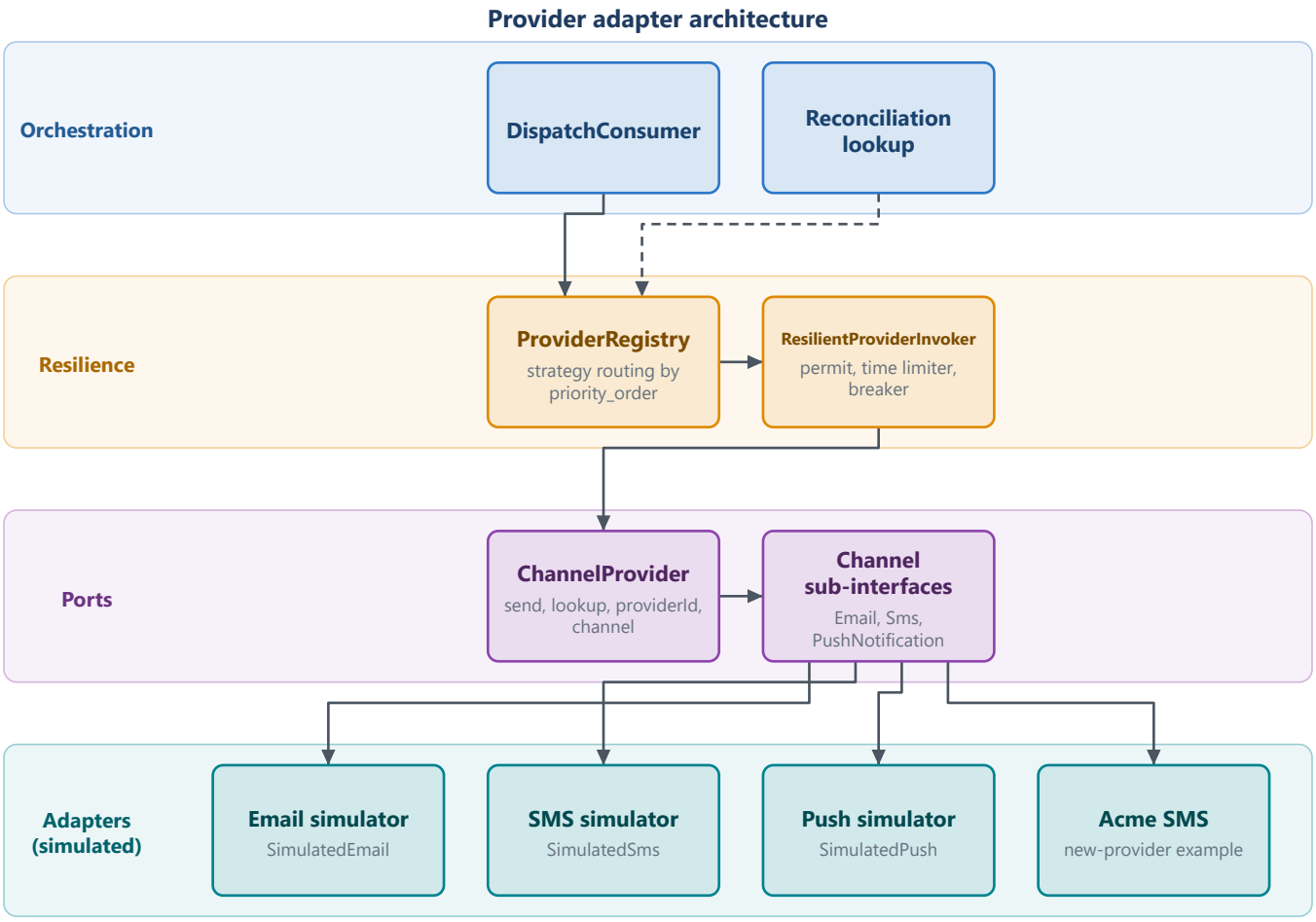


Figure 21. Ports and adapters for providers: orchestration only knows ChannelProvider and ProviderOutcome. A new vendor is a new bean plus configuration (ADR-0002, spec 0002). Implementation: backend/src/main/java/com/example/messaging/provider/ChannelProvider.java, backend/src/main/java/com/example/messaging/provider/ProviderRegistry.java, backend/src/main/java/com/example/messaging/provider/simulator/, specs/0002-new-provider-adapter.md

Class	Role
`ProviderRegistry`	Resolves the enabled providers for a channel in `priority_order` (strategy-based routing); backed by the `provider_config` table
`ResilientProviderInvoker`	Reserves a circuit-breaker permit, runs the call on the bounded provider executor with a time limiter, records the outcome and latency
`ProviderRequest`	Recipient address, rendered content, `idempotencyRef` (`messageId:attemptGroup`), channel options, timeout
`ProviderAdminService`	Enable or disable a provider, set rate limit and timeout, set a simulator fault mode (local and test only)
`FaultInjector`	Deterministic and probabilistic fault modes for simulators

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8.3 Resilience around every call

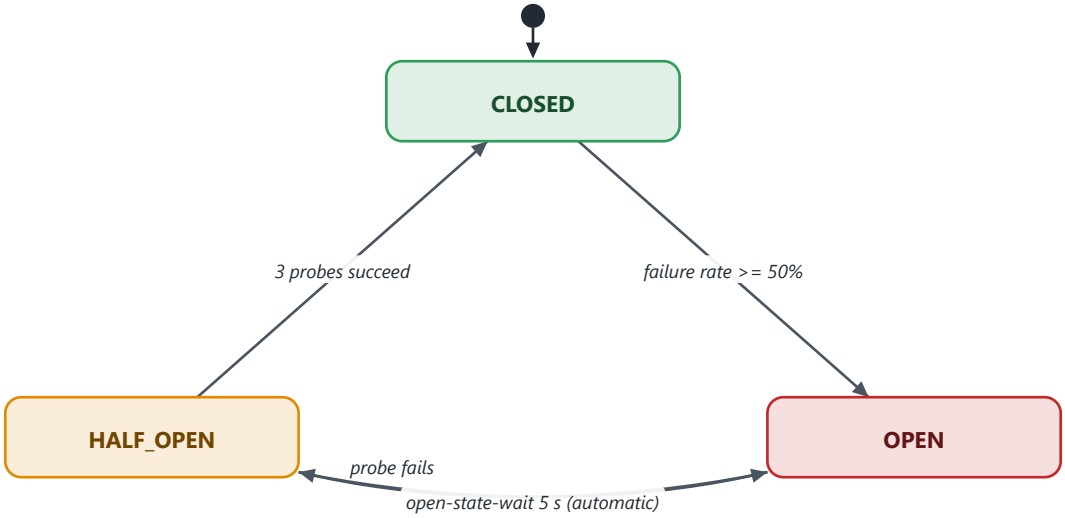


Figure 22. Circuit breaker around each provider (Resilience4j). Defaults: 50 percent failure rate over a 20-call window, minimum 10 calls, 5 s open wait, 3 half-open probes. The permit is reserved before the claim so a refusing breaker never burns an attempt. Implementation: backend/src/main/java/com/example/messaging/provider/ResilientProviderInvoker.java, backend/src/main/java/com/example/messaging/config/PipelineProperties.java, backend/src/main/resources/application.yml

Mechanism	Default	Where
Time limiter	Per-provider `timeout-ms` (`provider_config.timeout_ms`)	`ResilientProviderInvoker`, bounded executor (16 threads, queue 64)
Circuit breaker	Opens at 50 percent failures over a 20-call window (minimum 10 calls), 5 s open, 3 half-open probes	Resilience4j, configured from `messaging.pipeline.resilience.*`
Token bucket rate limit	`provider_config.rate_limit_per_sec`, burst = rate	`CoordinationService` (Redis Lua script, in-memory fallback)
Refusal handling	Open breaker or empty bucket requeues the delivery 0.1 to 5 s, consuming no attempt	`DispatchConsumer`

The breaker leaves `OPEN` by itself (`automaticTransitionFromOpenToHalfOpenEnabled`); otherwise a state pre-check would keep it open forever. `invoke(provider, request)` without a reservation returns `TRANSIENT` with code `CIRCUIT_OPEN` without touching the provider.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8.4 The simulators

No real provider is ever called. Four simulators stand in for vendors, all extending `SimulatedProviderBase`:

Provider id	Class	Channel	Callback key id
`simulated-email`	`SimulatedEmailProvider`	EMAIL	`simulated-email-key`
`simulated-sms`	`SimulatedSmsProvider`	SMS	`simulated-sms-key`
`simulated-push`	`SimulatedPushProvider`	PUSH	`simulated-push-key`
`acme-sms`	`AcmeSmsProvider`	SMS	`acme-sms-key`

`SimulatedProviderBase` behaves like a **compliant vendor**: it deduplicates on `idempotencyRef` before anything else, remembers what it accepted so that `lookup` works for reconciliation, and can post signed callbacks to the platform. Its fault modes (`FaultInjector.Mode`): `NONE`, `TRANSIENT`, `PERMANENT`, `RATE_LIMIT`, `TIMEOUT`, `AMBIGUOUS`, `ALTERNATING`, each with a probability. The `AMBIGUOUS` mode genuinely accepts the message and then "loses" the response, which is exactly the situation reconciliation exists for.

JAVA

```
case AMBIGUOUS:
    accept(ref); // the vendor did accept it, but the response is "lost"
    return ProviderResult.failure(ProviderOutcome.AMBIGUOUS, "SIM_LOST_RESPONSE",
            "accepted but response lost", elapsedMs(start));
```

A recipient address of the form `...+hardbounce@...` simulates a hard bounce (`HARD_BOUNCE`, which adds a suppression). `PERMANENT` mode answers `SIM_REJECTED` and does not suppress. Fault injection endpoints exist only in profiles `local` and `test`.

8.5 Worked example: adding a provider

`AcmeSmsProvider` is the worked "new provider" example (spec `specs/0002-new-provider-adapter.md`, skill `ai-engineering/skills/add-provider-adapter.md`). Adding a vendor is five small steps:

1. Implement `SmsProvider` (or the right sub-interface) as a Spring bean returning normalised outcomes.
2. Add a `ProviderPayloadMapper` bean that turns the vendor's webhook payload into a normalised event (Chapter 12), including its status vocabulary.
3. Insert a `provider_config` row (enabled, `priority_order`, rate limit, timeout, `hmac_key_ref`).
4. Add `messaging.callback.secrets.<keyId>` (a dummy value locally; a secret store in production).
5. Add mapper unit tests covering duplicate, out-of-order and unknown-status events (`EventNormalizerTest`).

Nothing in `service`, `messaging`, `scheduler`, `callback` or `domain` changes, and `NewProviderWithoutOrchestrationChangeTest` guards that those packages never name a concrete provider.

> **Spec drift recorded in the deviations**
>
> Spec 0002 still mentions the Acme event id as `eid` and the key id as `acme-key-1`. As implemented, Acme's payload uses `id` for the event id and the key id is `acme-sms-key` (seed `hmac_key_ref`). The implemented values are the ones in `docs/architecture/callbacks-normalization.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8.6 Tests that prove it

Test	Proves
`ProviderRegistryTest`	Routing order, disabled providers skipped
`ResilientProviderInvokerTest`	Timeouts, breaker transitions, reservation and release
`SimulatedProviderTest`, `FaultInjectorTest`	Provider-side idempotency, fault modes
`ProviderOutcomeIT`	Each outcome produces the right transition
`NewProviderWithoutOrchestrationChangeTest`	A new vendor needs no orchestration change
`ProviderApiIT`	Provider health and config endpoints

Reference material

ADR-0002: Provider abstraction

Source document: `docs/adr/0002-provider-abstraction.md`

Status: Accepted (2026-10-18)

Context

Three channels (EMAIL, SMS, PUSH) are served by external vendors with different payloads, auth, error vocabularies, rate limits and callback formats. Vendors change over time, and a second SMS vendor ("Acme") must be addable without touching orchestration (`DispatchService`, `RetryPolicy`, state machine). No real provider is ever called in this repo; simulators implement the same ports.

Decision

1. **Adapter**: one port per channel in package `provider`: `EmailProvider`, `SmsProvider`, `PushNotificationProvider`. Each adapter translates a channel-neutral request into the vendor call and maps vendor results to the closed enum `ProviderOutcome{ACCEPTED, TRANSIENT_FAILURE, PERMANENT_FAILURE, RATE_LIMITED, TIMEOUT, AMBIGUOUS}` wrapped in `ProviderResult`. Providers also expose `Optional<ProviderStatus> lookup(String idempotencyRef)` for reconciliation (ADR-0010).
2. **Strategy**: `ProviderRegistry` selects the provider for a channel from `provider_config` (`enabled`, `priority_order`) and the circuit state. Selection is data-driven, so failover or A/B is a config change.
3. **Single choke point**: all calls go through `ResilientProviderInvoker` (Resilience4j circuit breaker + time limiter on a bounded executor). Adapters must not implement their own retry; retry belongs to `RetryPolicy` (ADR-0006).
4. **Callback side**: each provider has a payload mapper registered with `EventNormalizer` (see `docs/architecture/callbacks-normalization.md`), so the rest of the system sees only `DeliveryEvent`.
5. **Worked extension**: `AcmeSmsProvider` (a 4th simulator) is added as a Spring bean implementing `SmsProvider`, a `provider_config` row (`provider_id=acme-sms`), a `messaging.providers.acme-sms.*` config block and one mapper. Zero changes to `DispatchService`, `RoutingService`, `MessageStateMachine`. The step-by-step is spec

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`specs/0002-new-provider-adapter.md`.

Alternatives considered

Option	Why not
One fat `Provider` interface with `send(channel, ...)`	Leaky: SMS and email have different options; violates interface segregation
Provider-specific code in orchestration (`switch(providerId)`)	Every new vendor edits the hot path; regression risk
Spring Cloud Stream binder-style generic plugin	Over-engineered for three channels
Call vendors through an API gateway/ESB	Hides outcome semantics we need (timeout vs rejected)

Consequences

- Positive: open/closed for providers; consistent failure classification; test doubles are trivial (`FaultInjector` drives outcomes).
- Negative: lowest-common-denominator request model; vendor-specific features go through `channelOptions` maps.
- Risk: misclassifying a vendor error (e.g. treating a vendor 500-after-accept as TRANSIENT) can cause duplicate sends; adapters must return AMBIGUOUS when the vendor may have accepted.

Validation

- Adapter conformance tests per provider (see source `provider` test package); `AcmeSmsProvider` tests must prove it is reachable only through `ProviderRegistry`.
- ArchUnit-style rule recommended: `service` and `messaging` must not depend on concrete provider classes (see `ai-engineering/checklists/code-review.md`).
- Metrics: `messaging.provider.calls{provider,channel,outcome}`, `messaging.provider.latency{provider}`, `messaging.provider.circuit.state{provider}`.

Spec 0002: New provider adapter

Source document: `specs/0002-new-provider-adapter.md`

Written before implementation; where it differs (event id field, key id), the implemented values are in Chapter 12.

- Status: **Implemented as a simulator** (reference implementation; no real vendor exists or is contacted). Rewritten on 2026-10-18 to match the code; the first draft planned an `AcmeClient`, an `AcmeCallbackMapper` and the key id `acme-key-1`, none of which exist.
- Owner (role): Backend
- Date: 2026-10-18
- Related: ADR-0002, ADR-0004, ADR-0010; `docs/architecture/callbacks-normalization.md`; `docs/architecture/callback-flow.md`; `docs/design/DEVIATIONS.md` (stage 2, item 7; bullet on `ProviderAdminService`); `ai-engineering/skills/add-provider-adapter.md`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. Context and problem

A second SMS vendor ("Acme", simulated) is needed for failover and rate-limit capacity. Goal: onboard it **without changing orchestration** (`DispatchConsumer`, `RetryPolicy`, `MessageStateMachine`, `ProviderRegistry` selection logic). This spec proves the extension point works; no real vendor is contacted.

2. Goals and non-goals

Goals: new adapter, payload mapper, config, tests; selectable via `provider_config`; failover works. Non-goals: real vendor integration, changing the SMS port, changing the lifecycle.

3. Requirements and what was built

Req	Requirement	Implementation (verified in source)
R1	Adapter implementing `SmsProvider`, id `acme-sms`, Spring bean	`provider/simulator/AcmeSmsProvider` (`@Component`, extends `SimulatedProviderBase implements SmsProvider`, `ID = "acme-sms"`, message id prefix `acme-`). It lives in `provider.simulator`, not `provider`
R2	Every result maps to `ProviderOutcome`; ambiguous after-send errors map to AMBIGUOUS; no retry inside the adapter	Shared by all simulators in `SimulatedProviderBase` + `FaultInjector`; `ProviderOutcome` lives in package `provider`
R3	Sends `idempotencyRef` (`messageId:attemptGroup`) as the client reference	`SimulatedProviderBase.send` de-duplicates on the idempotency reference like a compliant vendor
R4	`lookup(idempotencyRef)`	`SimulatedProviderBase.lookup`, used by `ReconciliationService` through `ResilientProviderInvoker.lookup`
R5	Callback mapper for Acme's own payload	`callback/AcmeSmsPayloadMapper`, a `ProviderPayloadMapper` bean found by `EventNormalizer`. Payload `{"ref","state","ts","id","code"}`: `ref` is Acme's own message id, `state` is `queued`/`sent`/`ok`/`fail`, `ts` is epoch seconds, `id` is the event id (the first draft said `eid`)
R5b	Signed callbacks	endpoint `/api/v1/callbacks/acme-sms`; key id **`acme-sms-key`** (seed `provider_config.hmac_key_ref`); dummy secret `messaging.callback.secrets.acme-sms-key` in `application-local.yml` and `application-test.yml`, `${CALLBACK_SECRET_ACME_SMS}` in `application-prod-like.yml`
R6	Configuration and `provider_config` row	seed row `prv-004` (`acme-sms`, `SMS`, **enabled = FALSE**, `priority_order` 2, 50 req/s, timeout 3000 ms, key ref `acme-sms-key`) in `db/seed/V900__synthetic_seed.sql`
R7	Fault injection through the common base	`FaultInjector` / `ProviderFaultController` (`PUT /api/v1/providers/{providerId}/fault-mode`, local/test only)
R8	No orchestration change	enforced by `NewProviderWithoutOrchestrationChangeTest#orchestrationPackagesNeverNameAConcreteProvider` (packages `service`, `messaging`, `scheduler`, `callback`, `domain` never name a concrete provider)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Acceptance criteria and evidence

AC	Criterion	Test (class#method)
AC-1	With `simulated-sms` disabled and `acme-sms` enabled through the config API, an SMS dispatches through Acme; attempt row has `provider_id=acme-sms`	`NewProviderWithoutOrchestrationChangeTest#acmeSmsIsDisabledByDefaultAndSwitchedOnPurelyThroughConfiguration`
AC-2	`priority_order` and breaker state drive failover between the two SMS vendors	`NewProviderWithoutOrchestrationChangeTest#priorityOrderAndCircuitStateDriveFailoverBetweenTheTwoSmsVendors`, `ProviderRegistryTest#anOpenCircuitFailsOverToTheNextProviderButIfAllAreOpenTheFirstIsStillReturned`
AC-3	Timeout leads to `DELIVERY_UNKNOWN` without a blind resend; `lookup` drives reconciliation	provider-independent: `FailureRecoveryIT#scenario04_providerTimeout_movesToDeliveryUnknown_reconcilesWithoutDuplicateSend`, `ReconciliationIT` (not Acme-specific)
AC-4	Acme callbacks normalise (`ok` to DELIVERED, `fail` to FAILED, `queued` to PROVIDER_ACCEPTED); duplicates and bad signatures handled	`EventNormalizerTest#acmeUsesItsOwnShapeWithEpochSecondsAndNoIdempotencyEcho`, `CallbackApiIT`, and the signed Acme callback step in `NewProviderWithoutOrchestrationChangeTest`
AC-5	Orchestration classes untouched	the source-level rule in R8 (a diffstat review was the original plan; the test replaces it)
AC-6	`GET /api/v1/providers` lists `acme-sms` with circuit state and rate limit	`ProviderApiIT`
AC-7	A brand new vendor defined only in test code is routed, called and normalised	`NewProviderWithoutOrchestrationChangeTest#aBrandNewVendorIsRoutedCalledAndNormalisedWithoutChangingAnyMainClass`

Executed results are recorded only in `docs/validation/VALIDATION_REPORT.md`; this spec does not claim a pass count.

5. Design (as built)

1. Adapter: `AcmeSmsProvider` is a thin subclass of the simulator base (acceptable for a simulator; a real vendor adapter should be an independent class that maps accepted to ACCEPTED, validation error to PERMANENT_FAILURE, throttle to RATE_LIMITED, connect refused before send to TRANSIENT_FAILURE, read timeout to TIMEOUT, unknown-after-send to AMBIGUOUS). There is no separate `AcmeClient`.
2. Mapper: `AcmeSmsPayloadMapper` implements `ProviderPayloadMapper` (parse and render), registered simply by being a bean; `EventNormalizer` rejects two mappers for one provider id (checked in `EventNormalizerTest`).
3. Data: the `acme-sms` row is in the synthetic seed (disabled); enabling is `PUT /api/v1/providers/acme-sms/config` by PLATFORM_ADMIN (audited), no restart.
4. Secrets: dummy local/test secret only; production reads `CALLBACK_SECRET_ACME_SMS` from the environment/secret store.
5. Metrics need no change (the `provider` tag distinguishes it). `ProviderAdminService` lives in package `provider` because it knows the simulator classes. Alternatives: dynamic plugin loading (rejected: over-engineered).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Failure modes

Classification-driven like every provider (ADR-0006, ADR-0010). Specific risk: a real vendor that ignores the idempotency reference would produce duplicates after a "not received" lookup; treat unknown lookup answers as unknown.

7. Security and privacy

Own HMAC key id bound to the provider (a key of another provider is rejected with the uniform 401); secrets only via config; free-text vendor reasons never reach audit or logs; raw payloads archived redacted; mapper is pure.

8. Performance and capacity

Per-provider token bucket from `provider_config.rate_limit_per_sec` (burst = rate); timeout from `provider_config.timeout_ms`.

9. Observability

`messaging.provider.calls{provider="acme-sms",...}`, `messaging.provider.latency`, `messaging.provider.circuit.state`; dashboards use the provider label.

10. Rollout and rollback

Disabled by default; enable through the provider config API after staging validation. Rollback: disable the provider; no schema contraction needed.

11. Open questions

Real vendor semantics for lookup and idempotency are unknown for any real vendor and must be verified against vendor documentation, not assumed.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

9. Email, SMS, and Push Workflows

9.1 One pipeline, three personalities

All three channels travel the same pipeline: submit, outbox, bus, consumer, provider, callback. What differs is the content rules enforced at authoring and render time, what an ambiguous result means, and what a receipt means.

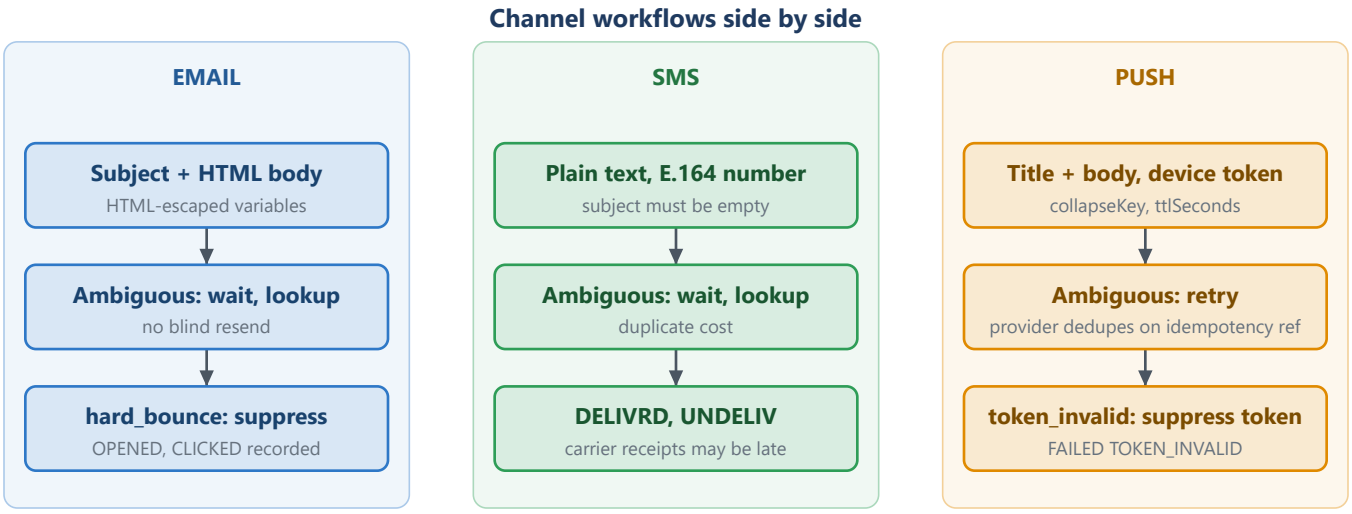


Figure 23. Email, SMS and push share the pipeline but differ in what the platform does after an ambiguous result and in what a receipt means. Implementation: backend/src/main/java/com/example/messaging/provider/EmailProvider.java, backend/src/main/java/com/example/messaging/provider/SmsProvider.java, backend/src/main/java/com/example/messaging/provider/PushNotificationProvider.java, backend/src/main/java/com/example/messaging/callback/SimulatedEmailPayloadMapper.java

Aspect	EMAIL	SMS	PUSH
Address used	`recipient.email`	`recipient.phone_e164`	`recipient.push_token`
Subject	Required, at most 255 characters; CR and LF stripped (header injection defence)	Must be empty	Required (the notification title), at most 65 characters
Body	HTML or text, at most 10000 characters authored, 200000 rendered; **variable values are HTML-escaped**	Plain text; encoding GSM-7 (160 or 153 characters per segment, extension characters cost 2) or UCS-2 (70 or 67); at most 10 segments	At most 240 characters
Channel options	`replyTo`, custom headers matching `X-[A-Za-z0-9-]+`, single line, at most 500 characters	`senderId`	`collapseKey`, `ttlSeconds`
Ambiguous outcome	`DELIVERY_UNKNOWN`, wait for callback, then `lookup`; retry only on "not received"	Same	Retry like a transient failure (the provider collapses duplicates)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Aspect	EMAIL	SMS	PUSH
Permanent failure signals	`hard_bounce` (adds suppression), `failed`, `rejected`	`UNDELIV`, `REJECTD`	`token_invalid` (suppresses the token), `failed`
Engagement events	`open`, `click`, `unsubscribe` (clears marketing opt-in)	none	`opened`, `clicked`
Why duplicates matter	Visible and reputational	Cost and annoyance	Collapse on the device

The limits come from `messaging.templates.*` (`MessagingProperties.Templates`): `maxVariables` 50, `maxVariableLength` 2000, `maxBodyLength` 10000, `maxRenderedEmailLength` 200000, `maxSmsSegments` 10, `maxPushTitleLength` 65, `maxPushBodyLength` 240, `maxSubjectLength` 255.

9.2 Email workflow

1. **Submit** with `channel=EMAIL`, a template whose channel is EMAIL, and variables. The submit transaction render-checks the template (output discarded) so a missing variable fails fast with `TEMPLATE_VARIABLE_MISSING` and nothing is persisted.
2. **Dispatch.** The consumer renders the pinned template again, HTML-escaping values, and calls the email provider with the subject, body and options.
3. **Receipts.** The simulated email vendor posts events with the payload shape `{"event","message_id","event_id","timestamp","meta":{"idempotencyRef"},"reason"}`. `delivered` becomes `DELIVERED`; `hard_bounce` becomes `BOUNCED` (status `FAILED`, address suppressed); `soft_bounce` becomes `DEFERRED` (rank 1, no state change); `open` and `click` are engagement only.
4. **Unsubscribe.** An `unsubscribe` event is recorded and clears the recipient's marketing opt-in, so the next MARKETING message is rejected `OPTED_OUT`.

JSON

```
{"event":"delivered","message_id":"em-77ab","event_id":"evt-1001","timestamp":"2026-10-18T10:15:30Z",
  "meta":{"idempotencyRef":"3f2c...:1"}}
```

9.3 SMS workflow

SMS adds **segment arithmetic** to authoring: a message that fits 160 GSM-7 characters is one segment, a longer one is `ceil(septets / 153)` segments, and any character outside the GSM alphabet (for example an emoji) switches the whole message to UCS-2 with 70 and 67 limits. The renderer computes this at validation time and rejects a body that would need more than ten segments. Phone numbers are stored in E.164 form (`phone_e164`) and masked in every response (`+1555***0101`).

Vendor status words are carrier-style: `ACCEPTD`, `SENT`, `DEFERRD`, `DELIVRD`, `UNDELIV`, `REJECTD` (simulated SMS) or `queued`, `sent`, `ok`, `fail` (Acme SMS, whose timestamp is epoch **seconds**). Carrier receipts can arrive late, which is why `SENT_TO_PROVIDER` without a receipt after `receipt-timeout` (15 minutes) becomes `DELIVERY_UNKNOWN` rather than failing.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

9.4 Push workflow

Push targets a device token. Two properties make it forgiving:

- **Idempotent collapse.** `collapseKey` lets a newer notification replace an older pending one on the device, and the provider deduplicates on the idempotency reference, so after a timeout the platform can safely retry.
- **Token lifecycle.** A `token_invalid` receipt is a permanent failure with reason `TOKEN_INVALID` and suppresses that recipient's push token, so the platform stops sending to a dead device.

`channelOptions.push.ttlSeconds` bounds how long the vendor holds an undelivered notification.

9.5 Fallback between channels

A request may list `fallbackChannels` (for example `["SMS"]` behind an EMAIL primary). Fallback follows strict rules:

1. It is attempted only after `PERMANENT_FAILURE` or exhausted retries of the primary channel.
2. Each fallback channel is evaluated through the **full** policy chain independently; fallback never bypasses consent, suppression or quiet hours (`PolicyService.evaluateFallbacks`).
3. The fallback gets a **fresh attempt budget** and a new idempotency reference group (`messageId:2`, `messageId:3`, ...), recorded as `channel_used` on the same message.
4. Chains are individually legal transitions: `PROCESSING -> FAILED -> QUEUED` after a permanent failure; `PROCESSING -> RETRY_PENDING -> QUEUED` after exhausted retries.

9.6 What the console shows

`MessageDetail` never contains rendered text or a full address. It carries the masked address (`r***@example.test`), `channelUsed`, the attempt list (provider id, outcome, provider message id, error code, latency) and the delivery events. The detail page renders the merged **history** (`GET /messages/{id}/history`: audit events, attempts and delivery events ordered in time) as the timeline widget.

9.7 Tests that prove it

Test	Proves
`TemplateRendererTest`	HTML escaping, subject CR/LF stripping, GSM-7 and UCS-2 segment counts, no expression evaluation
`ChannelDispatchIT`	Each channel dispatches end to end with its content rules
`SimulatedProviderTest`	Hard bounce address pattern, provider-side dedupe
`EventNormalizerTest`	Per-vendor status mapping, unknown status rejected
`ProviderOutcomeIT`	PUSH ambiguous is retried, EMAIL and SMS ambiguous become unknown
`FailureRecoveryIT#scenario04_*`	Ambiguous handling per channel, PUSH dedupe on the idempotency reference

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Reference material

Callback normalisation: vendor payloads and mapping

Source document: `docs/architecture/callbacks-normalization.md`

Vendor payloads and mapping (as implemented)

The four mappers live in package `callback` (`SimulatedEmailPayloadMapper`, `SimulatedSmsPayloadMapper`, `SimulatedPushPayloadMapper`, `AcmeSmsPayloadMapper`), each a `ProviderPayloadMapper` bean that `EventNormalizer` finds by provider id. Every mapper can also **render** a vendor payload from a canonical event, which is how the provider simulators and the tests produce callbacks that take the production path. Vendor statuses are matched case-insensitively; an unknown status, a missing status, a missing message reference (neither provider message id nor idempotency ref), a missing event id (payload and `X-Event-Id` header both absent), an unparsable timestamp or an id with characters outside `[A-Za-z0-9._:~=+/-]` makes the payload **malformed**: the endpoint answers `400 MALFORMED_REQUEST` (nothing is echoed back), counts `messaging.callbacks{result="malformed"}` and archives the raw payload (redacted) for diagnostics.

Provider (path id)	Payload shape	Message reference	Event id	Timestamp
`simulated-email`	`{"event","message_id","event_id","timestamp","meta":{"idempotencyRef"},"reason"}`	`message_id` and/or `meta.idempotencyRef`	`event_id` (else `X-Event-Id`)	ISO-8601 `timestamp`
`simulated-sms`	`{"status","msg_id","event_id","ts","ref","err"}`	`msg_id` and/or `ref`	`event_id` (else header)	ISO-8601 `ts`
`simulated-push`	`{"type","notification_id","id","time","ref","error"}`	`notification_id` and/or `ref`	`id` (else header)	ISO-8601 `time`
`acme-sms`	`{"ref","state","ts","id","code"}`	`ref` is Acme's own message id (no idempotency echo)	`id` (else header)	`ts` in epoch **seconds** (number)

Provider	Vendor status	`DeliveryEventType`
simulated-email	`queued` / `sent`	PROVIDER_ACCEPTED / SENT
simulated-email	`delivered`	DELIVERED
simulated-email	`hard_bounce`	BOUNCED (adds suppression of the address)
simulated-email	`soft_bounce`	DEFERRED
simulated-email	`failed` / `rejected`	FAILED / REJECTED
simulated-email	`open` / `click` / `unsubscribe`	OPENED / CLICKED / UNSUBSCRIBED (unsubscribe clears marketing opt-in)
simulated-sms	`ACCEPTD` / `SENT` / `DEFERRD`	PROVIDER_ACCEPTED / SENT / DEFERRED
simulated-sms	`DELIVRD`	DELIVERED
simulated-sms	`UNDELIV` / `REJECTD`	FAILED / REJECTED

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Provider	Vendor status	`DeliveryEventType`
simulated-push	`accepted` / `sent` / `deferred`	PROVIDER_ACCEPTED / SENT / DEFERRED
simulated-push	`delivered`	DELIVERED
simulated-push	`token_invalid`	FAILED with reason `TOKEN_INVALID` (suppresses the recipient's push token)
simulated-push	`failed`	REJECTED
simulated-push	`opened` / `clicked`	OPENED / CLICKED
acme-sms	`queued` / `sent`	PROVIDER_ACCEPTED / SENT
acme-sms	`ok` / `fail`	DELIVERED / FAILED

Example email payload:

JSON

```
{"event":"delivered","message_id":"em-77ab","event_id":"evt-1001","timestamp":"2026-10-18T10:15:30Z",
  "meta":{"idempotencyRef":"3f2c...:1"}}
```

maps to `NormalizedEvent{providerId=simulated-email, providerEventId=evt-1001, type=DELIVERED, providerMessageId=em-77ab, idempotencyRef=3f2c...:1, occurredAt=2026-10-18T10:15:30Z}`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

10. Template Management

10.1 Why templates are governed

A template error sent at scale is a public incident: a wrong fee in a trade confirmation, a broken link in a password reset, a marketing sentence inside a security alert. The platform treats template content as **controlled change**: versioned, previewed, approved by a second person, and pinned to each message at accept time.

10.2 Model

Concept	Table and key	Notes
Template	`template`, `UNIQUE(tenant_id, code, channel)`	Carries the **category**, fixed per template so that marketing content cannot be sent as TRANSACTIONAL (ADR-0012)
Template version	`template_version`, `UNIQUE(template_id, version_no, locale)`	Status, subject, body, declared `variables_json`, `created_by`, `approved_by`, `published_at`, optimistic `version`
Locale variants	Same `version_no`, different `locale`	`en`, `es`, ... Synthetic seed data ships templates in `en` and `es`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

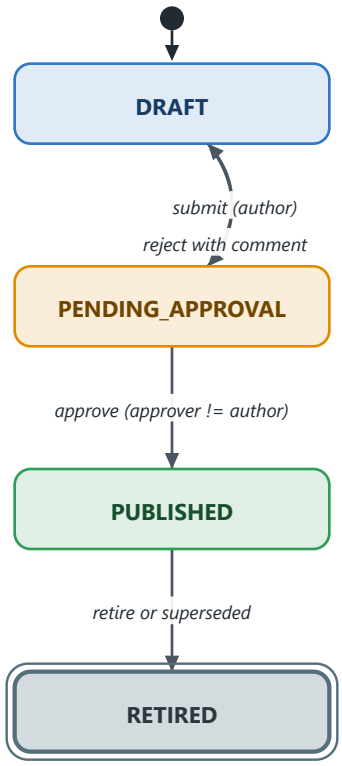


Figure 24. Template version lifecycle with four-eyes approval. Published versions are immutable; edits create a new version. Implementation: backend/src/main/java/com/example/messaging/domain/TemplateStatus.java, backend/src/main/java/com/example/messaging/service/TemplateService.java

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

10.3 Approval flow

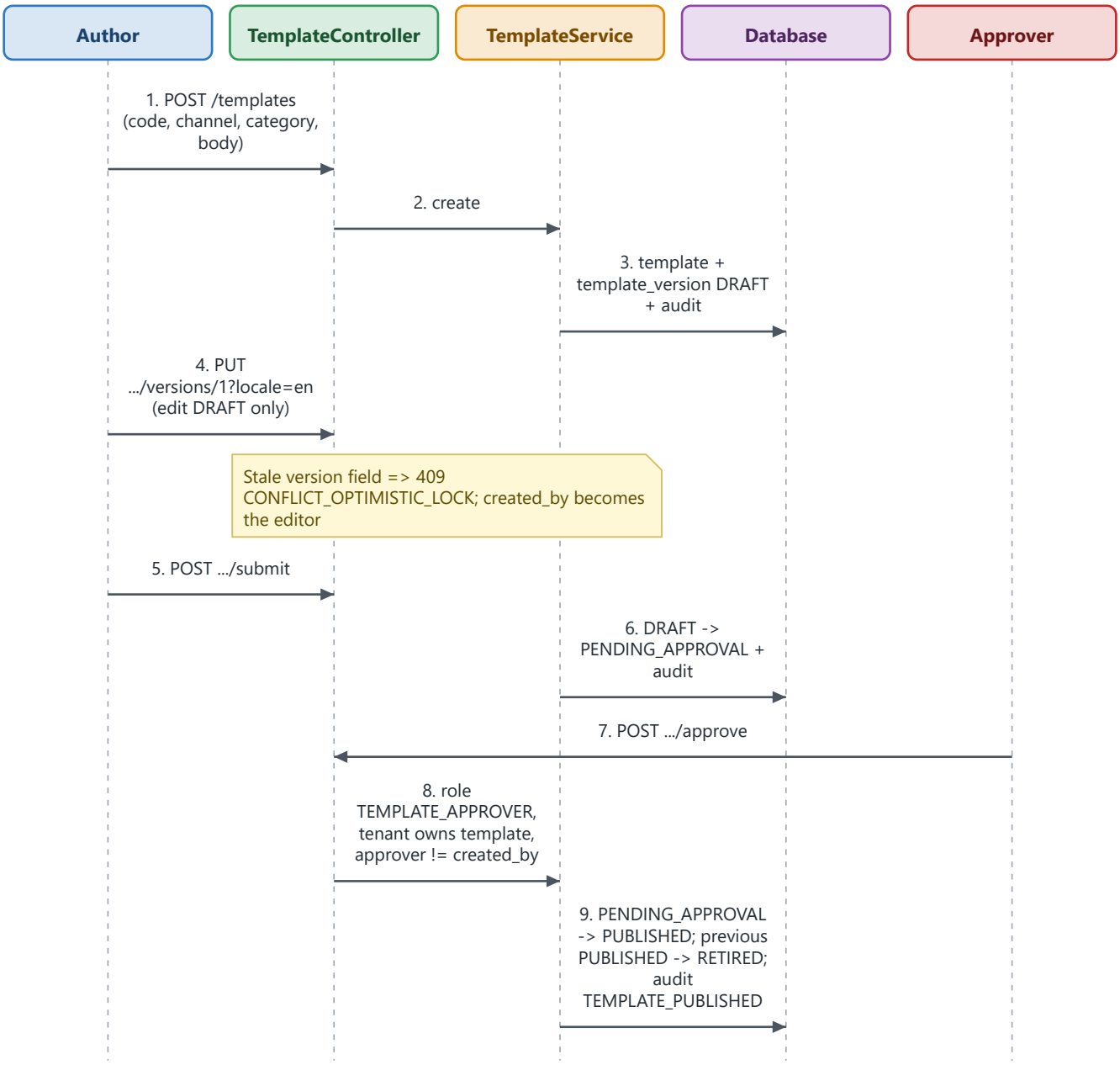


Figure 25. Template approval flow: author drafts, submits; a different approver publishes; the previous published version of the same locale is retired in the same step. Implementation: backend/src/main/java/com/example/messaging/web/TemplateController.java, backend/src/main/java/com/example/messaging/service/TemplateService.java, backend/src/main/java/com/example/messaging/security/AuthorizationService.java

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Controls enforced server side (`TemplateService`, `AuthorizationService`):

Control	Effect
Four eyes	The approver must differ from `created_by`. An edit overwrites `created_by` with the editor (author of record), so a second person editing a draft cannot sidestep four-eyes approval (DEVIATIONS)
Immutability	Only `DRAFT` versions can be edited; a published version never changes, edits require a new version
Optimistic locking	`PUT` carries the version field; a stale write is 409 `CONFLICT_OPTIMISTIC_LOCK`
Supersede	Approving a version retires the previously published version of the same locale in the same step
Duplicates	A duplicate code and channel, or a duplicate locale version, is 409 `DUPLICATE_RESOURCE`
Roles	Authors create and edit; approvers approve, reject and retire; `TENANT_ADMIN` can also author and retire
Audit	Every step is audited; `GET /api/v1/templates/{id}/history` returns the trail

10.4 The safe renderer

`TemplateRenderer` is deliberately not a general template engine. Its grammar is `{{name}}` or `{{name|default}}` and nothing else.

JAVA

```
// com.example.messaging.template.TemplateRenderer (class comment)
// Grammar: {{name}} or {{name|default}}. There is no expression evaluation of any kind: anything that is not a
// plain identifier inside the braces is a template error ({{ 7*7 }}, {{a.b}}, {{{x}}}).
// Only variables declared by the template are accepted. For EMAIL, values (and defaults) are HTML-escaped.
```

Property	Rule
No code execution	No expressions, includes, reflection or property paths; `${...}` is literal text
Declared variables only	An undeclared placeholder is an authoring error; an unsupplied required variable is `TEMPLATE_VARIABLE_MISSING`
Escaping	HTML escaping of values for EMAIL; CR and LF stripped from subjects and titles
Limits	50 variables, 2000 characters per value, 200 characters per default
Same code path for preview	`POST /api/v1/templates/preview` uses the same renderer and never sends

Authoring example (from the OpenAPI spec): `<p>Use code {{code}} to reset your password. It expires in {{minutes|15}} minutes.</p>` with `variables: [code, minutes]`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

10.5 Pinning (ADR-0007)

At accept time the platform resolves the latest `PUBLISHED` version for the template code, locale and channel (or the explicit version requested) and stores `template_code`, `template_version_no` and `template_locale` on the message. Variables are stored; the **rendered body is not**. Rendering happens again at dispatch with the pinned version. Consequences:

- A template retired or superseded after accept still renders for messages already accepted.
- An accepted message cannot change meaning because someone published a new version.
- Message storage holds no rendered personal content at rest.

10.6 Tests that prove it

Test	Proves
`TemplateRendererTest`	Grammar, escaping, error cases, limits
`TemplateLifecycleIT`	Draft, submit, approve, reject, retire; four-eyes; supersede; optimistic lock
`NotificationApiIT`	Pinning and missing-variable behaviour at submit
`SecurityMatrixTest`	Role matrix on template endpoints

Reference material

Template approval flow (design document)

Source document: `docs/architecture/template-approval-flow.md`

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter.

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter.

Controls

- Preview (`POST /templates/preview`) renders with sample data using the same `TemplateRenderer`, never sends.
- `TemplateRenderer` is an allow-listed `{{var}}` substitution: variables declared in `variables_json`; unknown variable = `TEMPLATE_RENDER_ERROR`; HTML escaping for email HTML; no expression evaluation, no includes, no reflection.
- Category binding: template category is fixed per template so marketing content cannot be sent as TRANSACTIONAL (ADR-0012).
- Published versions are immutable; edits require a new version (ADR-0007).
- Four-eyes: approver and author must differ; enforced server-side.
- Audit trail: `GET /api/v1/templates/{id}/history`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ADR-0007: Template versioning and pinning

Source document: `docs/adr/0007-template-versioning-and-pinning.md`

Status: Accepted (2026-10-18)

Context

Templates carry legal and regulatory wording (trade confirmations, security alerts). A template edit must never change what an already-accepted message says, must be reviewed (four-eyes), and must be traceable to the exact text a customer received. Rendering happens at dispatch time, possibly minutes after acceptance, so late edits are a real hazard.

Decision

- `template` (identity: tenant, code, channel, category) has many immutable-once-published `template_version` rows, unique per `(template_id, version_no, locale)`. Status `TemplateStatus{DRAFT, PENDING_APPROVAL, PUBLISHED, RETIRED}`.
- **Pin at accept time**: `SubmitMessageRequest.template.version = null` resolves to the latest PUBLISHED version for the locale **at acceptance** and writes `template_code`, `template_version_no`, `template_locale` onto `message_request`. An explicit version is validated to exist and be PUBLISHED (or RETIRED-but-still-allowed only if config permits; default: PUBLISHED only).
- **Render at dispatch with the pinned version**, even if that version has since been RETIRED. Retiring prevents *new* messages from pinning it; it does not break accepted ones. The render-check at submission validates variables (`VariableValidator`) but does not store the rendered body (data minimization; store variables, not body).
- **Edits**: only DRAFT versions are editable (optimistic lock via `If-Match`/`version`). Published content is immutable; a change is a new version.
- **Approval flow**: author submits (`DRAFT to PENDING_APPROVAL`); approver (different principal, four-eyes) publishes; on approve, the previously PUBLISHED version of the same locale becomes RETIRED; reject returns to DRAFT with comment. All steps audited.
- **Locale fallback**: requested locale, then language-only, then tenant default; the *resolved* locale is what gets pinned.

What happens to accepted messages if a template changes?

Event after acceptance	Effect on accepted (not yet sent) messages
New version published	None: they render with the pinned older version
Pinned version retired	None: still renders; audit notes pinned-retired send
Pinned version content found harmful	Operator cancels (`/cancel`, status VALIDATED/QUEUED/RETRY_PENDING) or bulk-cancels by filter; a retire alone does not stop them. Emergency lever: provider/consumer pause (`/ops/consumers/pause`)
Variables of the pinned version no longer satisfiable	Cannot happen: variable validation occurred at accept time against the same pinned version

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Alternatives considered

Option	Why not
Always render latest at dispatch	Silent content change after acceptance; unauditable
Snapshot rendered body at accept	PII/regulated content stored at rest; larger rows; violates minimization
Copy template body into each message	Same storage issue, no dedupe
Mutable templates with history table	Hard to prove what was sent

Consequences

- Positive: reproducible sends; safe rollout; audit by `(template_code, version_no, locale)`.
- Negative: stale wording may still go out for already-accepted messages (accepted trade-off; mitigated by cancel tooling).
- Risk: `TemplateRenderer` allow-list evolution must not break old versions: renderer changes are versioned and regression-tested against fixtures (`TemplateRendererTest`).

Validation

- Tests: pin-at-accept, retire-after-accept still renders, approver != author, retire-then-new-submissions-use-new (see source `service`/`template` tests).
- Security: template injection controls in `docs/security/threat-model.md` (allow-listed `{{var}}`, no expressions, HTML escaping).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

11. Customer Preferences and Suppression

11.1 Two different concepts

Concept	Meaning	Set by	Scope
Preference	What a recipient chose: marketing opt-in, per-channel enabled flags, quiet hours, time zone, fallback order	Recipient (through tenant admin or operator tooling)	Per recipient
Suppression	A hard block on an address and channel: hard bounce, spam complaint, regulatory request, token invalid	Platform automatically or an operator	Per tenant, channel and address hash

Preferences are soft and reversible; suppression is hard and protects the platform's sender reputation and legal position. Suppression blocks **every category** on that channel, including SECURITY_ALERT.

11.2 The data

Table	Key columns
`recipient`	`external_ref` unique per tenant, `email`, `phone_e164`, `push_token`, `locale`, `time_zone`, `deleted`, `version`
`recipient_preference`	`marketing_opt_in`, `email_enabled`, `sms_enabled`, `push_enabled`, `quiet_start`, `quiet_end`, `fallback_order`, `version`
`suppression`	`tenant_id`, `channel`, `address_hash`, `reason`, `UNIQUE(tenant_id, channel, address_hash)`

Suppression stores a **hash** of the address, not the address, so the table can block an address without becoming another copy of personal data (`Ids.addressHash`).

All data is synthetic: 60 recipients named "Recipient 001" with `recipient001@example.test` addresses and `+1555010xxxx` numbers, split across two tenants (`tenant-retail`, `tenant-wealth`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

11.3 The policy chain

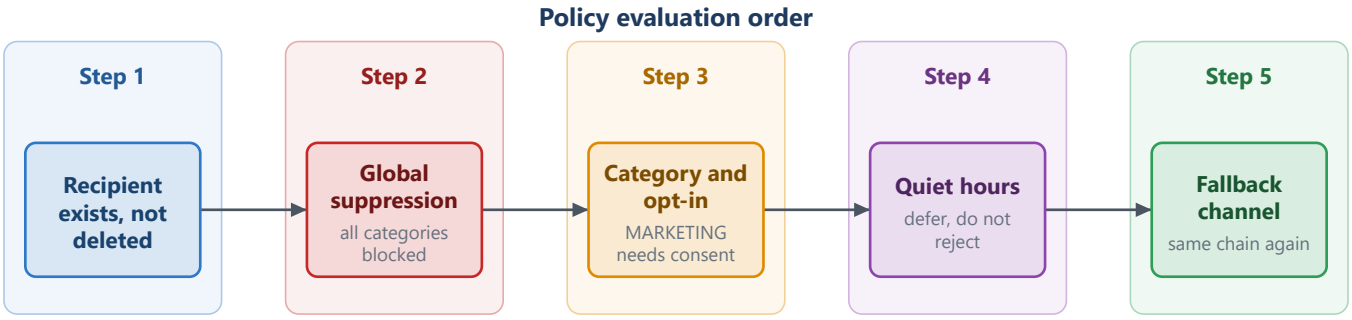


Figure 26. Ordered policy checks run at accept time and again at dispatch time; a fallback channel is evaluated through the same chain, so fallback never bypasses consent. Implementation: backend/src/main/java/com/example/messaging/service/PolicyService.java, backend/src/main/java/com/example/messaging/service/PolicyDecision.java, backend/src/main/java/com/example/messaging/service/PolicyReason.java

`PolicyService.decide` is a **pure static function** (recipient, preferences, suppressed flag, channel, category, priority, now) returning a `PolicyDecision` that is rejected, deferred (with an `until` instant) or allowed. Being pure, it is exhaustively testable by table (`PolicyServiceTest`). The order matters:

Step	Check	Result
1	Recipient missing or `deleted`	`REJECTED(RECIPIENT_DELETED)`
1b	No address for the channel	`REJECTED(NO_ADDRESS)`
2	Address on the suppression list for the channel	`REJECTED(SUPPRESSED)` for all categories
3	MARKETING requires an existing preference row with `marketingOptIn=true` **and** the channel enabled	`REJECTED(OPTED_OUT)`. SERVICE with the channel disabled is `REJECTED(CHANNEL_DISABLED)`. TRANSACTIONAL and SECURITY_ALERT ignore both flags; only suppression blocks them
4	Quiet hours in the recipient's time zone (default 21:00 to 08:00; windows may cross midnight; start equal to end disables them)	MARKETING and LOW or NORMAL SERVICE are **deferred** to the end of the window. An invalid time zone fails closed for MARKETING (`INVALID_PREFERENCES`) and open for the rest
5	Fallback channels	Each goes through steps 1 to 4 independently

Category semantics

Category	Consent needed	Quiet hours	Typical use
`SECURITY_ALERT`	No (still blocked by suppression)	Always sends	Login codes, fraud alerts
`TRANSACTIONAL`	No	Sends unless channel is SMS or PUSH and priority is LOW	Trade confirmations
`SERVICE`	Channel `enabled` flag	LOW and NORMAL deferred	Account notices
`MARKETING`	Explicit opt-in	Deferred	Newsletters

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

> **Defence in depth**
>
> Policy runs **twice**: at accept time (policy evaluated for the instant the message would actually be sent, so a message scheduled for tomorrow morning is not judged by tonight's quiet hours) and again at dispatch time, because preferences can change while a message waits in the queue. A message the recipient opted out of while it was queued is rejected at dispatch (`REJECTED`), never sent.

11.4 Managing preferences

Endpoint	Roles	Behaviour
`GET /api/v1/recipients`	OPERATOR, TENANT_ADMIN, PLATFORM_ADMIN, SENDER	Search by `q`, paged
`GET /api/v1/recipients/{id}/preferences`	same	Returns the preference with its `version`, plus `suppressedChannels`
`PUT /api/v1/recipients/{id}/preferences`	TENANT_ADMIN, OPERATOR, PLATFORM_ADMIN	Body includes `version`; a stale value is 409 `CONFLICT_OPTIMISTIC_LOCK`
`POST /api/v1/recipients/{id}/suppressions`	TENANT_ADMIN, OPERATOR	Adds a suppression `{channel, reason}`
`DELETE /api/v1/recipients/{id}/suppressions/{channel}`	TENANT_ADMIN, OPERATOR	Removes it

Quiet hours must be sent together (`quietStart` and `quietEnd`, `HH:mm`) with a valid `timeZone`; `fallbackOrder` has at most three channels. The console's `/preferences` page turns the 409 into an explicit "someone else changed this" conflict experience instead of silently overwriting.

Automatic suppression and opt-out

Event	Effect
Email `hard_bounce` receipt	Suppression of that address on EMAIL
Push `token_invalid` receipt	Suppression of that token on PUSH
Email `unsubscribe` receipt	`marketingOptIn` cleared
Simulator address `...+hardbounce@...`	Hard bounce at send time, suppression added

11.5 Tests that prove it

Test	Proves
`PolicyServiceTest`	The full decision table: deleted, suppressed, opt-out, quiet hours, fallback
`PreferenceApiIT`	Optimistic locking, validation, role checks
`TenantIsolationIT`	Preferences of another tenant are 404

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Test	Proves
`ProviderOutcomeIT`	Hard bounce adds suppression; fallback respects consent
`RetentionJobTest`	Preference and suppression retention and legal-hold handling

Reference material

ADR-0012: Consent and suppression policy

Source document: `docs/adr/0012-consent-and-suppression-policy.md`

Status: Accepted (2026-10-18)

Context

Sending marketing without consent, or to an address that hard-bounced or complained, creates legal exposure and damages sender reputation. Yet security alerts and transaction confirmations must reach the customer. The rules must be ordered, testable and impossible to bypass through fallback channels, bulk jobs or replay. (This is an engineering model for a synthetic system, not legal advice; real programs need compliance review, see `docs/security/prod-vs-reference.md`.)

Decision

`service.PolicyService` evaluates, in this order (CONTRACT 3.3; this ADR is the policy reference, there is no separate `policy.md`; implemented in `PolicyService`, reasons in `PolicyReason`):

1. Recipient exists in tenant and is not `deleted`.
2. **Suppression** (`suppression` table: hard bounce, complaint, regulatory) blocks **all categories** on that channel for that tenant: `REJECTED(SUPPRESSED)`. The scope is **per tenant**: the schema key is `UNIQUE(tenant_id, channel, address_hash)` and `SuppressionRepository` lookups include the tenant, so there is no cross-tenant (global) list in this implementation.
3. **Category rules**: MARKETING needs explicit channel opt-in and `marketingOptIn=true` else `REJECTED(OPTED_OUT)`. TRANSACTIONAL, SECURITY_ALERT, SERVICE ignore marketing opt-out but never step 2. Per-channel `enabled` applies only to SERVICE.
4. **Quiet hours** in the recipient's time zone (default 21:00-08:00): MARKETING and LOW/NORMAL SERVICE are deferred (still VALIDATED, `scheduled_at` moved to window end). SECURITY_ALERT always sends. TRANSACTIONAL sends unless channel is SMS/PUSH and priority is LOW.
5. **Fallback** channels are evaluated through steps 1-4 independently; fallback never bypasses consent.
6. Idempotency (ADR-0004).

Evaluation points: at submission **and again at dispatch time** (consumer), because consent can change between accept and send. A rejection at submission (opt-out, suppression, deleted recipient, no address, expiry before a deferred send) is persisted as a `REJECTED` message (ACCEPTED to REJECTED, audited) and answered **202** with `status=REJECTED` and `statusReason`, not a 4xx; this keeps idempotent replay and the audit trail consistent (`docs/design/DEVIATIONS.md`, stage 1). `POLICY_SUPPRESSED` stays in the error-code enum but is not returned by the

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

implemented API. A rejection decided at dispatch time moves the claimed message (PROCESSING) to `REJECTED` with `status_reason`. Hard bounce and complaint callbacks write `suppression` rows automatically. Suppression stores an `address_hash`, not the address, and is keyed `UNIQUE(tenant_id, channel, address_hash)`. Unsubscribe callbacks (`UNSUBSCRIBED`) update preferences with an audit event. Preference writes use optimistic locking (`version`, 409 `CONFLICT_OPTIMISTIC_LOCK`).

Alternatives considered

Option	Why not
Check consent only at submission	Stale consent at send time
A global (cross-tenant) suppression list	Shared reputation risk is real, but a global list leaks one tenant's recipient data signals to another and needs a platform-level owner. This implementation is tenant scoped; a platform-wide list would be a future ADR
Treat fallback as a new request	Would let callers bypass opt-out by listing channels
Block all categories on opt-out	Customers would miss security alerts

Consequences

- Positive: auditable ordered rules; one place to change policy; table-driven tests.
- Negative: dispatch-time evaluation adds reads (cached per-recipient preference snapshot with short TTL is an optimization, not a correctness dependency).
- Risk: category misuse (a marketing message flagged TRANSACTIONAL). Mitigation: category is bound to the template (`template.category`) and validated against the request, template approval reviews category, audit sampling.

Validation

- `PolicyServiceTest` table-driven over (category x channel x consent x suppression x quiet hours x priority); fallback-consent test; replay re-evaluates policy.
- Metrics: transitions to REJECTED by reason (`messaging.transitions{to="REJECTED"}`); see source for reason tags.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

12. Delivery Callbacks and Status Reconciliation

12.1 Why callbacks are hard

When a provider says "accepted", the story is not over. The real outcome (delivered, bounced, deferred, opened) arrives later as a **webhook** that the provider sends to a public URL. Webhooks are the platform's most exposed endpoint and the least reliable input: they can be forged, replayed, duplicated, delayed, reordered, or malformed. Meanwhile some sends end with no answer at all. The platform answers with two cooperating mechanisms: a hardened callback pipeline, and reconciliation for outcomes that never produce a callback.

12.2 The callback pipeline as implemented

`POST /api/v1/callbacks/{providerId}` is `permitAll` at the Spring Security layer and authenticated by HMAC inside `CallbackService`. The network is never trusted. The controller reads the body as **raw bytes** so the signature is computed over exactly what the provider sent.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

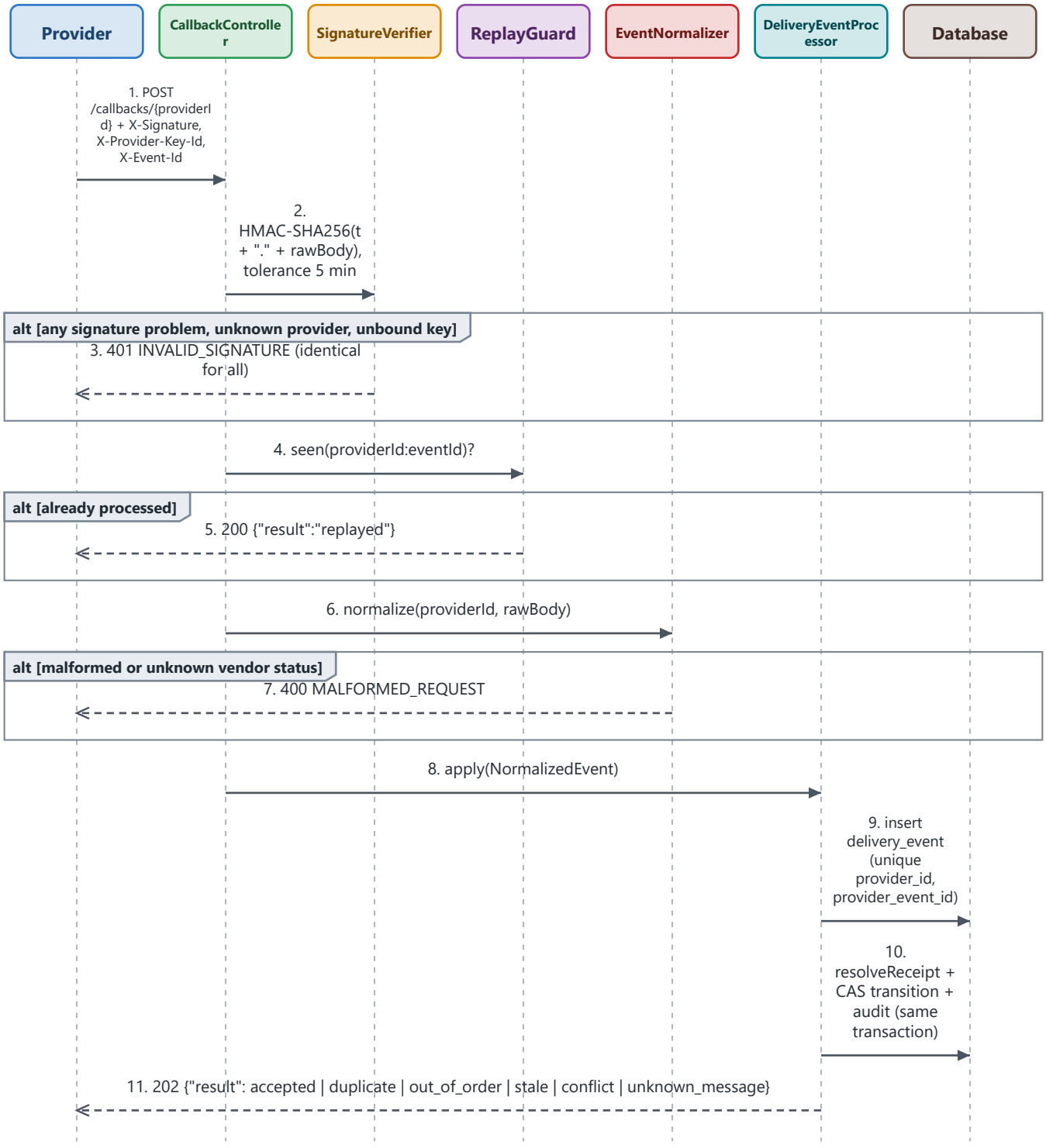


Figure 27. Delivery callback as implemented (see DEVIATIONS 7): signature first, then replay guard, normalisation, then a compare-and-set transition ranked by the monotonic rule. Implementation: backend/src/main/java/com/example/messaging/callback/CallbackController.java, backend/src/main/java/com/example/messaging/callback/CallbackService.java, backend/src/main/java/com/example/messaging/callback/SignatureVerifier.java, backend/src/main/java/com/example/messaging/callback/DeliveryEventProcessor.java

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Authentication

Element	Rule		
Headers	`X-Signature: t=<epoch>,v1=<hex>`, `X-Provider-Key-Id`, `X-Event-Id`		
Signed content	HMAC-SHA256 over `t + "." + rawBody`; raw bytes, never re-serialised JSON		
Comparison	Constant time (`MessageDigest.isEqual`, bitwise OR accumulation without early exit)		
Freshness	`	now - t	`at most messaging.callback.tolerance` (5 minutes)
Key choice	`X-Provider-Key-Id` must equal the provider's `provider_config.hmac_key_ref` (or `<ref>-next` during rotation); the secret comes from `messaging.callback.secrets.<keyId>`		
Rotation	Several `v1=` values are accepted, so two secrets can be active during a rotation		
Uniform failure	A bad signature, a stale timestamp, an unknown provider and an unbound key all return the same `401 INVALID_SIGNATURE`, so providers cannot be enumerated		

JAVA

```
// com.example.messaging.callback.SignatureVerifier (excerpt)
// bitwise OR keeps the loop free of early exits; isEqual is constant-time for equal lengths
match |= MessageDigest.isEqual(expected, c.getBytes(StandardCharsets.US_ASCII));
...
Mac mac = Mac.getInstance("HmacSHA256");
mac.init(new SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
```

Replay and duplicates (implemented behaviour, DEVIATIONS item 7)

Situation	HTTP	`result`	State change
Same `X-Event-Id` (key `providerId:eventId`) already processed within the 24 h replay window	**200**	`replayed`	none
New transport id but same `(provider_id, provider_event_id)` already stored	202	`duplicate`	none, the first row stays
Event applied or informational	202	`accepted`	status moved or event recorded
Rank-1 event after the outcome is known, or terminal receipt not applicable in the current state	202	`out_of_order`	recorded `applied=false`
Same terminal receipt again under a new event id	202	`stale`	recorded `applied=false`, `STALE`
Opposite terminal receipt after the first	202	`conflict`	first terminal kept, audit `DELIVERY_EVENT_CONFLICT`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Situation	HTTP	`result`	State change
Event for an unknown message	202	`unknown_message`	neither stored nor remembered (`delivery_event.message_id` is a mandatory foreign key)
Malformed payload or unknown vendor status	400	`malformed`	none
Signature problem	401	`invalid_signature`	none

Why 200 for a replay rather than 409 `REPLAYED_EVENT`? The provider must **stop retrying** an event we already processed; the error code `REPLAYED_EVENT` exists in the enum but is not used. The replay guard records an event **only after successful processing**, backed by the `replay_guard` table with an in-memory cache.

A response example for a replay:

JSON

```
{"result":"replayed","eventId":"evt-1001"}
```

Normalisation

`EventNormalizer` finds the `ProviderPayloadMapper` bean for the `{providerId}` path segment and produces a canonical `NormalizedEvent` (`providerId`, `providerEventId`, `type`, `providerMessageId`, `idempotencyRef`, `occurredAt`, `receivedAt`, `reasonCode`, `rawRef`). Normalisation is pure and side-effect free. The raw payload is archived best-effort to Mongo, redacted. Free-text vendor reasons are never copied into audit or log lines: only code-like reasons (`[A-Z][A-Z0-9_]{0,39}`) are kept.

Applying the event

`DeliveryEventProcessor` inserts the `delivery_event` row and applies `MessageStateMachine.resolveReceipt(currentStatus, type)` as a **compare-and-set** through `NotificationService.transition(..., viaReceipt=true)`, retried up to three times if the message moved concurrently, in the same transaction as the insert.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

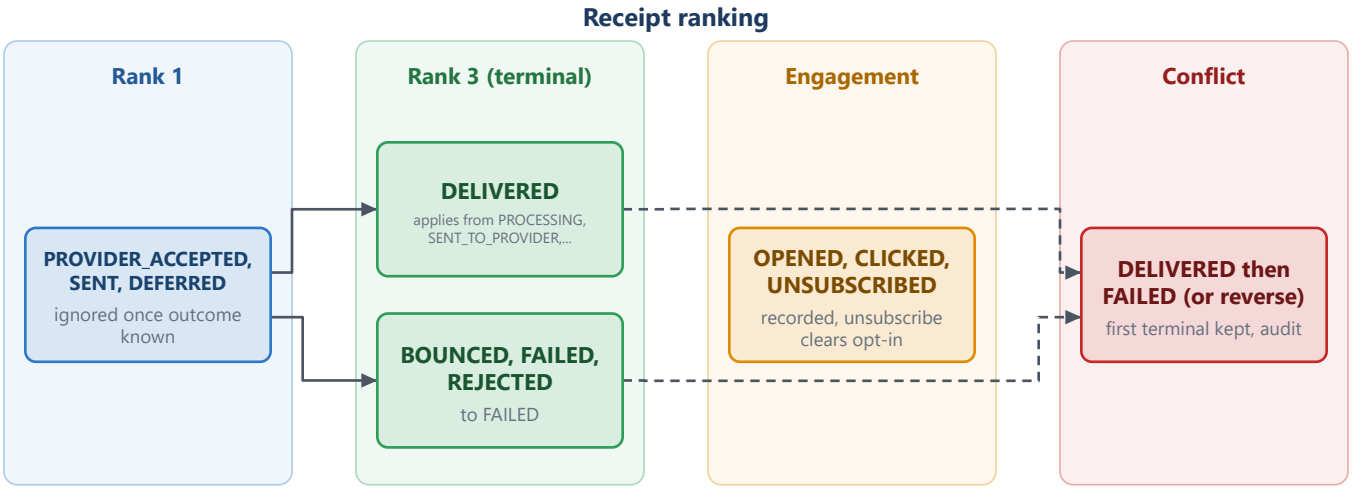


Figure 28. Monotonic rank rule: a lower rank never overwrites a higher rank; the first terminal receipt wins and a contradicting terminal receipt is audited as DELIVERY_EVENT_CONFLICT. Implementation: backend/src/main/java/com/example/messaging/domain/MessageStateMachine.java (resolveReceipt), backend/src/main/java/com/example/messaging/domain/DeliveryEventType.java

12.3 Ambiguous outcomes and reconciliation

Some sends never produce a clear answer: the provider call times out, the response is lost, the worker crashes between the provider's acceptance and the platform's database write, or the receipt callback never comes. The platform's rule (ADR-0010): **unknown is a state**, and it is resolved from provider evidence only.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

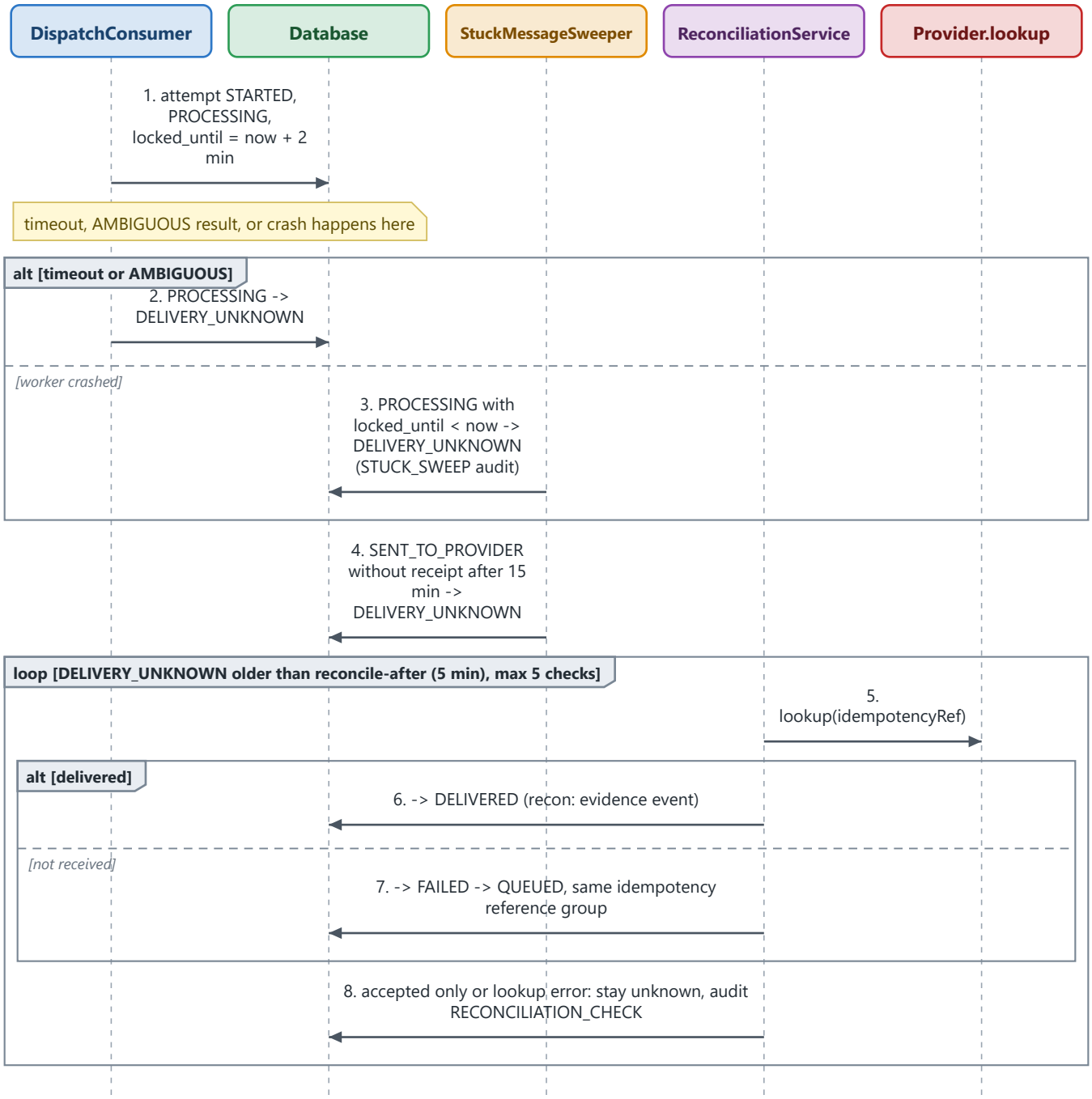


Figure 29. Ambiguous outcome reconciliation: unknown is a state. The sweeper and the reconciliation service resolve it from provider evidence only, never by guessing. Implementation: backend/src/main/java/com/example/messaging/scheduler/StuckMessageSweeper.java, backend/src/main/java/com/example/messaging/service/ReconciliationService.java, backend/src/main/java/com/example/messaging/provider/ChannelProvider.java (lookup)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Three detectors

Detector	Condition	Result	Class
Consumer	Provider result `TIMEOUT` or `AMBIGUOUS` (EMAIL, SMS)	`PROCESSING -> DELIVERY_UNKNOWN`	`DispatchConsumer`
Sweeper, crash	`PROCESSING` with `locked_until` older than now (lock-ttl 2 min)	`-> DELIVERY_UNKNOWN`, audit `STUCK_SWEEP`	`StuckMessageSweeper`
Sweeper, silence	`SENT_TO_PROVIDER` with no receipt after `receipt-timeout` (15 min)	`-> DELIVERY_UNKNOWN`	`StuckMessageSweeper`

Reconciliation

`ReconciliationService` runs every 15 seconds (`reconcile-interval-ms`), selecting `DELIVERY_UNKNOWN` messages older than `reconcile-after` (5 minutes), in batches of 50, and calls `ChannelProvider.lookup(idempotencyRef)`.

Lookup says	Action
Delivered	`DELIVERY_UNKNOWN -> DELIVERED`, with an evidence event `recon:<messageId>:<n>`
Failed or bounced	`-> FAILED`
Accepted, not yet delivered	Stay unknown; re-check later
Not received	`-> FAILED -> QUEUED` as a new attempt with the **same idempotency reference group**, so a compliant provider still deduplicates
Lookup error or provider without lookup	Stay unknown; increment the check count
After `reconcile-max-checks` (5)	Message remains in `GET /failed-messages` for an operator decision

Spacing and the maximum are derived from `RECONCILIATION_CHECK` audit events: no extra column is needed (DEVIATIONS item 9).

Invariants

1. A message in `DELIVERY_UNKNOWN` is never silently retried.
2. Reconciliation never creates `DELIVERED` without provider-originated evidence (a callback or a lookup result).
3. Late callbacks win over reconciliation (same rank rule; first terminal wins).
4. Every reconciliation action is audited with the lookup result.

Decision matrix by channel

Channel	After TIMEOUT or AMBIGUOUS	Why
EMAIL	Wait for callback, then `lookup`; retry only if "not received"	Duplicate emails are visible and reputational
SMS	Same	Duplicate cost and annoyance
PUSH	Retry allowed	The collapse key makes it idempotent on the device

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

12.4 Tests that prove it

Test	Proves
`SignatureVerifierTest`	Valid, tampered body, wrong key, stale timestamp, multiple `v1=` values
`ReplayGuardTest`	Replay window and keying
`EventNormalizerTest`	Mapping tables, unknown status, missing ids
`CallbackApiIT`	HTTP results (200 replayed, 202 variants, 400, 401) end to end
`ReconciliationIT`	Lookup outcomes, spacing, max checks, safe retry
`StuckMessageSweeperTest`	Lock expiry, receipt timeout
`FailureRecoveryIT#scenario03_*`, `scenario04_*`, `scenario05_*`	Duplicate, out-of-order and ambiguous cases, crash after provider accept
`tests/failure-recovery/callback-replay.mjs`, `provider-timeout.mjs`	Scripted replays against a started `local` application

Reference material

Callback normalisation (current, implemented)

Source document: `docs/architecture/callbacks-normalization.md`

Providers send heterogeneous webhooks. `callback.EventNormalizer` dispatches to a per-provider mapper (by `{providerId}` path segment) and always produces the same internal model, so `DeliveryEventProcessor` is provider-agnostic.

Canonical model `DeliveryEvent`

Field	Type	Source/rule
`providerId`	string	path variable, must match a `provider_config` row
`providerEventId`	string	provider's event id; falls back to `X-Event-Id` header; must be non-blank (else 400 `MALFORMED_REQUEST`, recorded for diagnostics)
`type`	`DeliveryEventType`	mapped from the vendor status (tables below); an unknown vendor status is not guessed: the payload is rejected as malformed (400)
`providerMessageId`	string	vendor message id returned at send time
`idempotencyRef`	string, optional	echoed `messageId:attemptGroup` when vendor supports metadata echo
`occurredAt`	instant	vendor timestamp converted to UTC; missing = `receivedAt`
`receivedAt`	instant	server `Clock`
`reasonCode`	string, optional	vendor error code, stored for ops, never shown raw to end customers

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Field	Type	Source/rule
`rawRef`	string	key of the archived raw payload in Mongo `provider_diagnostics` (best-effort)

Message lookup order: see "Processing after normalization" below (an unknown message is acknowledged as `unknown_message`, it is not stored).

Processing after normalization (`DeliveryEventProcessor`)

Message lookup order: see "Processing after normalization" below (an unknown message is acknowledged as `unknown_message`, it is not stored).

Result tag (`messaging.callbacks{result}`)	HTTP	Meaning	`delivery_event` row
`accepted`	202	applied (status moved) or informational/engagement event	`applied=true`
`duplicate`	202	`(provider_id, provider_event_id)` already stored (e.g. same payload event id under a different transport id, or after the replay window)	none (the first row stays)
`replayed`	200	the same `X-Event-Id` was already processed within `messaging.pipeline.replay-window` (24 h); no state change	none
`out_of_order`	202	a rank-1 event (SENT, ACCEPTED) after the outcome is known, or a terminal receipt that cannot apply in the current state	`applied=false`, `ignored_reason=OUT_OF_ORDER` / `STATE_NOT_RECEIVABLE`
`stale`	202	the same terminal receipt again under a new event id	`applied=false`, `ignored_reason=STALE`
`conflict`	202	opposite terminal receipt after the first one (DELIVERED then BOUNCED/FAILED, or the reverse): first terminal wins, audit `DELIVERY_EVENT_CONFLICT`	`applied=false`, `ignored_reason=CONFLICT`
`unknown_message`	202	see above	none
`malformed`, `invalid_signature`	400, 401	see `callback-flow.md`	none

The decision is `MessageStateMachine.resolveReceipt(currentStatus, type)` (monotonic rank rule, CONTRACT 3.4); the transition is a compare-and-set through `NotificationService.transition(..., viaReceipt=true)`, retried up to three times if the message moved concurrently, in the same transaction as the `delivery_event` insert. Free-text vendor reasons are never copied into the audit trail: only code-like reasons (`[A-Z][A-Z0-9_]{0,39}`) are kept.

Replay choice: a replayed `X-Event-Id` is answered **200** `{"result":"replayed"}` (not 409 `REPLAYED_EVENT`) because the provider must stop retrying an event we have already processed. The guard key is `providerId:eventId` (so two vendors may reuse an id), recorded only after successful processing, backed by the `replay_guard` table with an in-memory cache.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Authentication specifics: the key id must be the provider's `provider_config.hmac_key_ref` (or `<ref>-next` during rotation), the secret comes from `messaging.callback.secrets.<keyId>`, `t` must be within `messaging.callback.tolerance` (5 min), several `v1=` values are accepted so two secrets can be active. A signature failure, an unknown provider and an unbound key all give the same `401 INVALID_SIGNATURE` so providers cannot be enumerated.

Adding a mapper

Implement `ProviderPayloadMapper` and register it as a Spring bean (that is all: `EventNormalizer` collects the beans by provider id), add the vendor-status table rows to this file, add `messaging.callback.secrets.<keyId>` config (dummy value locally) matching `provider_config.hmac_key_ref`, add mapper unit tests with duplicate, out-of-order and unknown-status cases (see `EventNormalizerTest`). `NewProviderWithoutOrchestrationChangeTest` proves this with a vendor that exists only in the test. See `specs/0002-new-provider-adapter.md`.

Processing rules after normalization

See `callback-flow.md`: dedupe on `(provider_id, provider_event_id)`, rank rule, conflict audit. Normalization must be pure and side-effect free (testable without DB). Payload limits: body <= 256 KB (`RequestSizeFilter`); mapper must not log raw payload at INFO (may contain addresses).

Reconciliation flow (design document)

Source document: `docs/architecture/reconciliation-flow.md`

Decision matrix

Channel	After TIMEOUT/AMBIGUOUS	Why
EMAIL	wait for callback, then `lookup`, retry only if "not received"	duplicate emails are visible and reputational
SMS	same as EMAIL	duplicate cost and annoyance
PUSH	retry allowed (collapse key makes it idempotent)	duplicates collapse on the device

Invariants

1. A message in `DELIVERY_UNKNOWN` is never silently retried.
2. Reconciliation never creates DELIVERED without provider-originated evidence (callback or `lookup` result).
3. Late callbacks win over reconciliation (same rank rule); first terminal wins.
4. Every reconciliation action is audited with `lookup` result.

Tests:
`FailureRecoveryIT#scenario04_providerTimeout_movesToDeliveryUnknown_reconcilesWithoutDuplicateSend`, `#scenario05_consumerCrashAfterProviderAccept_sweeperMarksDeliveryUnknown`. ADR-0010. Runbook: `docs/operations/runbooks/consumer-crash-ambiguous-outcomes.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ADR-0010: Ambiguous outcomes and reconciliation

Source document: `docs/adr/0010-ambiguous-outcome-and-reconciliation.md`

Status: Accepted (2026-10-18)

Context

After a provider call times out, or the process crashes after the call but before persisting the result, we do not know whether the provider accepted the message. Retrying blindly may deliver a duplicate SMS (cost, customer annoyance, for security alerts possibly confusion); never retrying may drop an important message. Also, `SENT_TO_PROVIDER` messages may never get a delivery receipt.

Decision

Introduce the explicit state `DELIVERY_UNKNOWN` and a reconciliation step; "unknown" is a first-class, visible, alertable state, not an error to hide.

Triggers into DELIVERY_UNKNOWN

Trigger	Mechanism
Provider `TIMEOUT` / `AMBIGUOUS` outcome	`DispatchService` PROCESSING to DELIVERY_UNKNOWN
Consumer crash after provider call	`message_attempt` row written `STARTED` before the call; `StuckMessageSweeper` finds PROCESSING with stale `locked_until` (`messaging.dispatch.lock-ttl`)
No receipt within `receipt-timeout`	Sweeper: SENT_TO_PROVIDER to DELIVERY_UNKNOWN

Resolution

1. **Late callback** wins first: `DeliveryEventProcessor` applies DELIVERED or FAILED to a DELIVERY_UNKNOWN message (monotonic rank rule).
2. After `messaging.dispatch.reconcile-after`, `ReconciliationService` calls `provider.lookup(idempotencyRef)` (`idempotencyRef = messageId:attemptGroup`): | Lookup result | Action | |---|---| | accepted/delivered | record, transition to DELIVERED (or leave SENT_TO_PROVIDER semantics via DELIVERED only on confirmation) | | failed/bounced | FAILED (+ suppression if hard bounce) | | not received | **safe retry**: new attempt (QUEUED path via operator-free internal requeue), same `idempotencyRef` group | | lookup itself fails/unsupported | stay DELIVERY_UNKNOWN, backoff re-check; escalate to operator after `reconcile.max-checks` (see source) |
3. Channel policy `ambiguousOutcomePolicy`: EMAIL and SMS keep DELIVERY_UNKNOWN until reconciliation; PUSH may retry immediately (collapse key makes it idempotent).
4. Never blind-retry. Never mark DELIVERED without a provider-originated confirmation.

Conflicting evidence

If a retry was issued after "not received" and a late DELIVERED callback for the first attempt arrives, the first terminal wins; second-attempt callbacks are recorded `applied=false` and audited `DELIVERY_EVENT_CONFLICT`; the duplicate-send is counted for review (a rare, accepted residual risk given provider lookup inaccuracies).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Alternatives considered

Option	Why not
Always retry on timeout	Duplicate sends
Never retry on timeout	Message loss
Treat timeout as FAILED	Misleading; customers may still receive it
Two-phase send with provider "prepare/commit"	Not offered by real vendors
Assume delivered after N minutes	Invented data

Consequences

- Positive: honest status; bounded duplicate risk; operators have a filtered list (`GET /api/v1/failed-messages`).
- Negative: some messages sit in DELIVERY_UNKNOWN for minutes; depends on provider `lookup` fidelity.
- Metrics/alerts: transitions into DELIVERY_UNKNOWN (`messaging.transitions{to="DELIVERY_UNKNOWN"}`), age of oldest unknown (see slis-slos).

Validation

- `FailureRecoveryIT#scenario04_providerTimeout_movesToDeliveryUnknown_reconcilesWithoutDuplicateSend`, `#scenario05_consumerCrashAfterProviderAccept_sweeperMarksDeliveryUnknown`.
- Flow: `docs/architecture/reconciliation-flow.md`; runbook `docs/operations/runbooks/consumer-crash-ambiguous-outcomes.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13. Retry and Dead-Letter Flows

13.1 Principles

Retries are the most dangerous reliability feature: used carelessly they turn a small provider hiccup into a self-inflicted outage (a retry storm) or into duplicate messages. The platform's retry design follows five principles (ADR-0006):

1. Retry only what is **safe**: transient failures and rate limiting. Uncertain outcomes go to reconciliation (Chapter 12), never to a blind resend.
2. Spread retries in time: **exponential backoff with jitter** and a cap.
3. Bound everything: a maximum number of attempts per message, a poison threshold per envelope.
4. Park, do not discard: exhausted messages go to a **dead-letter queue** with a reason, visible to operators.
5. Recovery is deliberate: an **operator** replays, with an audit record and a fresh attempt budget.

A kid-friendly analogy: if the doorbell is not answered, you do not ring it 50 times in a row. You wait a little, ring again, wait longer, and after the fifth try you leave a note with the neighbour (the dead-letter queue) so a grown-up can decide what to do.

13.2 What is retried

`ProviderOutcome.isRetryable()` returns true only for `TRANSIENT_FAILURE` and `RATE_LIMITED`. `RetryPolicy.isRetryable(outcome)` is the single place the consumer asks.

Outcome	Automatic retry	Reason
`TRANSIENT_FAILURE`	Yes, with backoff	Provider 5xx or network error
`RATE_LIMITED`	Yes, with backoff	Provider 429
`PERMANENT_FAILURE`	No (fallback channel if allowed)	Invalid recipient, rejection
`TIMEOUT`, `AMBIGUOUS`	No for EMAIL and SMS (reconcile); yes for PUSH	The provider may already have it
Rate limiter refusal (our token bucket) or open circuit	Requeue 0.1 to 5 s, **no attempt consumed**	Not the message's fault

13.3 Backoff with jitter

JAVA

```
// com.example.messaging.service.RetryPolicy
public Duration ceiling(int attemptNo) {
    int shift = Math.min(Math.max(attemptNo - 1, 0), 30);
    long raw = baseMs * (1L << shift);
    if (raw < 0 || raw > capMs) raw = capMs;
    return Duration.ofMillis(raw);
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
}
/** Jittered backoff in [ceiling/2, ceiling]. */
public Duration backoff(int attemptNo, RandomGenerator random) {
    long ceil = ceiling(attemptNo).toMillis();
    double factor = 0.5 + 0.5 * random.nextDouble();
    return Duration.ofMillis(Math.max(1, (long) (ceil * factor)));
}
```

The formula is `next_attempt_at = now + min(cap, base x 2^(n-1)) x U(0.5, 1.0)`. With the shipped defaults (`backoff-base-ms` 1000, `backoff-cap-ms` 60000, `max-attempts` 5) the ceilings are 1, 2, 4, 8, 16 seconds for the first five retries, then 32 and 60 for configurations with more attempts.

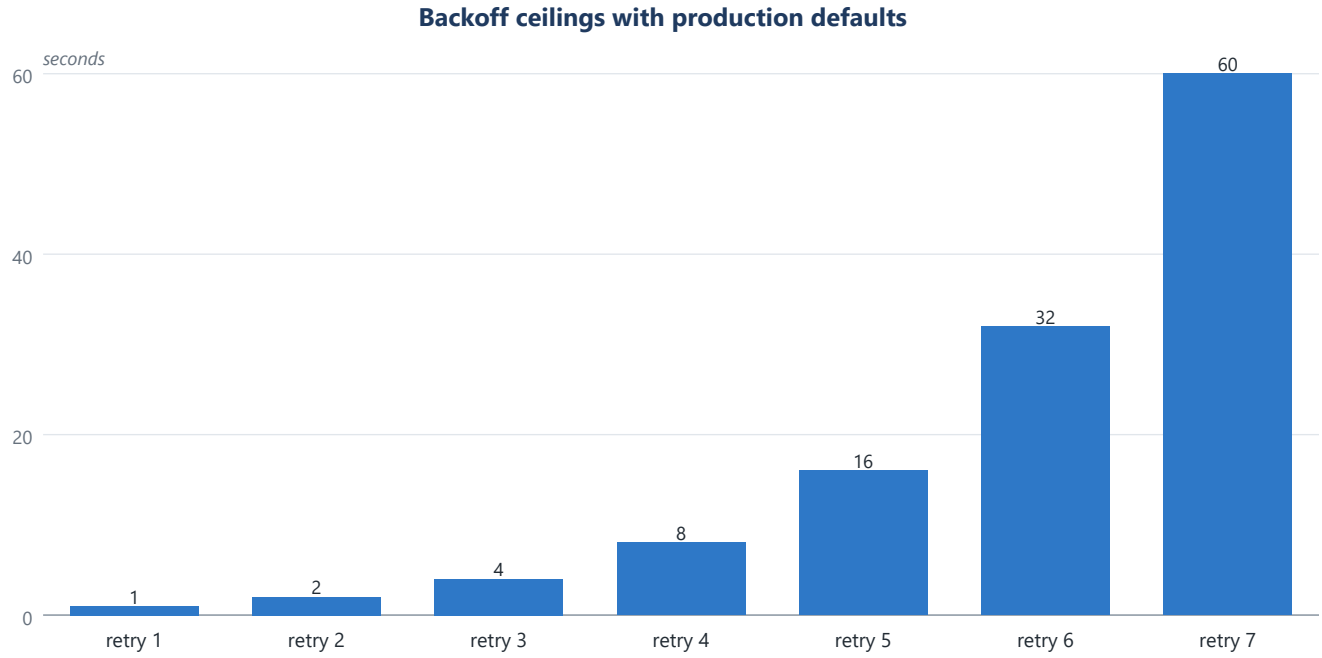


Figure 30. Un-jittered backoff ceiling min(cap, base x 2^(n-1)) for base 1000 ms and cap 60000 ms; the actual wait is the ceiling times a random factor in [0.5, 1.0]. Computed from the formula in RetryPolicy, not measured. Implementation: backend/src/main/java/com/example/messaging/service/RetryPolicy.java, backend/src/main/resources/application.yml

Jitter matters as much as the exponent: without it, thousands of messages that failed together retry together and re-create the overload. `RetryPolicy` takes a `RandomGenerator` so tests are deterministic (`RetryPolicyTest`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13.4 The retry loop

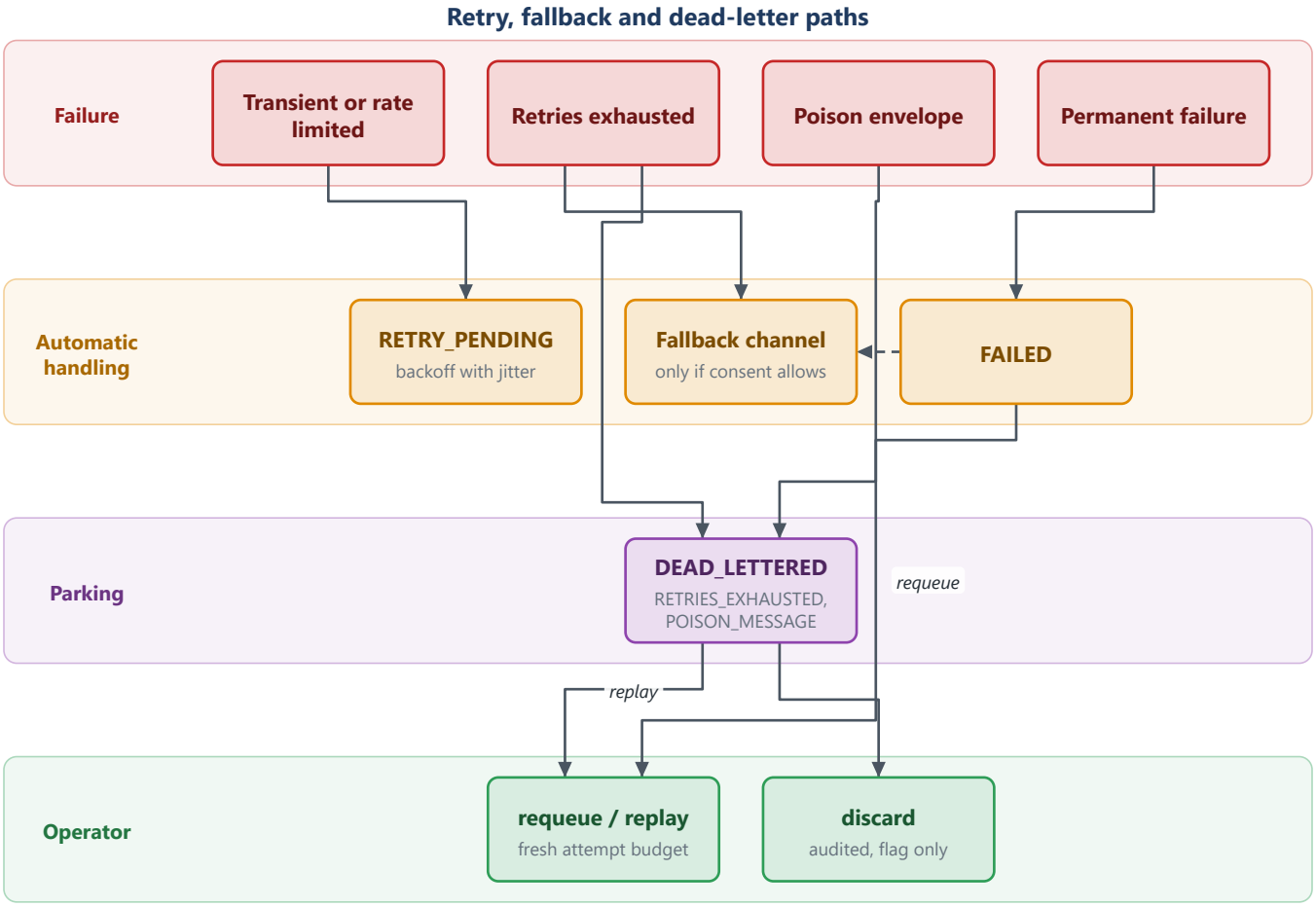


Figure 31. Where a message goes when delivery does not succeed. Replay and requeue are operator-only and audited; a replayed message receives a fresh attempt budget. Implementation: backend/src/main/java/com/example/messaging/service/RetryPolicy.java, backend/src/main/java/com/example/messaging/messaging/DispatchConsumer.java, backend/src/main/java/com/example/messaging/service/ReplayService.java, backend/src/main/java/com/example/messaging/service/DefaultDeadLetterOperations.java

1. The consumer maps a retryable outcome to `RETRY_PENDING`, sets `next_attempt_at`, increments `messaging.retry.scheduled{channel}`, and republishes the envelope with `publishDelayed` and a **new dispatchId** (`messageId:attemptNo`).
2. Delayed redelivery is done by the bus (Rabbit TTL tiers 5 s, 30 s, 2 m, 10 m; in-memory timers). `ScheduledDispatchPoller` (every second) is the safety net: it re-enqueues `RETRY_PENDING` messages whose backoff elapsed by more than `retry-grace` (5 s) but whose delayed envelope never arrived, repairs due `VALIDATED` messages that lack an outbox row, and re-publishes `QUEUED` messages whose envelope was lost (for example after an in-memory bus restart). A duplicate envelope is harmless because the consumer dedupes.
3. Both paths end in the same compare-and-set `RETRY_PENDING -> QUEUED`, so a retry can never run twice.
4. When attempts are exhausted: a **fallback channel** if the request allowed one and policy approves (`PROCESSING -> RETRY_PENDING -> QUEUED` with a fresh budget), otherwise `DEAD_LETTERED(RETRIES_EXHAUSTED)`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13.5 Dead-letter queue

Dead letters have **two representations** that are kept consistent: the bus DLQ (for poison envelopes) and the database status `DEAD_LETTERED` with a `dlq_reason` (the authoritative one, queryable through the API).

Reason (`DlqReason`)	Cause	Counter
`RETRIES_EXHAUSTED`	Attempts used up without success or fallback	`messaging.deadletter.total{reason}`
`POISON_MESSAGE`	Envelope references a missing message, cannot be deserialised, or keeps throwing on `poison-threshold` (5) deliveries	same
`PERMANENT_FAILURE`, `OPERATOR`	Defined in the `DlqReason` enum for permanent-failure parking and operator action; the automatic pipeline itself produces the two reasons above	same

`DispatchHandler.onPoison(envelope, deliveryCount, cause)` lets the bus tell the consumer when it dead-lettered an envelope on its own, so the consumer mirrors that into the database with legal transitions (`QUEUED -> PROCESSING -> FAILED -> DEAD_LETTERED`).

Operator tools

Endpoint	Roles	Effect
`GET /api/v1/dead-letters`	OPERATOR, TENANT_ADMIN, PLATFORM_ADMIN	Paged list with reason
`POST /api/v1/dead-letters/{messageId}/replay`	OPERATOR, PLATFORM_ADMIN	Policy pre-check (audited), CAS `DEAD_LETTERED -> QUEUED`, new outbox event
`POST /api/v1/dead-letters/{messageId}/discard`	OPERATOR, PLATFORM_ADMIN	Audited; stays `DEAD_LETTERED` with a `discarded` flag
`POST /api/v1/messages/{id}/requeue`	OPERATOR, PLATFORM_ADMIN	`FAILED` or `DEAD_LETTERED` to `QUEUED`
`GET /api/v1/failed-messages`	OPERATOR, TENANT_ADMIN, PLATFORM_ADMIN	`FAILED`, `DELIVERY_UNKNOWN`, `RETRY_PENDING` investigation list

Replay semantics worth remembering (DEVIATIONS stage 2): `NotificationService.requeueInternal` sets `max_attempts = attempt_count + messaging.dispatch.max-attempts`, so a replayed message gets a **fresh budget** instead of failing on its first retry; the outbox publisher must publish regardless of whether the message is already `QUEUED`. The console warns before replaying a message in `DELIVERY_UNKNOWN`, because it may already have been delivered.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13.6 Retry storms

A retry storm is retries amplifying an incident. The defences stack:

Defence	Effect
Full jitter and cap	Retries spread in time, never faster than the cap allows
Max attempts	Bounded amplification per message (at most 5 by default)
Shared token bucket per provider	Retries and first attempts share one rate limit
Circuit breaker	During an outage, calls stop entirely and requeues consume no attempt
Alert `RetryStorm`	Retries above half the accept rate pages the on-call (runbook `retry-storm.md`)
`retry-storm.js` k6 script	Designed to prove boundedness (NOT executed here)

13.7 Tests that prove it

Test	Proves
`RetryPolicyTest`	Ceiling, jitter bounds, overflow safety, retry classification
`FailureRecoveryIT#scenario01_*`	Provider outage, backoff windows, DLQ after N attempts, replay reaches `SENT_TO_PROVIDER` then `DELIVERED`
`FailureRecoveryIT#scenario08_*`	Poison messages dead-lettered with no infinite loop
`ReplayServiceIT`	Replay and requeue semantics, fresh budget, audit
`InMemoryMessageBusTest`	Delayed redelivery and DLQ behaviour
`tests/failure-recovery/provider-outage.mjs`	Scripted outage and recovery against a started `local` application

Reference material

ADR-0006: Retry and dead-letter policy

Source document: `docs/adr/0006-retry-and-dead-letter-policy.md`

Status: Accepted (2026-10-18)

Context

Provider failures are common and bursty. Naive immediate retry amplifies outages (retry storm), synchronized retries create thundering herds, and endless retry of an unprocessable message blocks capacity.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Decision

Classification drives behavior (`RetryPolicy`)

`ProviderOutcome`	Action
ACCEPTED	`SENT_TO_PROVIDER`
TRANSIENT_FAILURE, RATE_LIMITED	`RETRY_PENDING` with backoff; consumes an attempt, except provider-limiter exhaustion (our own token bucket empty), which requeues with short delay and consumes **no** attempt
PERMANENT_FAILURE	`FAILED`; fallback channel if allowed; hard bounce adds `suppression`
TIMEOUT, AMBIGUOUS	`DELIVERY_UNKNOWN`, no blind retry for EMAIL/SMS (ADR-0010)

Backoff: exponential with full jitter

`delay = min(cap, base * 2^(n-1))`, then `next_attempt_at = now + delay * U(0.5, 1.0)` (the contract's "full-jitter variant" keeps at least half the exponential delay so retries do not collapse to ~0). Config: `messaging.dispatch.backoff-base-ms`, `backoff-cap-ms`, `max-attempts`. Illustrative defaults: base 2 s, cap 5 min, 5 attempts. A provider `Retry-After` hint, if present, raises the delay to at least that value (never lowers).

Worked illustration (base 2 s, cap 300 s): n=1 → 2 s window (1-2 s); n=2 → 4 s (2-4 s); n=3 → 8 s (4-8 s); n=4 → 16 s (8-16 s); n=5 → 32 s (16-32 s).

Caps and bounds

- Max attempts per message and per channel group; total retry horizon also bounded by `expires_at` (expired => not sent, status `FAILED` with reason `EXPIRED`, see source).
- Retry never bypasses consent: before each attempt `PolicyService` is re-evaluated (cancel, expiry, suppression, quiet hours).
- Circuit breaker open => requeue with delay without consuming an attempt; breaker state exposed via `messaging.provider.circuit.state{provider}`.
- Global retry-storm guard: retries share the provider token bucket with first attempts; first attempts of higher priority are favoured by queue priority.

Dead letter

- `DlqReason{RETRIES_EXHAUSTED, POISON_MESSAGE, PERMANENT_FAILURE, OPERATOR}`; message status `DEAD_LETTERED`; broker DLQ holds the envelope; DB row holds the reason. Counter `messaging.deadletter.total{reason}`, gauge `messaging.queue.dlq.depth`.
- **Poison**: missing message, deserialization failure, or `poison-threshold` consecutive unexpected exceptions for the same `dispatchId` => DLQ with `POISON_MESSAGE` immediately; never infinite redelivery.
- **Replay**: only operators (`POST /api/v1/dead-letters/{messageId}/replay`, `POST /api/v1/messages/{id}/requeue`) via `ReplayService`; creates a new `dispatchId`/attempt group, re-runs policy, audits actor and reason. `discard` keeps status but flags the record. Replay of a message whose final provider outcome was ambiguous requires the reconciliation check first (see ADR-0010).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Alternatives considered

Option	Why not
Fixed-interval retry	Synchronizes load; no relief for providers
Unbounded retries	Poison loop, capacity loss
Broker-only retry (requeue on nack)	No attempt accounting, no per-class policy, immediate redelivery storms
Retry inside the provider adapter	Hidden attempts; breaks accounting and ADR-0002
Equal jitter vs decorrelated jitter	Decorrelated is fine; chosen variant is easier to test deterministically with an injected `Clock` and `Random`

Consequences

- Positive: bounded work, observable, tunable, operator control.
- Negative: worst-case latency for retried messages is minutes; critical alerts may use fallback channel sooner (policy step 5).
- Risk: a long provider outage fills `RETRY_PENDING` and then the DLQ; see runbooks `provider-outage.md`, `retry-storm.md`, `dlq-growth.md`.

Validation

- `RetryPolicyTest` (deterministic `Random`) asserts bounds per attempt; `FailureRecoveryIT#scenario01_providerOutage_retriesWithBackoff_thenDeadLetters_thenReplay`, `#scenario08_poisonMessage_goesToDlq_noInfiniteLoop`.

Runbook: DLQ growth

Source document: `docs/operations/runbooks/dlq-growth.md`

Symptoms and alerts

`DlqGrowth`, `PoisonMessages`; `messaging.queue.dlq.depth` rising; `messaging.deadletter.total{reason}`.

Impact

Messages not delivered and waiting for human decision. For `RETRIES_EXHAUSTED`, customers missed notifications.

Diagnose by reason

Reason	Typical cause	Check
`RETRIES_EXHAUSTED`	provider outage longer than retry horizon	provider history, `provider-outage.md`
`POISON_MESSAGE`	bug, bad deployment, corrupt envelope, message row missing	recent deploy; logs by `messageId`; `GET /api/v1/diagnostics/messages/{id}`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Reason	Typical cause	Check
`PERMANENT_FAILURE`	invalid addresses, suppressed or auth errors	error code distribution; `GET /api/v1/dead-letters?...`
`OPERATOR`	manual action	audit events

1. `GET $BASE/api/v1/dead-letters?page=0&size=100` then group by reason and provider.
2. `GET $BASE/api/v1/messages/{id}/history` for representative samples.
3. Correlate start time with deploys/config changes.

Mitigate

- Stop the source: roll back a bad release (`rollback-plan.md`), re-enable provider, fix config.
- For poison caused by a bug: do NOT replay until fixed.

Recover (replay policy: `data-retention-and-replay-policy.md`)

1. Decide per reason: replay, discard, or leave.
2. Replay in small batches (e.g., 50) with rate awareness: `POST /api/v1/dead-letters/{messageId}/replay` ; verify first batch outcome before continuing.
3. Messages past `expires_at` or whose consent changed will be rejected at replay (policy re-evaluation): expected.
4. Discard (`POST .../discard`) with a reason recorded for unfixable cases (stays `DEAD_LETTERED` with discarded flag).
5. For `DELIVERY_UNKNOWN`-origin messages ensure reconciliation was attempted before replay to avoid duplicates.

Verify

DLQ depth back to baseline; replayed messages progress to SENT_TO_PROVIDER/DELIVERED; audit shows actor and reason.

Prevent

Alert on reason mix, canary releases, schema-tolerant envelope, contract tests between publisher and consumer.

Proof: `FailureRecoveryIT#scenario08_poisonMessage_goesToDlq_noInfiniteLoop`, `#scenario01_...`.

Runbook: Retry storm

Source document: `docs/operations/runbooks/retry-storm.md`

Symptoms and alerts

`RetryStorm`; `rate(messaging_retry_scheduled_total)` high and sustained; provider calls per second above normal while success is low; provider `RATE_LIMITED` outcomes; CPU/DB load up; same recipients repeated.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Impact

Load amplification on a degraded provider and on our DB/broker; delays unrelated messages; potential provider blocking.

Diagnose

1. Provider outcomes: `messaging_provider_calls_total` by outcome: mostly TRANSIENT/TIMEOUT/RATE_LIMITED?
2. Check effective backoff: config `messaging.dispatch.backoff-base-ms`, `backoff-cap-ms`, `max-attempts` (a misconfigured base of 0 or cap too low causes storms). Verify via `GET /actuator/configprops` if exposed internally or deployment config.
3. Look for synchronization: retries clustered at the same second means jitter missing/disabled (bug) or clock mocked.
4. Is the circuit breaker open/closed? A breaker that never opens (threshold too high) allows hammering.
5. Look for requeue loops: rate-limiter requeues are cheap but could loop if token rate is 0 (`rate-per-sec=0`).

Mitigate

1. Disable or deprioritize the failing provider (`PUT /api/v1/providers/{id}/config`).
2. Increase `backoff-cap-ms` / `backoff-base-ms` (config rollout) and reduce `dispatch.concurrency`.
3. Pause consumers briefly (`POST /api/v1/ops/consumers/pause`) to let the provider recover; messages remain queued/RETRY_PENDING. Resume gradually.
4. Stop bulk jobs.

Recover

Resume consumers; watch for a second wave (all paused messages waking at once): the jitter spreads it; if not, resume in steps (concurrency 2, 4, 8).

Verify

Retry rate falls; success rate up; no growth in DLQ beyond expected.

Prevent

Full jitter (ADR-0006), breaker thresholds tested, retries share provider token bucket, max-attempt caps, load tests with fault injection (`PUT /providers/{id}/fault-mode` local/test only).

Proof: `RetryPolicyTest` (bounds), `FailureRecoveryIT#scenario01_...`, `#scenario10_...`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

14. Bulk Messaging with Spring Batch

14.1 When bulk is the right tool

A campaign to 100,000 recipients is not 100,000 API calls from a script; it is a **job**: validated, throttled, observable, stoppable and restartable. The platform uses **Spring Batch 5** (JDBC job repository in the same database) with a chunk-oriented step. The batch writer does not talk to providers; it submits each row through the same `NotificationService.submitInternal` path as the REST API, so every bulk message gets the same policy, template pinning, outbox, audit and dispatch behaviour. Bulk adds throughput management, not a second delivery path.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

14.2 Job design

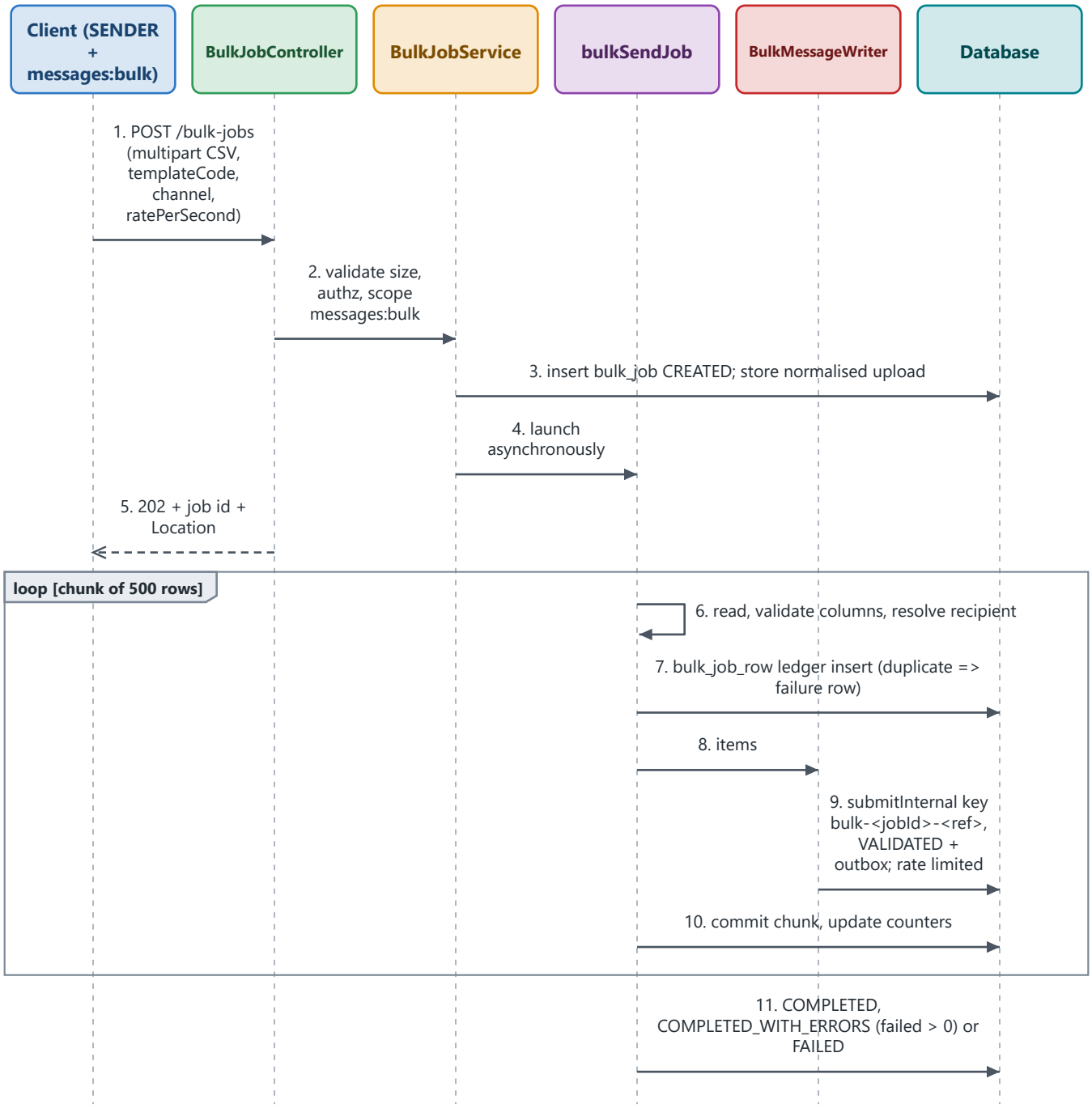


Figure 32. Bulk job with Spring Batch: a streamed CSV is read in chunks of 500, validated, de-duplicated against the (job_id, external_ref) ledger and written through NotificationService.submitInternal. Implementation: backend/src/main/java/com/example/messaging/batch/BulkJobService.java, backend/src/main/java/com/example/messaging/batch/BulkJobConfig.java, backend/src/main/java/com/example/messaging/batch/RecipientRowProcessor.java, backend/src/main/java/com/example/messaging/batch/BulkMessageWriter.java

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Element	Class	Role
Controller	`web/BulkJobController`	Upload, list, get, failures, stop, restart, generate-synthetic
Service	`batch/BulkJobService`	Validation, status bookkeeping, launching, stop, restart
Reader	`batch/RecipientCsvReaderFactory`, `BoundedLineReader`	Streams the CSV, never loads it whole; enforces line length and column limits
Row validation	`batch/CsvRowValidator`, `RecipientRowProcessor`	Columns, recipient resolution, in-job duplicate detection
Writer	`batch/BulkMessageWriter`	One `submitInternal` per item, transaction per chunk, rate limited
Rate limiting	`batch/TokenBucketRateLimiter`	Writer-side limit `ratePerSecond` on top of the provider token bucket
Listeners	`BulkJobListener`, `BulkSkipListener`	Final status, failure rows
Recovery	`BulkExecutionRecovery`, `BulkJobSweeper`	Close orphaned executions; mark stale jobs `ABANDONED`
Storage	`BulkUploadStorage`, `BulkJobStore`	Normalised upload files; job counters
Test data	`SyntheticDatasetGenerator`, `SyntheticRecipientProvisioner`	Generate N-row CSVs and recipients without any real data

The CSV header is `externalRef,locale,var_firstName,var_amount,...`; the `var_` columns are the variables of the chosen template. Uploads are normalised to UTF-8 with LF line ends, BOM and trailing blank lines removed, and stored as `<messaging.bulk.upload-dir>/<tenant>/<jobId>.csv` (a server-generated name). Files are deleted when the job completes (`delete-file-on-completion`) and otherwise by `RetentionJob`.

14.3 Counters, skips and duplicates

Rule	Detail
Counting	`processed = succeeded + failed + duplicate`
Policy rejections	Rows rejected by policy at submission (opt-out, suppression, ...) are persisted as `REJECTED` messages but counted as `failed` with reason `POLICY_REJECTED: <reason>`
Duplicates	Filtered (not skipped), recorded in `bulk_job_failure` as `DUPLICATE_EXTERNAL_REF: first seen on line N`; they do **not** count against the skip limit
Skip policy	Up to `messaging.bulk.skip-limit` (100) bad rows skipped and recorded; beyond that the job fails with a job-level failure `SKIP_LIMIT_EXCEEDED` (`line_no = 0`)
Final status	`COMPLETED_WITH_ERRORS` if and only if `failed > 0`; otherwise `COMPLETED`; `FAILED` when the step fails
Transient DB errors	Retried up to `retry-limit`
Rolled-back chunk	Leaves no counter and no message by design; details of rows rejected inside it are not kept, the job-level failure row explains why it failed

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

14.4 Authorization

Bulk is a privileged operation. Create, generate-synthetic, stop and restart need role `SENDER`, `TENANT_ADMIN` or `PLATFORM_ADMIN` **and** the scope `messages:bulk` for every caller, `PLATFORM_ADMIN` included. A refusal is audited as `BULK_JOB_DENIED` and answered 403. Reads (list, get, failures) need one of SENDER, TENANT_ADMIN, PLATFORM_ADMIN, OPERATOR, AUDITOR and are tenant scoped (404 across tenants).

14.5 Job states

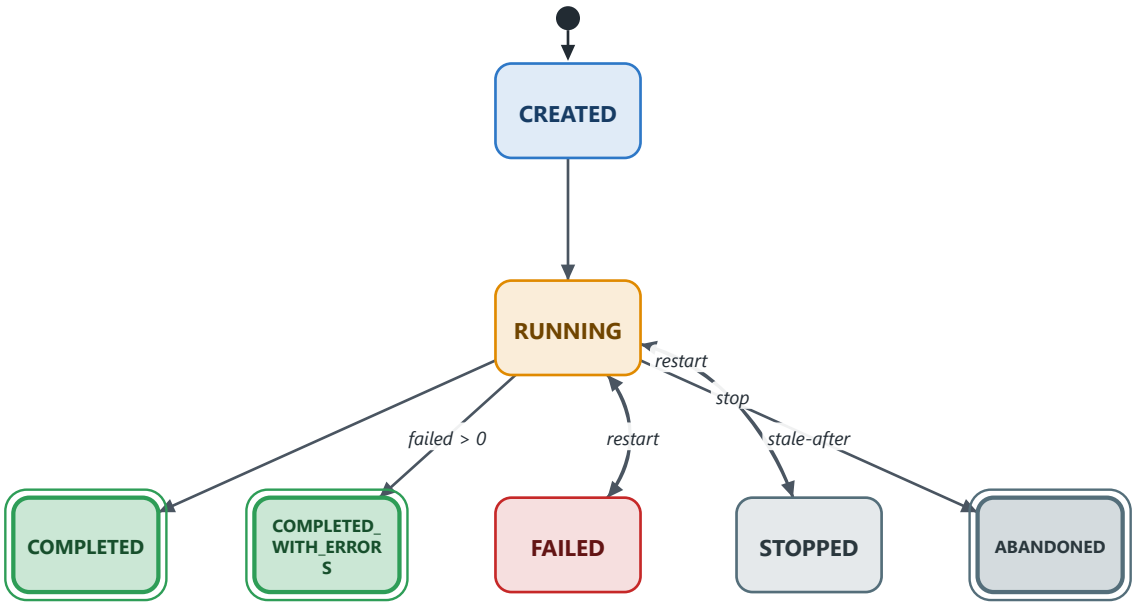


Figure 33. Bulk job status values. Restart is permitted only from FAILED or STOPPED; ABANDONED is set by the sweeper for jobs without progress and is terminal for the API. Implementation: backend/src/main/java/com/example/messaging/domain/BulkJobStatus.java, backend/src/main/java/com/example/messaging/batch/BulkJobService.java, backend/src/main/java/com/example/messaging/batch/BulkJobSweeper.java

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

14.6 Restart and the exactly-one guarantee

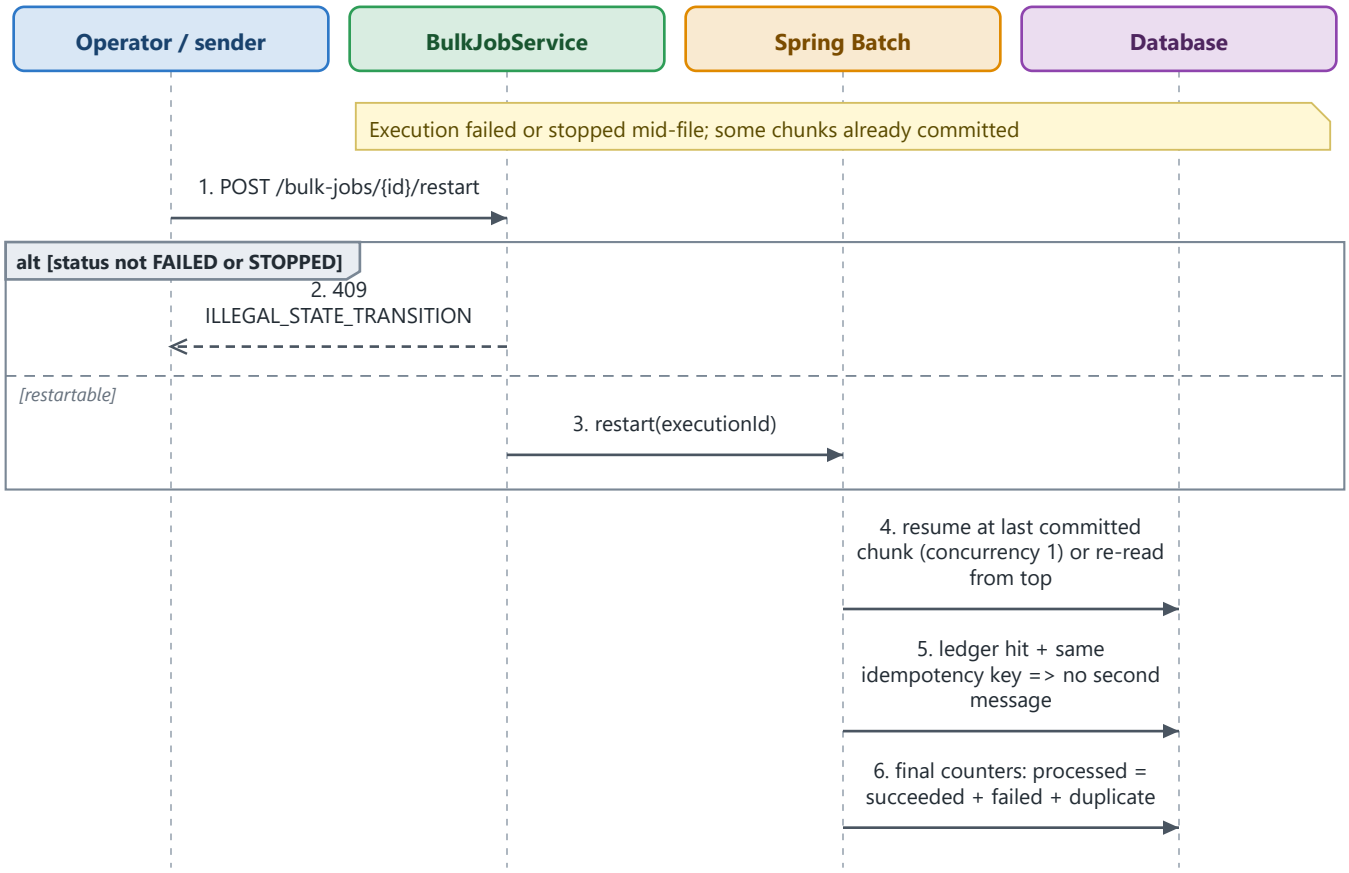


Figure 34. Bulk restart: a FAILED or STOPPED job is relaunched with the same parameters. With concurrency 1 the reader resumes from the last committed chunk; with more workers the file is re-read and the ledger plus deterministic idempotency keys make processed rows no-ops. Implementation: backend/src/main/java/com/example/messaging/batch/BulkJobService.java (restart), backend/src/main/java/com/example/messaging/batch/BulkExecutionRecovery.java, backend/src/main/java/com/example/messaging/batch/BulkJobStore.java

Two mechanisms make a restart safe:

Mechanism	Mechanics
Deterministic idempotency key	Each row is submitted with key `bulk-<jobId>-<externalRef>`, so submitting the same row again returns the original message instead of creating a second one
Row ledger	`bulk_job_row` has primary key `(job_id, external_ref)` plus the line number. A re-seen reference on the **same** line is a replay (safe); the same reference on **another** line is a duplicate

With `messaging.bulk.concurrency = 1` the reader checkpoints per chunk and a restart resumes from the last committed chunk. With more workers no exact position exists, so the restart re-reads the file from the top and the ledger plus the idempotency key make already processed rows no-ops. With several workers, which occurrence of a duplicated reference wins is not deterministic, but exactly one message is sent either way.

Restart is allowed only from `FAILED` or `STOPPED`; any other status is 409 `ILLEGAL_STATE_TRANSITION`. `stop` on a `RUNNING` job whose batch execution is not alive in this JVM closes the orphaned execution as `STOPPED` so it can be

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

restarted. Jobs run on the node that accepted the request (a single-node assumption for stop and orphan recovery); the sweeper marks jobs with no progress for `stale-after` as `ABANDONED`.

14.7 Throughput and fairness

Control	Default	Why
`messaging.bulk.concurrency`	4	Bounded writers (multi-threaded step behind a synchronised reader)
`chunk-size`	500	One transaction per chunk
`max-rows`	100000	Upper bound for one job
`max-upload-bytes`, `max-line-length`, `max-columns`	bounded	Reject abusive files before parsing
`ratePerSecond`	request parameter, capped by `max-rate-per-second`	Throttle intake so bulk does not flood the queue
Message priority	LOW or NORMAL by default	Bulk cannot starve CRITICAL and HIGH
`max-parallel-jobs`, `launch-queue-capacity`	bounded	Limit concurrent jobs

Extra meters: `messaging.bulk.progress{jobId}` (gauge), `messaging.bulk.writers.active`. Audit actions: `BULK_JOB_DENIED`, `BULK_JOB_ABANDONED`, `BULK_JOB_STOPPED`.

> **Synthetic generation**
>
> `POST /api/v1/bulk-jobs/generate-synthetic` creates and submits a job without an upload: it provisions the recipients it needs (`SYN-0000001...`, fictitious `example.test` addresses) for the caller's tenant and generates exactly the variable columns of the chosen template. With `createRecipients=false` every row fails with `RECIPIENT_NOT_FOUND`. `scripts/generate-dataset.sh` writes an N-row CSV from the command line.

14.8 Tests that prove it

Test	Proves
`BulkJobBatchIT`	End-to-end job: counters, failures, final status
`BulkJobApiIT`	Upload, list, get, failures, authorization and tenant scoping
`BulkJobRestartConcurrentIT`, `AbstractRestartIT`	Restart with concurrency, no duplicates
`FailureRecoveryBulkIT`	Scenario 9: writer failure mid-chunk, restart, message count equals valid unique rows
`CsvRulesTest`, `RowProcessorTest`	CSV rules and row validation
`TokenBucketRateLimiterTest`	Throttle
`BulkThroughputIT`	In-JVM throughput harness (local only; numbers are in the validation report, not here)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Test	Proves
`SyntheticDatasetGeneratorTest`, `BulkUploadStorageTest`, `BulkJobStoreTest`	Generator, storage, counters

Reference material

Bulk job flow (design document)

Source document: `docs/architecture/bulk-job-flow.md`

Implemented details (row ledger, counting, restart modes, authorization) are in sections 14.3 to 14.6 and DEVIATIONS "Bulk jobs and retention".

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter.

Properties

- **Restart safety**: deterministic idempotency keys make re-processing a chunk a no-op; `POST /bulk-jobs/{id}/restart` relaunches the same `JobParameters` and resumes from the last committed chunk.
- **Skip policy**: up to `messaging.bulk.skip-limit` invalid rows skipped and recorded; beyond it the job is `FAILED`. Transient DB errors retried 3 times.
- **Bounded concurrency**: `messaging.bulk.concurrency` (default 4), multi-threaded step only behind `SynchronizedItemStreamReader`.
- **Isolation of priorities**: bulk messages default to LOW/NORMAL priority so they cannot starve CRITICAL/HIGH queues; aging (spec 0001) adds starvation protection for LOW.
- **Throttle**: writer-side rate limit plus the normal provider token bucket; consent policy is applied again at dispatch.
- **Progress**: `messaging.bulk.progress{jobId}` gauge; `GET /api/v1/bulk-jobs/{id}` and `/failures` (paged).
- **Stop**: `POST /bulk-jobs/{id}/stop` marks `STOPPED` and the step ends after the current chunk; `ABANDONED` is set by an operator/sweeper for jobs that never finish (see runbook `bulk-job-stuck.md`).
- Schema: `bulk_job`, `bulk_job_failure`, Spring Batch tables (`V100__spring_batch.sql`).

Bulk test plan

Source document: `docs/operations/bulk-test-plan.md`

Repeatable plan for functional, failure-recovery and throughput testing of the bulk module (`com.example.messaging.batch`, `RetentionJob` for housekeeping). All data is synthetic. Numbers appear in this document **only where they were actually measured**; every such number states the command, date and machine caveat.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. What is tested, and where

Area	Test	Kind	Runs in
Happy path, validation failures reported (never echoed), duplicates across chunks/threads, skip limit => `FAILED`, retry of a transient DB error, bounded concurrency, writer rate limit, policy-rejected rows	`BulkJobBatchIT`	integration (Spring context, H2, real security chain)	`mvn verify`
Multipart upload, role AND scope check with audited 403, tenant isolation (404), size/row/content/UTF-8/line-length limits, hostile file names, stop/restart state rules, orphaned job stop+restart, sweeper `ABANDONED`, failures paging, synthetic jobs	`BulkJobApiIT`	integration	`mvn verify`
Reliability scenario 9: crash mid-chunk, only whole chunks committed, restart resumes from the last committed chunk, exactly one message per row	`FailureRecoveryBulkIT#scenario09_bulkJobRestart_resumesWithoutDuplicates` (+ interrupted twice, file removed, uninterrupted reconciliation)	integration, `concurrency=1`	`mvn verify`
Same crash with 4 workers: restart re-reads from the top, ledger + idempotency keys still give exactly-once	`BulkJobRestartConcurrentIT`	integration, `concurrency=4`	`mvn verify`
Row rules: validation, duplicate detection, recipient/template checks, formula injection	`RowProcessorTest`, `CsvRulesTest`	unit	`mvn test`
Upload storage: sanitising, traversal, limits, normalisation, cleanup on failure	`BulkUploadStorageTest`	unit	`mvn test`
Ledger/counters/failure records, races, rollback behaviour	`BulkJobStoreTest` (real schema on H2)	unit	`mvn test`
Token bucket (fake clock + real threads)	`TokenBucketRateLimiterTest`	unit	`mvn test`
Dataset generator: determinism, rates, defect kinds, 1,000,000 rows in a 24 MB heap	`SyntheticDatasetGeneratorTest`	unit	`mvn test`
Retention: terminal-state rule, cut-offs, children first, legal hold, dry run, batching/resume, other tables	`RetentionJobTest` (real schema on H2)	unit	`mvn test`
Throughput	`BulkThroughputIT`	measurement, **off by default** (`-Dbulk.measure=true`)	on demand

Maven wrapper command used in this project (JDK 21 and Maven are pinned, nothing is exported globally):

BASH

```
bash scripts/mvn.sh verify                                    # everything (unit + integration)
bash scripts/mvn.sh test -Dtest='Bulk*Test,RowProcessorTest,CsvRulesTest,SyntheticDatasetGeneratorTest,
  TokenBucketRateLimiterTest,RetentionJobTest'
bash scripts/mvn.sh verify -Dtest=NoSuchTest -Dsurefire.failIfNoSpecifiedTests=false
  -Dit.test='BulkJobBatchIT,BulkJobApiIT,FailureRecoveryBulkIT,BulkJobRestartConcurrentIT'
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. Datasets

Name	Rows	How	Use
tiny	10-100	built inline by the tests (`AbstractBulkIT.csv`)	functional
scenario	60 (55 distinct, 3 duplicates, 2 invalid)	`AbstractRestartIT.scenarioFile()`	restart/exactly-once
standard	20,000	`SyntheticDatasetGenerator`, dup 2 %, invalid 1 %, seed 5	throughput (measured, section 5)
large	100,000 (= default `messaging.bulk.max-rows`)	generator CLI	stress, not run for this document
huge	1,000,000	generator CLI	generator memory test only; running the job needs `messaging.bulk.max-rows` raised

Generator CLI (streams, constant memory):

```bash
bash scripts/generate-dataset.sh 100000 .run/recipients-100k.csv 0.02 0.01     # fat jar + PropertiesLauncher
# equivalent without the script, from compiled classes:
java -cp backend/target/classes com.example.messaging.batch.SyntheticDatasetGenerator 100000 .run/r.csv 0.02
  0.01 --seed=7
```

Options: `[--seed=N] [--refs=synthetic|seed] [--seed-recipients=40] [--start=1] [--vars=a,b,c]`.

- `--refs=synthetic` (default) writes `SYN-0000001...`. These recipients are **not** in the seed. They are created by `POST /api/v1/bulk-jobs/generate-synthetic` (idempotent top-up for the caller's tenant; switch off with `?createRecipients=false`), or in tests by `SyntheticRecipientProvisioner.ensure(tenant, n)`. A job run on such a file *without* provisioning reports every row as `RECIPIENT_NOT_FOUND` (a legitimate failure-path load test; set `messaging.bulk.skip-limit` high enough, otherwise the job fails by design after the limit).
- `--refs=seed` writes the seeded `CUST-001..CUST-040` of `tenant-retail` (max 40 rows, references must be unique).
- Defects (probability `invalidRate`): missing reference, unknown recipient (`NOPE-...`), wrong column count, formula injection (`=1+1`), 2,500-character field, bad locale. Duplicates (probability `duplicateRate`) repeat one of the last 1,000 valid references.

3. Expected behaviour (pass criteria)

1. Terminal status is `COMPLETED` (no failed rows) or `COMPLETED_WITH_ERRORS`; `FAILED` only if the skip limit was exceeded or a non-skippable error happened (then `bulk_job_failure` line 0 says why).
2. `processed = succeeded + failed + duplicate` at the end (and `= total` when the whole file was read).
3. `COUNT(message_request WHERE bulk_job_id = ?) = succeeded + (policy-rejected rows)`; `COUNT(DISTINCT idempotency_key)` equals the message count (no duplicate sends).
4. No `idempotency_record` of the job is left `IN_PROGRESS`.
5. `bulk_job_failure` holds one row per rejected line (`line_no`, reason code, SHA-256 of the line); no recipient data.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Stop leaves `STOPPED` after the current chunk; restart finishes the file without duplicates.
7. `messaging.bulk.writers.active` never exceeds `messaging.bulk.concurrency`.

4. How to run a throughput measurement

BASH

```
# 20,000 rows, 4 chunk workers, chunk size 500 (the defaults), result appended to
  backend/target/bulk-measurements.txt
bash scripts/mvn.sh verify -Dtest=NoSuchTest -Dsurefire.failIfNoSpecifiedTests=false \
    -Dit.test=BulkThroughputIT -Dbulk.measure=true -Dbulk.rows=20000 -Dbulk.concurrency=4 -Dbulk.chunk=500
```

The test provisions the recipients, generates the CSV, uploads it through the API, waits for the terminal status and asserts the invariants of section 3 before it prints one `BULK-MEASUREMENT ...` line. What the number covers: upload, validation, duplicate detection, template check, one `message_request` + outbox row + audit row + idempotency record per valid row, counters. It does **not** cover provider dispatch (that is the dispatch pipeline's throughput).

Variables worth sweeping and what to read:

Variable	Values	Look at
`messaging.bulk.concurrency`	1, 2, 4, 8	rows/s, `hikaricp_connections_active/pending`, lock waits on `bulk_job`
`messaging.bulk.chunk-size`	100, 500, 1000	rows/s, transaction length, memory
`ratePerSecond` of the job	none, 50, 500	achieved rate vs configured
`messaging.bulk.skip-limit` / defect rate	0 %, 1 %, 5 %	cost of rejected rows (chunk rollback + re-scan)
database	H2 (here), PostgreSQL (not run)	everything; H2 numbers are not a capacity statement

On a running application read: `messaging.bulk.progress{jobId}` (0..1), `messaging.bulk.writers.active`, `hikaricp_connections_*`, JVM heap, `GET /api/v1/bulk-jobs/{id}` counters, `messaging.queue.depth` of the dispatch queue.

Manual API flow against a running instance (not executed for this document)

BASH

```
# 1. dev token with the bulk scope (profile local/test only), then upload
TOKEN=$(curl -s -X POST localhost:18080/dev/token -H 'Content-Type: application/json' \
  -d '{"subject":"bulk-sender","tenant":"tenant-retail","roles":["SENDER"],"scopes":["messages:bulk"]}' |
    python -c 'import sys,json;print(json.load(sys.stdin)["accessToken"])')
curl -s -X POST localhost:18080/api/v1/bulk-jobs -H "Authorization: Bearer $TOKEN" \
  -F file=@.run/recipients-100k.csv -F name=load-100k -F templateCode=TRADE_CONFIRMATION \
  -F channel=EMAIL -F category=TRANSACTIONAL
# 2. poll
curl -s localhost:18080/api/v1/bulk-jobs/<id> -H "Authorization: Bearer $TOKEN"
# 3. synthetic shortcut (creates SYN recipients, generates and runs)
curl -s -X POST localhost:18080/api/v1/bulk-jobs/generate-synthetic -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"rows":20000,"duplicateRate":0.02,"invalidRate":0.01}'
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5. Measured results (actually run)

Machine: Windows 11, 8 logical CPUs, JDK 21 (HotSpot 21.0.7), 4 GB max heap, H2 2.x in memory (PostgreSQL mode), Spring `test` profile (Hibernate statistics on). **Caveat: the machine was shared with other Maven builds running at the same time, so absolute numbers are pessimistic and noisy; use them as order of magnitude and for relative comparison only.** Date 2026-10-18. Provider dispatch not included.

Rows	Concurrency	Chunk	Result	Rows/s	Notes
20,000 (dup 2 %, invalid 1 %)	4	500	`COMPLETED_WITH_ERRORS`, 19,369 sent, 195 failed, 436 duplicates, 252.1 s	**79**	upload accepted in 1.4 s
3,000	1	500	`COMPLETED_WITH_ERRORS`, 65.9 s	46	
3,000	4	500	`COMPLETED_WITH_ERRORS`, 51.6 s	58	scaling 1 -> 4 workers is only 1.26x
5,000	4	500	`COMPLETED_WITH_ERRORS`, 4,841 sent, 54 failed, 105 duplicates, 87.4 s	57	later snapshot of the backend **with the dispatch pipeline running** (outbox publisher + consumers dispatch the new messages while the job runs)

The first three rows were taken on a snapshot of the backend that had no outbox publisher yet (messages stayed `VALIDATED`); the last row is the same measurement after the dispatch module had landed.

Reading the numbers: the batch layer itself is not the bottleneck. A throw-away timing loop (not committed) that called `NotificationService.submitInternal` directly, with no batch at all, measured 21-53 ms per call on the same machine (first 200 calls are slower: JIT warm-up), i.e. 20-50 calls/s per thread. A bulk row costs one such call plus a few small statements, so 46-79 rows/s is what that submission path delivers on H2 with statistics enabled while the machine is busy. Speed-ups therefore belong in the submission path (statement count per message, template/recipient caching, a real database, no statistics) rather than in the batch step. The poor scaling with workers points to contention (single H2 writer, the `bulk_job` counter row, `BATCH_STEP_EXECUTION` updates at every chunk commit); a PostgreSQL run is needed to say anything about production scaling and was not possible here (no Docker).

Other measured facts:

- `SyntheticDatasetGeneratorTest#oneMillionRows_streamToDisk_inAnEmptyHeap_withinTheTimeBox`: 1,000,000 rows written (60+ MB) by a child JVM started with `-Xmx24m` in 6.2 s (the process would die with `OutOfMemoryError` if it kept rows in memory).
- `BulkJobBatchIT#writerRateLimit_pacesTheJob`: 60 rows at `ratePerSecond=50` finish in >= 0.9 s (about 1.1 s expected).

6. Not covered

PostgreSQL/Oracle runs, multi-node execution (jobs run on the node that accepted the request; see the module's known gaps), provider dispatch throughput, k6 (not available here), Testcontainers.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Runbook: Bulk job stuck

Source document: `docs/operations/runbooks/bulk-job-stuck.md`

Symptoms

`messaging.bulk.progress{jobId}` flat; job `RUNNING` but `processed` not changing; job `FAILED` after skip limit; `GET /api/v1/bulk-jobs/{id}` shows stale `updated_at`.

Diagnose

1. `GET $BASE/api/v1/bulk-jobs/{id}` : status, counters, `spring_batch_execution_id`.
2. `GET $BASE/api/v1/bulk-jobs/{id}/failures?page=0&size=100` : row-level reasons (invalid columns, unknown recipients, duplicates).
3. Is the app instance that was running the job alive? Instance restarted mid-job leaves Spring Batch execution `STARTED` (zombie) until marked failed.
4. Are writer-side throttles slow (`rate_limit_per_sec`)? Throughput = rate; 100k rows at 50/s = ~33 min (normal, not stuck).
5. Is backpressure (503 `max-queue-depth`) blocking the writer? Check queue depth.
6. DB: pool saturation or lock waits (`database-outage.md`).
7. Thread dump for the batch executor threads.

Mitigate / Recover

Situation	Action
Job exceeded skip limit due to bad file	Fix file; submit a new job (new job id => new idempotency keys, careful not to double-send rows already sent: use only unsent rows; failures endpoint lists rows)
Instance died mid-run	`POST /api/v1/bulk-jobs/{id}/stop` if still RUNNING/zombie (marks STOPPED/FAILED), then `POST /api/v1/bulk-jobs/{id}/restart` (only FAILED or STOPPED jobs can be restarted; step-by-step in `bulk-job-restart.md`): with `bulk.concurrency=1` it resumes from the last committed chunk, with more workers it re-reads the file and the ledger plus idempotency keys `bulk-<jobId>-<externalRef>` prevent double submission
Backpressure	wait or lower job rate; restart after queue drains
Wrong content/audience sent	`stop`, then cancel remaining messages: filter messages by `bulk_job_id` and cancel those in VALIDATED/QUEUED/RETRY_PENDING (`POST /api/v1/messages/{id}/cancel`)
Job permanently unusable	no API action: the sweeper marks a job without progress for `messaging.bulk.stale-after` (default 30m) as `ABANDONED` (audited `BULK_JOB_ABANDONED`, terminal); submit a new job with the unsent rows

Verify

After restart: counters advance, `processed = succeeded + failed + duplicate` at completion, `COUNT(message_request WHERE bulk_job_id=?)` equals succeeded count (no duplicates).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Prevent

Chunked commits, deterministic keys, skip limit, max rows, throttles, bulk priority LOW, startup sweeper for zombie executions (see source).

Proof: `FailureRecoveryIT#scenario09_bulkJobRestart_resumesWithoutDuplicates`.

Runbook: Bulk job restart

Source document: `docs/operations/runbooks/bulk-job-restart.md`

Referenced by the `BulkJobStalled` alert (`observability/prometheus/alerts.yml`). Companion triage document: `bulk-job-stuck.md`. Written from the implemented behaviour recorded in `docs/design/DEVIATIONS.md` (section "Bulk jobs and retention") and the code in `backend/src/main/java/com/example/messaging/batch/BulkJobService.java`, not from a real incident. Set `BASE` and `TOKEN` as described in `README.md` of this folder.

Symptoms and alerts

- Alert `BulkJobStalled`: `messaging_bulk_progress` for a `jobId` unchanged for 10 minutes while between 0 and 1.
- `GET $BASE/api/v1/bulk-jobs/{id}` shows `RUNNING` with counters not moving, or `FAILED` / `STOPPED`.
- `GET $BASE/api/v1/bulk-jobs/{id}/failures` contains a job-level row (`line_no = 0`), e.g. `SKIP_LIMIT_EXCEEDED` or `STEP_FAILED: <exception type>`.

Who may act

`stop` and `restart` need role SENDER, TENANT_ADMIN or PLATFORM_ADMIN **and** the token scope `messages:bulk` for every caller, PLATFORM_ADMIN included. A refusal is answered 403 and audited as `BULK_JOB_DENIED`. Reads are tenant scoped (another tenant's job is 404). A PLATFORM_ADMIN needs an `X-Tenant-Id` header to act on a tenant's job.

State rules (enforced in `BulkJobService`)

Action	Allowed from	Otherwise
`POST /api/v1/bulk-jobs/{id}/stop`	`RUNNING` (with a batch execution id)	409 `ILLEGAL_STATE_TRANSITION`
`POST /api/v1/bulk-jobs/{id}/restart`	`FAILED` or `STOPPED`	409 `ILLEGAL_STATE_TRANSITION`
neither	`COMPLETED`, `COMPLETED_WITH_ERRORS`, `ABANDONED` (terminal for the API)	409

Restart also needs the uploaded CSV to still exist (`<messaging.bulk.upload-dir>/<tenant>/<jobId>.csv`). The file is deleted when a job completes (`delete-file-on-completion`) and later by `RetentionJob`; if it is gone the restart is refused with 409 and you must submit a new job (see "File gone" below). If too many bulk jobs are already queued the restart answers 503 with `Retry-After: 30`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Diagnose

1. `GET $BASE/api/v1/bulk-jobs/{id}`: status, `total/processed/succeeded/failed/duplicate`.
2. `GET $BASE/api/v1/bulk-jobs/{id}/failures?page=0&size=100`: row reasons. `line_no = 0` is the job-level cause.
3. Is the job simply slow? Throughput is bounded by `ratePerSecond`: 100,000 rows at 50/s is about 33 minutes. A flat counter is only a problem if it stays flat.
4. Is the node that accepted the job still alive? Jobs run on the node that accepted the request (single-node assumption). If that process died the job stays `RUNNING` in the database although no worker runs it.
5. Backpressure or database trouble: `GET $BASE/api/v1/ops/queues`, `database-outage.md`.
6. `messaging.bulk.writers.active` (gauge) shows whether writers are active.

Mitigate and recover

Situation	Action
Job `RUNNING`, worker alive but wedged	`POST .../stop`, wait for `STOPPED`, then `POST .../restart`
Job `RUNNING` but its node died (orphan)	`POST .../stop`: the orphaned execution is closed as `STOPPED` and audited with `orphaned=true`; then `POST .../restart`
Job `FAILED` after a transient cause (DB blip, backpressure)	fix the cause, then `POST .../restart`
Job `FAILED` with `SKIP_LIMIT_EXCEEDED`	the file has too many bad rows: fix the file and submit a **new** job. A new job has new idempotency keys (`bulk-<newJobId>-<externalRef>`), so rows already sent by the old job would be sent again: remove them from the new file first
Job `ABANDONED` (sweeper: no progress for `messaging.bulk.stale-after`, default 30m)	terminal: cannot be restarted. Submit a new job with the unsent rows
File gone	submit a new job; use the messages list filtered by the old job's `bulk_job_id` to find what was already sent
Wrong audience or content	`stop`, then cancel remaining messages that are still `VALIDATED`/`QUEUED`/`RETRY_PENDING` with `POST /api/v1/messages/{id}/cancel`

What a restart does (and does not) guarantee

- With `messaging.bulk.concurrency = 1` the reader checkpoints per chunk and the restart resumes from the last committed chunk.
- With more workers there is no exact position: the restart re-reads the file from the top. The `bulk_job_row` ledger and the idempotency key `bulk-<jobId>-<externalRef>` turn already processed rows into no-ops, so each `externalRef` is sent at most once. Which occurrence of a duplicated reference wins is not deterministic in that case (exactly one is sent either way).
- A chunk that was rolled back leaves no counter and no message by design; details of rows rejected inside a rolled-back chunk are not kept.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Verify

- Status moves to `RUNNING` then a terminal state; `processed` increases.
- At completion `processed = succeeded + failed + duplicate`; `COMPLETED_WITH_ERRORS` iff `failed > 0`. Policy rejections (opt-out, suppression) count as `failed` with reason `POLICY_REJECTED: <reason>`.
- No duplicates: the number of `message_request` rows with the job's `bulk_job_id` equals `succeeded + (policy-rejected rows)`; there is exactly one message per `externalRef`.
- Audit trail shows `BULK_JOB_STOPPED` / `BULK_JOB_RESTARTED` (`GET $BASE/api/v1/audit-events?entityType=BULK_JOB&entityId={id}`).

Escalate

Platform on-call if the restart answers 409 "cannot be restarted: ..." repeatedly or the batch repository tables (`BATCH_*`) look inconsistent.

Prevent

Chunked commits, deterministic idempotency keys, skip limit, `max-rows`, writer rate limit, stale-job sweeper, `ratePerSecond` sized to the provider limits.

Evidence in the repository

`FailureRecoveryBulkIT#scenario09_bulkJobRestart_resumesWithoutDuplicates`, `FailureRecoveryBulkIT#interruptedTwice_stillExactlyOncePerRow`, `FailureRecoveryBulkIT#restartOfAFailedJobWhoseFileWasRemoved_isRefused`, `BulkJobRestartConcurrentIT#multiThreadedJob_restartedFromTheTop_producesNoDuplicates`, `BulkJobApiIT#stopAndRestart_followTheStateRules`, `BulkJobApiIT#orphanedRunningJob_canBeStoppedThenRestarted`, `BulkJobApiIT#sweeper_abandonsStaleJobs_only`. Executed results, if any, are recorded only in `docs/validation/VALIDATION_REPORT.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

15. Database ER Diagram

15.1 Design stance

PostgreSQL 16 is the **single system of record** (ADR-0005). The schema is owned by Flyway migrations in `backend/src/main/resources/db/migration` (JPA runs with `ddl-auto: validate`). SQL is deliberately **portable**: only `VARCHAR`, `BIGINT`, `INT`, `BOOLEAN`, `TIMESTAMP` and plain constraints, with no partial indexes, JSONB or `ON CONFLICT`, so the same migrations run on H2 in PostgreSQL mode for local runs and tests, and map mechanically to Oracle (`docs/database/oracle-mapping.md`, reference DDL not executed).

Migration	Content
`V1__tenant_recipient.sql`	`tenant`, `recipient`, `recipient_preference`, `suppression`
`V2__templates.sql`	`template`, `template_version`
`V3__messages.sql`	`message_request`, `message_attempt`, `provider_config`, `delivery_event`
`V4__bulk_audit_idempotency_outbox.sql`	`bulk_job`, `bulk_job_failure`, `audit_event`, `idempotency_record`, `processed_message`, `outbox_event`, `replay_guard`
`V50__dispatch_indexes.sql`	Indexes for the dispatch pipeline queries
`V100__spring_batch.sql`	Spring Batch metadata tables
`V200__bulk_ledger_legal_hold.sql`	`bulk_job_row`, `legal_hold`, unique failure line index, retention age indexes

15.2 Entity-relationship diagrams

Four figures cover all 19 application tables: an overview of the foreign-key structure, then three detail figures for identity and templates, the message core, and the reliability and bulk tables.

> **Reading the diagrams**
>
> Only key and relationship columns are drawn in the detail figures; the migrations are authoritative. `outbox_event.aggregate_id` and `processed_message.dispatch_id` are logical references only (no foreign key), by design: dedupe and outbox rows must be insertable and purgeable independently of the message row. `bulk_job.spring_batch_execution_id` references the Spring Batch metadata tables logically.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

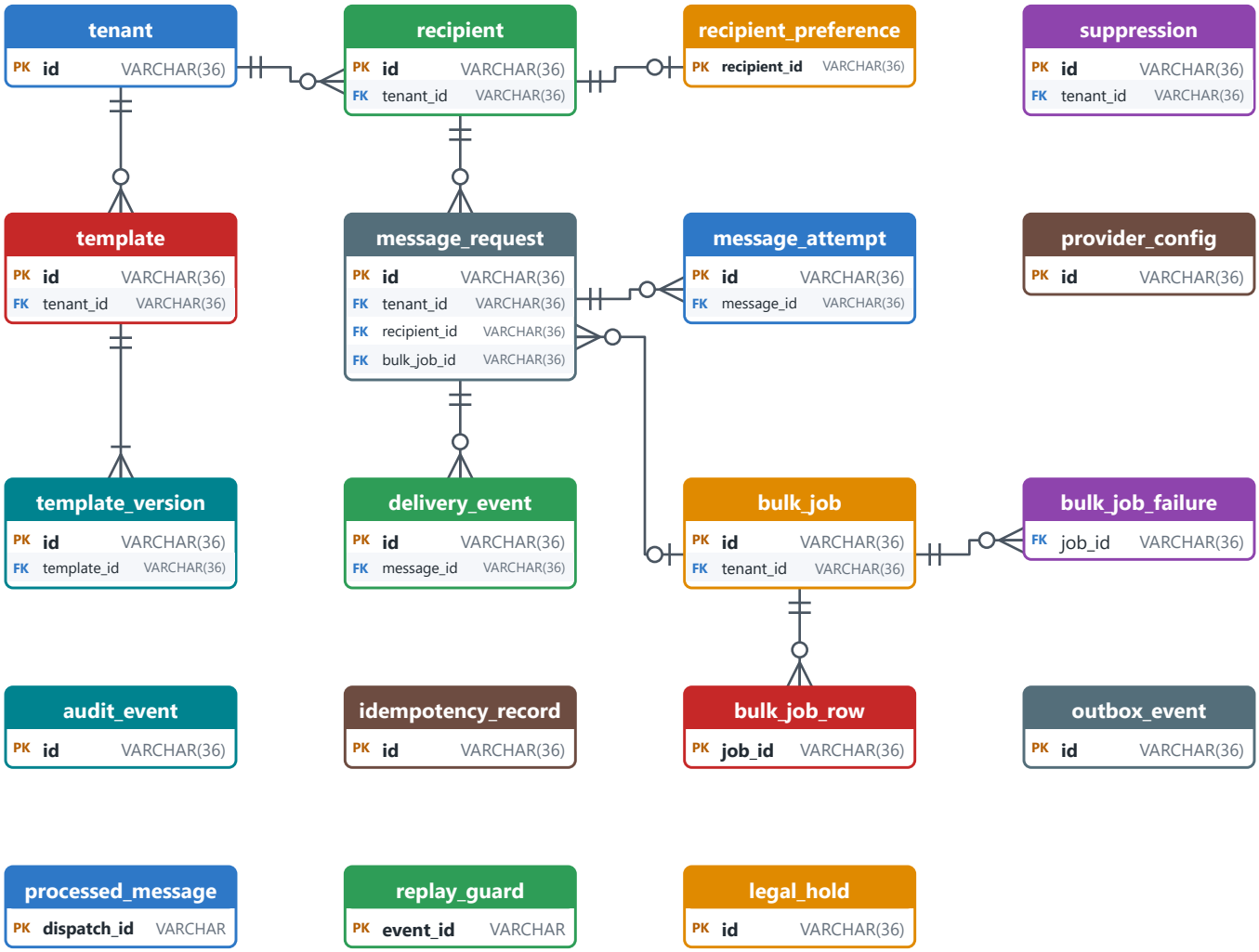


Figure 35. ER overview: all 19 application tables created by Flyway migrations V1 to V200 with their foreign-key relationships (columns abbreviated to keys; the next three figures show the details). Spring Batch metadata tables from V100 are not drawn. Implementation: backend/src/main/resources/db/migration/V1__tenant_recipient.sql ... V200__bulk_ledger_legal_hold.sql

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

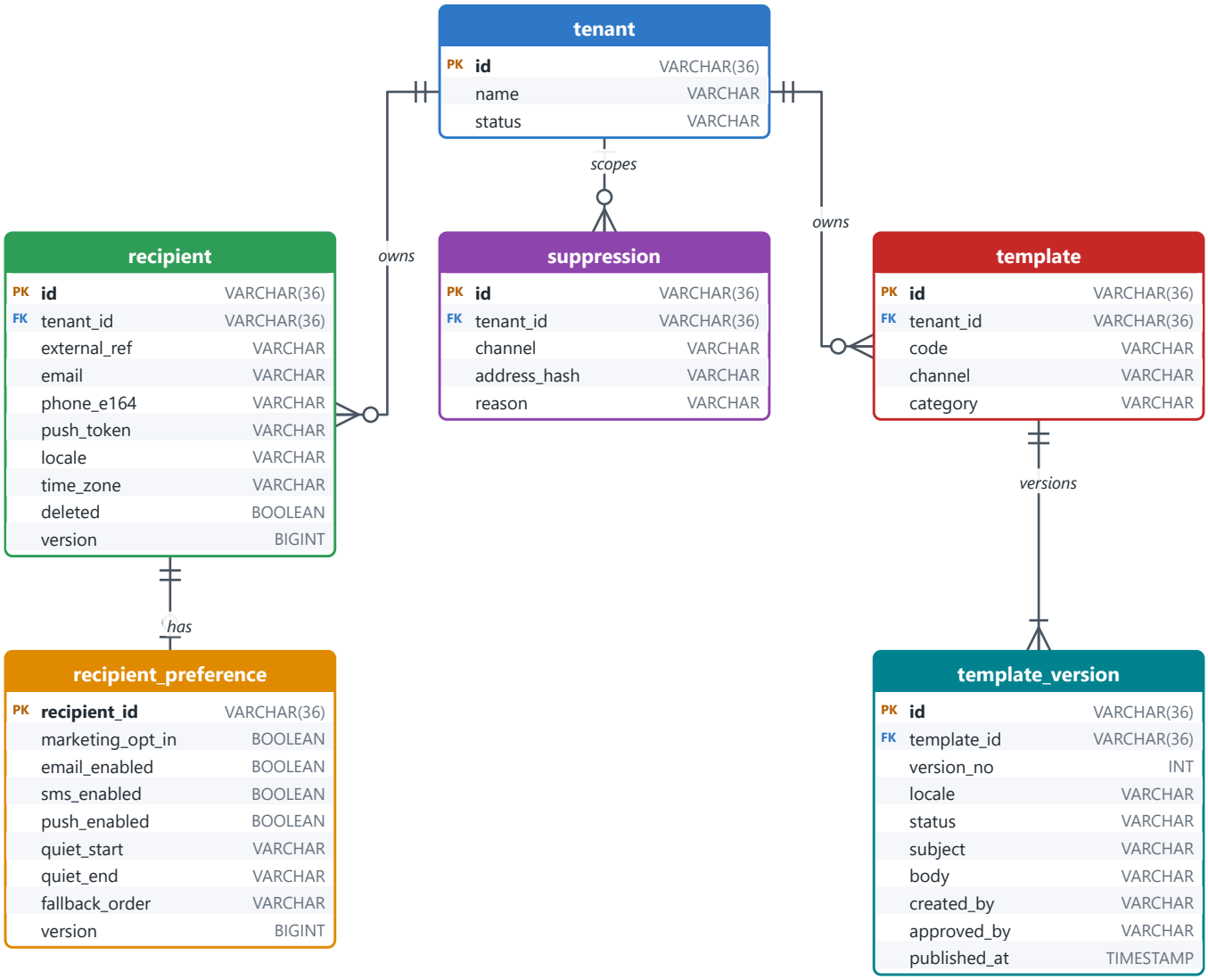


Figure 36. ER detail 1 of 3: tenants, recipients, preferences, suppressions and templates. UNIQUE(tenant_id, external_ref) on recipient, UNIQUE(tenant_id, channel, address_hash) on suppression, UNIQUE(tenant_id, code, channel) on template, UNIQUE(template_id, version_no, locale) on template_version. Implementation: backend/src/main/resources/db/migration/V1__tenant_recipient.sql, V2__templates.sql

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

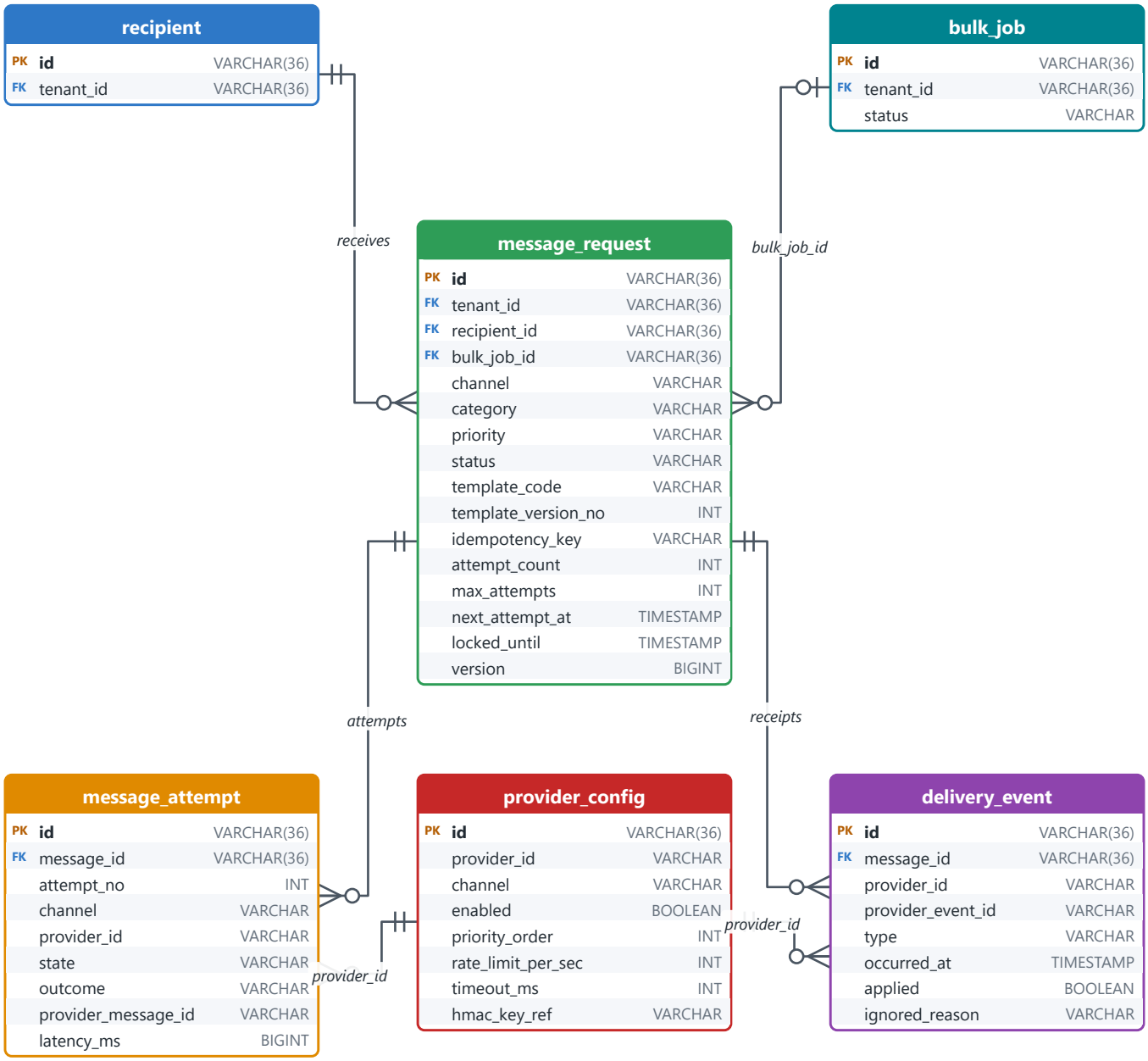


Figure 37. ER detail 2 of 3: the message and what hangs off it. UNIQUE(tenant_id, idempotency_key) on message_request, UNIQUE(message_id, attempt_no) on message_attempt, UNIQUE(provider_id, provider_event_id) on delivery_event. The template is referenced logically by (template_code, template_version_no, template_locale). Implementation: backend/src/main/resources/db/migration/V3__messages.sql

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.



Figure 38. ER detail 3 of 3: bulk jobs and their ledgers, plus the reliability and compliance tables (outbox, idempotency, dedupe, replay guard, audit, legal hold). outbox_event.aggregate_id and processed_message.dispatch_id are logical references only. Implementation: backend/src/main/resources/db/migration/V4__bulk_audit_idempotency_outbox.sql, V200__bulk_ledger_legal_hold.sql

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

15.3 Table catalogue

Table	Purpose	Constraints that carry business meaning
`tenant`	Tenant root	`status`
`recipient`	Synthetic recipients	`UNIQUE(tenant_id, external_ref)`; `deleted` flag; `version`
`recipient_preference`	Marketing opt-in, channel flags, quiet hours, fallback order	One row per recipient; `version` for optimistic locking
`suppression`	Hard blocks by address hash	`UNIQUE(tenant_id, channel, address_hash)`
`template`	Template identity and category	`UNIQUE(tenant_id, code, channel)`
`template_version`	Versioned, localised content	`UNIQUE(template_id, version_no, locale)`
`message_request`	The message and its lifecycle	`UNIQUE(tenant_id, idempotency_key)`; `status`, `attempt_count`, `max_attempts`, `next_attempt_at`, `locked_until`, `channel_used`, `bulk_job_id`, `dlq_reason`, `discarded`; `version`
`message_attempt`	One row per provider attempt	`UNIQUE(message_id, attempt_no)`; `state` STARTED or FINISHED; outcome, provider message id, error code, latency
`provider_config`	Provider routing and limits	`UNIQUE(provider_id)`; `priority_order`, `rate_limit_per_sec`, `timeout_ms`, `hmac_key_ref`, `version`
`delivery_event`	Normalised provider receipts	`UNIQUE(provider_id, provider_event_id)`; `applied`, `ignored_reason`, `raw_ref`
`bulk_job`	Bulk job and counters	`version`; counters total, processed, succeeded, failed, duplicate
`bulk_job_failure`	Per-line failures	Unique `(job_id, line_no)`; `line_no = 0` means a job-level failure
`bulk_job_row`	Row ledger	Primary key `(job_id, external_ref)`
`audit_event`	Append-only audit	Indexes by entity and by tenant and time
`idempotency_record`	API idempotency	`UNIQUE(tenant_id, idem_key)`; `IN_PROGRESS` or `COMPLETED`; stored response
`processed_message`	Consumer dedupe	`dispatch_id` unique
`outbox_event`	Transactional outbox	`status` NEW or SENT; `available_at` for deferred rows
`replay_guard`	Callback replay memory	`event_id` primary key
`legal_hold`	Retention exemption	Scope TENANT or MESSAGE; active while `released_at IS NULL`

Beyond the contract's column list, `message_request` gained `client_reference`, `options_json`, `metadata_json`, `fallback_channels`, `fallback_index`, `locked_until`, `dlq_reason`, `discarded` and `status_changed_at`; `provider_config.version`, `message_attempt.state`, `outbox_event.available_at` and `bulk_job.template_code/channel/category` were added too (DEVIATIONS).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

15.4 Access patterns and indexes

The hot queries drive the index set. Plan shapes in `docs/database/query-plans-and-indexes.md` are labelled ILLUSTRATIVE unless a block is marked `EXECUTED`; see `docs/validation/VALIDATION_REPORT.md` for any executed plans.

Pattern	Index
Message list by tenant and status, newest first	`idx_msg_tenant_status_created (tenant_id, status, created_at DESC)`
Recipient history	`idx_msg_recipient_created (recipient_id, created_at)`
Retry poller	`idx_msg_status_next_attempt (status, next_attempt_at)`
Scheduled dispatch	`idx_msg_status_scheduled (status, scheduled_at)`
Stuck sweeper	`idx_msg_status_locked (status, locked_until)` and `idx_msg_status_changed (status, status_changed_at)`
Correlation and bulk	`idx_msg_correlation`, `idx_msg_bulk_job`, `idx_msg_client_reference (tenant_id, client_reference)`
Outbox claim	`idx_outbox_status_available (status, available_at)`, `idx_outbox_aggregate_status`
Audit by entity and action	`idx_audit_entity`, `idx_audit_entity_action`, `idx_audit_tenant_time`
Reconciliation and callbacks	`idx_attempt_provider_msg (provider_id, provider_message_id)`, `idx_event_message`
Retention scans	`idx_msg_created`, `idx_audit_occurred`, `idx_event_received`, `idx_processed_at`, `idx_replay_seen`

15.5 Transactions and the connection pool

The central rule: **no database transaction is held across a provider call.** The consumer commits the claim, calls the provider outside any transaction, then opens a new short transaction for the outcome. HikariPool exhaustion is therefore bounded by DB time, not provider latency. A `DataSource` fault wrapper in `FailureRecoveryIT` proves that a database outage yields 503 with nothing persisted (scenario 6).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

15.6 Other stores

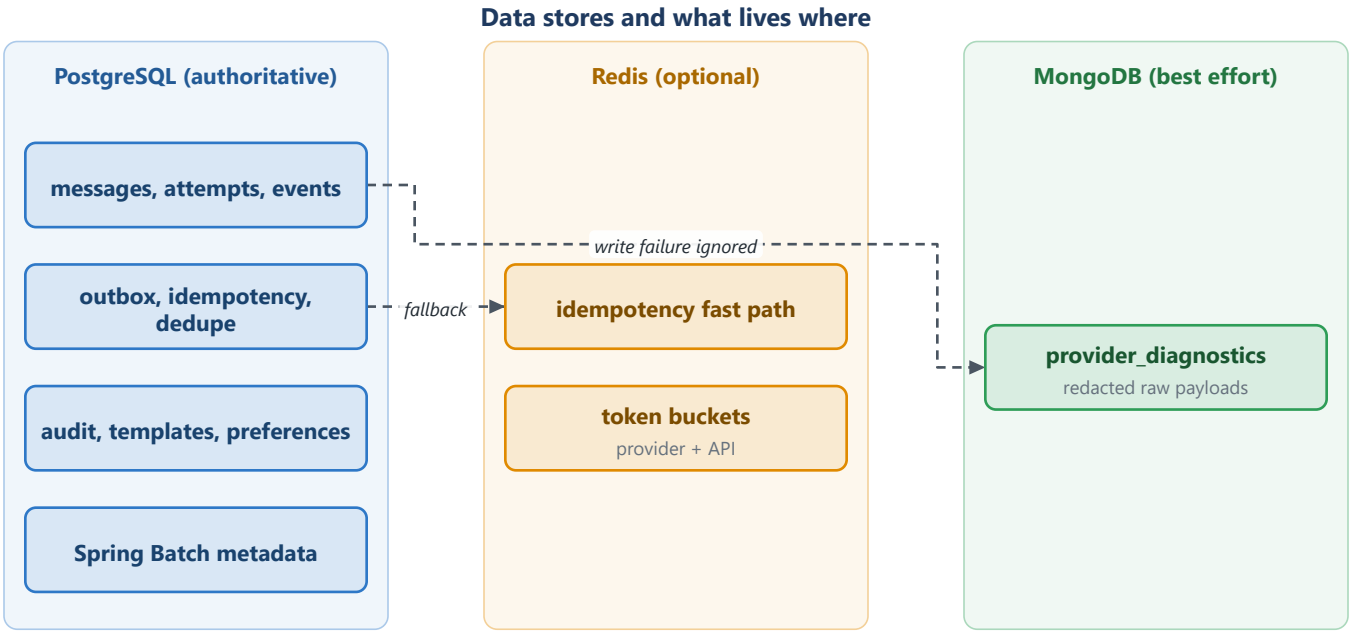


Figure 39. PostgreSQL is the only system of record. Redis is a fast path with database fallback; MongoDB holds redacted provider diagnostics and may be down without failing a message. Implementation: backend/src/main/java/com/example/messaging/coordination/ResilientCoordination.java, backend/src/main/java/com/example/messaging/diagnostics/ProviderDiagnosticsStore.java, docs/database/

15.7 Retention and legal hold

`RetentionJob` (cron `0 30 2 * * *`, batch-size and pause bounded, dry-run supported) purges by age: messages after `message-days` (90), audit after `audit-days` (365), events after `event-days` (30), `processed_message` older than the replay horizon, sent outbox rows, `replay_guard` rows older than `replay-window`, and bulk upload files. A **legal hold** (`legal_hold`, scope TENANT or MESSAGE) exempts data until released. Meters: `messaging.retention.deleted{table}`, `messaging.retention.run{dryRun}`; audit action `RETENTION_PURGE`.

Reference material

Data model

Source document: `docs/database/data-model.md`

Authoritative DDL: Flyway migrations in `backend/src/main/resources/db/migration` (see source). This document explains purpose, constraints and access patterns. ER diagram: `docs/architecture/er-diagram.md`. Portable-SQL rule (ADR-0005): types `VARCHAR`, `BIGINT`, `INT`, `BOOLEAN`, `TIMESTAMP`; no partial indexes, JSONB or `ON CONFLICT`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Conventions

- `id VARCHAR(36)` UUID (`common.Ids`). Timestamps are UTC `TIMESTAMP` (`created_at`, `updated_at`) written from the injected `Clock`.
- `version BIGINT` (optimistic locking, JPA `@Version`) on: `recipient`, `recipient_preference`, `template_version`, `message_request`, `bulk_job`.
- JSON-ish columns (`payload_json`, `variables_json`, `detail_json`) are text with application-side validation.
- Tenant-owned tables carry `tenant_id` and every repository method used by the web layer filters on it (ADR-0011).

Table catalogue

Table	Purpose	Key constraints	Written by	Retention
`tenant`	Business-unit boundary	PK id	admin/seed	permanent
`recipient`	Synthetic person and addresses	UNIQUE(tenant_id, external_ref)	seed/API	`deleted` soft flag; purge per policy
`recipient_preference`	Consent, channel enablement, quiet hours	PK recipient_id, `version`	PreferenceService	with recipient
`suppression`	Hard blocks by address hash	UNIQUE(tenant_id, channel, address_hash)	callbacks, operators	long (reputation)
`template`	Template identity	UNIQUE(tenant_id, code, channel)	TemplateService	permanent
`template_version`	Versioned content per locale	UNIQUE(template_id, version_no, locale)	TemplateService	permanent (audit of what was sent)
`message_request`	The message and its lifecycle	UNIQUE(tenant_id, idempotency_key); `version`	NotificationService, DispatchService, callbacks	`messaging.retention.message-days`
`message_attempt`	One provider try	UNIQUE(message_id, attempt_no)	DispatchService	with message
`provider_config`	Provider enablement and limits	UNIQUE(provider_id)	PLATFORM_ADMIN	permanent
`delivery_event`	Normalized callback events	UNIQUE(provider_id, provider_event_id)	DeliveryEventProcessor	`event-days`
`bulk_job`, `bulk_job_failure`	Bulk send tracking	PK; `version`	batch	with message retention
`audit_event`	Append-only audit	no update/delete by app	AuditService	`audit-days` (longest)
`idempotency_record`	API dedupe	UNIQUE(tenant_id, idem_key)	IdempotencyService	`expires_at`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Table	Purpose	Key constraints	Written by	Retention
`processed_message`	Consumer dedupe	UNIQUE(dispatch_id)	DispatchConsumer	short (days)
`outbox_event`	Reliable publish	status NEW/SENT	submission, OutboxPublisher	purge SENT after days
`replay_guard`	Callback replay cache	PK event_id	ReplayGuard	short (>= tolerance window, hours)
`BATCH_*`	Spring Batch metadata	per Spring	Spring Batch	with job history

Indexes (CONTRACT 4) and their query

Index	Serves
`message_request(tenant_id, status, created_at DESC)`	message list filtered by status, console dashboards
`message_request(recipient_id, created_at)`	recipient history
`message_request(status, next_attempt_at)`	retry poller
`message_request(status, scheduled_at)`	scheduled poller
`message_request(correlation_id)`	support lookup
`message_request(bulk_job_id)`	bulk job drill-down
`audit_event(entity_type, entity_id, occurred_at)`	`/history` timelines
`audit_event(tenant_id, occurred_at)`	audit search

Further: the unique constraints above double as indexes. Candidate indexes not in CONTRACT: `outbox_event(status, created_at)`, `delivery_event(message_id)`, `message_attempt(message_id)` covered by unique, `suppression` lookup by hash covered by unique. See `query-plans-and-indexes.md`.

State and invariants (enforced where)

Invariant	Enforced by
status transitions legal	`MessageStateMachine` + CAS updates (not a DB check constraint, because the table is data in code)
one attempt number per message	UNIQUE(message_id, attempt_no)
one provider event applied once	UNIQUE(provider_id, provider_event_id)
idempotent submission	UNIQUE(tenant_id, idempotency_key), UNIQUE(tenant_id, idem_key)
consent not stored with content	variables stored, rendered body never stored
append-only audit	no update/delete repository methods; DB privileges recommended in prod (REVOKE UPDATE, DELETE)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Transactional boundaries

- Submission: one transaction (message, outbox, idempotency, audit).
- Dispatch: separate short transactions: (1) dedupe+CAS to PROCESSING, (2) attempt STARTED, (3) outcome+transition. **No transaction is held across the provider call.**
- Callback: one transaction per event (insert event + CAS transition + audit).
- See `transactions-and-connection-pooling.md`.

Seed data (profiles local, test only)

2 tenants (`tenant-retail`, `tenant-wealth`), 60 synthetic recipients, 8 templates (en/es), 4 provider configs. No real data.

Query plans and indexes

Source document: `docs/database/query-plans-and-indexes.md`

Status of the plans below: ILLUSTRATIVE. They show the shape a planner is expected to choose and are not output from an executed run unless a block is explicitly labelled `EXECUTED (engine, date)`. Executed H2/PostgreSQL plans, if produced later, are to be appended under "Executed plans" with the exact command. No PostgreSQL instance was available during authoring.

Hot queries

#	Query intent	Frequency	Path
Q1	Retry/scheduled poller: due messages	every 200 ms to 1 s	`status`, `next_attempt_at`
Q2	Outbox claim	every 200 ms	`outbox_event` NEW
Q3	Message list by tenant+status, newest first, paged	UI	`(tenant_id, status, created_at DESC)`
Q4	Recipient history	support	`(recipient_id, created_at)`
Q5	Message history timeline	UI	audit + attempts + events by message id
Q6	Sweeper: PROCESSING with stale lock	every N s	`status` (+ `locked_until`, see source)
Q7	Ops summary counts last 24h	dashboard (cache!)	tenant, created_at, status

Q1: due retries (illustrative)

```sql
SELECT id FROM message_request
WHERE status = 'RETRY_PENDING' AND next_attempt_at <= :now
ORDER BY next_attempt_at
FETCH FIRST 200 ROWS ONLY;   -- LIMIT 200 on PG/H2
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Illustrative plan (PostgreSQL):

```
Limit
  -> Index Scan using ix_msg_status_next_attempt on message_request
       Index Cond: ((status = 'RETRY_PENDING') AND (next_attempt_at <= $1))
```

Trade-off: non-partial index `(status, next_attempt_at)` includes all terminal rows (95%+ of the table). Selectivity on `status='RETRY_PENDING'` is high so it works, but the index is large. PostgreSQL partial index (`WHERE status IN ('RETRY_PENDING','VALIDATED')`) would be smaller but is banned by the portable-SQL rule; Oracle can emulate via function-based index returning NULL for terminal statuses. Revisit if poller latency grows. Mitigation: purge terminal rows by retention.

Q3: tenant list (illustrative)

SQL

```
SELECT ... FROM message_request
WHERE tenant_id = :t AND status = :s
ORDER BY created_at DESC LIMIT 50 OFFSET 0;
```

```
Limit -> Index Scan Backward using ix_msg_tenant_status_created
```

Matches index order, no sort. Without `status` filter the planner uses the index prefix `tenant_id` then a sort (or a different index): add `(tenant_id, created_at DESC)` only if that query dominates (cost: write amplification). `OFFSET` deep paging cost grows linearly: recommend keyset pagination for exports (see oracle-mapping).

Q5: history, avoiding N+1

Bad: load message, then `attempts` lazily, then each event lazily. Good: three indexed queries by message id (or one `UNION ALL` projection) merged in memory:

SQL

```
SELECT 'AUDIT' k, occurred_at ts, action label FROM audit_event WHERE entity_type='MESSAGE' AND entity_id=:id
UNION ALL SELECT 'ATTEMPT', started_at, outcome FROM message_attempt WHERE message_id=:id
UNION ALL SELECT 'EVENT', occurred_at, type FROM delivery_event WHERE message_id=:id
ORDER BY ts;
```

Needs `delivery_event(message_id)` index (not in CONTRACT list; add in a migration, see source). List endpoints use `@EntityGraph`/projections (no lazy loading in loops); verify with Hibernate statistics in a test asserting statement count.

Index trade-offs

Index	Benefit	Cost
Unique `(tenant_id, idempotency_key)`	correctness + lookup	insert cost, required
`(tenant_id, status, created_at DESC)`	main list	updated on every status change (status is in the key): high churn on hot rows; HOT updates impossible when indexed column changes (PostgreSQL), so expect bloat and tune autovacuum

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Index	Benefit	Cost
`(status, next_attempt_at)`	pollers	same churn
`(status, scheduled_at)`	scheduled poller	same churn
`(correlation_id)`	support lookups	small
`audit_event` indexes	timelines	audit table grows fastest; partition by month in prod

Rule of thumb adopted: each additional index on `message_request` needs a named query and an owner; removal candidates are reviewed quarterly using `pg_stat_user_indexes` (`idx_scan = 0`).

Verification commands (to run when PostgreSQL available; NOT executed here)

```sql
EXPLAIN (ANALYZE, BUFFERS) <query>;
SELECT relname, idx_scan FROM pg_stat_user_indexes ORDER BY idx_scan;
SELECT n_dead_tup, last_autovacuum FROM pg_stat_user_tables WHERE relname='message_request';
```

H2: `EXPLAIN ANALYZE SELECT ...` (plan text differs; use only to catch full scans).

Executed plans

None recorded at the time of writing.

Transactions and connection pooling

Source document: `docs/database/transactions-and-connection-pooling.md`

Isolation

Operation	Isolation	Technique	Why
Submission	READ COMMITTED (PG default)	unique constraint on idempotency; single tx	uniqueness gives serializability where needed without SERIALIZABLE retries
Status transitions	READ COMMITTED	CAS `UPDATE ... WHERE id=? AND status=?` check affected rows == 1, plus `@Version`	cheap, no row locks held beyond the statement
Outbox claim	READ COMMITTED	CAS update of a batch of ids (or `FOR UPDATE SKIP LOCKED` as optimization, not on H2 reliably: see source)	multiple instances without double claim
Callback apply	READ COMMITTED	insert unique event, CAS	duplicate = unique violation, treated as success
Reports/summary	READ COMMITTED (read-only tx)	read replica optional	slight staleness fine

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Operation	Isolation	Technique	Why
Retention deletes	READ COMMITTED	chunked	bounded lock footprint

Not used: SERIALIZABLE (retry complexity), REPEATABLE READ (PG snapshot could mask CAS expectations; unnecessary). H2 `MODE=PostgreSQL` default is READ COMMITTED; MVCC differences: assert behavior with `*ContainerIT` on real PG (not executed here).

Anti-patterns avoided: holding a transaction across the provider call; `SELECT` then `UPDATE` without CAS (lost update); catching `DataIntegrityViolationException` inside the same transaction and continuing (PG aborts the transaction: use a new transaction or check-then-insert in a savepoint, see source `IdempotencyService`/ `ReplayGuard`).

Why no transaction over the provider call

Provider latency can be seconds; holding a DB connection that long would exhaust the pool (pool size is small) and amplify a provider slowdown into a DB-wide outage. Dispatch uses three short transactions (claim, attempt STARTED, outcome).

HikariCP sizing

Formula (starting point, then measure): `pool = (core_count * 2) + effective_spindle_count`, but for this app size from concurrency:

Concurrent DB users per instance = API threads actively in DB + consumer workers + schedulers + batch threads, but each holds a connection only during short transactions.

Source	Concurrency	Connection hold
Tomcat request threads	e.g. 50 max, ~20 in DB at once at peak	~5-20 ms
Dispatch workers	`dispatch.concurrency` = 8	2 short tx per message
Schedulers (outbox, poller, sweeper, retention)	~4	short
Batch	`bulk.concurrency` = 4	chunk tx up to 1-2 s (500 rows)

Illustrative sizing: `maximum-pool-size` 20-30 per instance, `minimum-idle` = max (fixed-size pool recommended), `connection-timeout` 2-3 s (fail fast, return 503 rather than queue clients forever), `max-lifetime` below DB/proxy idle kill (e.g. 20 min), `leak-detection-threshold` 10-20 s in non-prod. Total across instances must stay under PG `max_connections` minus headroom: `instances * pool <= ~70% of max_connections` (or use PgBouncer in transaction mode; note prepared statements and session state caveats).

Little's law check: connections needed = throughput (tx/s) x avg hold time (s). 1000 tx/s x 10 ms = 10 connections. If hold time grows 10x (DB slow), required connections 10x: pool saturates, so alert on `hikaricp_connections_pending > 0` sustained and `hikaricp_connections_acquire_seconds` p99.

Separating pools: bulk/batch and retention use the same DB but may use a dedicated small pool (`maximumPoolSize` 4-5) so that bulk jobs cannot starve API connections (option; see source for whether implemented).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Monitoring

Metric	Meaning	Action
`hikaricp_connections_active`	in use	near max = saturation
`hikaricp_connections_pending`	waiting threads	>0 sustained = pool or DB slow
`hikaricp_connections_acquire_seconds`	wait time	p99 > connection-timeout/2 investigate
`hikaricp_connections_timeout_total`	failures	paging alert
`hikaricp_connections_usage_seconds`	hold time	spikes = slow SQL or tx held too long

Diagnosis commands: `docs/operations/performance-investigation-guide.md`.

Deadlocks and lock waits

Consistent ordering: message row before attempt/event rows; audit inserts last. Unique-violation retries are bounded. PG: `SELECT * FROM pg_locks l JOIN pg_stat_activity a USING (pid) WHERE NOT granted;` to inspect (not executed here).

MongoDB usage

Source document: `docs/database/mongodb-usage.md`

Scope

MongoDB holds one collection, `provider_diagnostics`: sanitized provider request/response snippets per attempt and the archive of raw callback payloads. It never holds lifecycle state, idempotency data, consent, or content bodies. Package `diagnostics` (`ProviderDiagnosticsStore`, `DiagnosticsController`, `GET /api/v1/diagnostics/messages/{id}`, roles OPERATOR/PLATFORM_ADMIN). Local/test: in-memory `de.bwaldvogel:mongo-java-server` 1.47.0. Prod-like: real MongoDB replica set (**not executed here**).

Why Mongo here and not PostgreSQL

Reason	Detail
Variable shape	Each provider returns a different payload; schema-less storage avoids a migration per vendor field
Volume and value	High write volume, low per-row value, short life: keeps hot OLTP tables and WAL small
Independent failure domain	Diagnostics outage must not affect sending (ADR-0005)
Cheap TTL	TTL index expires documents without cron/DELETE

Honest counter-argument: PostgreSQL JSONB could do this with one less datastore; we chose Mongo to keep the portable-SQL rule (no JSONB) and to demonstrate a polyglot boundary with explicit consistency trade-offs.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Document shape (illustrative)

JSON

```
{
  "_id": "uuid",
  "tenantId": "tenant-retail",
  "messageId": "…",
  "attemptNo": 1,
  "providerId": "sim-sms",
  "kind": "REQUEST|RESPONSE|CALLBACK",
  "providerEventId": "evt-1001",
  "capturedAt": {"$date": "2026-10-18T10:15:30Z"},
  "expireAt":   {"$date": "2026-11-17T10:15:30Z"},
  "payload": { "...": "redacted via common.Redaction" },
  "correlationId": "…"
}
```

Never stored: message body, full phone/email (masked or hashed via `Redaction`), credentials, signature secrets.

Consistency model

- **Best-effort, after the fact**: written after the PostgreSQL transaction outcome is known (or asynchronously). A failed write increments `messaging.diagnostics.write.failures` and is logged at WARN; processing continues.
- No distributed transaction. Possible states: DB has the event but Mongo lacks the document (`delivery_event.raw_ref` dangling), or Mongo has a document for an attempt whose DB transaction rolled back (orphan). Both are tolerated: diagnostics are advisory.
- Reads handle missing documents (`404` or empty list in `/diagnostics/messages/{id}`), never an error.

Reconciliation risks

Risk	Mitigation
Dangling `raw_ref`	UI shows "diagnostics unavailable"; no functional dependency
Orphan documents	TTL expiry; periodic report comparing `messageId` set vs DB (optional job; see source)
Replay depending on Mongo payload	Forbidden: replay and reconciliation use PostgreSQL + provider `lookup` only
Sensitive data leakage	Redaction at write, field allow-list, TTL, access limited to OPERATOR/PLATFORM_ADMIN, tenant filter on read
Clock skew on TTL	TTL uses `expireAt` set by app `Clock`

Indexes

Index	Purpose
`{messageId: 1, capturedAt: 1}`	per-message diagnostics view
`{tenantId: 1, capturedAt: -1}`	tenant-scoped browsing
`{providerId: 1, providerEventId: 1}` (unique, sparse)	dedupe callback archive (best-effort)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Index	Purpose
`{expireAt: 1}` with `expireAfterSeconds: 0`	TTL

Retention: default 30 days (config; illustrative), shorter than `delivery_event` retention.

Migration / evolution

- Schema additions need no migration (additive fields; readers tolerate absence). Use a `schemaVersion` field when payload structure changes.
- Moving to PostgreSQL JSONB or object storage later: dual-write behind `ProviderDiagnosticsStore` port, backfill is optional (data is expendable), cut over reads.
- Moving off Mongo entirely is safe because nothing depends on it for correctness.

Operations

Write concern `w=1` is adequate (data is expendable), read preference primary. Alert on `messaging.diagnostics.write.failures` rate; runbooks treat this as informational unless sustained. Backups are optional for this data.

Retention and archival

Source document: `docs/database/retention-and-archival.md`

Config: `messaging.retention.{message-days,audit-days,event-days}`; executor: `scheduler.RetentionJob`. Values below are illustrative defaults for a reference system, not regulatory advice (actual retention is a compliance decision; see `docs/security/data-minimization-and-retention.md`).

Data	Default	Rationale	Mechanism
`message_request` (+ `message_attempt`) terminal rows	90 days	support window, dispute handling	batched delete by `created_at` and terminal status, chunk 1000, sleep between chunks
`delivery_event`	90 days (`event-days`)	follows message	batched delete
`audit_event`	400 days (`audit-days`)	longer than data it describes	batched delete or archive-then-delete
`idempotency_record`	`expires_at` (e.g. 24-72 h)	clients retry within hours	delete expired
`processed_message`	7 days	must exceed max broker redelivery/replay horizon	delete
`outbox_event` SENT	7 days	debug	delete
`replay_guard`	24 h (>= 2x tolerance)	callback replay window	delete
Mongo `provider_diagnostics`	30 days	TTL index	automatic
`suppression`	indefinite	reputation/regulatory	manual review

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Data	Default	Rationale	Mechanism
`template_version`	indefinite	proof of what was sent	none

Rules

1. Never delete non-terminal messages (`RETRY_PENDING`, `DELIVERY_UNKNOWN`, etc.) regardless of age: they are alerted on instead.
2. Delete children before parents or use cascading only where safe; order: attempts/events, then message, then audit last.
3. Idempotency retention must exceed the outbox/replay horizon, otherwise a replayed old submission could create a new message.
4. `processed_message` retention must exceed maximum possible redelivery delay, including DLQ replay window. Replaying a message creates a new `dispatchId`, so replay is not blocked by dedupe.
5. Deletes are chunked (avoid long locks/bloat): `DELETE ... WHERE id IN (SELECT id ... FETCH FIRST n ROWS ONLY)`, committing per chunk; each run logs counts and emits audit `RETENTION_PURGE` (counts only).

Archival (optional tier)

Before deletion, export aggregates (counts by day/tenant/channel/status/outcome) to a reporting table or object storage as Parquet/CSV; no PII. Full-row archival requires the same controls as production data (encryption, access governance) and is out of scope for the reference.

Partitioning path

When volume makes batched deletes costly, partition `message_request`, `message_attempt`, `delivery_event`, `audit_event` by month and drop partitions. PostgreSQL requires the partition key in unique constraints; the idempotency uniqueness already lives in `idempotency_record` (small, separately purged) which is why message-level uniqueness is not the only line of defense. Oracle interval partitioning: see `oracle-mapping.md`.

Right-to-erasure (design note)

Recipient `deleted` soft flag blocks sending (policy step 1). Hard erasure: null address columns and remove `recipient_preference`, keep `address_hash` suppression rows and anonymized audit records. Not implemented as an automated workflow here; listed in `prod-vs-reference.md`.

Schema evolution

Source document: `docs/database/schema-evolution.md`

Flyway (`backend/src/main/resources/db/migration`, `V<n>__description.sql`). Rolling deployments mean version N and N+1 of the app run against the same schema simultaneously, so every migration must be compatible with both.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Rules

1. Migrations are immutable after merge. Fix forward with a new version.
2. Portable SQL only (H2 PostgreSQL mode + PostgreSQL; Oracle reference DDL updated in the same PR).
3. One concern per migration; small; no data backfill of large tables inside a DDL migration.
4. Seed data only via profile-scoped location (`local`, `test`).
5. Never `DROP`/rename in the same release that stops using the column.

Expand / migrate / contract

Phase	Release	Action	Compatible with old code?
Expand	N	Add nullable column / new table / new index (create index concurrently on PG in prod via a manual runbook step if table is large)	yes
Dual-write	N	App writes old and new	yes
Backfill	N or ops job	Chunked update (`WHERE new_col IS NULL`, 1000 rows, sleep), idempotent, resumable	yes
Switch reads	N+1	App reads new	old N still works because data present
Contract	N+2	Add `NOT NULL`/constraint after backfill verified, drop old column	only once N is retired

Worked example: add `message_request.expires_reason`

1. `V12__add_expires_reason.sql`: `ALTER TABLE message_request ADD COLUMN expires_reason VARCHAR(64);` (nullable, no default) (example number illustrative).
2. Release N writes it; readers tolerate null.
3. No backfill needed. No contract step (stays nullable by design).

Worked example: rename `payload_json` to `variables_json`

Never rename in place. Expand: add `variables_json`; dual-write; backfill with a chunked job; switch reads; contract after one full release: drop `payload_json`.

Risky operations

Operation	Risk	Safer approach
`ADD COLUMN ... NOT NULL DEFAULT` on big table	rewrite/lock (older PG; Oracle fast with default metadata)	add nullable, backfill, add constraint
Add index on `message_request`	blocks writes (non-concurrent)	PG `CREATE INDEX CONCURRENTLY` (outside Flyway transaction; manual/`executeInTransaction=false` script, not portable: document per environment)
Change type	rewrite	add new column, copy, switch
Add unique constraint	fails on duplicates	detect duplicates first with a query in the migration pre-check

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Operation	Risk	Safer approach
Drop column	breaks old pods	wait for N+2

Testing

- Flyway runs on H2 for every integration test context (validates syntax portability).
- A migration test boots an empty DB, migrates to head, and asserts key constraints exist (see source `*MigrationTest`).
- Recommended for prod-like: migrate a snapshot copy with production-like volumes and time it (**not executed here**).

Rollback

Schema rollback is "roll forward". Expand-only migrations are what make an app rollback safe: the previous app version ignores new columns. See `docs/operations/rollback-plan.md`.

Review checklist

Cross-reference: `ai-engineering/standards/db-migrations.md` and `.github/instructions/sql-migrations.instructions.md`.

Oracle mapping

Source document: `docs/database/oracle-mapping.md`

Reference DDL lives in `backend/src/main/resources/db/oracle/` (reference only; **not executed** here, no Oracle instance available). This document lists the mechanical changes from the portable PostgreSQL/H2 schema and the semantic differences to watch.

Type mapping

Portable (PG/H2)	Oracle 19c+	Notes
`VARCHAR(36)` id	`VARCHAR2(36 CHAR)` or `RAW(16)`	keep VARCHAR2 for simplicity; RAW(16) halves index size (needs converter)
`VARCHAR(n)`	`VARCHAR2(n CHAR)`	Oracle treats `''` as NULL: never rely on empty-string vs null distinction; app normalizes blank to NULL
`VARCHAR(4000)` JSON text	`VARCHAR2(4000 CHAR)`; > 4000 bytes use `CLOB`	with `MAX_STRING_SIZE=EXTENDED` 32767 possible. For validated JSON use `CHECK (col IS JSON)` (Oracle-only)
long text (`response_body`)	`CLOB`	avoid selecting CLOBs in list queries; mark `LAZY` in JPA
`BIGINT`	`NUMBER(19)`	
`INT`	`NUMBER(10)`	

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Portable (PG/H2)	Oracle 19c+	Notes
`BOOLEAN`	`NUMBER(1)` with `CHECK (x IN (0,1))`	Hibernate maps via `@Convert` or dialect (Oracle 23ai has native BOOLEAN)
`TIMESTAMP` (UTC)	`TIMESTAMP(6)` or `TIMESTAMP(6) WITH TIME ZONE`	store UTC; use `WITH TIME ZONE` only if you need the original offset (not needed: user time zones live in `recipient.time_zone`). Avoid `TIMESTAMP WITH LOCAL TIME ZONE` surprises (session TZ) unless deliberate. Set `-Doracle.jdbc.timezoneAsRegion=false` and `hibernate.jdbc.time_zone=UTC`
`now()` / `CURRENT_TIMESTAMP`	`SYSTIMESTAMP`	app passes `Clock` values instead: preferred
`DESC` index column	works, creates function-based index	fine

Identity and sequences

Ids are application-generated UUID strings, so no identity columns are needed. For `message_attempt` numbering and any surrogate numeric keys, use `CREATE SEQUENCE ... CACHE 1000` (not `GENERATED ALWAYS AS IDENTITY` if portability to 11g-style matters; identity is fine on 12c+). Hibernate: `@SequenceGenerator(allocationSize=50)` must equal sequence `INCREMENT BY`.

Upsert: MERGE vs portable pattern

Portable code uses "insert, catch unique violation, then update" or CAS `UPDATE ... WHERE`. Oracle option for hot paths:

```sql
MERGE INTO replay_guard t
USING (SELECT :event_id AS event_id FROM dual) s ON (t.event_id = s.event_id)
WHEN NOT MATCHED THEN INSERT (event_id, seen_at) VALUES (s.event_id, SYSTIMESTAMP);
```

PostgreSQL equivalent (not used because of portability rule) is `INSERT ... ON CONFLICT DO NOTHING`. Prefer the unique-violation approach in the app (`DataIntegrityViolationException`): identical semantics on both engines; MERGE under concurrency can still raise ORA-00001 and must be retried.

Pagination

PostgreSQL/H2: `LIMIT :size OFFSET :offset`. Oracle 12c+: `OFFSET :offset ROWS FETCH NEXT :size ROWS ONLY` (Hibernate generates it via `OracleDialect`). Keyset pagination (`WHERE (created_at, id) < (:ts, :id) ORDER BY created_at DESC, id DESC FETCH FIRST :n ROWS ONLY`) is preferred for deep paging; Oracle needs the expanded form `created_at < :ts OR (created_at = :ts AND id < :id)`. Top-N: `FETCH FIRST 1 ROW ONLY`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Concurrency primitives

Need	PostgreSQL	Oracle
Claim batch without blocking	`SELECT ... FOR UPDATE SKIP LOCKED`	same syntax (11g+, officially documented 12c+); our CAS `UPDATE` pattern avoids both
Optimistic lock	`version` column	same
Default isolation	READ COMMITTED (locking reads)	READ COMMITTED with statement-level snapshot (read consistency); `SERIALIZABLE` raises ORA-08177

Hints and plans

Do not embed hints in portable code. On Oracle, if the optimizer picks poorly for the retry poller, prefer fixing statistics (`DBMS_STATS`), adding histograms on `status` (skewed), or SQL Plan Baselines over `/*+ INDEX(...) */`. Bind peeking plus skewed `status` is a classic issue: terminal statuses dominate (>95% of rows). If hints are unavoidable they go in a vendor-specific repository override, not in shared JPQL.

Partitioning (Oracle Enterprise option)

Table	Partitioning	Benefit
`message_request`, `message_attempt`, `delivery_event`, `audit_event`	interval partition on `created_at`/ `occurred_at` by day or month	retention = `DROP PARTITION` instead of mass DELETE; partition pruning for date filters
`outbox_event`, `processed_message`, `replay_guard`	short-lived: interval partition by day	cheap purge

Local indexes are preferred; global unique indexes (e.g., `UNIQUE(tenant_id, idempotency_key)`) must include the partition key or be global (global indexes need maintenance on partition drop: `UPDATE GLOBAL INDEXES`). PostgreSQL can do the same with declarative partitioning, but unique constraints must then include the partition key: a real design constraint if partitioning is adopted (idempotency lookups would need the date or a separate small dedupe table).

Other differences

Topic	PostgreSQL	Oracle
Identifiers	lower case	upper case unless quoted; keep names <= 30 chars for 12.1 compat (128 on 12.2+)
DDL transactionality	transactional	DDL auto-commits; Flyway migrations must be small and idempotent
Boolean/empty string	real boolean; '' != NULL	emulated; '' = NULL
Case-insensitive search	`ILIKE`, `citext`	`UPPER(col) LIKE`, function-based index
Locking readers	readers don't block	readers don't block
Sequences/serial	identity/sequence	sequence/identity
JDBC driver / pool	PG driver	ojdbc11, set `oracle.jdbc.fanEnabled`, UCP optional; Hikari fine

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Topic	PostgreSQL	Oracle
`LIMIT` in subquery	allowed	use `FETCH FIRST` in subquery
String aggregation	`string_agg`	`LISTAGG`

Migration checklist (PG to Oracle)

1. Translate DDL using the table above; run the reference DDL.
2. Switch Hibernate dialect, driver, `hibernate.jdbc.time_zone=UTC`.
3. Re-run integration tests against Oracle (Testcontainers `gvenzl/oracle-free` is an option; **not executed here**).
4. Compare plans for the five hot queries (`query-plans-and-indexes.md`).
5. Validate blank-string handling for optional columns.

ADR-0005: Database choice

Source document: `docs/adr/0005-database-postgresql-vs-mysql-oracle-and-nosql.md`

Status: Accepted (2026-10-18)

Context

Core data is relational and transactional: message state, attempts, idempotency, outbox, audit, templates, preferences. We need strong consistency for state transitions, unique constraints for dedupe, and ad-hoc operational queries. Many enterprises run Oracle; the SQL must stay portable. Provider diagnostics and raw callback payloads are variable-shape, write-heavy, low-value-per-row and expire quickly.

Decision

1. **PostgreSQL 16** is the primary store. Local/test use **H2 2.x in** `MODE=PostgreSQL`.
2. **Portable SQL rule**: Flyway migrations must run on H2 and PostgreSQL: no partial indexes, no JSONB, no `ON CONFLICT`, only `TIMESTAMP`, `VARCHAR`, `BIGINT`, `CLOB`-like `VARCHAR(4000)`. This keeps an **Oracle** port mechanical: see `docs/database/oracle-mapping.md` and the reference DDL in `backend/src/main/resources/db/oracle/`.
3. **MongoDB** is used only for the `provider_diagnostics` collection (raw provider request/response snippets and callback payload archive). It is best-effort: a failed write increments `messaging.diagnostics.write.failures` and never fails message processing. `delivery_event.raw_ref` points at the Mongo document. See `docs/database/mongodb-usage.md`.
4. State and unique-constraint dedupe never live in Mongo or Redis.

Alternatives considered

Option	Pros	Cons	Verdict
MySQL 8	Ubiquitous	Weaker transactional DDL, historically looser constraint behavior, less aligned with Oracle-style analytics SQL	Not chosen; schema would also port

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Option	Pros	Cons	Verdict
Oracle	Common in enterprises; partitioning, advanced features	Licensing, no local parity without containers, proprietary hints/MERGE	Supported via mapping doc, not default
MongoDB for everything	Flexible schema	No multi-document uniqueness+state guarantees as simple as SQL; harder joins for ops views	Rejected for core data
Cassandra/Dynamo	Scale	Weak ad-hoc query, no cross-row transactions	Overkill
PostgreSQL JSONB for diagnostics	One datastore	Violates portable-SQL rule; bloats hot tables	Rejected; Mongo is isolated

Consequences

- Positive: ACID transitions with CAS; H2 gives fast tests; portable DDL.
- Negative: no JSONB convenience (JSON stored as `VARCHAR(4000)` text such as `payload_json`, `detail_json`); upsert is written as update-then-insert or guarded insert; two datastores to operate.
- Risks: H2 vs PostgreSQL divergence (locking, planner): mitigate with `*ContainerIT` PostgreSQL tests in `-Pcontainers` (not executed here) and explicit review of every migration (`ai-engineering/standards/db-migrations.md`).
- Mongo consistency risk: diagnostics can be missing for a message; reconciliation never depends on them.

Validation

- Flyway runs on H2 in every `AbstractIntegrationTest` context. PostgreSQL execution: **NOT executed** here.
- Link: `docs/database/data-model.md`, `docs/database/query-plans-and-indexes.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

16. Security and Audit Architecture

16.1 Security principles

1. **Never trust the network.** Every protected call carries a JWT; callbacks are authenticated by HMAC even though the HTTP layer lets them through.
2. **Tenant isolation by construction.** The tenant comes from the token, never from the request body, and every repository query is tenant scoped.
3. **Least privilege.** Seven roles and one scope; endpoint access is a table, not an if-statement scattered through code.
4. **Minimise data.** No content or personal data on the bus, rendered text not stored, addresses masked in every response, redaction in logs, audit and Mongo.
5. **Secure by default.** No default Spring profile, so a forgotten profile cannot enable the dev token issuer; actuator, OpenAPI docs and Prometheus are restricted unless a local profile opens them.

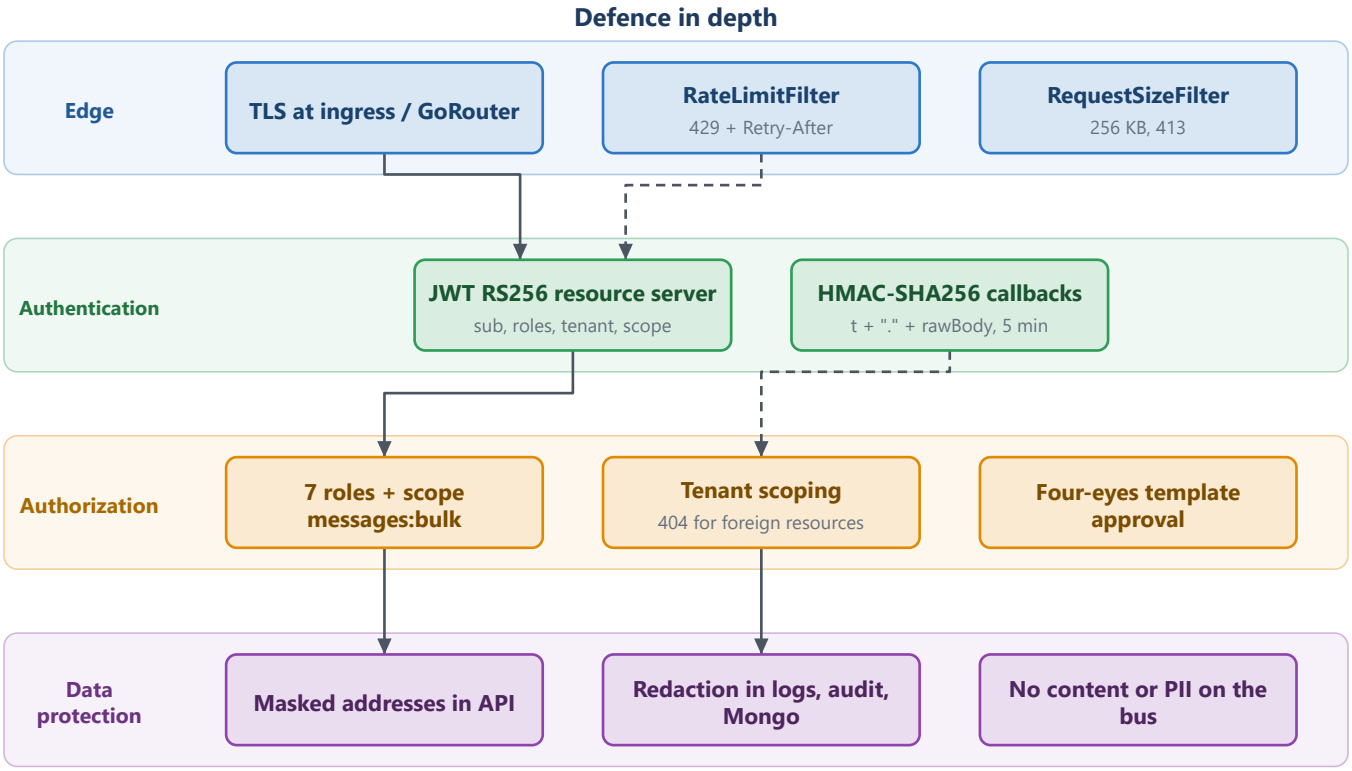


Figure 40. Security controls by layer. Callbacks are permitAll at the Spring Security layer but authenticated by HMAC in the service, so the network is never trusted. Implementation: backend/src/main/java/com/example/messaging/config/SecurityConfig.java, backend/src/main/java/com/example/messaging/security/, backend/src/main/java/com/example/messaging/callback/SignatureVerifier.java, backend/src/main/java/com/example/messaging/common/Redaction.java

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

16.2 Authentication

Surface	Mechanism	Implementation
API	Spring Security 6 OAuth2 resource server, **JWT RS256**. Claims: `sub`, `roles` (array), `tenant` (string; `*` only for platform admin), `scope`	`config/SecurityConfig`, `config/JwtConfig`, `security/JwtAuthConverter`
Production identity	`spring.security.oauth2.resourceserver.jwt.issuer-uri` from the environment	`application-prod-like.yml`
Local identity	`POST /dev/token` mints RS256 JWTs from an in-memory generated key; validated with the matching public key (`NimbusJwtDecoder.withPublicKey`); enabled only when `messaging.security.dev-token-enabled=true` (profiles `local`, `test`)	`security/DevTokenController`; `DevTokenDisabledIT` proves it is off otherwise
Provider callbacks	HMAC-SHA256 (Chapter 12)	`callback/SignatureVerifier`
Identity provider unreachable	An infrastructure failure (for example an unreachable JWKS endpoint) becomes 503 `SERVICE_UNAVAILABLE` with `Retry-After` ; the request is still denied, never treated as authenticated	`security/IdentityProviderUnavailable Filter`

The Spring Security rules, in order (`SecurityConfig`): `OPTIONS` denied; `/api/v1/callbacks/**` and `/actuator/health/**` open; `/actuator/prometheus` restricted to `PLATFORM_ADMIN` and `OPERATOR` unless `metrics-public`; every other `/actuator/**` for `PLATFORM_ADMIN`; `/dev/token` open (and only exists locally); OpenAPI docs open only when `docs-public`; `/api/**` authenticated.

16.3 Authorization

Role matrix

Capability	SENDER	TENANT_ADMIN	PLATFORM_ADMIN	OPERATOR	TEMPLATE_AUTHOR	TEMPLATE_APPROVER	AUDITOR
Submit message, create bulk job (needs scope `messages:bulk`)	yes	yes	yes				
Read messages, status, history	yes	yes	yes	yes	yes	yes	yes
Cancel message	yes	yes	yes	yes			
Requeue, replay or discard dead letter			yes	yes			
View dead letters, failed messages		yes	yes	yes			
Delivery events		yes	yes	yes			yes
Audit events		yes	yes				yes
Create and edit template drafts, submit		yes (create)			yes		
Approve, reject template						yes	

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Capability	SENDER	TENANT_ADMIN	PLATFORM_ADMIN	OPERATOR	TEMPLATE_AUTHOR	TEMPLATE_APPROVER	AUDITOR
Retire template		yes				yes	
Preview template		yes		yes	yes		
Edit preferences, suppressions		yes	yes	yes			
Provider config, pause consumers			yes				
Queue stats			yes	yes			

The table mirrors the endpoint role list of the contract and the `ACCESS` constants of the Angular console; `SecurityMatrixTest` and `SecurityIT` exercise it, and the console's guards are UX only.

Tenant isolation

`TenantContext` holds the tenant claim for the request. Non-admin principals see only their tenant's data; a resource owned by another tenant answers **404, not 403**, so its existence is not disclosed. `PLATFORM_ADMIN` carries `tenant=*` and may pass `X-Tenant-Id` to scope a call. `TenantIsolationIT` proves the 404 behaviour for GET, cancel, history and status; ADR-0011 records the shared-schema decision and lists row-level security as the production hardening.

Object-level checks

`AuthorizationService` performs resource-level checks beyond roles: the tenant owns the template, the approver differs from the author, and the bulk scope is present. A refused bulk action is audited (`BULK_JOB_DENIED`).

16.4 Data protection

Control	Mechanism
Masked responses	`MessageDetail` carries `maskedAddress` only (`r***@example.test`, `+1555***0101`); `common/Redaction.maskEmail`, `maskPhone`, `maskToken`
No content at rest	Variables are stored; the rendered body is not (ADR-0007)
No content on the bus	`DispatchEnvelope` holds identifiers only
Log redaction	`Redaction` masks emails, long numbers and bearer tokens in free text; structured logs carry `correlationId`, `tenant`, `messageId` only; `LoggingPrivacyIT` asserts it
Mongo diagnostics	Redacted payloads only; write failures never fail processing (`messaging.diagnostics.write.failures`)
Free-text vendor reasons	Never copied into audit or log lines; only code-like reasons
Suppression storage	Address hash, not the address
Request limits	256 KB body, rate limit, page size at most 200, bulk row and file limits

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Control	Mechanism
Secrets	Never in files: environment placeholders, `cf set-env` or user-provided services, Kubernetes Secret templates with dummy values; dummy callback secrets are `local-dev-secret-not-for-production`

16.5 Audit architecture

The audit trail is **append-only** and written in the same transaction as the change it records (`service/AuditService`, table `audit_event`: `actor`, `actor_role`, `action`, `entity_type`, `entity_id`, `correlation_id`, `detail_json`, `occurred_at`). Because it shares the transaction, there is no state change without an audit row and no audit row for a change that rolled back.

Group	Actions (selection from `domain/AuditAction`)
Message	`MESSAGE_ACCEPTED`, `MESSAGE_REJECTED`, `MESSAGE_TRANSITION`, `MESSAGE_CANCELLED`, `MESSAGE_REQUEUED`, `MESSAGE_DISCARDED`, `MESSAGE_REPLAYED`, `MESSAGE_FALLBACK`, `REPLAY_PRECHECK`
Delivery	`DELIVERY_EVENT_APPLIED`, `DELIVERY_EVENT_CONFLICT`
Template	`TEMPLATE_CREATED`, `TEMPLATE_VERSION_CREATED`, `TEMPLATE_VERSION_EDITED`, `TEMPLATE_SUBMITTED`, `TEMPLATE_APPROVED`, `TEMPLATE_REJECTED`, `TEMPLATE_RETIRED`
Preferences	`PREFERENCE_UPDATED`, `SUPPRESSION_ADDED`, `SUPPRESSION_REMOVED`
Operations	`PROVIDER_CONFIG_UPDATED`, `CONSUMERS_PAUSED`, `CONSUMERS_RESUMED`, `STUCK_SWEEP`, `RECONCILIATION_CHECK`
Bulk and retention	`BULK_JOB_CREATED`, `BULK_JOB_STOPPED`, `BULK_JOB_RESTARTED`, `BULK_JOB_DENIED`, `BULK_JOB_ABANDONED`, `RETENTION_PURGE`

Reads: `GET /api/v1/audit-events` (AUDITOR, TENANT_ADMIN, PLATFORM_ADMIN; filters `entityType`, `entityId`, `action`, `actor`, `from`, `to`) and the merged per-message timeline `GET /api/v1/messages/{id}/history`, which interleaves audit events, attempts and delivery events in time order. Retention keeps audit 365 days by default, with legal hold support (Chapter 15).

16.6 Threats and where they are addressed

Threat	Control	Test or evidence
Forged callback	HMAC with 5-minute tolerance, constant-time compare	`SignatureVerifierTest`, `CallbackApiIT`, AC-12
Replayed callback	Replay guard, unique event key	`ReplayGuardTest`, scenario 3
Cross-tenant access (BOLA)	Tenant scoped queries, 404 policy	`TenantIsolationIT`
Template injection	Allow-listed grammar, no expressions	`TemplateRendererTest`
Header injection in email	CR and LF stripped, custom header regex	`TemplateRendererTest`, validation in `NotificationService`
PII in logs	Redaction, no content stored	`LoggingPrivacyIT`, `RedactionTest`
Abuse and resource exhaustion	Rate limit, body size, bounded pools, backpressure	`RateLimitIT`, `BackpressureIT`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Threat	Control	Test or evidence
Token forgery	RS256 signature validation, issuer in production	`JwtAuthConverterTest`, `SecurityIT`
Dev endpoints in production	No default profile, flag, `DevTokenDisabledIT`	`DevTokenDisabledIT`
Privilege escalation through template edit	`created_by` overwritten by editor, four eyes	`TemplateLifecycleIT`

16.7 Honest boundary

This is a reference implementation. `docs/security/prod-vs-reference.md` lists what a production deployment still needs (row-level security, a real identity provider, WAF, secret manager, mutual TLS, formal pen testing, compliance review). CI definitions for OWASP dependency-check, npm audit and Trivy exist but were **not executed**.

Reference material

Threat model

Source document: `docs/security/threat-model.md`

Scope: the messaging platform API, dispatch pipeline, callbacks, console, data stores. Synthetic data; reference implementation. Test locations: `tests/security` (HTTP-level scripts) and `backend/src/test/java/.../security|callback|web` (see source for exact class names). STRIDE letters: S spoofing, T tampering, R repudiation, I information disclosure, D denial of service, E elevation of privilege.

Assets and trust boundaries

Assets: message content (variables, rendered text in flight), recipient addresses and consent, provider credentials and HMAC secrets, audit trail, template content, availability of critical alerts. Boundaries: internet to edge; edge to app; app to data stores; app to providers; providers to callback endpoint.

Threat table

#	Threat	STRIDE	Attack example	Mitigation	Code location	Test
T1	Unauthorized submission	S, E	Call `POST /messages` without token, expired token, or with `ROLE_AUDITOR`	Resource-server JWT (RS256, issuer/audience/exp validated); URL role rules; method security	`config.SecurityConfig`, `web.MessageController`	`tests/security`: no token 401; wrong role 403; expired 401
T2	Cross-tenant access (BOLA)	I, T	Tenant B requests tenant A message id, template, job, recipient	Tenant from JWT claim only; `findByIdAndTenantId`; 404 not 403; admin scoping via `X-Tenant-Id` only for `tenant=*`	`security.TenantContext`, `security.AuthorizationService`, `repository`	cross-tenant matrix on every id-based endpoint expects 404

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Threat	STRIDE	Attack example	Mitigation	Code location	Test
T3	Forged callbacks	S, T	Attacker POSTs fake DELIVERED to `/callbacks/{providerId}`	HMAC-SHA256 over `t.rawBody`, constant-time compare, key by `X-Provider-Key-Id`, 5 min tolerance	`callback.SignatureVerifier`	`SignatureVerifierTest`; `tests/security` bad/missing/expired signature expects 401 + `messaging.callbacks{result="invalid_signature"}`
T4	Callback and request replay	S, T	Capture a valid signed callback and resend	`X-Event-Id` replay cache, unique `(provider_id, provider_event_id)`, timestamp window	`callback.ReplayGuard`, `callback.DeliveryEventProcessor`	`ReplayGuardTest`; `FailureRecoveryIT#scenario03_...`
T5	Credential exposure	I	Secrets in git, logs, images, error messages	No secrets in files (dummy dev secrets only, labelled), env/secret store, `Redaction`, log scrubbing, secret scanning in CI (config only, not executed here)	`common.Redaction`, `config.PropertiesConfig`, `docs/security/secrets-management.md`	`RedactionTest`; repo grep rule in review checklist
T6	Content leakage	I	Rendered body or address in logs, API, Mongo, bus	Envelope has ids only; variables stored not body; API returns masked addresses; Redaction in diagnostics; no body at INFO	`messaging.DispatchEnvelope`, `web` DTOs, `diagnostics`	contract test: `MessageDetail` JSON has no `body`/full address; log capture test for PII patterns
T7	Template injection	T, E, I	`{{` expressions, HTML/JS in variables, SpEL/OGNL gadget	Allow-listed `{{var}}` substitution, no evaluation, HTML escape for email HTML, variable allow-list and length limits, four-eyes approval	`template.TemplateRenderer`, `template.VariableValidator`, `service.TemplateService`	`TemplateRendererTest` (script tags, nested braces, unknown vars, 10k-char value)
T8	Duplicates (accidental or malicious)	T, D	Replaying POST to spam a recipient	Idempotency key, rate limits, per-recipient frequency policy (config, see source)	`service.IdempotencyService`, `security.RateLimitFilter`	`FailureRecoveryIT#scenario02_...`
T9	Abuse / DoS	D	Flood API, huge bodies, giant bulk CSV, slow loris, provider exhaustion	256 KB body limit (413), per-client token bucket 429, queue-depth backpressure 503, bulk `max-rows`, bounded executors, provider rate limiter, timeouts	`security.RequestSizeFilter`, `security.RateLimitFilter`, `messaging`, `batch`	`RateLimitFilterTest`, `RequestSizeFilterTest`, `FailureRecoveryIT#scenario10_...`
T10	Privilege escalation	E	SENDER calls `/ops/consumers/pause`, edits provider config, approves own template, sets `tenant` claim	Role matrix, four-eyes (approver != author), fault-mode endpoint disabled outside local/test, dev token disabled outside local/test, JWT signature verification	`SecurityConfig`, `AuthorizationService`, `security.DevTokenController` (profile-gated)	`tests/security`: role matrix; `DevTokenControllerProfileTest` (prod-like profile returns 404)
T11	Unauthorized bulk sends	E, T	Low-privilege caller uploads CSV to blast recipients	`ROLE_SENDER` + scope `messages:bulk`, tenant scoping of recipients, max rows, rate cap, consent policy re-applied at dispatch, audit with `created_by`	`batch.BulkJobController`, `service.PolicyService`	`BulkJobSecurityTest` (missing scope 403), policy tests for opted-out rows

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Threat	STRIDE	Attack example	Mitigation	Code location	Test
T12	Repudiation	R	Operator replays/discards DLQ then denies	Append-only `audit_event` with actor, role, correlation id; no update/delete in app; DB privileges in prod	`service.AuditService`	audit assertion in replay/discard/cancel tests
T13	Tampering with state	T	Operator forces illegal transition	State machine + 409; ops actions limited to allowed edges	`domain.MessageStateMachine`	`MessageStateMachineTest`
T14	Log injection / header injection	T	Newline in `X-Correlation-Id`	Correlation id validated (charset, length) else regenerated	`common.CorrelationIdFilter`	`CorrelationIdFilterTest`
T15	SSRF	I	Callback or channelOptions URL causes server fetch	No server-side fetch of caller-supplied URLs; provider endpoints from config only	`provider`	code review rule
T16	Mass assignment	T, E	Client sets `status`, `tenantId` in request body	DTO records with explicit fields; unknown properties rejected	`web` DTOs	`MALFORMED_REQUEST` test
T17	Dependency/ supply chain	T	Vulnerable library	Spring Boot BOM pinned, dependency scan in CI (config only, not executed)	`pom.xml`, `.github/workflows`	not executed
T18	Sensitive actuator exposure	I	Public `/actuator/prometheus`, heap dump	Only health + prometheus exposed, prometheus network-restricted in prod	`application*.yml`, network policy	static check

Residual risks

Documented in `prod-vs-reference.md`: no WAF, no real KMS, no mTLS enforced by default, JWT revocation not implemented, no automated pen-test, no DLP.

Review cadence

Update on every new endpoint (checklist `ai-engineering/checklists/security-review.md`). Sensitive changes need independent review (multi-agent workflow).

OWASP API Top 10 (2023) analysis

Source document: `docs/security/owasp-api-top10-2023-analysis.md`

Self-assessment of the reference implementation against the 2023 list. "Implemented" means designed in CONTRACT and implemented in source by the backend agent (verify in code; status of tests in `docs/validation/VALIDATION_REPORT.md`). This is not a certification or pen-test.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ID	Risk	Exposure here	Controls	Gaps / notes
API1:2023	Broken Object Level Authorization	Message, template, job, recipient ids in paths	Tenant from JWT; tenant-scoped repositories; 404 for foreign; UUID ids (non-guessable but not relied upon)	RLS in PostgreSQL not used (portable SQL): defense in depth missing
API2:2023	Broken Authentication	Bearer JWT; dev token endpoint; callbacks via HMAC	RS256 validation (issuer-uri in prod); `/dev/token` only `local`/`test`; short-lived tokens; HMAC for callbacks	No token revocation, no refresh handling (IdP responsibility); no brute-force controls (no passwords here)
API3:2023	Broken Object Property Level Authorization	DTO exposure and mass assignment	Response DTOs never include rendered body, full address; masked addresses; request DTOs whitelist fields; `status`/`tenantId` not writable	Field-level role differences (e.g., auditor vs operator) limited
API4:2023	Unrestricted Resource Consumption	Submit floods, bulk CSV, page size, provider spend	256 KB body limit, rate limit 50 rps / burst 100, page size <= 200, bulk max rows and rate, queue backpressure 503, bounded executors, timeouts	No per-tenant monthly quota/cost cap (listed in prod gap)
API5:2023	Broken Function Level Authorization	Ops, provider config, fault mode, replay	Role matrix in CONTRACT 5; admin endpoints `PLATFORM_ADMIN`; fault-mode local/test only	Role matrix tested per endpoint in `tests/security`
API6:2023	Unrestricted Access to Sensitive Business Flows	Bulk marketing sends, OTP/security alert abuse	Bulk needs scope; consent policy; per-recipient frequency (see source); idempotency; audit	No CAPTCHA/anti-automation (not user-facing); anomaly detection absent
API7:2023	Server Side Request Forgery	Provider URLs	No caller-supplied URLs fetched; provider endpoints from config	Egress allow-list is a deployment control (not executed)
API8:2023	Security Misconfiguration	Actuator, CORS, error detail, headers, profiles	Minimal actuator exposure; `ApiError` without stack traces; CORS restricted to console origin (see source); security headers (see source); profile separation; no default creds	TLS terminates at ingress (not configured here); verify headers in `tests/security`
API9:2023	Improper Inventory Management	Versions, debug endpoints	Single `/api/v1`; OpenAPI committed and asserted against springdoc; dev endpoints profile-gated; `API-Version` header	Old version retirement process described in `docs/leadership/api-contract-guide.md`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ID	Risk	Exposure here	Controls	Gaps / notes
API10:2023	Unsafe Consumption of APIs	Provider responses and callbacks treated as untrusted	Callback HMAC + schema mapping; response size/time limits; outcome enum mapping (unknown => AMBIGUOUS, not success); provider data never executed or rendered unescaped	Real vendors' TLS pinning, IP allow-lists not modeled

Other relevant practices

- Input validation: Bean Validation on DTOs, max lengths, enum parsing returns `MALFORMED_REQUEST`.
- Error handling: consistent `ApiError`, no internal details, `correlationId` for support.
- Logging: structured JSON, MDC `correlationId`, `tenant`, `messageId`; PII redaction (`common.Redaction`).
- Security testing: `tests/security` scripts, unit tests; SAST/DAST/dependency scanning pipeline stages are configured statically, not executed here.

Service-to-service authentication

Source document: `docs/security/service-to-service-auth.md`

Inbound: calling services (SENDER clients)

Aspect	Design
Mechanism	OAuth 2.0 **client credentials** grant against the enterprise IdP; the platform is an OAuth2 resource server validating RS256 JWTs (Spring Security 6)
Claims	`sub` (client id), `roles` (array, e.g. `ROLE_SENDER`), `tenant` (one tenant id), `scope` (e.g. `messages:send`, `messages:bulk`), `iss`, `aud`, `exp`
Token lifetime	Short (illustrative: 5-15 min); clients cache and refresh before expiry
Validation	issuer-uri discovery (prod), signature, `exp`/`nbf`, audience (configure `aud` check; see source `SecurityConfig`/`JwtConfig`), clock skew 60 s
Authorization	tenant + roles + scopes (ADR-0011)
Local	`POST /dev/token` mints RS256 tokens from an in-memory key for personas; disabled outside `local`/`test` (T10)
Rate limiting	per `sub` token bucket

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Example client call (placeholders, no real secrets):

```
POST {idp}/oauth2/token  grant_type=client_credentials  scope=messages:send   (client authenticates with
  private_key_jwt or secret from its own secret store)
POST /api/v1/messages   Authorization: Bearer <access_token>   Idempotency-Key: <uuid>
```

Prefer `private_key_jwt` or mTLS-bound tokens (RFC 8705) over shared client secrets.

Optional: mTLS

Option	When	Notes
TLS at ingress only	default reference	simplest; clients identified by JWT
mTLS at ingress/mesh (client certs)	regulated environments, internal service mesh	cert identity (SPIFFE/URI SAN) mapped to client; combine with JWT (defense in depth); certificate rotation automation required; certificate-bound access tokens (`cnf` claim) prevent token theft replay

Not implemented/executed in this repo; an Ingress annotation/mesh policy example may exist under `infrastructure/` (static only).

Outbound: platform to providers

Aspect	Design
Credentials	Per-provider API key/OAuth client from the secret store; never in `provider_config` (only `hmac_key_ref`)
Transport	TLS 1.2+ with default trust store; certificate pinning not modeled
Timeouts	`messaging.providers.<id>.timeout-ms`; connect and read timeouts distinct (see source)
Network	Egress allow-list / proxy
Idempotency	`idempotencyRef` sent in header/body where vendor supports

Simulators are in-process and need no credentials.

Inbound: provider callbacks

Authenticated by HMAC (`X-Signature: t=<epoch>,v1=<hex>` over `t + "." + rawBody`), `X-Provider-Key-Id` selects the secret, tolerance 5 min, replay cache on `X-Event-Id`. Optional additions in prod: provider source IP allow-list at edge, mTLS if the vendor supports. Callback endpoints are `permitAll` in the filter chain by design and protected by the verifier: tests must prove that an unsigned request never reaches `DeliveryEventProcessor`.

Internal components

Within the single JVM: no network auth. Between the platform and data stores: per-store credentials, TLS (`prod-like`), least-privilege DB user (DML only for the app; DDL by a separate migration user used by Flyway in CI/CD), Rabbit user limited to the messaging vhost, Redis ACL user limited to key prefix `msg:*`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Threats addressed

T1, T3, T4, T10 in `threat-model.md`.

Secrets management

Source document: `docs/security/secrets-management.md`

Inventory

Secret	Used by	Local value	Prod source
DB credentials	`spring.datasource.*`	H2 in-memory (none)	Secret store injected as env vars
RabbitMQ credentials	`RabbitMessageBus`	n/a (memory bus)	Secret store
Redis password	coordination adapter	n/a	Secret store
Mongo URI/credentials	`MongoConfig`	in-memory	Secret store
Callback HMAC secrets per `keyId`	`callback.SignatureVerifier`	`local-dev-secret-not-for-production` (dummy)	Secret store, referenced by `provider_config.hmac_key_ref`
Provider API credentials	provider adapters	none (simulators)	Secret store
JWT signing key	IdP (prod); dev: in-memory generated RS256 key per run	ephemeral	IdP-managed; app only has `issuer-uri`/JWKS
OTLP headers/tokens	tracing exporter	disabled	Secret store

Rules

1. No secret in git, images, `application*.yml`, test fixtures, logs, docs, or CI files. Only the labelled dummy value above is allowed, and only for `local`/`test`.
2. Secrets are injected through environment variables or mounted files (`*_FILE` convention), mapped in `PropertiesConfig`; the app fails fast at startup in `prod-like` if a required secret is missing or equals the dummy value (see source for the startup guard).
3. `hmac_key_ref` stores a *reference name*, not the secret.
4. Secrets are never echoed by `/actuator/env` (exposure disabled; Spring sanitizes keys such as `*secret*`, `*password*`, `*token*`).
5. `.env.example` (or `env.example`) holds only placeholders like `CHANGE_ME`.
6. Logs: `Redaction` masks values for keys matching `(?i)(secret|password|token|authorization|api[-_]?key|signature)`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Deployment mapping

Platform	Mechanism	Notes (not executed here)
Kubernetes	`Secret` objects (encrypted at rest with KMS provider) or External Secrets Operator / CSI driver from a vault	RBAC limit `get secret`; never in Helm values committed to git
Cloud Foundry	user-provided service instances / CredHub	bound as env vars in manifest
CI	CI credentials store; masked variables	no secrets in pipeline files

Rotation

Secret	Procedure
Callback HMAC	Two active `keyId`s: provider starts signing with new `keyId` (`X-Provider-Key-Id`), platform accepts both for overlap window, retire old after traffic confirms. Metric `messaging.callbacks{result="invalid_signature"}` must stay flat
DB password	Create second DB user, roll deployment, revoke old; Hikari `max-lifetime` ensures reconnect
Provider API keys	Provider-specific dual-key overlap
JWT keys	IdP JWKS rotation; resource server caches JWKS and refreshes on unknown `kid`

Compromise response: revoke, rotate, search logs/audit for use, post-incident review (`docs/operations/post-incident-review-template.md`).

Detection

- Secret scanning (gitleaks or equivalent) as CI/pre-commit stage; configured statically, **not executed** here.
- Review checklist item: "no new secret-like literals" (`ai-engineering/checklists/security-review.md`).
- AI-assisted coding rule: never paste real secrets into prompts; assistants must use placeholders (see `AGENTS.md`).

Data minimisation and retention

Source document: `docs/security/data-minimization-and-retention.md`

All data in this repository is synthetic. The design nevertheless applies production-grade minimization so the pattern is demonstrable. Not legal advice; real programs need privacy/compliance review.

Data classification

Class	Examples	Handling
Restricted	Rendered message body, provider credentials, HMAC secrets, push tokens	Body is never persisted by the platform; secrets in secret store; push tokens not returned by API

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Class	Examples	Handling
Confidential	Email, phone, recipient name	Stored only in `recipient`; API returns masked (`r***@example.test`, `+1555***0101`); logs mask; bus has ids only; `suppression` stores hash only
Internal	message ids, statuses, metrics, audit	Normal controls, tenant-scoped
Public	OpenAPI spec, docs	n/a

Minimization decisions

Decision	Where	Rationale
Store template variables, not rendered body (`payload_json`)	`message_request`, ADR-0007	Avoid storing regulated wording with personal data; re-render from pinned version for support if needed
Variables validated against template declaration; unknown keys rejected	`VariableValidator`	Prevent arbitrary data hoarding
Envelope carries ids only	`DispatchEnvelope`	Broker is a wider access surface
Mask addresses in all API responses	`web` DTO mappers	Operators rarely need full address
Hash addresses for suppression	`suppression.address_hash`	Block without retaining address
Redact provider diagnostics	`common.Redaction` before Mongo write	Avoid copying PII to second store
Correlation id instead of business ids in logs where possible	MDC	Traceability without content
Mongo TTL	`expireAt`	Short life for raw payloads
Audit stores `detail_json` with ids and field names, not values, for preference/template changes	`AuditService`	Audit without duplicating sensitive values (see source for exact fields)

Retention

See `docs/database/retention-and-archival.md` for numbers. Principles: shortest retention that supports support/dispute/audit needs; audit outlives data it describes; non-terminal rows are never purged silently; purge actions are counted and audited.

Data subject requests (design)

Request	Approach (not automated in the reference)
Access	Search by `external_ref` and recipient id; masked views via API; full export is a controlled DB task
Erasure	Mark `deleted` (stops sending), null contact columns, delete preferences, anonymize audit actors where required, keep hash suppression
Consent change	`PUT /recipients/{id}/preferences` with optimistic lock, audited

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Test data rules

No real names, emails, phone numbers: `Recipient 001`, `recipient001@example.test`, `+1555010xxxx`. CI and docs must not contain real data. `SyntheticDatasetGenerator` produces the bulk files.

Logging rules

Never log: bodies, full addresses, tokens, signatures, secrets, raw callback payloads at INFO. Allowed: ids, statuses, counts, masked addresses, error codes. Enforced by review checklists and a log capture test (see `docs/security/threat-model.md` T6).

Production versus reference

Source document: `docs/security/prod-vs-reference.md`

This repository is an independent reference implementation with synthetic data and simulated providers. It is not production-ready and makes no compliance claim. This list is deliberately blunt.

Area	Reference state	What production needs
Compliance review	None. Consent model is an engineering abstraction	Legal/privacy review per jurisdiction and channel (e.g., telephony and email marketing rules, data residency, retention schedules); documented lawful basis; records of processing
Access governance	Roles defined in code; JWT from dev issuer locally	Enterprise IdP, joiner/mover/leaver, periodic access reviews, break-glass process, privileged access management, separation of duties for ops roles
Real provider integration	Simulators only	Vendor contracts, sandbox testing, real error taxonomy mapping, webhook IP allow-lists, volume tiers, deliverability monitoring (SPF/DKIM/DMARC, sender reputation, short code registration)
Disaster recovery	Not exercised; no multi-region	RPO/RTO targets, tested backup/restore of PostgreSQL, Rabbit definitions + quorum queues, cross-AZ/region design, DR drills, runbook validation
Penetration testing	None executed	Independent pen-test, DAST, authenticated API fuzzing, threat model review with security team
Static/dep scanning	Config only, not executed	SAST, SCA, container scanning, secret scanning enforced as gates with triage SLAs
Secrets/KMS	Env-based; dev dummy secret	Vault/KMS-backed, rotation automation, HSM for signing where required
Encryption	TLS assumed at ingress; no at-rest config in repo	At-rest encryption (DB/volume), TLS everywhere incl. data stores, field-level encryption for contact data where mandated
Tenant isolation	Shared schema, app-enforced	PostgreSQL RLS as defense in depth; per-tenant quotas; possibly schema/DB per regulated tenant
Capacity	Not load-tested on real infra; k6 scripts not executed	Load and soak tests on prod-like infra, capacity model, autoscaling validation, chaos tests
Observability	Metrics/dashboards defined; not rendered; alerts illustrative	Real SLOs agreed with stakeholders, paging rotation, dashboards in use, trace sampling policy, log retention and access control

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Area	Reference state	What production needs
Rate limiting	App-level per client	Edge/WAF/API gateway limits, bot protection, per-tenant spend quotas
Audit	Append-only by convention	WORM storage/DB privileges, SIEM forwarding, tamper evidence
Data lifecycle	Retention job, no erasure workflow	Subject-request workflows, legal hold, verified purge
Supply chain	Pinned BOM	SBOM, signed images, provenance (SLSA), base-image patch cadence
Release process	Static pipeline files; no CI run claimed	Protected branches, required reviews, environment promotion, canary/blue-green, automated rollback
JWT handling	Validation yes; revocation no	Token introspection or short lifetimes + revocation events; mTLS-bound tokens
Operational maturity	Runbooks written, never exercised in an incident	Game days, on-call training, PIR culture, error-budget governance
Accessibility/UX	Basic Angular Material	Accessibility audit (WCAG), localization review
Cost controls	None	Per-channel cost tracking, anomaly alerts on send volume

What the reference does demonstrate

Design for failure (ADR-0003/4/6/9/10), tenant isolation tests, HMAC callbacks with replay protection, bounded resources, auditability, template governance, documented honesty about unexecuted assets.

How to use this document

Treat each row as an input to a real project's risk register (`docs/leadership/risk-register.md` template) rather than as a checklist already satisfied.

ADR-0011: Multi-tenancy and authorization

Source document: `docs/adr/0011-multi-tenancy-and-authorization.md`

Status: Accepted (2026-10-18)

Context

Several business units (synthetic tenants `tenant-retail`, `tenant-wealth`) share one deployment. One tenant must never read, mutate, enumerate or infer another tenant's messages, templates, recipients or jobs. Humans (operators, approvers) and service clients (senders) have different powers.

Decision

- **Shared database, shared schema, `tenant_id` discriminator** on every tenant-owned table; tenant is derived from the verified JWT claim `tenant` (never from a request body). `*` is allowed only with `ROLE_PLATFORM_ADMIN`, who may scope with `X-Tenant-Id`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- `security.TenantContext` (request-scoped) is populated by the security layer; repositories take tenant as an explicit parameter (`findByIdAndTenantId`). There is no "find by id" for tenant-owned entities in the repository layer used by the web layer.
- **404, not 403**, for resources owned by another tenant (no existence oracle). 403 is reserved for role failures within the caller's own tenant.
- Authorization layers:
  1. URL/method rules in `SecurityConfig` (coarse role check).
  2. Method-level checks and `AuthorizationService` (resource-level: tenant ownership, template four-eyes approver != author, bulk requires `ROLE_SENDER` + scope `messages:bulk`).
  3. Frontend guards are UX only and never trusted.
- Roles: `PLATFORM_ADMIN, TENANT_ADMIN, TEMPLATE_AUTHOR, TEMPLATE_APPROVER, OPERATOR, SENDER, AUDITOR`. The authoritative role-by-endpoint matrix is CONTRACT section 5.
- Callbacks are tenant-less at the network layer: tenant is resolved from the message found by `providerMessageId`/`idempotencyRef`, after signature verification.
- Audit: every privileged action writes `audit_event` with `actor`, `actor_role`, `tenant_id`, `correlation_id`.
- Rate limits are per client (`sub`) with tenant-level ceilings.

Alternatives considered

Option	Why not (for reference scale)
Schema-per-tenant	Migration fan-out, pool explosion, but stronger isolation; valid evolution for regulated tenants
Database-per-tenant	Strongest isolation, highest cost
Row-level security (PostgreSQL RLS)	Not portable to H2/Oracle under the portable-SQL rule; recommended as defense-in-depth on PostgreSQL (see `docs/security/prod-vs-reference.md`)
Tenant from header/body	Spoofable
403 for foreign tenant	Leaks existence (BOLA recon)

Consequences

- Positive: simple ops, one schema, enumeration-resistant.
- Negative: a missing `tenant_id` predicate is a data-leak bug; mitigated by repository signature convention, cross-tenant tests for every endpoint, and the review checklist (`ai-engineering/ai-code-review/06-incorrect-authorization-cross-tenant.md`).
- Noisy neighbour: per-client and per-tenant rate limits, bulk job caps, provider token buckets.

Validation

- `tests/security` cross-tenant matrix (every GET/PUT/POST by id with tenant B token against tenant A resource expects 404); controller slice tests with `@WithMockUser`-style principals (see source `security` and `web` tests).
- OWASP mapping: API1 BOLA, API5 BFLA in `docs/security/owasp-api-top10-2023-analysis.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

17. Cloud Foundry Deployment

> **Execution status: NOT executed**
>
> No Cloud Foundry foundation or `cf` CLI was available on the authoring machine. What did run: the dependency-free manifest validator (`node cloud-foundry/validate-manifest.mjs`) and `bash -n` on the CF scripts. Everything about real `cf push`, bindings, routes and rolling deploys is documented guidance and is unverified here.

17.1 Why Cloud Foundry is a first-class target

Many large enterprises run application platforms where developers push an artifact and the platform builds, runs and routes it. The platform is a **twelve-factor application**: configuration comes from the environment, backing services are attached resources, processes are stateless and disposable, logs go to stdout. That makes Cloud Foundry and Kubernetes equally viable, and ADR-0008 records the decision to ship manifests for both while treating neither as the one true runtime.

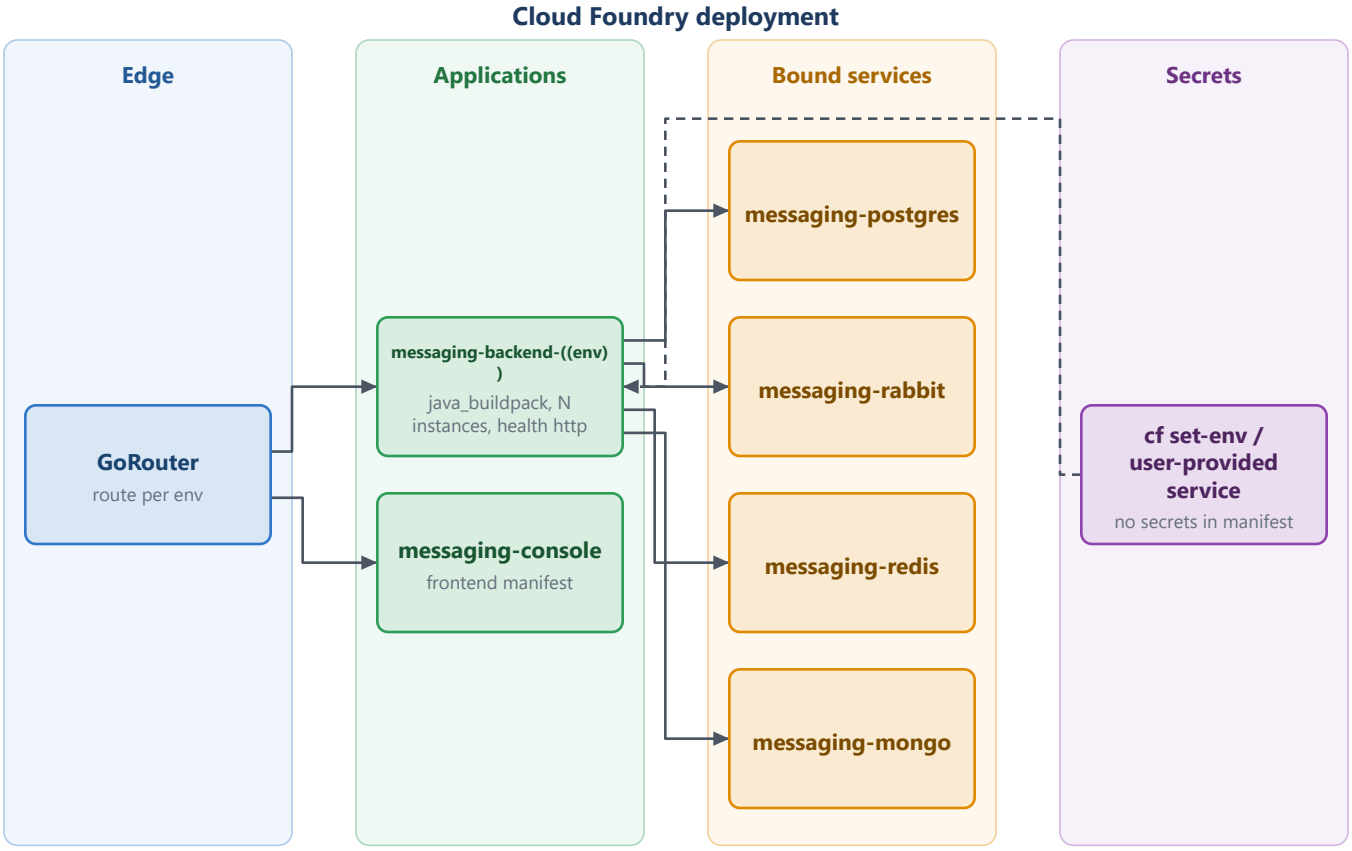


Figure 41. Cloud Foundry topology. NOT executed: no foundation was available. The manifest was parsed and structurally checked by validate-manifest.mjs. Implementation: cloud-foundry/manifest.yml, cloud-foundry/manifest-frontend.yml, cloud-foundry/scripts/

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

17.2 The manifest

`cloud-foundry/manifest.yml` is parameterised with `((env))`, `((instances))`, `((memory))` and `((apps_domain))` taken from `cloud-foundry/vars/dev.yml` and `test.yml`.

YAML

```yaml
applications:
  - name: messaging-backend-((env))
    path: ../backend/target/messaging-platform.jar
    buildpacks: [java_buildpack]
    instances: ((instances))
    memory: ((memory))
    health-check-type: http
    health-check-http-endpoint: /actuator/health/liveness
    timeout: 180
    env:
      SPRING_PROFILES_ACTIVE: prod-like,rabbit,cloud
      JBP_CONFIG_OPEN_JDK_JRE: '{ jre: { version: 21.+ } }'
      MESSAGING_BUS_TYPE: rabbit
      MESSAGING_COORDINATION_TYPE: redis
      MESSAGING_SECURITY_DEV_TOKEN_ENABLED: 'false'
      SERVER_SHUTDOWN: graceful
    services: [messaging-postgres, messaging-rabbit, messaging-redis, messaging-mongo]
```

Element	Why
`health-check-type: http` on `/actuator/health/liveness`	A failing liveness check restarts the instance; `timeout: 180` is the startup budget for Flyway and JPA
`JBP_CONFIG_SPRING_AUTO_RECONFIGURATION` disabled	Explicit configuration, no hidden auto-reconfiguration
`JAVA_OPTS: -Duser.timezone=UTC -XX:+ExitOnOutOfMemoryError`	UTC everywhere; an out-of-memory instance exits and is replaced rather than limping
`SERVER_SHUTDOWN: graceful`	In-flight deliveries drain within 30 s before the instance stops
Dev token disabled	The dev issuer can never be enabled by a forgotten setting in a deployed space
Services bound by name	The four backing services (PostgreSQL, RabbitMQ, Redis, MongoDB) are created beforehand
No secrets in the file	JWT issuer and callback HMAC secrets come from `cf set-env` or a user-provided service; the validator flags literal values on `*password*`, `*secret*`, `*token*` keys

`cloud-foundry/manifest-frontend.yml` pushes the Angular static assets with the `staticfile_buildpack`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

17.3 Operating model

Topic	Practice
Org and space	One space per environment (`dev`, `test`)
Push pipeline	`cf push` uploads the artifact; a buildpack produces a droplet; the droplet runs in a container
Service bindings	`VCAP_SERVICES` mapped to Spring properties by java-cfenv (**repo gap:** the backend POM must include it for the `cloud` profile to bind automatically; otherwise map to `SPRING_*` variables with `cf set-env`)
Rolling deploy	`cf push ... --strategy rolling`: schema migrations must be backward compatible with the previous release while both run
Blue-green	`cloud-foundry/scripts/blue-green.sh`: push `-green`, smoke it on a temporary route, switch the production route, keep blue for rollback
Scaling	`cf scale -i N`: consumers are in-process, so more instances means more concurrent consumers on the shared queue (watch DB pool size per instance)
Rollback	`cloud-foundry/scripts/rollback.sh APP <revision>` wrapping `cf rollback --strategy rolling`; or droplet pinning; or re-map the blue route
Smoke	`cloud-foundry/scripts/smoke.sh` asserts liveness, readiness and that unauthenticated `/api/v1/me` is 401

Scripts: `deploy.sh` (validate manifests, rolling push, smoke), `smoke.sh`, `blue-green.sh`, `rollback.sh`.

17.4 Environments and profiles

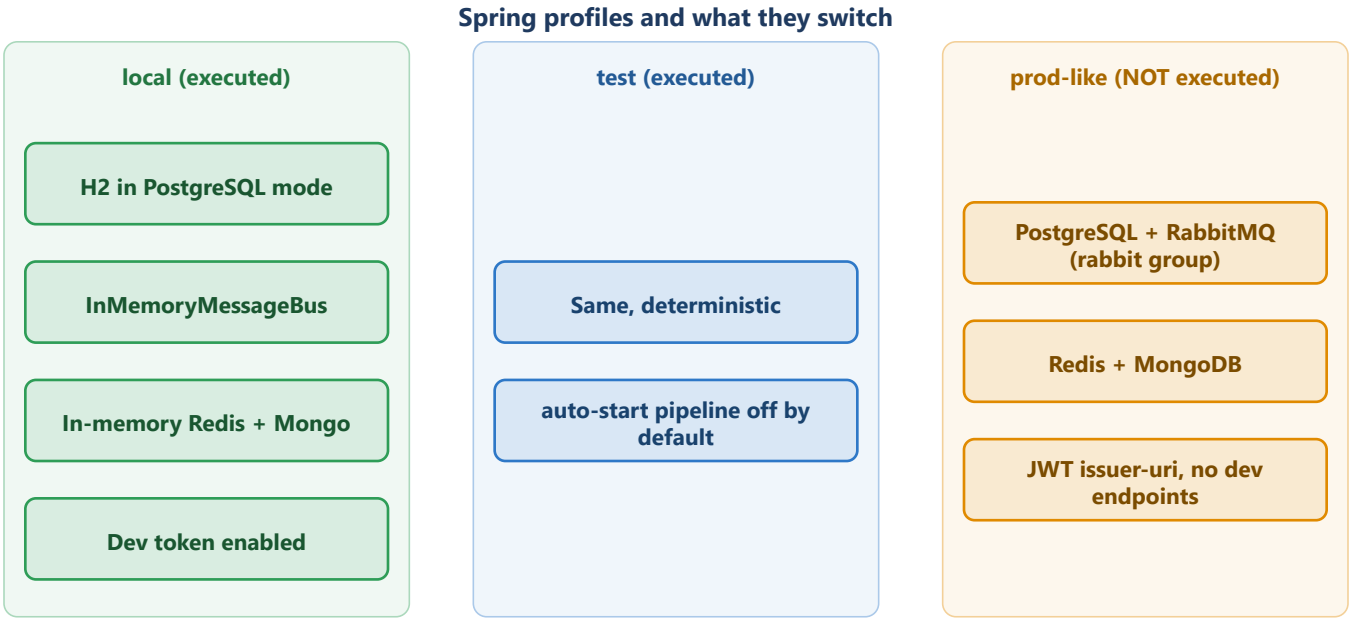


Figure 42. The same jar runs everywhere; profiles select adapters. Only the local profile (single JVM) was executed. Implementation: backend/src/main/resources/application.yml, application-local.yml, application-test.yml, application-prod-like.yml

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

17.5 Cloud Foundry versus Kubernetes in one table

Concern	Cloud Foundry	Kubernetes
Unit of deploy	App plus buildpack	Container image plus manifests
Learning curve for app teams	Low	Higher
Scaling	`cf scale`, platform autoscaler	HPA, custom metrics (KEDA option)
Health	`health-check-type: http`	Liveness, readiness and startup probes
Network policy	Application security groups	`NetworkPolicy`
Secrets	CredHub or user-provided service	`Secret`, external secret operator
Portability	Platform specific	Broad

Reference material

Cloud Foundry guide

Source document: `docs/operations/cloud-foundry-guide.md`

Execution status: NOT executed. No Cloud Foundry foundation or cf CLI was available. What was run: the dependency-free manifest validator (`node cloud-foundry/validate-manifest.mjs`) and `bash -n` on the CF scripts. Everything about real `cf push`, bindings, routes and rolling deploys is documented guidance and unverified here.

Files

Path	Purpose
`cloud-foundry/manifest.yml`	backend app (java_buildpack, Java 21, rolling-compatible)
`cloud-foundry/manifest-frontend.yml`	Angular static assets via `staticfile_buildpack`
`cloud-foundry/vars/{dev,test}.yml`	per-space variables (`env`, `instances`, `memory`, `apps_domain`)
`cloud-foundry/scripts/{deploy,smoke,blue-green,rollback}.sh`	operational scripts (`bash -n` checked only)
`cloud-foundry/validate-manifest.mjs`	offline manifest linter (parses YAML subset; checks keys, memory format, health check, Java 21 pin, service bindings, inline-secret heuristics)

Operating model

- **Org / space.** An org groups teams/quota; a space is an environment (`dev`, `test`) with its own apps, routes, services and roles (SpaceDeveloper, SpaceManager, Auditor). Use one space per environment and keep manifests environment-neutral using `--vars-file`.
- **Push pipeline.** `cf push` uploads source/artifact, a **buildpack** detects/compiles it into a **droplet** (the runnable artifact), and the droplet runs in a **container** (Diego cell). Apps are stateless and disposable; the filesystem is ephemeral.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Buildpacks.** `java_buildpack` runs a jar with a JRE it provides. Java 21 is selected by `JBP_CONFIG_OPEN_JDK_JRE: '{ jre: { version: 21.+ } }'`. The buildpack computes JVM memory settings from the container limit (memory calculator), so set `memory` deliberately (1G dev, 1.5G test). Confirm your foundation's java-buildpack version offers JRE 21 (`cf buildpacks`).
- **Health checks.** `health-check-type: http` against `/actuator/health/liveness`; `timeout: 180` is the startup budget (Flyway + JPA), `health-check-invocation-timeout: 5`. A failing check restarts the instance.
- **Routes.** `messaging-backend-<env>.<apps_domain>`; the gorouter load-balances instances and removes unhealthy ones. TLS terminates at the router; the manifest's frontend sets `FORCE_HTTPS`.
- **Service bindings and `VCAP_SERVICES`.** `messaging-postgres`, `messaging-rabbit`, `messaging-redis`, `messaging-mongo` are marketplace or user-provided service instances created beforehand (`cf create-service ...` / `cf cups ...`). Binding injects credentials as JSON in the `VCAP_SERVICES` env var (`cf env APP` shows it; treat it as sensitive).
  - Use **java-cfenv** (`io.pivotal.cfenv:java-cfenv-boot`) to map `VCAP_SERVICES` to Spring properties (datasource, rabbit, redis, mongo). Spring Cloud Connectors is end-of-life; do not use it. The manifest disables the buildpack's Spring auto-reconfiguration (`JBP_CONFIG_SPRING_AUTO_RECONFIGURATION`) because java-cfenv is the supported path.
  - **Repo gap:** the backend POM must include java-cfenv for the `cloud` profile to pick up bindings automatically. If it does not, map `VCAP_SERVICES` to the `SPRING_*` variables with `cf set-env` or a small startup shim. Not verified.
- **Secrets.** Never put secrets in the manifest (the validator flags `*password*`, `*secret*`, `*token*` env keys with literal values). Use `cf cups messaging-secrets -p '{"callbackSecret":"CHANGE_ME_LOCAL_ONLY"}'` (dummy example) and bind it, or CredHub-backed services.

Deploying

```bash
bash scripts/mvn.sh -f backend/pom.xml -DskipTests package
cf login -a <api> -o <org> -s dev          # interactive; do not script passwords on the command line
bash cloud-foundry/scripts/deploy.sh dev    # validates manifests, rolling push, smoke
# equivalent manual command:
cf push -f cloud-foundry/manifest.yml --vars-file cloud-foundry/vars/dev.yml --strategy rolling
```

Prerequisites in the target space: the four service instances, a route/domain, quota for `instances x memory`.

Rolling deploy

`--strategy rolling` creates a new process revision and replaces instances gradually, honoring health checks, so there is no downtime if at least one instance stays healthy. Constraint: schema migrations must be backward compatible (old and new versions overlap). Requires CF API v3 / CLI v8+.

Blue-green (manual)

`cloud-foundry/scripts/blue-green.sh <env>` pushes `-green` without the production route, smoke tests it on a temporary route, maps the production route to it, unmaps blue, and keeps blue for rollback. Use it when you need full pre-switch verification or incompatible changes (with care for the shared database). Alternative: `cf push --strategy canary` on foundations that support it (check your CF version; unverified here).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Scaling

```bash
cf scale messaging-backend-test -i 3          # horizontal
cf scale messaging-backend-test -m 2G -k 2G   # vertical (restarts)
cf app messaging-backend-test                 # instance state, CPU, memory
```

Consumers are in-process; more instances means more concurrent consumers against the shared RabbitMQ queue (check `messaging.dispatch.concurrency` and DB pool size per instance).

Rollback

1. Revisions (preferred): `cf revisions APP` then `bash cloud-foundry/scripts/rollback.sh APP <revision>` (wraps `cf rollback APP --version N --strategy rolling`).
2. Droplets (older CLI/API): `cf droplets APP`, `cf set-droplet APP <droplet-guid>`, `cf restart APP --strategy rolling`.
3. Blue-green: re-map the route to the retained blue app. Config/env changes are not part of a droplet: a revision rollback restores the previous droplet and environment snapshot as the platform defines it; verify env after rollback.

Logs, events, debugging

```bash
cf logs messaging-backend-dev --recent        # recent buffer
cf logs messaging-backend-dev                 # live tail (Loggregator)
cf events messaging-backend-dev               # push/crash/scale history
cf ssh messaging-backend-dev                  # if SSH is enabled in the space
cf env messaging-backend-dev                  # includes VCAP_SERVICES (sensitive)
```

Logs are JSON on stdout; forward them via the platform's syslog drain if needed.

Troubleshooting

Symptom	Likely cause	Check / fix
`Staging error: no compatible JRE`	buildpack lacks Java 21	`cf buildpacks`; pin a newer buildpack with `-b`, or ask the platform team
Instances crash after start (`CRASHED`, `OOMKilled`)	memory too low for heap + metaspace + threads	raise `memory`; check memory calculator output in staging logs
`Instance never became healthy`	liveness failing, slow start, DB unreachable	`cf logs --recent`; raise `timeout`; verify bindings (`cf services`, `cf env`)
502/404 from router	route not mapped, app unhealthy, wrong port	`cf routes`, `cf app`
Cannot connect to datasource	missing java-cfenv mapping or binding not restaged	`cf restage APP` after binding changes; confirm `cloud` profile
Rolling push stuck	new revision failing health check	`cf logs`; `cf rollback` or `cf cancel-deployment APP`
Flyway failure	migrator lacks privileges	see `infrastructure/postgres/init.sql` role model

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Cloud Foundry vs Kubernetes

Topic	Cloud Foundry	Kubernetes
Unit of deploy	app (source/jar) staged to a droplet	container image
Developer interface	`cf push` + manifest	manifests/Helm/Kustomize + CI
Config	env vars, bound services (`VCAP_SERVICES`)	ConfigMap/Secret, env, volumes
Networking	gorouter routes, container networking policies	Service, Ingress, NetworkPolicy
Scaling	`cf scale` (manual; autoscaler service optional)	HPA/KEDA, cluster autoscaler
Rolling/blue-green	`--strategy rolling`, route mapping	Deployment strategies, Argo Rollouts/Flagger
Customization	opinionated, fewer knobs (buildpacks)	very flexible, more operational surface
Stateful dependencies	marketplace services	operators/StatefulSets/managed services
Best fit	standardized 12-factor apps, platform-team-run	heterogeneous workloads, custom infra needs
This project ships both: `cloud-foundry/` for the CF path and `infrastructure/kubernetes	helm` for Kubernetes. They are not tested against live platforms here.	

Local practice path (guidance only; availability changes, verify before relying)

- **Korifi** (CNCF/Cloud Foundry community) provides the CF API on top of Kubernetes and can run on kind/minikube. It is the actively developed successor in the CF-on-Kubernetes space. Needs a local Kubernetes cluster (Docker required) and enough RAM; install steps change between releases, follow its current README.
- **cf-for-k8s** was the earlier approach and has been archived/deprecated; do not start new work on it.
- **Pivotal Web Services** was shut down; do not plan on it. Public free CF trials (for example vendor trial accounts such as SAP BTP trial) come and go and have usage limits; check terms and current availability.
- **Without any foundation:** use `node cloud-foundry/validate-manifest.mjs` and practice reading manifests, `VCAP_SERVICES` JSON handling and the commands above in the CF CLI documentation.

ADR-0008: Cloud Foundry versus Kubernetes

Source document: `docs/adr/0008-cloud-foundry-vs-kubernetes.md`

Status: Accepted (2026-10-18)

Context

Enterprises run Java services on either Cloud Foundry (buildpack push, platform-managed routing/services) or Kubernetes (container, declarative manifests). The reference must demonstrate that the application is portable and show operational thinking for both. **No Docker, kubectl, Helm or `cf` CLI is available in the authoring**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

environment, so all deploy assets are statically validated only.

Decision

- Build a **platform-neutral 12-factor application**: config via env vars, `/actuator/health/liveness|readiness`, SIGTERM-triggered graceful shutdown (30 s `SmartLifecycle`), stateless instances, logs to stdout, backing services via URLs.
- Ship **both** deployment descriptors: `cloud-foundry/manifest.yml` (+ per-environment var files) and `infrastructure/{docker,compose,kubernetes,helm}`. Neither is executed here.
- The Kubernetes path is the primary documented target for the reference (HPA, PDB, NetworkPolicy, probes); CF is documented as an equivalent for teams on that platform.
- Platform-specific code is avoided: no Spring Cloud Kubernetes, no `VCAP_SERVICES` parsing in business code (CF binds are mapped to the same env vars in the manifest).

Concern	Cloud Foundry	Kubernetes
Packaging	Java buildpack, JAR	OCI image (multi-stage, non-root)
Scaling	`cf scale`, autoscaler service	HPA on CPU/custom queue-depth metric (KEDA optional)
Zero-downtime	blue-green via route mapping / rolling `--strategy rolling`	rolling update + PDB + readiness
Health	`health-check-type: http` on readiness	liveness/readiness/startup probes
Secrets	user-provided service / CredHub	Secret + external secrets operator (see secrets-management)
Network policy	Container networking policies	NetworkPolicy
Batch (bulk jobs)	same app instances; restartable	same; optional separate Deployment with `bulk` profile

Alternatives considered

Option	Why not
Kubernetes only	Ignores teams on CF; also acceptable but less illustrative
CF only	Less flexibility for sidecars, KEDA, policy tooling
Serverless functions	Long-lived consumers and connection pools fit poorly

Consequences

- Positive: portability; one JAR/image; same runbooks.
- Negative: two descriptor sets to keep in sync (drift risk, mitigated by a checklist item in `release-readiness-checklist.md`).
- Honest limits: probes, resource requests, HPA targets are starting points that have **not** been load-tested on a cluster.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Validation

- Static: YAML parse/lint of manifests (see `docs/validation/VALIDATION_REPORT.md` when generated). Runtime: NOT executed.
- Graceful shutdown is testable in-JVM (`DispatchConsumer` lifecycle test, see source).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

18. Docker and Kubernetes Architecture

Execution status: NOT executed

Docker, `kubectl`, `helm` and a cluster were not available. Dockerfiles, the Compose file, kustomize overlays and the Helm chart were reviewed statically and parsed by `scripts/validate-infra.sh` (which prints SKIPPED for absent tools). `docker build`, `docker compose up`, `kubectl apply` and `helm install` were never run.

18.1 Container images

Image	File	Highlights
Backend	`infrastructure/docker/Dockerfile.backend`	Multi-stage: `maven:3.9.9-eclipse-temurin-21` build, then `eclipse-temurin:21.0.5_11-jre-jammy` runtime; Spring Boot **layered** jar extraction for cache-friendly layers; non-root user `10001`; `tini` as init; OCI labels `version` and `revision` from build args `VERSION`, `GIT_SHA`
Frontend	`infrastructure/docker/Dockerfile.frontend`, `nginx.conf`	Angular production build served by `nginx-unprivileged`; proxies `/api` to the backend

Tags are immutable: `<version>-<git sha7>` (for example `0.1.57-3fa9c21`), never `latest`.

18.2 Docker Compose stack

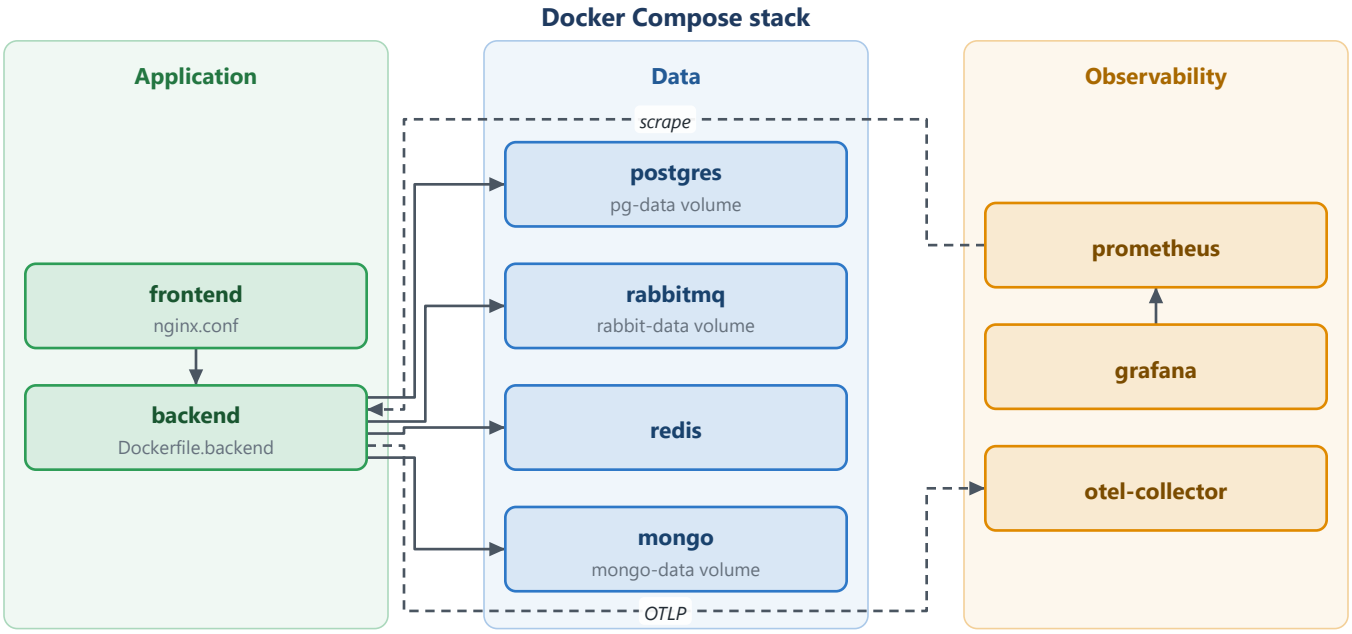


Figure 43. Services declared in docker-compose.yml. NOT executed: Docker is not installed on the authoring machine; the file was parsed by scripts/validate-infra.sh only. Implementation: infrastructure/compose/docker-compose.yml, infrastructure/docker/Dockerfile.backend, infrastructure/docker/Dockerfile.frontend

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`infrastructure/compose/docker-compose.yml` declares postgres 16.4, rabbitmq 3.13.7 (management, definitions imported), redis 7.4.1, mongo 7.0.14, the backend, the frontend, prometheus v2.54.1, grafana 11.2.2 and otel-collector-contrib 0.111.0, with named volumes `pg-data`, `rabbit-data`, `mongo-data`, `prom-data`, `grafana-data`. Images are pinned to explicit tags; services have resource limits and health checks; backend and frontend run with a read-only root filesystem and dropped capabilities. The compose backend runs profile `prod-like,rabbit`, so the dev token is **disabled** there. Environment values come from `.env` (copied from the example file, with dummy `CHANGE_ME_LOCAL_ONLY` placeholders). The Compose host ports are API 18080, UI 14200, Prometheus 19090 and Grafana 13000.

18.3 Kubernetes architecture

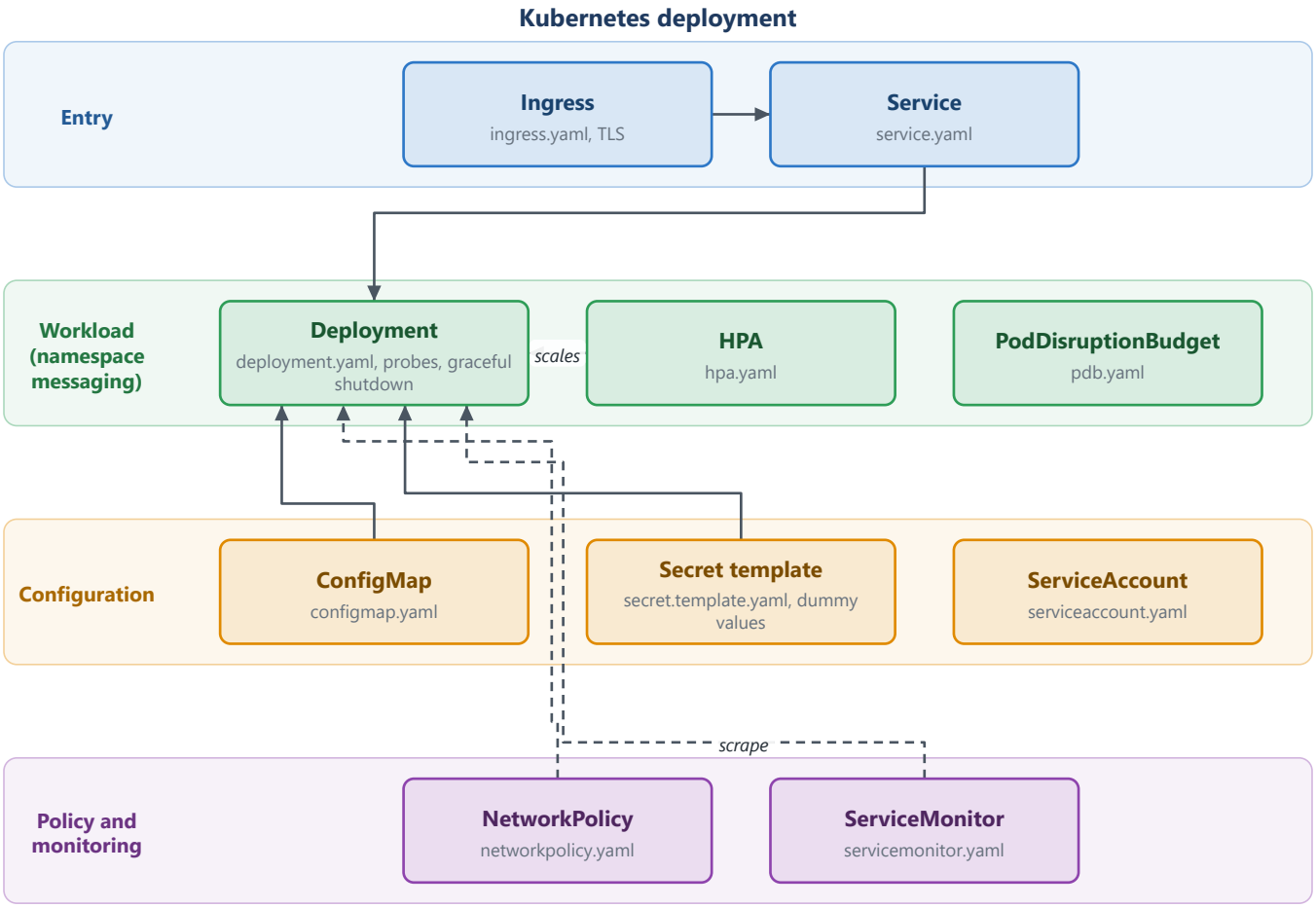


Figure 44. Kubernetes topology from the kustomize base and Helm chart. NOT executed: no cluster, kubectl or Helm was available; the YAML was parsed and structurally checked only. Implementation: infrastructure/kubernetes/base/, infrastructure/kubernetes/overlays/, infrastructure/helm/messaging-platform/

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Manifests (`infrastructure/kubernetes/base`)

File	Content (as written)
`deployment.yaml`	2 replicas; `terminationGracePeriodSeconds: 60` (above the 30 s application drain plus a preStop sleep); `runAsNonRoot`, `readOnlyRootFilesystem`; requests `500m` CPU and `768Mi`, limits `2` CPU and `1Gi`; startup, liveness (`/actuator/health/liveness`) and readiness (`/actuator/health/readiness`) probes; Prometheus scrape annotations
`hpa.yaml`	HorizontalPodAutoscaler 2 to 8 replicas at 70 percent average CPU
`pdb.yaml`	`minAvailable: 1`
`service.yaml`, `ingress.yaml`	ClusterIP service; ingress with TLS
`configmap.yaml`, `secret.template.yaml`	Non-secret configuration; **dummy** secret template (values replaced by a secret manager in real use)
`networkpolicy.yaml`	Ingress only from the ingress controller; egress to PostgreSQL, RabbitMQ, Redis, MongoDB, providers and the OTel collector
`serviceaccount.yaml`, `servicemonitor.yaml`	Dedicated service account; Prometheus Operator scrape definition

Overlays: `overlays/dev` (adds in-cluster dependencies in `dependencies.yaml` for experiments) and `overlays/prod-like`. A Helm chart (`infrastructure/helm/messaging-platform`) mirrors the base with `values.yaml` and `values-prod-like.yaml`.

Scaling the right thing

The API and the consumers run in the same pods, which is simplest. If consumer saturation must not affect API latency, split them into separate Deployments by profile; scale consumers on queue depth (KEDA or a custom metrics adapter) and the API on CPU or request rate. More consumers only help while the provider rate limit allows; the token bucket still caps throughput.

Graceful termination

Rollouts and node drains send `SIGTERM`. The application stops intake, drains in-flight deliveries for up to 30 seconds (`spring.lifecycle.timeout-per-shutdown-phase`), and unacknowledged deliveries return to the queue. The 60 second grace period leaves room for the drain and the preStop sleep that lets the endpoint drop out of the load balancer first.

18.4 Environments

Env	How	Profile
Local, no Docker	`bash scripts/run-local.sh --wait`	`local` (the only one executed here), API on 18080, Angular dev server on 14200
Local containers	Docker Compose	`prod-like,rabbit`
Kubernetes	kustomize or Helm	`prod-like,rabbit`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Env	How	Profile
Cloud Foundry	`cf push`	`prod-like,rabbit,cloud`

Reference material

Deployment guide (Docker Compose, environments, checklists)

Source document: `docs/operations/deployment/README.md`

Execution status: NOT executed. Docker is not installed on the authoring machine. The compose file, Dockerfiles, nginx config and init scripts were reviewed statically and parsed (`bash scripts/validate-infra.sh`); `docker compose config`, `docker build` and `docker compose up` were SKIPPED. Treat every command below as untested until run on a Docker host.

Environments

Env	How	Profile	Notes
Local, no Docker	`bash scripts/run-local.sh --wait`	`local` (H2, in-memory bus/Mongo, dev token)	port 18080; Angular dev server 14200
Local containers	docker compose (below)	`prod-like,rabbit`	real PostgreSQL/RabbitMQ/Redis/Mongo; dev token disabled, so `/dev/token` and the k6 scripts need a real JWT issuer or a local-profile backend
Kubernetes	`docs/operations/kubernetes-guide.md`	`prod-like,rabbit`	
Cloud Foundry	`docs/operations/cloud-foundry-guide.md`	`prod-like,rabbit,cloud`	

Profile `rabbit` is added because CONTRACT section 2 names it for `RabbitTopologyConfig`; drop it if the backend only keys off `prod-like`.

Compose quick start (NOT executed)

BASH

```
cp .env.example .env          # all values are dummy placeholders (CHANGE_ME_LOCAL_ONLY...)
docker compose --env-file .env -f infrastructure/compose/docker-compose.yml up -d --build
docker compose --env-file .env -f infrastructure/compose/docker-compose.yml ps
# optional load profile
docker compose --env-file .env -f infrastructure/compose/docker-compose.yml --profile load run --rm k6
```

Endpoints: API `http://localhost:18080`, UI `http://localhost:14200`, RabbitMQ UI `http://localhost:15672` (loopback), Prometheus `http://localhost:19090`, Grafana `http://localhost:13000`. For debugging ports use `infrastructure/compose/docker-compose.override.example.yml` (copy to `docker-compose.override.yml`, add a second `-f`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

What the stack contains

postgres:16.4, rabbitmq:3.13.7-management (definitions imported), redis:7.4.1, mongo:7.0.14, backend, frontend (nginx-unprivileged), prometheus v2.54.1, grafana 11.2.2, otel-collector-contrib 0.111.0, k6 0.54.0 (profile `load`). All images are pinned to explicit tags (digest pinning recommended for production). Services have resource limits and healthchecks; backend and frontend run with a read-only root filesystem, dropped capabilities and `no-new-privileges`. Networks: `data` (internal, no external route), `edge`, `observability`. Volumes persist database, broker and metrics data.

Data-store initialization

Component	File	Behavior
PostgreSQL	`infrastructure/postgres/init.sh` + `init.sql`	creates `messaging_migrator` (DDL, runs Flyway), `messaging_app` (DML only), `messaging_readonly`; passwords from env
RabbitMQ	`infrastructure/rabbitmq/definitions.json` + `rabbitmq.conf`	vhost `messaging`; exchanges `messaging.dispatch`, `messaging.retry`, `messaging.dlx`; priority queue `messaging.dispatch.q` (`x-max-priority: 9`), `messaging.dlq`, retry delay queues 5s/30s/2m/10m that dead-letter back to dispatch; policies for backpressure and retention
MongoDB	`infrastructure/mongo/init-mongo.js`	app user, `provider_diagnostics` indexes, 30-day TTL on `receivedAt`

Caveats to verify on first run: RabbitMQ definitions contain no users (the default user from env is created and, because `default_vhost = messaging`, gets permissions on that vhost); the Flyway user wiring uses `SPRING_FLYWAY_USER/PASSWORD`; `audit_event` append-only grants must be tightened after migrations (`REVOKE UPDATE, DELETE ... FROM messaging_app`).

Backend container

Multi-stage build (Maven 3.9 + Temurin 21 builder, Temurin 21 JRE runtime), layered Spring Boot jar, non-root uid 10001, tini as PID 1 so SIGTERM reaches the JVM for graceful shutdown, `HEALTHCHECK` on `/actuator/health/liveness`, flags `-XX:MaxRAMPercentage=70 -XX:+ExitOnOutOfMemoryError`. Assumes the Boot jar supports `-Djarmode=tools` extraction (Boot 3.3+; project uses 3.5.16).

Frontend container

Node 22 build (`ng build --configuration production`) then `nginxinc/nginx-unprivileged` on port 8080 with CSP, `X-Frame-Options`, `nosniff`, `Referrer-Policy`, `Permissions-Policy`, SPA fallback, immutable caching for hashed assets, and `/api/` reverse proxy to `backend:18080`. HSTS is commented out: enable it where TLS terminates in front.

Pre-deployment checklist

- ☐ Image tags are immutable CI versions (`docs/operations/ci-cd.md`), scanned with Trivy
- ☐ Real secrets come from a secret manager; none in `.env` committed (it is git-ignored)
- ☐ `messaging.security.dev-token-enabled=false` and issuer URI configured
- ☐ Flyway migrations backward compatible with the previous release (rolling/blue-green overlap)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- ☐ Alerts and dashboards loaded; runbooks reachable (`docs/operations/runbooks/`)
- ☐ Rollback path rehearsed (compose: previous tag; K8s: `rollout undo`; CF: revisions)

Kubernetes guide

Source document: `docs/operations/kubernetes-guide.md`

Execution status: NOT executed. No kubectl, kustomize, helm or cluster was available. The manifests were only parsed as YAML and checked with in-repo heuristics (`bash scripts/validate-infra.sh`: deployment has probes/non-root/read-only FS/preStop/spread; Helm templates have balanced `{{ }}` and `end` counts). `kubectl kustomize`, `kubeconform`, `helm lint` and `helm template` were SKIPPED. The Helm chart has never been rendered.

Layout

```
infrastructure/kubernetes/base/           Deployment, Service, HPA, PDB, Ingress, ConfigMap, Secret template,
  NetworkPolicy, ServiceAccount, ServiceMonitor
infrastructure/kubernetes/overlays/dev/    1 replica + StatefulSets for postgres/rabbitmq/redis/mongo (dummy
  creds, dev only)
infrastructure/kubernetes/overlays/prod-like/  3 replicas, bigger resources, DoNotSchedule zone spread, PDB
  minAvailable 2, dummy Secret removed
infrastructure/helm/messaging-platform/    chart (values.yaml, values-prod-like.yaml)
```

Design choices

- **Probes.** `startupProbe` and `livenessProbe` use `/actuator/health/liveness`; `readinessProbe` uses `/actuator/health/readiness`. Startup allows up to 3 minutes (36 x 5s) for Flyway/JPA before liveness takes over.
- **Graceful shutdown.** `preStop: sleep 10` lets the endpoint/ingress drop the pod before SIGTERM; Spring graceful shutdown then drains in-flight work (CONTRACT: 30s). `terminationGracePeriodSeconds: 60` exceeds sleep + drain. Unacked RabbitMQ messages return to the queue.
- **Security context.** non-root uid 10001, `readOnlyRootFilesystem`, all capabilities dropped, seccomp RuntimeDefault, no service account token mounted, `/tmp` as an `emptyDir`.
- **Topology spread.** zone and hostname spread with `ScheduleAnyway` (dev) or `DoNotSchedule` for zones in prod-like.
- **HPA.** CPU 70% plus an **example** External metric `messaging_queue_depth` (target average 2000). It requires an external-metrics provider (prometheus-adapter or KEDA) mapping the Prometheus gauge; without one, remove that metric block (the dev overlay removes it; Helm enables it only with `autoscaling.queueDepth.enabled`). Consumer capacity also depends on DB pool and RabbitMQ prefetch, so scaling pods on queue depth is not free.
- **PDB.** `minAvailable` 1 (base), 2 (prod-like), 0 in dev so single-replica drains do not block.
- **NetworkPolicy.** default-deny plus an allow policy: ingress only from `ingress-nginx` and `monitoring` namespaces on 18080; egress to DNS, dependency pods, and the OTLP collector. Adjust namespace labels to your cluster; the Helm egress block is port-based and should be narrowed.
- **Secrets.** `secret.template.yaml` and chart `secrets.values` contain dummy placeholders only. Real clusters should use `secrets.existingSecret` (External Secrets / Vault / Sealed Secrets). The prod-like overlay deletes

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

the dummy Secret; you must supply `backend-secrets` yourself.

- **Frontend.** Only the backend is deployed by these manifests. The frontend image (`infrastructure/docker/Dockerfile.frontend`) proxies `/api` to host `backend:18080` (compose naming); in Kubernetes the Ingress routes `/api` directly to the backend Service instead, and serving the SPA needs an extra Deployment or CDN (not included).
- **Dependencies.** The dev StatefulSets are examples (single replica, small PVC); RabbitMQ topology from `infrastructure/rabbitmq/definitions.json` must be imported separately. Use operators or managed services in real environments. PostgreSQL role setup is in `infrastructure/postgres/init.sql`.

Usage (commands NOT executed)

BASH

```
kubectl kustomize infrastructure/kubernetes/overlays/dev | kubeconform -strict -ignore-missing-schemas -summary
kubectl apply -k infrastructure/kubernetes/overlays/dev
kubectl -n messaging-dev rollout status deploy/backend
kubectl -n messaging-dev port-forward svc/backend 18080:18080

helm lint infrastructure/helm/messaging-platform -f infrastructure/helm/messaging-platform/values-prod-like.
  yaml
helm upgrade --install msg infrastructure/helm/messaging-platform -n messaging-prod-like --create-namespace \
  -f infrastructure/helm/messaging-platform/values-prod-like.yaml --set image.tag=0.1.57-3fa9c21
```

The `messaging-dev` namespace in the overlay must exist (`kubectl create namespace messaging-dev`); Kustomize does not create it.

Troubleshooting

Symptom	Check
Pod `CrashLoopBackOff`	`kubectl logs`, DB/Rabbit reachability, `backend-secrets` keys present, Flyway privileges
Never Ready	readiness group includes dependencies; `kubectl exec`-less check via `port-forward` + `/actuator/health`
Rollout stuck	maxUnavailable 0: new pod failing readiness; check events
HPA `unknown` external metric	no external-metrics API; remove or install adapter/KEDA
Ingress 404	host `messaging.example.test` and class `nginx` are placeholders; TLS secret `messaging-tls` must exist
Egress blocked	NetworkPolicy labels (`app.kubernetes.io/component: dependency`) and namespace names

Deployment views

Source document: `docs/architecture/deployment-views.md`

Honesty note: none of the container, Kubernetes, Helm or Cloud Foundry assets below were executed in the authoring environment (no Docker/kubectl/cf). Only the `local` profile (single JVM, H2, in-memory bus) was run. These diagrams describe intended topology.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. Local (executed): single JVM

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter.

2. Prod-like on Kubernetes (not executed)

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter. Notes: split API and consumers into separate Deployments if consumer saturation should not affect API latency (profile-controlled consumers: see source `messaging.*` config); scale consumers on queue depth (KEDA or custom metric adapter, an option), API on CPU/RPS. Single replica count assumptions in `infrastructure/helm` values are starting points.

3. Cloud Foundry (not executed)

A Mermaid source diagram that lives in the repository document is replaced in this PDF by the native figures in this chapter. `cloud-foundry/manifest.yml`: health check `http` on `/actuator/health/readiness`, env vars map bound services to `SPRING_DATASOURCE_URL` etc.; rolling deploy via `cf push --strategy rolling` (not executed).

4. Network and trust zones

Zone	Components	Rules
Public edge	Ingress / GoRouter	TLS, WAF/rate limit (prod-vs-reference gap), only `/api/**`, `/actuator/health/**` exposed; `/actuator/prometheus` internal only
App	API/consumer pods	No inbound from data zone; egress allow-list
Data	PostgreSQL, Rabbit, Redis, Mongo	Inbound only from app zone; credentials from secret store
Provider egress	Providers (simulated here)	Egress via allow-list/proxy; callbacks arrive via edge and are HMAC-verified

5. Environments

Env	Profile	Notes
dev laptop	`local`	all in-memory, dev token enabled
CI	`test`	deterministic, H2; `-Pcontainers` optional
staging/prod	`prod-like` + `rabbit`	PostgreSQL, Redis, Mongo, issuer-uri, no dev endpoints

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

19. CI/CD Pipeline

> **Execution status: NOT executed**
>
> None of the three pipeline definitions has run on a CI server. Their YAML was parsed locally by `bash scripts/validate-infra.sh` (using the in-repo parser `scripts/lib/yaml-lite.mjs`); the Jenkinsfile Groovy was not linted. Steps that call Docker, Trivy, `cf`, kubeconform, helm or shellcheck were never run on the authoring machine.

19.1 One pipeline, three syntaxes

The same stage map is expressed for GitHub Actions (`.github/workflows/ci.yml`, `release.yml`), Jenkins (`Jenkinsfile`) and Bamboo (`bamboo-specs/bamboo.yaml`), because enterprises differ in what they run, and because a pipeline described three times shows that the stages are the design, not the tool.

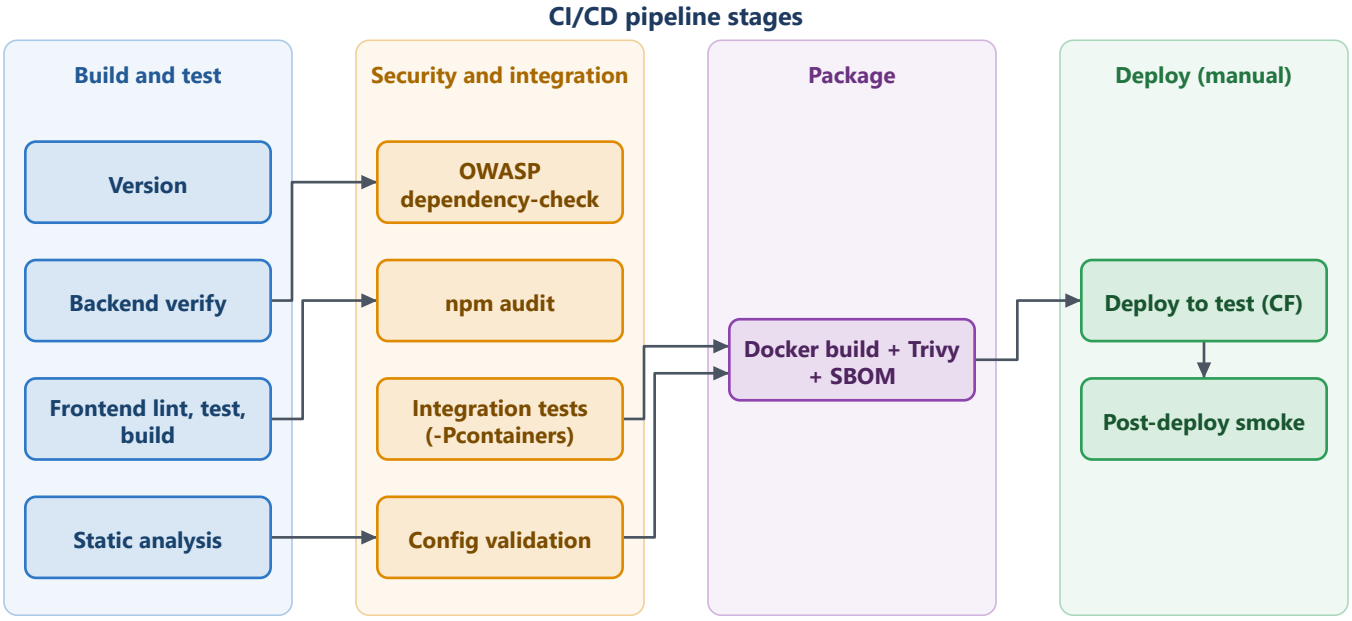


Figure 45. Pipeline stage map shared by GitHub Actions, Jenkins and Bamboo definitions. NOT executed on any CI server; YAML parsed locally, Jenkinsfile Groovy not linted. Implementation: .github/workflows/ci.yml, .github/workflows/release.yml, Jenkinsfile, bamboo-specs/bamboo.yaml

19.2 Stages and gates

Stage	Purpose	Merge-blocking
Version	Compute `<base>.<run number>` (base `0.1`)	n/a
Backend build and test	`mvn verify` (unit and integration tests on H2, in-memory adapters)	Yes
Frontend	Lint, Karma unit tests, production build	Yes
Static analysis	Profile `-Pstatic-analysis`	Yes

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Stage	Purpose	Merge-blocking
OWASP dependency-check	Profile `-Psecurity-scan`, optional `NVD_API_KEY`	Yes
npm audit	Frontend dependency vulnerabilities	Yes
Integration tests	Profile `-Pcontainers`: Testcontainers for PostgreSQL, RabbitMQ, Redis, MongoDB	Yes in CI
Config validation	Compose, kustomize, kubeconform, helm, CF manifest, shellcheck, promtool, repository checks	Yes
Docker build, Trivy, SBOM	Image build; Trivy HIGH or CRITICAL (unfixed ignored); CycloneDX SBOM	Yes
Deploy to test	Manual, only from `main`, secrets-gated	n/a
Post-deploy smoke	`cloud-foundry/scripts/smoke.sh`: liveness, readiness, unauthenticated `/api/v1/me` is 401	Gate for the deploy
Release	Tag `v*.*.*` publishes images with that exact semantic version	n/a

A skipped deploy (no secrets configured) reports a clear "skipped" summary and is **not** a pass.

19.3 Artifact versioning and immutability

- Build version: `<base>.<run number>`, for example `0.1.57`.
- Image tag: `<version>-<git sha7>`, for example `0.1.57-3fa9c21`; Docker tags cannot contain `+`.
- Images carry OCI labels `version` and `revision`. Deploy by immutable tag or digest, never `latest`.
- Bump `VERSION_BASE` deliberately in the workflow, the Jenkinsfile and the Bamboo variable together.

19.4 Secrets in the pipeline

Name	Used for	Where
`NVD_API_KEY`	Faster OWASP dependency-check (optional)	GitHub secret, Jenkins credential `nvd-api-key`, Bamboo variable
`CF_API`, `CF_USERNAME`, `CF_PASSWORD`, `CF_ORG`, `CF_SPACE`	Test deploy	GitHub environment `test`, Jenkins `cf-test-creds`, Bamboo variables
`GITHUB_TOKEN`	Image push in the release workflow	Provided by Actions

No real secret appears in any pipeline file; Bamboo `CHANGE_ME_LOCAL_ONLY` entries are dummy placeholders.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

19.5 What can be reproduced locally

```bash
bash scripts/validate-infra.sh          # static validations; prints SKIPPED for absent tools
node cloud-foundry/validate-manifest.mjs
```

The backend and frontend test suites can be run locally (`bash scripts/mvn.sh clean verify`, `npm test` in `frontend/messaging-console`); the results that were actually produced are recorded in `docs/validation/VALIDATION_REPORT.md`.

19.6 Known gaps recorded by the pipeline author

- Profiles `static-analysis`, `security-scan` and `containers` must exist in `backend/pom.xml`; the static-analysis job fails fast if its profile id is missing. No Checkstyle or SpotBugs configuration files were added under `backend/config/`.
- `cloud-foundry/manifest.yml` expects `backend/target/messaging-platform.jar`; align the jar name with the POM `finalName`.
- Third-party actions are tag-pinned, not SHA-pinned; pin to SHAs in a real organisation.
- The Jenkins agent needs labels `linux && docker`, tools `jdk21` and `nodejs22`, plus trivy and cf CLIs.

19.7 Release readiness and rollback

The release gate is a checklist, not a feeling: `docs/operations/release-readiness-checklist.md` (tests, security, migrations, observability, runbooks) and `docs/operations/rollback-plan.md` (every release is rollback-safe by construction: expand-only schema changes and backward-compatible messages and API, so rollback means deploying the previous immutable version rather than restoring data).

Reference material

CI/CD documentation

Source document: `docs/operations/ci-cd.md`

Execution status: NOT executed. None of the three pipeline definitions has run on a CI server. Their YAML was parsed locally (`bash scripts/validate-infra.sh`, in-repo parser `scripts/lib/yaml-lite.mjs`); the Jenkinsfile Groovy was not linted. Steps that call Docker, Trivy, cf, kubeconform, helm or shellcheck were never run on the authoring machine.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Stage map

Stage	GitHub Actions (`.github/workflows/ci.yml`)	Jenkins (`Jenkinsfile`)	Bamboo (`bamboo-specs/bamboo.yaml`)
Version	job `version`	stage `Version`	`${bamboo.version_base}.${bamboo.buildNumber}` in job script
Backend build + unit tests	`backend-build-test` (`mvn verify`)	`Build and test` / `Backend verify`	stage Build and Test / job Backend Verify
Frontend lint/test/build	`frontend`	`Frontend lint, test, build`	job Frontend Build
Static analysis	`static-analysis` (`-Pstatic-analysis`)	`Static analysis`	job Static Analysis
OWASP dependency-check	`owasp-dependency-check` (`-Psecurity-scan`, optional `NVD_API_KEY`)	`OWASP dependency-check` (`withCredentials` id `nvd-api-key`)	job Dependency Check
npm audit	`npm-audit`	`npm audit`	job Dependency Check
Integration tests	`backend-integration-tests` (`-Pcontainers`)	`Integration tests (Testcontainers)`	job Integration Tests
Config validation	`config-validation` (compose, kustomize, kubeconform, helm, CF manifest, shellcheck, promtool, repo checks)	`Config validation`	job Config Validation
Docker build + Trivy + SBOM	`docker-build-scan` (matrix backend/frontend)	`Docker build and scan`	stage Package / job Docker Images
Deploy to test (manual)	`deploy-test` (`workflow_dispatch`, environment `test`, secrets-gated)	`Deploy to test` (param + `input` gate, credentials id `cf-test-creds`)	stage Deploy Test (manual)
Post-deploy smoke	final step in `deploy-test` (`cloud-foundry/scripts/smoke.sh`)	`Post-deploy smoke`	task in Deploy Test CF
Release	`.github/workflows/release.yml` on tag `v*.*.*`	not defined	deployment project placeholder

Artifact versioning

- Build version: `<base>.<run number>` where `base` is `0.1` (`VERSION_BASE`), e.g. `0.1.57`.
- Image tag: `<version>-<git sha7>`, e.g. `0.1.57-3fa9c21`. Docker tags cannot contain `+`, so build metadata is joined with `-`.
- Release builds: tag `vMAJOR.MINOR.PATCH` publishes images tagged with exactly that semantic version (`release.yml`).
- Images carry OCI labels (`version`, `revision`) via build args `VERSION` and `GIT_SHA`. Deploy by immutable tag (ideally digest), never `latest`.
- Bump `VERSION_BASE` (major/minor) deliberately in the workflow, Jenkinsfile and Bamboo variable together.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Secrets and credentials

Name	Used for	Where defined
`NVD_API_KEY`	faster OWASP dependency-check (optional)	GitHub secret / Jenkins credential `nvd-api-key` / Bamboo variable `nvd_api_key_password`
`CF_API`, `CF_USERNAME`, `CF_PASSWORD`, `CF_ORG`, `CF_SPACE`	CF test deploy	GitHub environment `test` secrets / Jenkins `cf-test-creds` / Bamboo variables
`GITHUB_TOKEN`	GHCR push in release	provided by Actions

No real secret appears in any pipeline file. Bamboo `CHANGE_ME_LOCAL_ONLY` entries are dummy placeholders. The deploy job checks that secrets exist and reports a clear "skipped" summary when they do not: a skipped deploy is not a pass.

Gates

Merge-blocking: backend verify, frontend, config validation, static analysis, Trivy HIGH/CRITICAL (unfixed ignored), OWASP, npm audit. Deploy is manual and only from `main`. Smoke tests assert liveness, readiness and that unauthenticated `/api/v1/me` returns 401.

Known gaps and assumptions

- Profiles `static-analysis`, `security-scan`, `containers` must exist in `backend/pom.xml`. The static-analysis job fails fast if its profile id is missing; the other two would only warn, so confirm them (lead action). No Checkstyle/SpotBugs config files were added under `backend/config/`.
- `deploy-test` pushes `backend/target/*.jar` and `cloud-foundry/manifest.yml` expects `backend/target/messaging-platform.jar`; align the jar name with `finalName` in the POM.
- Third-party actions are tag-pinned, not SHA-pinned. Pin to SHAs in a real organization.
- The Docker builds assume `frontend/messaging-console/package-lock.json` exists and the Angular output path is `dist/messaging-console/browser`.
- Jenkins agent needs labels `linux && docker`, tools `jdk21` and `nodejs22`, plus trivy and cf CLIs.
- SBOMs: GitHub uses anchore/sbom-action (CycloneDX JSON); Jenkins/Bamboo use `trivy image --format cyclonedx`.

Reproduce locally (what can run without CI)

```bash
bash scripts/validate-infra.sh          # static validations; prints SKIPPED for absent tools
node cloud-foundry/validate-manifest.mjs
```

Release readiness checklist

Source document: `docs/operations/release-readiness-checklist.md`

Tick each item with evidence (link/command output) in the release PR. Items marked (N/E) cannot be executed in the authoring environment; in a real pipeline they must be.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Code and tests

- ☐ `mvn -B verify` passes (unit + integration `*IT` on H2). Evidence: CI log or `docs/validation/VALIDATION_REPORT.md`.
- ☐ `FailureRecoveryIT` all 10 scenarios green (see `docs/architecture/reliability-scenarios.md`).
- ☐ Angular: `ng test --watch=false --browsers=ChromeHeadless` and production build pass.
- ☐ `*ContainerIT` (PostgreSQL, RabbitMQ, Redis, Mongo) with `-Pcontainers` (N/E here).
- ☐ OpenAPI committed spec matches springdoc output (test); breaking-change review done.
- ☐ No new TODO/FIXME without ticket; no `System.out`.

Data and schema

- ☐ Flyway migrations are expand-only for rolling deploy (`docs/database/schema-evolution.md`); Oracle reference DDL updated.
- ☐ Large-table index/DDL plan reviewed; migration timing rehearsed on prod-sized copy (N/E).
- ☐ Retention config unchanged or reviewed.

Security

- ☐ Security checklist done (`ai-engineering/checklists/security-review.md`); independent review for auth/tenant/callback/template changes.
- ☐ Dependency, container, secret scans clean or triaged (N/E here).
- ☐ No dev endpoints enabled in `prod-like` (`dev-token-enabled=false`, no `/dev/**`, no fault-mode).
- ☐ Secrets rotated/available per `docs/security/secrets-management.md`.

Reliability and performance

- ☐ Load test vs baseline (k6/harness) no regression > agreed threshold (N/E for k6).
- ☐ Config review: concurrency, prefetch, backoff, pool sizes sane for target environment.
- ☐ Graceful shutdown verified (rolling restart loses no messages: in-flight redelivered, dedupe holds).
- ☐ Capacity/headroom review including DB connections vs instances.

Observability and operations

- ☐ Dashboards and alert rules updated for new metrics; alert-to-runbook links valid.
- ☐ Runbooks updated for new failure modes; on-call briefed.
- ☐ Feature flags / config defaults documented; rollback plan reviewed (`rollback-plan.md`).

Documentation

- ☐ ADR added/superseded for decisions; spec status updated (`specs/`).
- ☐ CHANGELOG/release notes with migration notes and behavior changes.
- ☐ Contract doc (`docs/design/CONTRACT.md` or DEVIATIONS) in sync.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Deployment

- ☐ Staging soak with synthetic traffic and fault injection (N/E here).
- ☐ Canary plan: % traffic/instances, success criteria (error rate, dispatch delay p95, DLQ growth), abort criteria.
- ☐ Post-deploy verification script (smoke: submit, status, callback via simulator) passes.
- ☐ Go/no-go recorded by release owner and on-call.

Rollback plan

Source document: `docs/operations/rollback-plan.md`

Principle: every release must be rollback-safe by construction (expand-only schema, backward-compatible messages and API) so rollback is "deploy previous version", not "restore data". Procedures below are design-level; not executed here.

Decision criteria (during canary or after full rollout)

Roll back when any holds for > 5 min (illustrative): API 5xx burn alert firing; dispatch delay p95 > 2x baseline; DLQ growth with new reason; any suspected duplicate-send or wrong-recipient/content behavior (roll back immediately); provider success rate regression correlated with release; data-consistency invariant violated (see `ai-engineering/context-engineering.md` invariants).

Compatibility requirements (checked in release readiness)

Surface	Rule
Database	Previous app version must run on the new schema: only additive migrations; no drop/rename in same release
Broker	`DispatchEnvelope` changes are additive; consumers ignore unknown fields; old consumers must handle new envelopes (rolling mixed versions)
API	Additive only within `/api/v1`; removal needs version/deprecation
Config	New keys have safe defaults; removing a key waits one release
Templates/policies	Data-driven; no code coupling
Redis/Mongo	Key/collection shapes versioned; additive

Procedures

Kubernetes (not executed)

1. `kubectl rollout undo deployment/messaging-api` (or Helm `helm rollback messaging <revision>`).
2. Watch `kubectl rollout status`; verify readiness; check queue depth and DLQ.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Cloud Foundry (not executed)

1. Blue-green: map route back to the previous app (`cf map-route` / `cf unmap-route`), or `cf push` previous droplet (`cf rollback <app>` if the CLI version supports it).

Feature flag / config rollback

Prefer disabling the feature via config/flag (e.g., priority aging off, provider disabled) which is faster than redeploy.

Data handling after rollback

Situation	Action
Messages accepted by new version	remain valid; old version processes them (compatibility rule)
Messages in `DELIVERY_UNKNOWN` created during bad window	reconcile as usual; do not bulk-fail
Wrongly sent content	cannot be recalled: incident process, customer comms owned by business; stop further sends: pause consumers, cancel queued messages by filter
DLQ growth during bad window	analyze; replay after fix per replay policy
Schema expanded columns unused	leave until next release; contract phase later

Forward-fix vs rollback

Forward-fix only when the fix is trivial, tested, and risk of staying on the bad version is low. Otherwise roll back first.

Verification after rollback

Smoke test, SLIs normalized, backlog draining, no new error classes, audit entry for rollback decision (who/when/why).

Post-rollback

PIR (`post-incident-review-template.md`), add regression test, update checklists/invariants if a class of bug was missed.

Pull request checklist

Source document: `docs/leadership/pr-checklist.md`

The PR template (`.github/pull_request_template.md`) embeds this list. Reviewer and author both tick.

Author

- ☐ Linked spec/issue; scope matches; PR small enough to review (< ~400 changed lines or split).
- ☐ ADR added/updated if a cross-cutting decision changed; no silent ADR override.
- ☐ Tests: unit + integration for new behavior; regression test for each bug fix; no `@Disabled` without ticket.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- ☐ State machine: any status change goes through `MessageStateMachine` and is covered by the transition matrix test.
- ☐ Idempotency considered for every new write path (API, consumer, callback, provider).
- ☐ Tenant isolation: every new repository read/write takes tenant; cross-tenant test added.
- ☐ Authorization: role + resource-level check; endpoint added to the role matrix test.
- ☐ No transaction held across provider/remote calls; no unbounded queues/threads/collections.
- ☐ Timeouts on every remote call; resources closed (try-with-resources).
- ☐ Logging: no secrets, bodies, full addresses; MDC fields set; levels sane.
- ☐ Metrics: names per CONTRACT section 8 or documented additions.
- ☐ DB: migration is expand-only, portable (H2+PG), indexed for new queries, N+1 checked.
- ☐ API: backward compatible; OpenAPI updated; error codes from the list.
- ☐ Docs/runbooks updated; config keys documented with defaults.
- ☐ Not-executed items called out honestly in the PR description.

Reviewer

- ☐ Understand the intent; read tests first.
- ☐ Walk through failure paths: crash after each step, duplicate delivery, concurrent actors.
- ☐ Verify invariants list (`ai-engineering/context-engineering.md`).
- ☐ Security-sensitive? Request independent security review (CODEOWNERS).
- ☐ Performance: queries per request, pool use, allocation in hot paths.
- ☐ Naming/readability; no dead code; no speculative abstraction.
- ☐ Approve only when all above are satisfied; leave actionable comments classified blocking vs nit.

See also `ai-engineering/checklists/code-review.md` and worked cases in `code-review-scenarios.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

20. Observability and Performance

> **Validation status**
>
> Prometheus, alert, recording-rule, Grafana and OpenTelemetry files were validated by **parsing only** (`bash scripts/validate-infra.sh`). `promtool` is not installed, so the PromQL was not checked by Prometheus, and no dashboard was rendered by Grafana. The k6 scripts were **not executed**. Throughput and latency numbers are quoted only from `docs/validation/` (see `docs/validation/VALIDATION_REPORT.md` and `docs/validation/performance-results.md` if present); this chapter's capacity charts are **illustrative arithmetic**.

20.1 Three signals

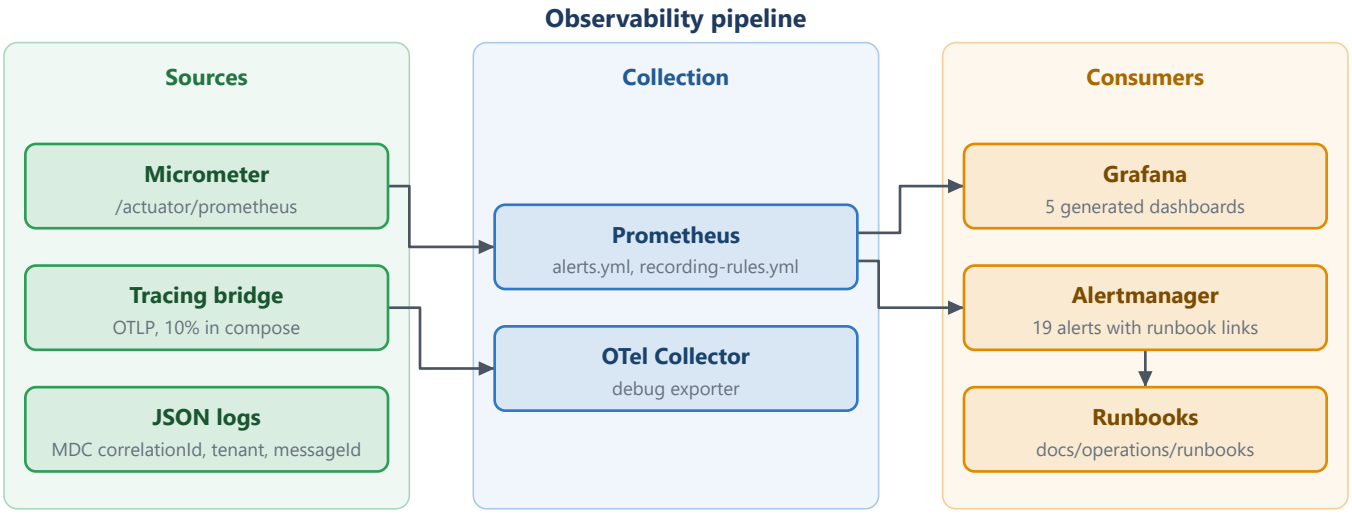


Figure 46. Signals leave the backend as Prometheus metrics, OTLP traces and JSON logs. Config files were parsed only: PromQL was not checked by promtool and no dashboard was rendered. Implementation: observability/prometheus/, observability/otel/otel-collector.yaml, observability/grafana/, backend/src/main/java/com/example/messaging/metrics/MessagingMetrics.java

Signal	Source	Destination	Notes
Metrics	Micrometer, `/actuator/prometheus`	Prometheus (`observability/prometheus/prometheus.yml`)	Percentile histograms enabled for `http.server.requests`, `messaging.dispatch.delay`, `messaging.provider.latency`
Traces	`micrometer-tracing-bridge-otel` plus OTLP exporter	OTel Collector (`observability/otel/otel-collector.yaml`), `debug` exporter by default	Disabled locally; 10 percent sampling to `otel-collector:4318` in Compose
Logs	JSON console logs	stdout, shipped by the platform agent	MDC keys `correlationId`, `tenant`, `messageId`; no content or addresses

Correlation is end to end: `CorrelationIdFilter` accepts or generates `X-Correlation-Id`, echoes it in the response and in every `ApiError`, copies it into the MDC, stores it on the message (`correlation_id`) and carries it in the `DispatchEnvelope`, so a customer complaint can be followed from the API call through the queue to the provider

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

attempt.

20.2 Metrics catalogue

The meter names are an API: dashboards and alerts depend on them (`metrics/MessagingMetrics`). Micrometer's Prometheus naming turns dots into underscores, adds `_total` to counters and `_seconds` to timers.

Meter	Type	Tags	Prometheus series
`messaging.messages.accepted`	counter	`channel`, `priority`, `tenant`	`messaging_messages_accepted_total`
`messaging.transitions`	counter	`from`, `to`	`messaging_transitions_total`
`messaging.dispatch.delay`	timer		`messaging_dispatch_delay_seconds_*`
`messaging.provider.calls`	counter	`provider`, `channel`, `outcome`	`messaging_provider_calls_total`
`messaging.provider.latency`	timer	`provider`	`messaging_provider_latency_seconds_*`
`messaging.provider.circuit.state`	gauge	`provider` (0 closed, 1 half-open, 2 open)	`messaging_provider_circuit_state`
`messaging.queue.depth`	gauge	`priority`	`messaging_queue_depth`
`messaging.queue.inflight`, `messaging.queue.dlq.depth`	gauge		`messaging_queue_inflight`, `messaging_queue_dlq_depth`
`messaging.queue.oldest.age`	gauge	seconds (added in stage 2)	`messaging_queue_oldest_age`
`messaging.retry.scheduled`	counter	`channel`	`messaging_retry_scheduled_total`
`messaging.deadletter.total`	counter	`reason`	`messaging_deadletter_total`
`messaging.callbacks`	counter	`result` (accepted, duplicate, replayed, invalid_signature, out_of_order, stale, conflict, unknown_message, malformed)	`messaging_callbacks_total`
`messaging.bulk.progress`	gauge	`jobId`	`messaging_bulk_progress`
`messaging.idempotency.hits`	counter		`messaging_idempotency_hits_total`
`messaging.ratelimit.rejected`	counter	`scope`	`messaging_ratelimit_rejected_total`
`messaging.redis.fallback`	counter		`messaging_redis_fallback_total`
`messaging.diagnostics.write.failures`	counter		`messaging_diagnostics_write_failures_total`
`messaging.reconciliation`	counter	`result`	`messaging_reconciliation_total`
`messaging.bulk.writers.active`, `messaging.retention.deleted`, `messaging.retention.run`	gauge, counter	`table`, `dryRun`	additive

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Plus Spring, HikariCP and JVM defaults (`hikaricp_connections_*`, `http_server_requests_seconds`). Two implementation notes from the deviations record: Micrometer holds gauge suppliers weakly, so `MessagingMetrics` keeps strong references to the suppliers handed to `bindQueueStats` and `bindCircuitState` (otherwise gauges turned `NaN` after a garbage collection); and `publishPercentiles(0.5, 0.95, 0.99)` on the dispatch and provider timers supplies the p95 values shown by `/providers` and `/ops/summary`.

20.3 SLIs, SLOs and alerts

All targets are **illustrative**, not commitments of any organisation, and not derived from measured production traffic. The table summarises `docs/operations/slis-slos.md`.

SLI	Illustrative SLO (30 days)
API availability on `POST /api/v1/messages`	99.9 percent
API latency	99 percent under 300 ms, p95 under 150 ms
Acceptance durability (no accepted message lost)	99.99 percent
Dispatch delay	p95 under 5 s CRITICAL, 30 s HIGH, 5 min NORMAL
Provider submission success	99 percent per provider
Callback processing	99.9 percent under 10 s
Backlog health	oldest queued CRITICAL under 30 s, NORMAL under 15 min
Unresolved ambiguity	`DELIVERY_UNKNOWN` older than 30 min under 0.1 percent of volume
DLQ	depth under 100, no growth for more than 1 h

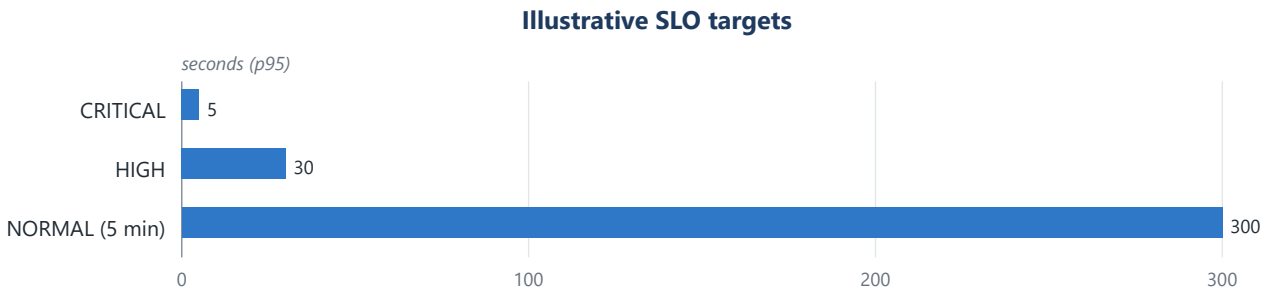


Figure 47. ILLUSTRATIVE dispatch-delay p95 targets by priority from the SLI table; stakeholder-agreed values would replace them. Implementation: docs/operations/slis-slos.md, observability/prometheus/alerts.yml

`observability/prometheus/alerts.yml` defines **19 alerts**, each carrying a `runbook_url`, with multi-window burn-rate alerting for the 99.9 percent availability objective (14.4x over 1 h and 5 min pages; 6x over 6 h and 30 min raises a ticket). Examples: `ProviderErrorRatioHigh`, `ProviderCircuitOpen`, `QueueDepthHigh`, `DispatchDelayHigh`, `DlqGrowth`, `RetryStorm`, `DeliveryUnknownGrowth`, `CallbackInvalidSignatureSpike`, `HikariPoolSaturation`, `RedisFallbackActive`, `BulkJobStalled`. `scripts/lib/static-checks.mjs` fails if a `messaging_*` series in the observability files is not derived from the meter catalogue.

Five Grafana dashboards are generated by `observability/grafana/build-dashboards.mjs` and committed: API and SLO, Dispatch and Queues, Providers, Bulk and Batch, JVM and DB Pool.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

20.4 Performance engineering approach

Performance work follows a fixed method (`docs/operations/performance-investigation-guide.md`): measure first, find the saturated resource (CPU, DB pool, queue, provider limit), change one thing, measure again. The structural choices that keep latency and throughput predictable are in the design, not in tuning:

Design choice	Performance effect
202 after one short transaction, no provider call on the request thread	Submit latency independent of provider latency
No transaction across a provider call	Connection pool bounded by DB time
Separate provider executor	A hanging provider cannot starve workers or Tomcat
Priority queues, prefetch 16, 8 workers	Urgent traffic first; bounded in-flight work
Token bucket per provider	The platform never exceeds a vendor quota
Streaming CSV reader and chunked commits	Bulk memory is constant, not proportional to file size
Indexes matched to hot queries	Poller, outbox and list queries avoid scans
No N+1	Fetch joins, projections; `RepositoryQueryCountIT`
Backpressure	Overload becomes a clean 503 rather than a collapse

Illustrative capacity arithmetic

The high-level design sketches capacity as arithmetic, to show **how to reason**, not what the system achieves. Assume a 300 ms provider call and 8 workers per instance:

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

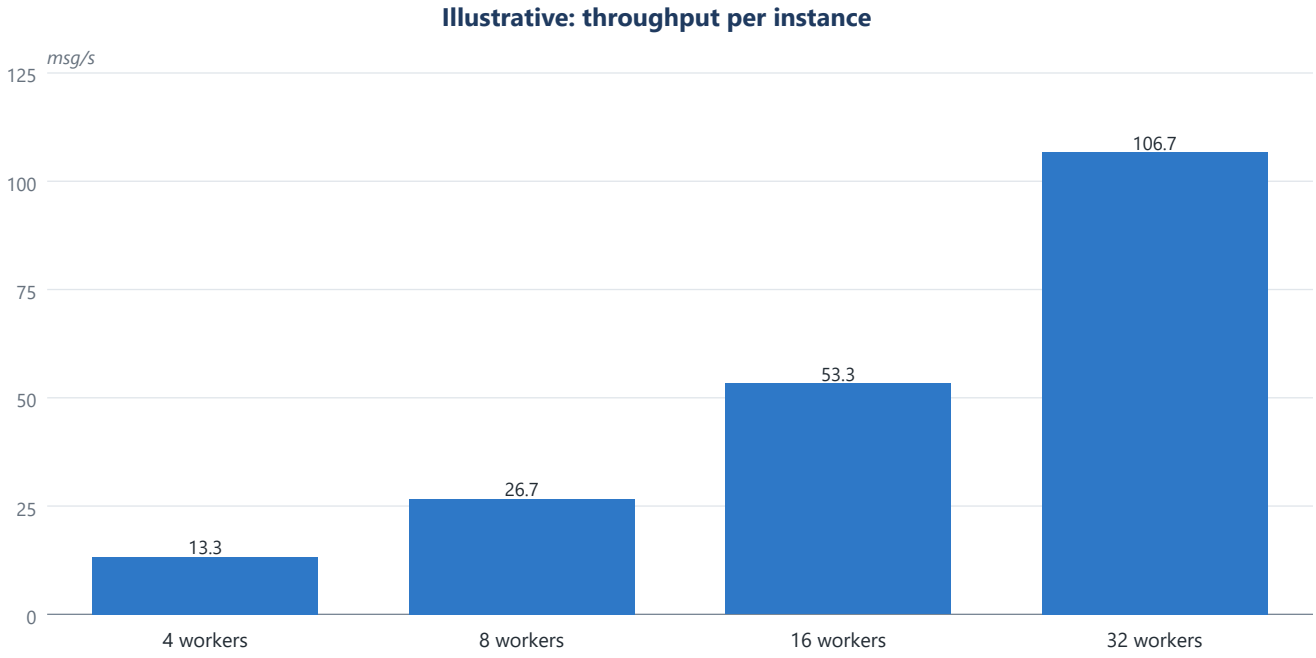


Figure 48. ILLUSTRATIVE capacity arithmetic, not a measurement: throughput of one instance = workers / provider latency, for a 300 ms provider call (workers divided by 0.3 s). Implementation: docs/leadership/hld.md section 6, backend/src/main/resources/application.yml (messaging.dispatch.concurrency)

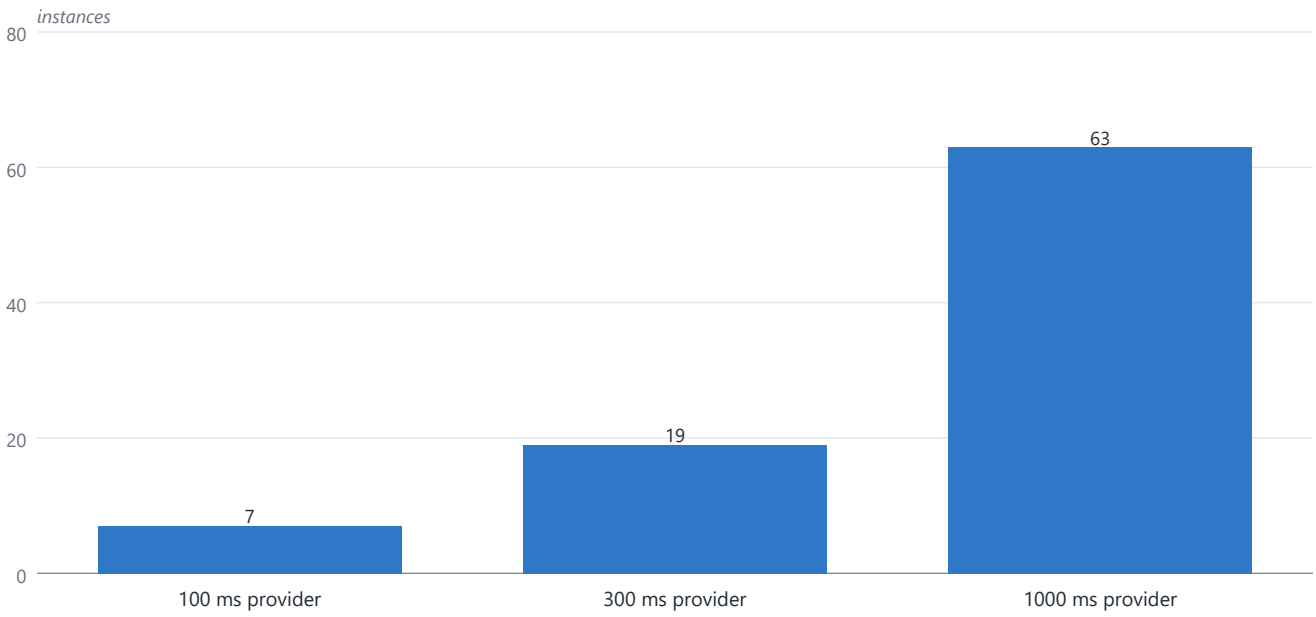


Figure 49. ILLUSTRATIVE: instances needed to sustain 500 msg/s with 8 workers each, as provider latency varies. Real capacity is bounded by provider quota first. Computed, not measured. Implementation: docs/leadership/hld.md section 6

The second chart assumes a target of 500 messages per second sustained and shows why provider latency, not platform code, dominates the instance count, and why a vendor rate limit usually binds before the platform does. None of these numbers was measured.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

20.5 Load tests and what was and was not run

Tool	Status	What it is for
`performance/node-load/load-driver.mjs`	Runnable locally; a result exists only if recorded in `docs/validation/`	Zero-dependency driver with p50, p95, p99 for submit, read, mixed and idempotent scenarios
In-JVM tests (`BulkThroughputIT`)	Run locally on H2 and the in-memory bus as part of the Maven build; any measured number is quoted only from `docs/validation/`	Throughput of the bulk pipeline in one JVM
`performance/k6/*.js` (8 scripts)	**NOT executed**	submit-high-volume, concurrent-consumers, provider-rate-limit, bulk-processing, db-throughput, backlog-recovery, retry-storm, dashboard-queries

> **How to read any local number**
>
> Local runs use H2 and the in-memory bus; they say nothing about PostgreSQL or RabbitMQ capacity. Simulated providers have artificial latency and fault models, so the "provider rate limit" scenario exercises the platform's limiter and requeue logic, not a real vendor.

Reference material

Observability

Source document: `docs/operations/observability.md`

Validation status. Prometheus, alert, recording-rule, Grafana and OTel files were validated by parsing only (`bash scripts/validate-infra.sh`: YAML/JSON syntax plus structural checks). `promtool` is not installed here, so the PromQL was **not** checked by Prometheus, and no dashboard has been rendered by Grafana. Nothing was scraped from a running backend by these files. Treat all thresholds as illustrative starting points.

Signals

Signal	Source	Destination
Metrics	Micrometer, `/actuator/prometheus`	Prometheus (`observability/prometheus/prometheus.yml`)
Traces	`micrometer-tracing-bridge-otel` + OTLP exporter	OTel Collector (`observability/otel/otel-collector.yaml`), `debug` exporter by default
Logs	JSON console logs with MDC `correlationId`, `tenant`, `messageId`	stdout (ship with your platform agent)

Locally, tracing export is disabled; in compose the backend sends OTLP/HTTP to `otel-collector:4318` at 10% sampling.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Metric naming

CONTRACT section 8 meter names become Prometheus names by Micrometer rules: dots become underscores, counters gain `_total`, timers gain `_seconds` (with `_count`, `_sum`, `_bucket`).

CONTRACT meter	Prometheus series
`messaging.messages.accepted{channel,priority,tenant}`	`messaging_messages_accepted_total`
`messaging.transitions{from,to}`	`messaging_transitions_total`
`messaging.dispatch.delay`	`messaging_dispatch_delay_seconds_{count,sum,bucket}`
`messaging.provider.calls{provider,channel,outcome}`	`messaging_provider_calls_total`
`messaging.provider.latency{provider}`	`messaging_provider_latency_seconds_*`
`messaging.provider.circuit.state{provider}`	`messaging_provider_circuit_state` (0 closed, 1 half-open, 2 open)
`messaging.queue.depth{priority}` / `.inflight` / `.dlq.depth`	`messaging_queue_depth`, `messaging_queue_inflight`, `messaging_queue_dlq_depth`
`messaging.retry.scheduled{channel}`	`messaging_retry_scheduled_total`
`messaging.deadletter.total{reason}`	`messaging_deadletter_total`
`messaging.callbacks{result}`	`messaging_callbacks_total`
`messaging.bulk.progress{jobId}`	`messaging_bulk_progress`
`messaging.idempotency.hits`, `messaging.ratelimit.rejected{scope}`, `messaging.redis.fallback`, `messaging.diagnostics.write.failures`	`*_total` counters

`scripts/lib/static-checks.mjs` fails if any `messaging_*` name in observability files is not derived from section 8.

Assumptions to verify against the running backend

1. Histogram buckets (`*_bucket`) exist only if percentile histograms are enabled for `http.server.requests`, `messaging.dispatch.delay` and `messaging.provider.latency` (`management.metrics.distribution.percentiles-histogram.*`). Without them the p95 recording rules and three alerts (`DispatchDelayHigh`, `ApiLatencyP95High`, provider latency panel) return no data.
2. `messaging.bulk.progress` is treated as a 0..1 fraction for `BulkJobStalled`. If it is a row count, change the `< 1` bound.
3. CONTRACT has no "oldest message age" meter even though the ops API reports one. Dispatch-delay p95 and queue-depth slope are used as proxies. Suggested addition (not applied, record in DEVIATIONS.md if adopted): gauge `messaging.queue.oldest.age`.

Recording rules (`recording-rules.yml`)

SLIs: API request/error rate and error ratio over 5m/30m/1h/6h, API p95, dispatch delay p95, accept rate, total queue depth, provider call rate, failure ratio and p95 latency per provider, invalid-signature rate.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Alerts (`alerts.yml`, 19 alerts)

Alert	Meaning	Severity	Runbook
ProviderErrorRatioHigh	>20% provider calls failing 5m	page	provider-outage
ProviderCircuitOpen	breaker state 2 for 2m	page	provider-outage
ProviderRateLimited	sustained RATE_LIMITED outcomes	ticket	provider-rate-limit
QueueDepthHigh	>10k queued for 10m	page	queue-backlog
QueueBacklogGrowing	depth slope >5/s for 15m	ticket	queue-backlog
DispatchDelayHigh	p95 accepted-to-provider >30s	page	queue-backlog
DlqGrowth	DLQ growth or depth >100	page	dlq-growth
RetryStorm	retries >50% of accept rate	page	retry-storm
DeliveryUnknownGrowth	transitions to DELIVERY_UNKNOWN rising	page	consumer-crash-ambiguous-outcomes
CallbackInvalidSignatureSpike	invalid HMAC signatures	page	callback-signature-failures
ApiErrorBudgetFastBurn / SlowBurn	multi-window burn rate for 99.9% SLO (14.4x and 6x)	page / ticket	high-api-latency
ApiLatencyP95High	p95 >300ms	page	high-api-latency
BackendDown	scrape target down 2m	page	high-api-latency
HikariPoolSaturation	active/max >90% or pending >5	page	database-outage
JvmGcPauseHigh	GC >10% of wall time	ticket	high-api-latency
RedisFallbackActive	coordination falls back from Redis	ticket	redis-outage
BulkJobStalled	no bulk progress for 10m	ticket	bulk-job-restart
DiagnosticsWriteFailures	Mongo diagnostics writes failing	ticket	database-outage

Every alert carries `runbook_url: docs/operations/runbooks/<name>.md`; the eleven runbook names are fixed and checked by the static checks. The runbook documents themselves are authored elsewhere in `docs/operations/runbooks/`.

Grafana

Provisioned automatically in compose (datasource uid `prometheus`, folder "Messaging Platform"). Dashboards are generated by `node observability/grafana/build-dashboards.mjs` and committed: API and SLO, Dispatch and Queues, Providers, Bulk and Batch, JVM and DB Pool. Open `http://localhost:13000` (admin password from `.env`). Not rendered/verified here.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Local usage

```bash
docker compose --env-file .env -f infrastructure/compose/docker-compose.yml up -d prometheus grafana
  otel-collector   # NOT executed here
# Backend run locally via scripts/run-local.sh is scraped through job "messaging-backend-local"
  (host.docker.internal:18080).
```

Without Docker, scrape manually: `curl http://localhost:18080/actuator/prometheus | grep '^messaging_'`.

Tuning notes

- Burn-rate windows follow the multi-window pattern: page when both a short and long window exceed the budget burn.
- Keep alert `for:` durations above the scrape/evaluation interval (15s) and above normal batch bursts.
- Label cardinality: `messaging.bulk.progress{jobId}` creates a series per job; drop old jobs or aggregate if the job count grows.

SLIs, SLOs and alerting

Source document: `docs/operations/slis-slos.md`

All targets are illustrative examples for this reference implementation. They are not commitments of any organization and were not derived from measured production traffic. Real SLOs come from stakeholder agreement and measured baselines.

SLI / SLO table

#	SLI	Definition (good / valid events)	Source metric	Illustrative SLO (30 d window)
1	API availability	non-5xx responses / all responses on `/api/v1/messages` POST (excluding 4xx, 503 backpressure counted as bad)	`http_server_requests_seconds_count{uri="/api/v1/messages", method="POST"}` by `status`	99.9%
2	API latency	POST /messages under 300 ms	`http_server_requests_seconds_bucket`	99% < 300 ms; p95 < 150 ms
3	Acceptance durability	accepted (202) messages that reach a post-VALIDATED state within 60 s of acceptance, or are scheduled in future	`messaging.transitions{to="QUEUED"}` vs `messaging.messages.accepted`	99.99% (no accepted message lost)
4	Dispatch delay (CRITICAL/HIGH)	accepted to provider submit	`messaging.dispatch.delay` timer (priority tag if present, see source)	p95 < 5 s for CRITICAL, p95 < 30 s for HIGH, p95 < 5 min NORMAL

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	SLI	Definition (good / valid events)	Source metric	Illustrative SLO (30 d window)
5	Provider submission success	attempts with outcome ACCEPTED / attempts excluding PERMANENT_FAILURE for invalid recipients	`messaging.provider.calls{outcome}`	99% per provider (5 min rolling for alerting)
6	Delivery-status processing	callbacks processed (any result except invalid_signature) within 10 s of receipt	`messaging.callbacks{result}`, callback handler `http_server_requests_seconds`	99.9% < 10 s
7	Backlog health	oldest queued message age	`messaging.queue.depth{priority}`, ops endpoint `oldest-message age` (`GET /api/v1/ops/queues`)	CRITICAL < 30 s, NORMAL < 15 min
8	Unresolved ambiguity	messages in DELIVERY_UNKNOWN older than 30 min	derived from `messaging.transitions{to="DELIVERY_UNKNOWN"}` minus resolutions (or DB gauge, see source)	< 0.1% of daily volume
9	DLQ	DLQ depth growth	`messaging.queue.dlq.depth`, `messaging.deadletter.total{reason}`	depth < 100 and no growth for > 1 h

Note: delivery rate to the end handset/mailbox is mostly outside platform control; track it as an indicator, not an SLO.

Error budget policy (example)

Budget status	Action
> 50% remaining	normal feature work
25-50%	review top contributors weekly; risky changes need extra review
< 25%	freeze non-reliability changes except security fixes; focus on reliability backlog
exhausted	freeze all releases except fixes restoring the SLO; mandatory post-incident review

Burn-rate alerting (multi-window): page when 14.4x burn over 1 h and 5 min; ticket at 6x over 6 h and 30 min (standard multi-window pattern; thresholds illustrative).

Alert examples (Prometheus-style; static, not loaded into a Prometheus here)

YAML

```yaml
- alert: ApiAvailabilityBurn
  expr: |
    sum(rate(http_server_requests_seconds_count{uri="/api/v1/messages",method="POST",status=~"5.."}[5m]))
    / sum(rate(http_server_requests_seconds_count{uri="/api/v1/messages",method="POST"}[5m])) > 0.0144
  for: 5m
  labels: {severity: page}
- alert: ProviderSubmissionSuccessLow
  expr: |
    sum by (provider) (rate(messaging_provider_calls_total{outcome="ACCEPTED"}[5m]))
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
    / sum by (provider) (rate(messaging_provider_calls_total[5m])) < 0.95
  for: 5m
  labels: {severity: page}
- alert: ProviderCircuitOpen
  expr: messaging_provider_circuit_state == 2
  for: 2m
  labels: {severity: page}
- alert: QueueBacklogGrowing
  expr: sum(messaging_queue_depth) > 1000 and deriv(sum(messaging_queue_depth)[10m]) > 0
  for: 10m
  labels: {severity: ticket}
- alert: DlqGrowth
  expr: increase(messaging_deadletter_total[15m]) > 20
  labels: {severity: ticket}
- alert: PoisonMessages
  expr: increase(messaging_deadletter_total{reason="POISON_MESSAGE"}[15m]) > 0
  labels: {severity: ticket}
- alert: CallbackSignatureFailures
  expr: increase(messaging_callbacks_total{result="invalid_signature"}[5m]) > 10
  labels: {severity: page}
- alert: RetryStorm
  expr: sum(rate(messaging_retry_scheduled_total[5m])) > 50
  for: 10m
  labels: {severity: ticket}
- alert: RedisFallbackActive
  expr: increase(messaging_redis_fallback_total[5m]) > 0
  labels: {severity: ticket}
- alert: DiagnosticsWriteFailures
  expr: rate(messaging_diagnostics_write_failures_total[10m]) > 0
  for: 30m
  labels: {severity: info}
- alert: HikariPoolSaturated
  expr: hikaricp_connections_pending > 0
  for: 3m
  labels: {severity: page}
- alert: DeliveryUnknownAging
  expr: messaging_delivery_unknown_oldest_age_seconds > 1800   # gauge name: see source; may not exist
  labels: {severity: ticket}
```

Micrometer-to-Prometheus naming: dots become underscores, counters get `_total`, timers become `_seconds_*` (verify exact names against `/actuator/prometheus` of a running instance; the shipped alert rules in `observability/alerts` are the executed-nowhere source of truth).

Dashboard panels (suggested)

Accepted rate by channel/priority; status transitions; dispatch delay percentiles; queue depth by priority + DLQ; provider calls by outcome and circuit state; callbacks by result; retry scheduled; bulk progress; Hikari active/pending; JVM heap/GC pause; HTTP latency by route.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Review cadence

Monthly SLO review; adjust only with data. Link: runbooks in `docs/operations/runbooks/`.

Performance test plan

Source document: `docs/operations/performance-tests.md`

Execution status: NOT executed. No k6, Docker or load environment exists on the authoring machine. The k6 scripts in `performance/k6/` are unrun. The Node driver `performance/node-load/load-driver.mjs` is runnable locally but no measured result is recorded in this repository yet. Do not quote any throughput or latency figure from this repo until the results table in `performance/README.md` is filled from a real run.

Illustrative SLOs (starting points, not measured guarantees)

SLI	Objective	Where measured
API availability (non-5xx on `/api`)	99.9% monthly	recording rule `sli:api_error_ratio:*`
Submit latency	p95 < 300 ms, p99 < 800 ms	k6 threshold on `POST /messages`; `sli:api_latency_p95_seconds:rate5m`
Read latency (lists)	p95 < 500 ms	k6 `db-throughput.js`, `dashboard-queries.js`
Dispatch delay (accepted to provider submit)	p95 < 30 s for NORMAL priority	`messaging.dispatch.delay` timer
Provider failure ratio	< 20% sustained (alert threshold)	`sli:provider_failure_ratio:rate5m`

Scenarios and what each proves

Script	Question it answers	Pass signals	Typical failure meaning
submit-high-volume	Does the accept path hold at 150 req/s with idempotency + outbox writes?	202 rate > 99%, p95 < 300 ms	pool saturation, slow idempotency claim, missing index
concurrent-consumers	How does drain rate scale with `messaging.dispatch.concurrency`?	queue depth sample < 5000, stable in-flight	consumer starvation, provider limiter too tight
provider-rate-limit	Does a provider rate limit degrade gracefully (requeue, no 5xx, no attempt burn)?	no 5xx, retries bounded	attempts consumed on rate limit, hot loops
bulk-processing	Throughput of the Spring Batch job end to end	terminal COMPLETED(*WITH_ERRORS), rows/s above floor*	chunk/commit tuning, skip policy cost
db-throughput	Mixed read/write DB behaviour	read p95 < 500 ms while writing	N+1, missing index, Hikari pending > 0

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Script	Question it answers	Pass signals	Typical failure meaning
backlog-recovery	Time to drain a backlog after consumers resume	drain < 10 min for 5000 msgs (illustrative)	slow consumers, DLQ spill
retry-storm	Are retries bounded (backoff + jitter) under total provider failure?	API 202 stays healthy, retries do not exceed accept rate x attempts, system recovers	retry amplification, synchronized retries
dashboard-queries	Can 30 operators poll the console without hurting dispatch?	p95 < 400 ms on ops endpoints	heavy aggregate queries needing caching

Method

1. Warm the JVM (the Node driver does a 2 s warm-up; for k6 discard the first minute).
2. Run one scenario at a time on an otherwise idle machine; note CPU count and the active profile (H2 + in-memory bus behaves very differently from PostgreSQL + RabbitMQ).
3. Capture: tool output, Grafana dashboards "API and SLO", "Dispatch and Queues", "JVM and DB Pool", backend logs for errors.
4. Compare against thresholds; investigate with the matching runbook in `docs/operations/runbooks/`.
5. Record the run in `performance/README.md` results table (date, commit, environment, command, outcome).

Honesty notes

- Local runs use H2 and the in-memory bus; those numbers say nothing about PostgreSQL/RabbitMQ capacity.
- Simulated providers have artificial latency and fault models; "provider rate limit" here exercises our limiter and requeue logic, not a real provider.
- Fault-injection endpoints exist only in profiles `local`/`test`, so retry-storm and provider-rate-limit cannot run against `prod-like`.

Performance investigation guide

Source document: `docs/operations/performance-investigation-guide.md`

Concrete commands for investigating a running instance. Commands are standard JDK/Linux tools shown for reference; `<pid>` is the JVM process id (`jcmd -l` lists). **None of these were executed against a deployed instance in authoring; a local-run subset can be tried against the `local` profile (port 18080).** Heap dumps contain message variables and recipient data: handle as Confidential, restrict access, delete after use.

0. Method

1. Define the symptom with a metric (not "feels slow"). 2. Locate the bottleneck class: CPU, memory/GC, threads/locks, DB, queue/downstream, network. 3. Change one thing. 4. Verify with the same metric. 5. Record in the PIR.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. First look (metrics)

Question	Metric / endpoint
Request latency/error	`http_server_requests_seconds_{count,sum,bucket}` by `uri`,`status`
JVM heap	`jvm_memory_used_bytes{area="heap"}`, `jvm_memory_max_bytes`
GC	`jvm_gc_pause_seconds_{count,sum,max}`, `jvm_gc_live_data_size_bytes`
Threads	`jvm_threads_live_threads`, `jvm_threads_states_threads{state}`, `tomcat_threads_busy_threads`
CPU	`process_cpu_usage`, `system_cpu_usage`
DB pool	`hikaricp_connections_{active,idle,pending,max}`, `hikaricp_connections_acquire_seconds`, `hikaricp_connections_usage_seconds`
Queue	`messaging_queue_depth`, `messaging_queue_inflight`, `messaging_dispatch_delay_seconds`, `GET /api/v1/ops/queues`
Provider	`messaging_provider_latency_seconds`, `messaging_provider_calls_total{outcome}`

2. CPU

```
top -H -p <pid>                      # per-thread CPU (Linux); note the hottest thread ids (decimal)
printf '%x\n' <tid>                  # convert to hex to match "nid=0x..." in jstack
jcmd <pid> Thread.print > threads.txt # find the thread by nid
jcmd <pid> JFR.start name=cpu settings=profile duration=60s filename=cpu.jfr
jfr print --events jdk.ExecutionSample cpu.jfr | head -200
jfr summary cpu.jfr
```

Open `cpu.jfr` in JDK Mission Control or IntelliJ profiler. Look for: JSON (de)serialization, regex catastrophic backtracking in validators, template rendering loops, logging (synchronous appenders), crypto (HMAC is cheap, RSA verify cached via JWT decoder). Windows equivalent: Task Manager/Process Explorer + same `jcmd` (JDK path: see CONTRACT section 0).

3. Memory and GC

```
jcmd <pid> GC.heap_info
jcmd <pid> GC.class_histogram | head -30       # suspects by instances/bytes
jcmd <pid> GC.heap_dump /secure/tmp/heap.hprof # STOP-THE-WORLD; PII risk
jstat -gcutil <pid> 1000 10                    # occupancy and GC counts every 1 s
jcmd <pid> VM.flags | tr ' ' '\n' | grep -E 'Xmx|Xms|UseG1|UseZGC|MaxRAMPercentage'
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

GC logging at startup (recommended in prod): `-Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=20m`. Read: pause times (G1 target 200 ms by default), allocation rate, "Pause Full" events (bad), humongous allocations (large byte arrays: e.g., unbounded CSV reads or large JSON bodies; we cap body at 256 KB and stream bulk CSV). Typical findings and fixes:

Finding	Likely cause	Fix
Heap grows steadily, old gen not reclaimed	unbounded cache, queue, per-request collections, `InMemoryMessageBus` queue too big (capacity 5000 by design)	bound it; inspect with histogram/MAT dominator tree
Frequent young GCs, high allocation	per-message large object creation, string concatenation, JSON trees	profile allocations (`jdk.ObjectAllocationSample` in JFR)
Container OOM-killed without Java OOM	native/off-heap/threads exceed limit	set `-XX:MaxRAMPercentage=70`, check metaspace/direct buffers (`jcmd <pid> VM.native_memory summary` with `-XX:NativeMemoryTracking=summary`)

4. Threads and locks

```
jcmd <pid> Thread.print -l            # includes locked ownable synchronizers
jstack <pid>                          # same family; take 3 dumps 5-10 s apart and diff
jcmd <pid> Thread.print | grep -c 'HikariPool.getConnection'   # threads waiting for DB connections
jcmd <pid> Thread.dump_to_file -format=json threads.json       # JDK 21: also shows virtual threads
```

Patterns: many threads `WAITING` in `HikariPool.getConnection` => pool exhausted/slow DB; many `BLOCKED` on one monitor => contended lock; consumer workers all in provider socket read => slow provider with no effective timeout (check time limiter); `RUNNABLE` in regex/serialization => CPU hotspot. Starvation checks: provider calls must use the dedicated bounded executor (not Tomcat threads, not the consumer pool itself); scheduler threads must not run long tasks; batch executor bounded.

5. Database

SQL

```
-- PostgreSQL (not executed here)
SELECT pid, state, wait_event_type, wait_event, now()-query_start AS runtime, left(query,100)
FROM pg_stat_activity WHERE state <> 'idle' ORDER BY runtime DESC;
SELECT query, calls, mean_exec_time, rows FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10; --
  needs extension
EXPLAIN (ANALYZE, BUFFERS) <slow query>;
SELECT relname, seq_scan, idx_scan, n_dead_tup FROM pg_stat_user_tables ORDER BY seq_scan DESC;
```

Application side: enable Hibernate statistics in a test and assert statement counts (N+1 detection), slow query log (`spring.jpa.properties.hibernate.session.events.log.LOG_QUERIES_SLOWER_THAN_MS=100`, verify property name for the Hibernate version in use). Pool: see `docs/database/transactions-and-connection-pooling.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Queue lag

`GET /api/v1/ops/queues`; RabbitMQ: `rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers` and management UI publish/deliver/ack rates. Rule: if `unacked` equals `consumers x prefetch` and ready grows, consumers are the bottleneck; if consumers idle and ready grows, check paused consumers, pollers (outbox) or prefetch misconfiguration. Little's law: throughput = in-flight / latency; with 8 workers and 400 ms provider latency max ~20 msgs/s per instance.

7. Network and downstream

Provider latency histograms; Resilience4j state (`messaging_provider_circuit_state`); DNS/TLS handshake time; connect vs read timeouts; retry amplification.

8. In-JVM load harness

`performance/jvm-harness` and `backend/src/test/.../perf/LoadHarnessIT` can be run locally; record numbers only from executed runs, with date and hardware (see `docs/validation/VALIDATION_REPORT.md` and `performance/README.md`).

9. Checklist artifact

`ai-engineering/checklists/performance-review.md`.

Runbook: High latency and resource saturation

Source document: `docs/operations/runbooks/high-latency-and-resource-saturation.md`

Detailed command reference: `../performance-investigation-guide.md`. This runbook is the triage decision tree.

Symptoms and alerts

`http_server_requests_seconds` p95/p99 up for POST /messages; API availability burn; `jvm_gc_pause_seconds` high; CPU > 85%; `hikaricp_connections_pending` > 0; Tomcat busy threads at max.

Triage tree

1. **Is it everywhere or one route?** Compare `http_server_requests_seconds` by `uri`. One route => slow query or downstream. All routes => resource saturation.
2. **DB pool?** `hikaricp_connections_pending > 0` or `acquire_seconds` high => `database-outage.md` (pool/lock/slow query).
3. **GC?** `jvm_gc_pause_seconds_max`, `jvm_memory_used_bytes{area="heap"}` near max, frequent full GCs => memory pressure.
4. **Threads?** `jvm_threads_live`, `tomcat_threads_busy`/ `executor` queue sizes; thread dump shows many threads BLOCKED/WAITING on the same lock or `HikariPool.getConnection`.
5. **CPU?** process CPU high with low GC => hot loop/regex/serialization/crypto (HMAC and JWT verify are cheap; JSON and rendering at volume can be hot); profile with JFR.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. **Downstream?** `messaging.provider.latency`, Redis/Mongo latency (diagnostics write on request thread? it must not be).
7. **Noisy tenant?** `messaging.messages.accepted` by tenant; rate limit rejections.

Immediate mitigation

Finding	Action
Heap exhaustion	restart one instance at a time after capturing heap histogram/dump (`jcmd <pid> GC.class_histogram` , `jcmd <pid> GC.heap_dump <file>` store securely: heap contains PII-like data), increase memory only with justification
Thread starvation by blocking calls	confirm provider calls run on the dedicated bounded executor with timeouts; reduce `dispatch.concurrency` if downstream slow; check no provider calls on Tomcat threads
CPU	scale out replicas; capture JFR (`jcmd <pid> JFR.start duration=60s filename=...`) for root cause
Slow query	add index/kill query; see `query-plans-and-indexes.md`
Noisy tenant/bulk	throttle/stop bulk job; tighten tenant rate limit
Bad release	rollback (`../rollback-plan.md`)

Verify

p95/p99 back under SLO; pool pending 0; GC pause normal; no BLOCKED thread hot spots.

Prevent

Load tests with GC/heap profile, timeouts everywhere, bounded pools, resource limits, perf review checklist (`ai-engineering/checklists/performance-review.md`).

Runbook: Queue backlog

Source document: `docs/operations/runbooks/queue-backlog.md`

Symptoms and alerts

`QueueBacklogGrowing`; `messaging.queue.depth{priority}` rising; oldest-message age above SLO; `messaging.dispatch.delay` p95 up; API returns 503 `Retry-After` when `max-queue-depth` exceeded.

Impact

Dispatch delay; CRITICAL traffic protected by priority but affected at extreme depth; clients see 503 with `Retry-After` at the backpressure threshold.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Diagnose (decide: producers too fast, consumers too slow, or downstream throttled)

1. `GET $BASE/api/v1/ops/queues` : depth per priority, in-flight, consumers, retry-pending, oldest age.
2. Producers: `rate(messaging_messages_accepted_total[5m])` by tenant/channel (bulk job? `GET /api/v1/bulk-jobs`).
3. Consumers: are workers alive (`messaging.queue.inflight` > 0)? Paused (`/ops/consumers/pause` used)? Thread dump if stuck (see performance guide: `jcmd <pid> Thread.print`).
4. Downstream: `messaging.provider.latency`, `RATE_LIMITED` outcomes, circuit state. A slow provider with fixed concurrency means throughput = concurrency / latency.
5. DB: `hikaricp_connections_pending`.
6. Broker health (Rabbit management UI/`rabbitmqctl list_queues name messages consumers`) in real environments.

Mitigate

Cause	Action
Bulk job flooding	Stop or throttle job: `POST /api/v1/bulk-jobs/{id}/stop`; reduce `rate_limit_per_sec`; ensure bulk priority LOW
Consumers paused	`POST /api/v1/ops/consumers/resume`
Consumers too few and provider can take more	Increase replicas or `messaging.dispatch.concurrency` stepwise (e.g., 8 to 12) while watching provider `RATE_LIMITED` and DB pool
Provider slow/rate-limited	More consumers will NOT help: see `provider-outage.md` / `provider-rate-limit.md`
Stuck consumers	Restart instances one by one (graceful shutdown returns unacked to queue)
Starvation of LOW	Priority aging (spec 0001) if enabled; otherwise accept

Recover

Watch depth and age trend down. Do not purge queues: messages are persisted in DB but purging the broker loses envelopes (recoverable only by operator requeue from DB states: avoid).

Verify

Oldest age within SLO; `dispatch.delay` p95 normal; no unexpected 503s.

Escalate

If CRITICAL oldest age > 60 s for 10 min: page platform lead.

Prevent

Capacity model (consumers x per-message latency vs provider limits), bulk throttles, alerts on trend not only level, load test (`performance/`, not executed here), backpressure tuned.

Proof: `FailureRecoveryIT#scenario10_providerRateLimitAndBackpressure_noAttemptConsumed_apiReturns503`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

21. AI-Assisted Development Workflow

> **Provenance, stated plainly**
>
> This repository was authored with **Claude Code** (an agentic CLI), orchestrated by a lead agent that delegated file-disjoint sections to sub-agents, all working from one shared contract (`docs/design/CONTRACT.md`). **No GitHub Copilot session, Copilot agent mode, Copilot code review or any other third-party agent was run to produce these artifacts.** The files `.github/copilot-instructions.md`, `.github/instructions/`, `.github/prompts/` and `AGENTS.md` are *authored configuration* showing how such tools would be configured; they have not been exercised by those tools here. Where a workflow below describes an independent security review, it describes the **intended process**, not a record that it happened.

21.1 What "AI-assisted" means here

AI coding assistants are fast and fluent and will happily produce plausible code that violates the invariants a messaging platform lives on: a missing idempotency check, a status written without the compare-and-set, a retry on an uncertain outcome, a log line with an address in it. The workflow in this repository is therefore built on one idea: **make the invariants explicit, make the checks mechanical, and keep humans on the decisions.**

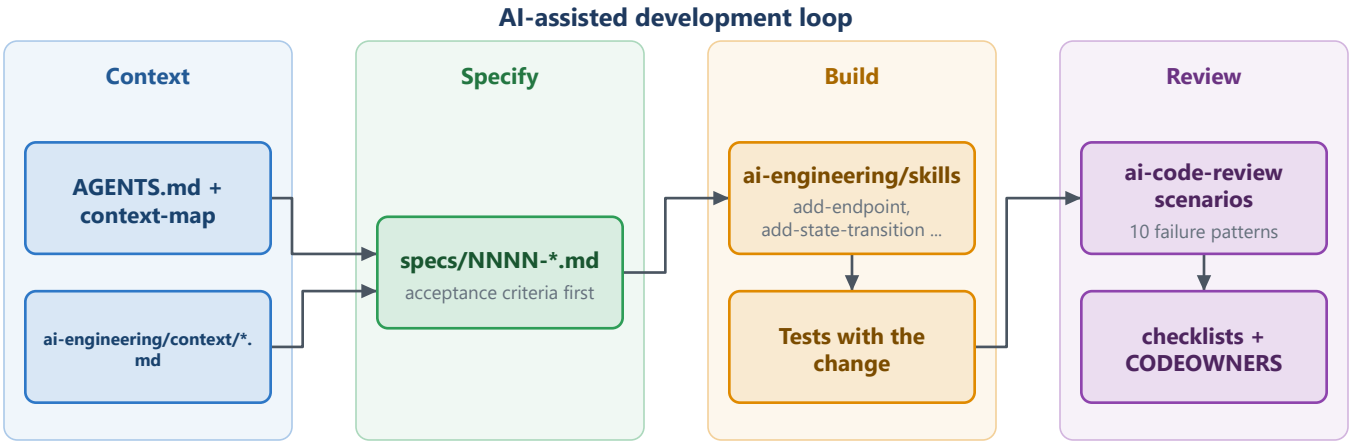


Figure 50. Spec-driven loop used to build this repository with Claude Code: context, spec, small change, tests, independent review, then merge. Humans own decisions and invariants. Implementation: ai-engineering/spec-driven-workflow.md, ai-engineering/context-engineering.md, ai-engineering/multi-agent-workflow.md, .github/copilot-instructions.md

Principle	Mechanism in the repository
One source of truth	`docs/design/CONTRACT.md` is the build contract; deviations are appended to `docs/design/DEVIATIONS.md`, never silently changed
Load only what the task needs	`ai-engineering/context-map.md` and seven short `ai-engineering/context/*.md` modules (api, architecture, db, lifecycle, observability, security, testing), each with a "load when" header
Specify before building	`specs/spec-template.md`; worked specs `0001-priority-aging.md` and `0002-new-provider-adapter.md` with acceptance criteria first
Reusable playbooks	`ai-engineering/skills/`: add-endpoint, add-provider-adapter, add-state-transition, diagnose-queue-backlog, review-security, write-failure-test, write-migration

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Principle	Mechanism in the repository
Standards that a model can follow	`ai-engineering/standards/`: Java and Spring, Angular, API design, DB migrations
Checklists as gates	`ai-engineering/checklists/`: code review, security review, test generation, performance review, definition of done, release readiness
Learn from realistic mistakes	`ai-engineering/ai-code-review/`: ten worked examples of flawed AI-style code, the fix, the rationale and the regression test
Independent review of sensitive areas	`ai-engineering/multi-agent-workflow.md`: authors never approve their own authentication, tenancy, callback, template, state-machine or retry changes; `.github/CODEOWNERS` encodes it
Honesty rules	Never claim execution that did not occur; label "not executed"; no secrets; no real data; mark illustrative numbers

21.2 The invariants an assistant must not break

`ai-engineering/context-engineering.md` keeps a short invariants list that every prompt includes. The version below is the working summary:

Invariant	Why it exists
Every status change goes through `NotificationService.transition` with the expected `from`	CAS ownership; the state machine is the only authority
Accepted means persisted: state, outbox and audit in one transaction	No message lost between database and broker
The envelope carries identifiers only	No content or PII on the bus
Never retry a `TIMEOUT` or `AMBIGUOUS` outcome blindly for EMAIL and SMS	Duplicate customer messages
No transaction is held across a provider call	Pool exhaustion
Tenant comes from the token, resources of other tenants are 404	Isolation without disclosure
No rendered body, address or secret in logs, audit or Mongo	Data minimisation
Fallback channels pass the full policy chain	Consent is never bypassed
Everything bounded	Threads, queues, retries, body sizes, bulk concurrency

21.3 Spec-driven loop (idea to production in 11 steps)

`ai-engineering/spec-driven-workflow.md` walks one change from idea to production with a worked example. The shape:

Step	Output	Gate
1 Problem and personas	Spec sections 1 to 3	Human sign-off
2 Acceptance criteria (Given, When, Then)	AC list with ids	Testable
3 Design and ADR if cross-cutting	ADR, diagrams	Design review (Gate A)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Step	Output	Gate
4 Context selection	Modules loaded	Token-lean
5 Task breakdown	Tasks with dependencies	Small, reviewable
6 Implement with tests	Code and tests together	Tests executed
7 Self-review with checklist	Checklist ticked	`checklists/code-review.md`
8 Independent review for sensitive areas	Review notes	Gate B and security gate
9 Performance review for hot paths	Findings	`checklists/performance-review.md`
10 Documentation and honesty labels	Docs, deviations	No invented behaviour
11 Release readiness	Checklist	Gate C

21.4 How the agents were organised for this repository

A lead agent fixed the contract first, then delegated sections whose files do not overlap (backend foundation, messaging core, batch module, Angular console, infrastructure, documentation, PDFs). Each agent recorded where it had to differ from the contract in `DEVIATIONS.md`, which is why that file doubles as the authoritative record of implemented behaviour (Appendix B). Two concrete benefits are visible in the deviations: agents discovered real design flaws by implementing (the consumer must check before it claims; the outbox publisher must tolerate an already `QUEUED` message; Micrometer gauges are weak references), and the record kept those discoveries from being lost.

21.5 What AI assistance is good and bad at here

Good at	Needs a human or a check
Boilerplate controllers, DTO records, repositories, migrations in the portable subset	Choosing the transaction boundary
Exhaustive table-driven tests (state matrix, policy table)	Deciding what must never happen (the invariants)
Consistent documentation and diagrams from code	Judging whether a number is measured or illustrative
Spotting N+1 queries and missing indexes when asked	Security review of its own changes (must be independent)
Drafting runbooks	Validating runbooks against a real incident

21.6 Prompts and skills

The five Copilot-format prompt files in `.github/prompts/` (`debug-queue-backlog`, `generate-tests`, `perf-review`, `review-security`, `spec-to-tasks`) and the plain-Markdown playbooks in `ai-engineering/skills/` carry the same guidance in tool-neutral form so that any assistant, or a human, can follow them. Path-specific instruction files under `.github/instructions/` (`angular`, `java`, `security`, `sql-migrations`, `tests`) scope the rules to the files they govern.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Reference material

AI engineering workspace

Source document: `ai-engineering/README.md`

How this repository is set up so that humans and AI coding assistants can work on it safely and consistently: context files, standards, checklists, reusable prompts/skills, a multi-agent workflow, and worked review examples.

Provenance statement (read first)

This repository was authored with **Claude Code** (an agentic CLI), orchestrated by a lead agent that delegated sections to sub-agents, working from `docs/design/CONTRACT.md`. **No GitHub Copilot session, Copilot agent mode, Copilot code review, or any other third-party agent was run to produce these artifacts.** The files under `.github/copilot-instructions.md`, `.github/instructions/`, `.github/prompts/` and `AGENTS.md` are *authored configuration* demonstrating how such tools would be configured; they have not been exercised by those tools here. Where this workspace describes a workflow (e.g., independent security review), it describes the intended process, not a record that it happened.

Contents

Path	Purpose
`context-map.md`	Which doc to load for which task; repository map
`context/*.md`	Seven short, token-lean context modules with "load when" headers
`standards/*.md`	Java/Spring, Angular, API design, DB migration standards
`checklists/*.md`	Code review, security review, test generation, performance review, definition of done, release readiness
`debugging-workflows.md`	Symptom-driven debugging recipes
`templates/`	Feature spec template pointer and handoff-note template
`context-engineering.md`	Selection, token optimization, drift prevention, invariants list
`skills/*.md`	Reusable task playbooks
`multi-agent-workflow.md`	Roles, gates, independent review
`spec-driven-workflow.md`	Idea to production in 11 steps, with a worked example
`ai-code-review/`	10 worked examples: flawed code, fix, rationale, regression test
`../specs/`	Specs (0001 priority aging, 0002 new provider adapter, template)
`../AGENTS.md`	Cross-tool agent instructions
`../.github/copilot-instructions.md`, `instructions/`, `prompts/`	Copilot-format configuration

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Tool-specific format differences

Mechanism	File(s)	Scope	Loaded	Notes
AGENTS.md (open convention read by many agents)	`AGENTS.md` at repo root (nested ones allowed in subfolders)	whole repo / subtree (nearest file wins)	automatically by supporting agents	plain Markdown, no front matter; keep short: build/test commands, conventions, boundaries
GitHub Copilot repository instructions	`.github/copilot-instructions.md`	whole repo	automatically in Copilot chat/agent/review where supported	plain Markdown; natural-language rules; keep under a couple of pages
Copilot path-specific instructions	`.github/instructions/*.instructions.md`	files matching `applyTo` glob in YAML front matter	automatically when the matching files are in context	front matter `applyTo: "**/*.java"`; optional `excludeAgent`
Copilot prompt files	`.github/prompts/*.prompt.md`	user-invoked reusable prompt (`/name` in chat)	on demand	front matter (e.g., `mode`/`agent`, `description`, `tools`) varies by Copilot version: verify against current docs
Claude Code memory	`CLAUDE.md` (project root, nested, and user-level), imports with `@path`	repo/subtree/user	automatically at session start	Markdown; hierarchical; this repo does not require a CLAUDE.md because `AGENTS.md` carries the shared rules (a one-line `CLAUDE.md` importing `@AGENTS.md` is a common bridge)
Claude Code skills / slash commands	`.claude/skills`, `.claude/commands`	on demand	not used here; the equivalent playbooks live in `ai-engineering/skills/` as plain Markdown so any tool (or human) can read them	
Cursor/other	`.cursor/rules`, etc.	tool specific	out of scope	`AGENTS.md` is the portable baseline

Design choice: single source of truth in `ai-engineering/context/*` and `docs/design/CONTRACT.md`; tool-specific files are thin pointers plus the highest-value rules, to avoid duplication and drift (see `context-engineering.md`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

How to use

1. Start from `context-map.md` and load only the modules your task needs.
2. For features follow `spec-driven-workflow.md`; write the spec from `../specs/spec-template.md`.
3. Before opening a PR run the relevant checklist from `checklists/`.
4. For sensitive changes (auth, tenancy, callbacks, templates, state machine, retries) follow independent review in `multi-agent-workflow.md`.
5. Never silently override an ADR: write a superseding ADR.

Honesty rules for AI-authored content

Never claim execution that did not occur; label "not executed"; no secrets (dummy values only); no real data; do not assert vendor behavior as fact; mark illustrative numbers.

Spec-driven workflow

Source document: `ai-engineering/spec-driven-workflow.md`

An 11-step flow, followed by a worked example: **delivery-priority policy (priority aging / starvation prevention)**, whose full spec is `specs/0001-priority-aging.md`. The example is a *design exercise*: it describes a feature to build, not one that exists in the code at the time of writing (check source and `docs/validation/VALIDATION_REPORT.md` for status).

The 11 steps

#	Step	Output	Gate
1	Idea and problem framing	problem statement, who is hurt, evidence	owner agrees it is worth doing
2	Requirements	numbered requirements, non-goals	review
3	Acceptance criteria	Given/When/Then, measurable	testable?
4	Design	spec design section, ADR if cross-cutting, failure modes, security/perf notes	Gate A design review
5	Task decomposition	ordered tasks, each independently mergeable, with named tests	sizing sanity
6	Test plan first	named tests (unit, integration, failure), data, matrices	tests precede or accompany code
7	Implementation	small PRs per task	Gate B code review
8	Security and performance review	threat-model row, independent review if sensitive, perf notes	security gate
9	Documentation	docs, runbooks, config, ADR status, API docs	accuracy check
10	Release	readiness checklist, flag/canary plan, rollback plan	Gate C
11	Operate and learn	dashboards/alerts, SLO check, PIR if incidents, spec status Implemented, backlog follow-ups	review at 2 weeks

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Worked example: priority aging / starvation prevention

1. Idea

Strict priority queues (`CRITICAL(9)`, `HIGH(6)`, `NORMAL(3)`, `LOW(1)`) can starve LOW messages when higher priorities are continuously busy (e.g., a large HIGH burst). LOW includes legitimate service messages that would never go out.

2. Requirements (summary)

R1 A message waiting longer than a priority-specific threshold is promoted one level (aging), up to a ceiling below CRITICAL. R2 CRITICAL is never displaced by aged traffic more than a bounded share. R3 Promotion is deterministic, audited and observable. R4 Feature-flagged, default off. R5 No change to consent/policy semantics (aging affects order only). R6 Works with RabbitMQ priority and the in-memory bus. Non-goals: dynamic per-tenant priorities, preemption of in-flight work.

3. Acceptance criteria (summary)

AC-1 LOW waiting > `lowToNormalAfter` becomes NORMAL (effective priority) exactly once per threshold; AC-2 never exceeds `maxEffectivePriority=HIGH`; AC-3 original priority is retained for reporting; AC-4 with flag off behavior is unchanged; AC-5 under a sustained HIGH flood at 90% capacity, a LOW message is dispatched within `lowToNormalAfter + normal wait` (bounded); AC-6 CRITICAL p95 dispatch delay does not regress by > 10% in the harness.

4. Design (summary)

- Store `effective_priority` and `priority_promoted_at` (nullable, expand-only migration). `priority` remains the requested value.
- A `PriorityAgingJob` (scheduler) finds QUEUED/RETRY_PENDING messages older than thresholds by `(status, created_at)`-bounded query, and re-publishes an envelope at the higher priority with a new `dispatchId`; the old envelope becomes a no-op because consumer CAS `QUEUED to PROCESSING` plus `processed_message` ensures only one executes (the stale lower-priority copy is acked and skipped).
- Bus semantics: RabbitMQ cannot reprioritize in place, so re-publish is the mechanism; in-memory bus identical.
- Weighted fair dequeue alternative considered (rejected: complex, broker-specific). See spec for alternatives.

5. Task decomposition

T1 migration + entity fields; T2 `AgingPolicy` pure class with thresholds config + unit tests; T3 `PriorityAgingJob` + CAS bookkeeping; T4 consumer skip logic test; T5 metrics + audit; T6 flag/config + docs; T7 harness scenario; T8 runbook update.

6. Test plan (names)

`AgingPolicyTest#promotesAfterThreshold_onlyOneLevelPerThreshold`, `#neverExceedsCeiling`, `PriorityAgingIT#lowMessageUnderHighFlood_isDispatchedWithinBound`, `#staleLowEnvelope_isSkippedAfterPromotion_noDoubleSend`, `#flagOff_noPromotion`, `#criticalDelayNotRegressed_harness`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. Implementation

Small PRs per task using the PR template; each runs checklists.

8. Security and performance review

Security: aging must not let a SENDER force HIGH by delaying; ceiling prevents CRITICAL abuse; priority field remains validated; audit entry `PRIORITY_PROMOTED` has ids only. Independent review not strictly required (not authn/consent), but the double-send risk touches idempotency: reviewer must verify invariant 3. Performance: aging query bounded by `FETCH FIRST n`, uses `(status, created_at)`-style index (new index decision documented), runs every N seconds with batch cap; metric `messaging.priority.promotions{from,to}` (new metric requires CONTRACT/DEVIATIONS note).

9. Docs

Update `docs/architecture/dispatch-retry-flow.md`, runbook `queue-backlog.md`, config docs, ADR (new ADR-0013 if accepted: priority aging), `context/lifecycle.md`/observability if metric added.

10. Release and rollback

Ship dark (flag off); enable in staging with fault/load; canary on one instance; success criteria: LOW wait bounded, CRITICAL delay unchanged, no duplicates. Rollback: disable flag (no schema rollback needed; columns nullable and ignored).

11. Operate

Dashboard panel: promotions rate, wait by priority; alert if LOW oldest age exceeds threshold even with aging on.

Second example

Adding a provider: `specs/0002-new-provider-adapter.md`.

Context engineering

Source document: `ai-engineering/context-engineering.md`

How to give an AI assistant (or a new engineer) the right information at the lowest token cost, and keep it correct over time.

1. Selection: load the minimum sufficient context

- Start from the task type in `context-map.md`; load 1-3 modules (each is ~300-600 tokens), not the whole docs tree.
- Prefer the *authoritative* short artifact over a long explanation: transition table (lifecycle.md) over the ADR; ADR only when rationale is needed.
- Load source files only for the classes being changed, plus their tests.
- Pull large sources (CONTRACT, OpenAPI) by section/`grep`, not whole.
- Order matters: put invariants and task goal first, reference material after.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. Token optimization

Technique	Example
Tables over prose	transition table instead of paragraphs
Names over descriptions	`ResilientProviderInvoker` rather than re-explaining circuit breaking
Link, don't paste	"see ADR-0010"
One canonical home per fact	metric names only in CONTRACT 8 + observability.md
Summarize history	handoff notes, not transcripts
Strip noise	exclude generated files, lockfiles, build output, `node_modules`, `target` from context
Chunk big tasks	one spec task per session
Ask for diffs	request patch-sized outputs

3. Repetition avoidance

Tool-specific files (`AGENTS.md`, `.github/copilot-instructions.md`, path instructions) are *thin*: top rules + pointers to `ai-engineering/context/*`. Do not copy tables into three places. If a fact appears in two files, one of them must be a link.

4. Drift prevention

1. Source-of-truth map in `context-map.md` ("Where truth lives").
2. Contract tests tie docs to code where possible: OpenAPI vs springdoc; transition matrix test vs `MessageStateMachine`; metric names test vs CONTRACT 8.
3. Deviations recorded in `docs/design/DEVIATIONS.md`, ADR superseded rather than edited.
4. PR checklist item: "docs/context updated".
5. Periodic audit: grep context modules for each class name and metric name; failing lookups are drift.
6. Labelled uncertainty: docs say "see source" instead of guessing behavior.
7. Every doc mentioning a result cites the command and whether it ran.

5. Invariants list (the things that must always be true)

Reviewers and assistants must verify these against any change:

1. Status changes only via `MessageStateMachine` + CAS; transitions match the table (CONTRACT 3.1).
2. Accepted means durable: message + outbox + idempotency + audit commit in one transaction; publish happens after commit.
3. Consumer processing is idempotent (`processed_message`, CAS QUEUED->PROCESSING); callbacks idempotent by `(provider_id, provider_event_id)`.
4. No transaction spans a provider call; every remote call has a timeout; pools and queues are bounded.
5. Provider acceptance never implies delivery; timeouts/ambiguous never become failure or blind resend for EMAIL/SMS; `DELIVERY_UNKNOWN` resolves only via provider-originated evidence.
6. Retries are bounded, jittered, re-evaluate policy; poison messages go to DLQ.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. Consent: suppression > category rules > quiet hours; fallback and replay never bypass consent; evaluated at accept and dispatch.
8. Tenant comes from the verified JWT only; every tenant-owned read/write is tenant-scoped; foreign resource => 404.
9. Rendered body, full addresses, secrets never stored/logged/returned/put on the bus; variables stored not body.
10. Template version pinned at accept; published versions immutable; approver != author.
11. Callback signature verification never bypassed; replay protection on.
12. Dev-only endpoints exist only in `local`/`test`.
13. Audit is append-only and written for every privileged action and transition.
14. Docs never claim unexecuted validation.

6. Prompting patterns that work here

- "Given invariants 1, 3, 5, review this diff for violations; list each with file:line and a failing test idea."
- "Write the failing test first for scenario N from reliability-scenarios.md."
- "Propose 2 designs with trade-offs, recommend one, cite ADRs affected; do not implement." Anti-patterns: pasting the entire repo; asking for "production ready" without acceptance criteria; letting the assistant rewrite tests to pass; accepting invented APIs without compiling.

7. Session hygiene

New session per spec task; end with a handoff note; verify by running tests, not by assistant assertion.

Multi-agent workflow

Source document: `ai-engineering/multi-agent-workflow.md`

A role-based way to split work among AI agents (and humans). **Provenance note**: this repository itself was produced by a Claude Code lead agent delegating file-disjoint sections to sub-agents under a shared contract. The role model below generalizes that practice; it does not claim that every gate was run with separate agents here.

Roles

Role	Responsibilities	Deliverables	Depends on	Must not
Requirements	clarify problem, personas, acceptance criteria, non-goals	spec sections 1-3 (`specs/`), AC list	stakeholders	design the solution
Architecture	design, ADRs, interfaces, failure modes, trade-offs	ADR(s), design section, diagrams, contract updates/DEVIATIONS	requirements	override accepted ADRs silently
Backend	implement services, persistence, messaging, providers; unit/integration tests	code, migrations, OpenAPI changes	architecture, contract	change API/schema outside spec

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Role	Responsibilities	Deliverables	Depends on	Must not
Angular	implement console screens, services, interceptors, tests	frontend code and tests	OpenAPI, backend endpoints	trust client-side authorization
Testing	test plan, matrices, failure-recovery tests, data builders	tests, scripts, traceability	requirements, backend	weaken assertions to pass
Security	threat model update, authz/tenant/callback review, secrets and logging review	review notes, tests, threat-model rows	design, code	approve own authored change
Performance	capacity thinking, query plans, load harness, profiling	findings, indexes proposals, benchmark notes (executed only)	backend	claim unmeasured numbers
Documentation	docs, runbooks, README accuracy, honesty labels	docs, validation report text	all	invent behavior ("see source")

Flow and review gates

```
Requirements -> Architecture -> [Gate A: design review] -> Backend/Angular (parallel by contract)
   -> Testing (continuous) -> [Gate B: code review + checklists] -> Security (sensitive only) -> Performance
     (hot paths)
   -> Documentation -> [Gate C: release readiness] -> Release
```

Gate	Entry	Exit criteria
A design review	spec + ADR draft	invariants list respected; failure modes enumerated; ADR references; open questions resolved
B code review	PR with tests	`checklists/code-review.md`; tests green (executed); AC traceable
Security gate	sensitive change flagged	independent reviewer completes `security-review.md`, findings fixed or accepted by owner
C release readiness	all merged	`docs/operations/release-readiness-checklist.md`

Sensitive changes require independent review

Sensitive = authn/authz, tenant scoping, callback verification, template engine, state machine, retry/idempotency/reconciliation, consent policy, secrets/logging of data, deployment of dev endpoints. Rules:

1. The reviewer is a different agent session/person from the author, with fresh context (only the diff, spec, invariants, and checklists: not the author's reasoning).
2. Reviewer attempts to break it with failing tests (`skills/review-security.md`).
3. Author cannot approve; disagreements escalate to the architecture role/owner.
4. Result is recorded in the PR (findings table, verdict).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

No silent ADR override

Any change contradicting an accepted ADR requires: a new ADR that supersedes (status, rationale, consequences), update of the old ADR header, entry in `docs/design/DEVIATIONS.md` if it contradicts CONTRACT, and architecture-role approval. Agents must stop and escalate rather than "work around" an ADR in code.

Coordination mechanics

- Contract-first: `docs/design/CONTRACT.md` is shared truth; agents write disjoint file sets (as done in this repo) to avoid merge conflicts.
- Handoff notes between sessions (`templates/handoff-note-template.md`).
- Conflict rule: contract wins; if contract is wrong, record deviation, do not fork silently.
- Evidence rule: reports state what was executed (command + result) and what was not.
- Parallelism: Backend and Angular can proceed in parallel against the OpenAPI spec; Testing writes failure tests from the reliability table in parallel with Backend; Documentation trails but reviews continuously.

Failure modes of multi-agent work (and countermeasures)

Failure	Countermeasure
Divergent assumptions (names, states)	contract + invariants; automated contract tests
Duplicated/contradictory docs	single home per fact, thin tool files
Overconfident claims	validation report from real outputs only
Test weakening	review of test diffs; mutation sanity check
Context loss	handoff notes, specs as memory
Reviewer shares author's blind spots	fresh-context independent review

Debugging workflows

Source document: `ai-engineering/debugging-workflows.md`

Symptom-driven recipes for humans and assistants. Always: reproduce or capture evidence first, form a hypothesis, test it, change one thing, add a regression test. Never "fix" by loosening a security control or ADR invariant.

W1: A message is stuck (not terminal)

1. `GET /api/v1/messages/{id}/history` and `/status`: note status, `status_reason`, attempts, `next_attempt_at`, `scheduled_at`.
2. By status: | Status | Likely cause | Next check | |---|---|---| | VALIDATED, scheduled future | not due / quiet-hours deferral | `scheduled_at`, recipient time zone | | VALIDATED, due | outbox publisher stalled | oldest `outbox_event` NEW age; scheduler logs; backpressure | | QUEUED | consumers paused/idle/saturated | `GET /ops/queues`, pause state, thread dump | | PROCESSING | stale lock/crash | `locked_until`; sweeper running? | | RETRY_PENDING | backoff or poller issue | `next_attempt_at` vs now; poller query/index | | SENT_TO_PROVIDER | no receipt | provider callbacks; `receipt-timeout`; signature failures | | DELIVERY_UNKNOWN | ambiguity |

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

reconciliation lookup logs; `consumer-crash-ambiguous-outcomes.md` |

3. Fix cause; do not hand-edit status in DB (bypasses state machine and audit).

W2: Duplicate message reported

Count `message_attempt` rows and `idempotencyRef`; check `Idempotent-Replay`; check two different `Idempotency-Key` s from client; check DELIVERY_UNKNOWN retry path and provider idempotency support (see TS-1 in `docs/leadership/troubleshooting-scenarios.md`).

W3: Test fails intermittently

Suspect: real clock, random, shared H2 db, async not awaited, ordering. Reproduce with repeat (`-Dsurefire.rerunFailingTestsCount` is NOT a fix). Replace sleeps with await conditions; inject Clock; ensure unique db per context; check thread leaks between tests.

W4: Cross-tenant leak suspected

Reproduce with tenant B token; find repository method without tenant parameter; check admin header path; add matrix test first, then fix.

W5: Callback returns 401

Compare keyId, secret config, timestamp skew, raw body mutation by proxy; see `docs/operations/runbooks/callback-signature-failures.md`. Never disable verification.

W6: Slow endpoint

Metric by uri, statement count test, `EXPLAIN`, Hikari metrics, thread dump (`docs/operations/performance-investigation-guide.md`).

W7: Flyway failure

Read the failing version and error; on H2 vs PG difference check portable-SQL rule; never edit a merged migration: add a new one; local reset: delete local H2 file db if used (in-memory resets on restart).

W8: Build/CI failure on Windows/Git Bash

Use full JDK path via session settings (never export JAVA_HOME globally); local app on port 18080 (8080 occupied); pass Windows-style paths to node with `cygpath -m`.

AI-assisted debugging rules

- Provide the assistant: symptom, evidence (ids, metrics), relevant context modules only.
- Ask for hypotheses ranked with evidence needed, not immediate patches.
- Verify claims by running code/tests; reject fixes that add retries/sleeps/catch-all as a first move.
- Record the root cause and the regression test in the PR.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Context map

Source document: `ai-engineering/context-map.md`

Task to context

Task	Load (in order)	Skip
Change lifecycle/status logic	`context/lifecycle.md`, ADR-0003, `docs/architecture/dispatch-retry-flow.md`	db, frontend
Add/modify endpoint	`context/api.md`, `context/security.md`, `standards/api-design.md`, CONTRACT section 5	observability
DB change/migration	`context/db.md`, `standards/db-migrations.md`, `docs/database/schema-evolution.md`	api
Provider work	`context/architecture.md`, ADR-0002, `specs/0002-new-provider-adapter.md`, `docs/architecture/callbacks-normalization.md`	
Retry/DLQ/reconciliation	`context/lifecycle.md`, ADR-0006, ADR-0010, `docs/architecture/reconciliation-flow.md`	
Security review	`context/security.md`, `docs/security/threat-model.md`, `checklists/security-review.md`	
Write tests	`context/testing.md`, `checklists/test-generation.md`, `docs/architecture/reliability-scenarios.md`	
Incident/debug	`debugging-workflows.md`, `context/observability.md`, runbooks	
Angular work	`standards/angular.md`, CONTRACT section 10	backend context
Performance	`checklists/performance-review.md`, `docs/operations/performance-investigation-guide.md`	
New feature	`spec-driven-workflow.md`, `specs/spec-template.md`	

Always available (small): `AGENTS.md`, the invariants list in `context-engineering.md`. Binding source: `docs/design/CONTRACT.md` (large: load only the section you need: 3 lifecycle/policy, 4 schema, 5 API, 6 dispatch, 7 batch, 8 metrics, 9 config).

Repository map

```
<root>/
  AGENTS.md                     cross-tool agent rules
  backend/                      Spring Boot 3.5.16, Java 21, single module messaging-platform
    src/main/java/com/example/messaging/{common,config,security,domain,repository,service,template,provider,
      messaging,callback,batch,diagnostics,web,scheduler,metrics}
    src/main/resources/{db/migration,db/oracle,openapi,application*.yml}
    src/test/java/...           mirrors main packages; AbstractIntegrationTest; FailureRecoveryIT
  frontend/messaging-console/   Angular 20 standalone/Signals app
  infrastructure/               docker, compose, kubernetes, helm, rabbitmq, postgres, mongo (static
    validation only)
  cloud-foundry/                manifest (not executed)
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
observability/                prometheus, grafana, otel, alerts
tests/                        end-to-end, api-http, security, failure-recovery
performance/                  k6 (not executed), jvm-harness
docs/
  design/                     CONTRACT.md (initial plan), DEVIATIONS.md (what was built; wins),
    CONSISTENCY_LOG.md, requirements-and-assumptions.md
  adr/                        0001..0012 + README
  architecture/               flows (Mermaid), ER, deployment, semantics, reliability scenarios
  database/ security/ operations/ (runbooks/) leadership/ api/ interview/ pdf/ traceability/ validation/
ai-engineering/               this workspace
specs/                        spec-driven artifacts
scripts/ tools/pdf/           helper scripts
.github/                      copilot-instructions, instructions/, prompts/, PR template, CODEOWNERS,
  workflows
```

Package responsibilities (one line each)

`common` ids/errors/redaction; `config` Spring config and properties records; `security` principal, tenant, filters; `domain` enums, entities, state machine; `repository` JPA; `service` use cases and policy; `template` safe renderer; `provider` ports, simulators, resilience; `messaging` bus, consumer, DLQ, replay; `callback` webhook auth and normalization; `batch` bulk job; `diagnostics` Mongo store; `web` controllers/DTOs; `scheduler` outbox, poller, sweeper, retention; `metrics` meters.

Where truth lives

Fact	Source of truth
States, transitions	`domain.MessageStateMachine` (mirrors CONTRACT 3.1, ADR-0003)
Schema	Flyway migrations
API shape	`messaging-api-v1.yaml`
Metric names	CONTRACT section 8 / `metrics.MessagingMetrics`
Validation results	`docs/validation/VALIDATION_REPORT.md` only

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

22. Failure and Recovery Scenarios

22.1 Designing for failure

A messaging platform is judged by what it does when things go wrong. The design is organised around **ten failure scenarios**, each analysed along the same five axes and each tied to a named automated test:

Axis	Question
Detection	Which metric, alert or API view tells an operator?
Customer impact	What does a customer or client actually experience?
Recovery	What brings the system back, automatically or by operator action?
Data consistency	What invariant holds throughout (no loss, no duplicate, no regression)?
Prevention	Which design decision limits the blast radius?
Proof	Which test demonstrates it?

The test methods below are real: they exist in `backend/src/test/java/com/example/messaging/pipeline/FailureRecoveryIT.java` (scenarios 1 to 8 and 10) and `backend/src/test/java/com/example/messaging/batch/FailureRecoveryBulkIT.java` (scenario 9). Whether they pass is stated only in `docs/validation/VALIDATION_REPORT.md`, never in the design text.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

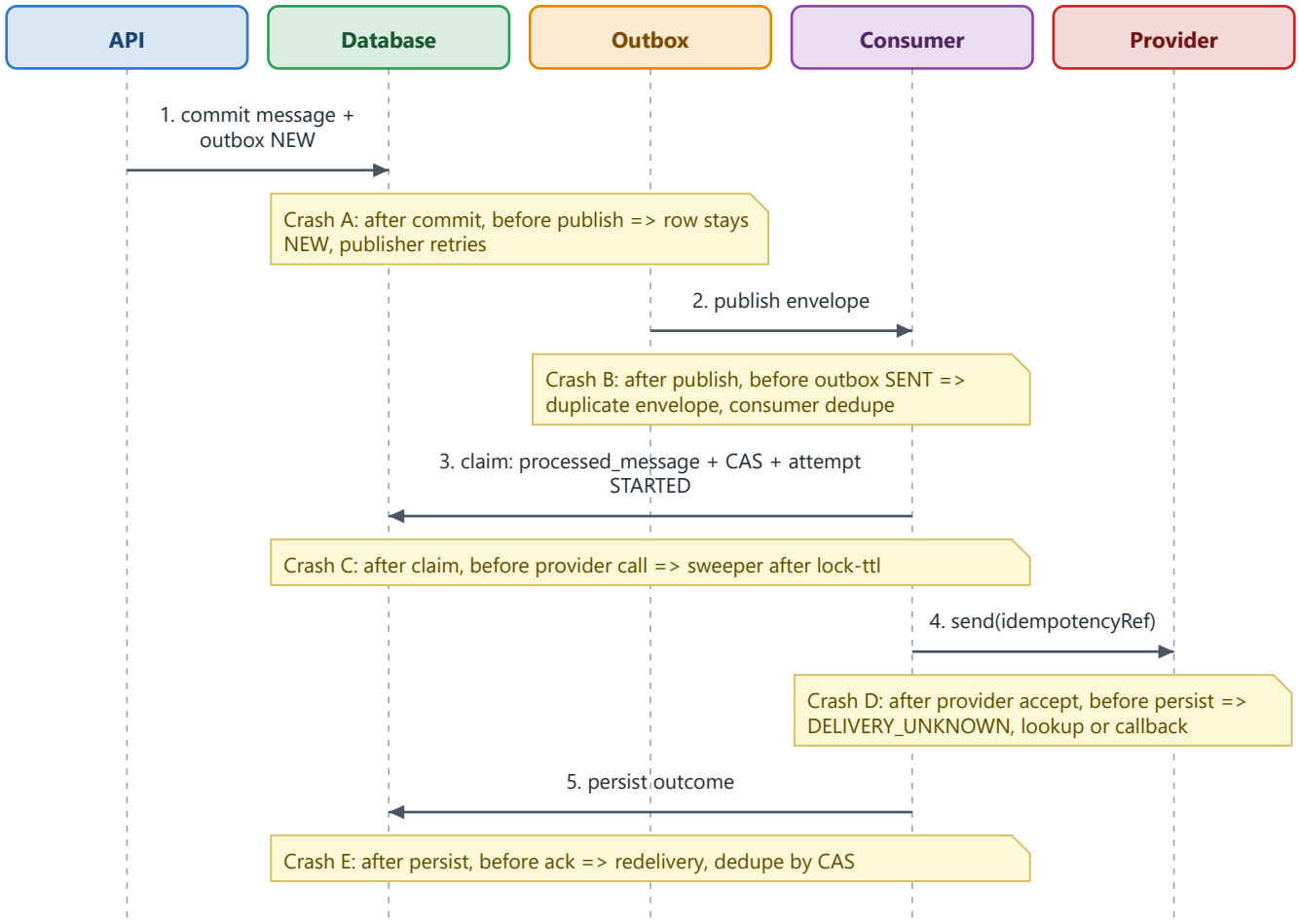


Figure 51. Crash windows along the pipeline and what recovers each one. No window loses an accepted message; the dangerous window (after provider accept, before persist) ends in DELIVERY_UNKNOWN rather than a blind resend. Implementation: backend/src/main/java/com/example/messaging/scheduler/StuckMessageSweeper.java, backend/src/main/java/com/example/messaging/messaging/DispatchConsumer.java, backend/src/main/java/com/example/messaging/scheduler/OutboxPublisher.java

Evidence from the executed end-to-end run

The real HTTP end-to-end run recorded in `docs/validation/e2e-run-output.txt` (Appendix E; 254 passed, 0 failed, 0 skipped) exercised several of these scenarios against the packaged jar on the `local` profile. Its section 8 shows, as PASS lines: transient failure to `RETRY_PENDING` and then delivery after the fault cleared; permanent failure with exactly one attempt; provider timeout to `DELIVERY_UNKNOWN` with exactly one attempt while unknown; reconciliation finding `NOT_RECEIVED` and safely re-sending; an ambiguous answer reconciled as still accepted (no duplicate send) and resolved by a late `DELIVERED` callback; rate limiting (429) retried; six SMS through a 2 per second limiter all dispatched; an outage ending in `DEAD_LETTERED` after 5 attempts and an operator replay after recovery. Section 9 shows duplicate, replayed, out-of-order and conflicting callbacks, and section 11 shows the bulk stop and restart finishing a 20,000-row job exactly once. Scenarios 5 to 7 (a worker crash, a database outage, a Redis outage) are simulated in-JVM only and are not part of that HTTP run.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

22.2 Scenario summary

#	Scenario	Detection	Recovery	Consistency invariant	Proof
1	Provider outage then recovery	`ProviderCircuitOpen`, `ProviderErrorRatioHigh`, rising `messaging.retry.scheduled`	Breaker half-opens; exhausted messages `DEAD_LETTERED`; operator replay	One attempt row per try; no status skipping	`FailureRecoveryIT#scenario01_providerOutage_retriesWithBackoff_thenDeadLetters_thenReplay`
2	Duplicate API submission	`messaging.idempotency.hits`; `Idempotent-Replay: true`	Automatic	Exactly one message and one idempotency row per (tenant, key)	`#scenario02_duplicateSubmission_sameKey_returnsOriginal_noSecondMessage`
3	Duplicate and out-of-order callbacks	`messaging.callbacks{result}`	Automatic: ignore or record `applied=false`	Status never regresses; first terminal wins	`#scenario03_duplicateAndOutOfOrderCallbacks_applyMonotonically`, `#scenario03b_receiptThatOvertakesTheConsumerIsApplied`
4	Provider timeout or ambiguous result	`messaging.transitions{to="DELIVERY_UNKNOWN"}`, `GET /failed-messages`	Late callback or `lookup`; "not received" triggers safe retry	No blind resend; provider call count stays 1 until lookup says not received	`#scenario04_providerTimeout_movesToDeliveryUnknown_reconcilesWithoutDuplicateSend`
5	Consumer crash after provider acceptance	`PROCESSING` older than lock TTL; sweeper transition	`StuckMessageSweeper` then callback or lookup	Attempt row `STARTED` remains; not retried blindly	`#scenario05_consumerCrashAfterProviderAccept_sweeperMarksDeliveryUnknown`, `#scenario05b_sentToProviderWithoutReceiptBecomesUnknownAfterTheReceiptTimeout`
6	Database outage	`hikaricp_connections_pending`, readiness DOWN, 503 rate	Connections recover; publisher resumes; nacked messages redelivered	Fail closed: no 202 without commit	`#scenario06_databaseOutage_submissionFailsClosed_thenRecovers`
7	Redis outage	`messaging.redis.fallback`, Lettuce errors	`ResilientCoordination` falls back and switches back	Idempotency still enforced by the DB unique key	`#scenario07_redisOutage_fallsBackToDb_noDuplicates`
8	Poison message	`messaging.deadletter.total{reason="POISON_MESSAGE"}`	Operator inspects, fixes, replays or discards	`DEAD_LETTERED` with bounded redeliveries	`#scenario08_poisonMessage_goesToDlq_noInfiniteLoop`
9	Bulk job interrupted and restarted	`messaging.bulk.progress{jobId}` flat	`POST /bulk-jobs/{id}/restart`	One message per valid row; counters reconcile	`FailureRecoveryBulkIT#scenario09_bulkJobRestart_resumesWithoutDuplicates`
10	Provider rate limit, backpressure, backlog	`messaging.provider.calls{outcome="RATE_LIMITED"}`, `messaging.queue.depth`, `messaging.ratelimit.rejected`	Token bucket paces; backlog drains; 503 + `Retry-After` beyond the threshold	Rate-limit requeue consumes no attempt; no loss	`#scenario10_providerRateLimitAndBackpressure_noAttemptConsumed_apiReturns503`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

22.3 How the scenarios are driven

`FailureRecoveryIT` extends the pipeline test base (`AbstractPipelineIT`) and runs the **real asynchronous pipeline** with simulators. The faults are injected by test seams that production code exposes without weakening it:

Seam	Used for
`FaultInjector` modes (`messaging.simulator.*`)	Transient, permanent, rate limit, timeout, ambiguous responses
Adjustable `Clock` (system time plus an offset)	Crossing lock TTLs, receipt timeouts and `reconcile-after` without sleeping
Programmable `DispatchHook`	Worker "death" after the provider call; injected processing failures
`DataSource` wrapper that fails `getConnection`	Database outage (scenario 6)
Switchable coordination backend wrapped by the production `ResilientCoordination`	Redis outage (scenario 7)

Backoff is shortened to a 40 ms base and 200 ms cap with `max-attempts` 4 in that context; production defaults are in `application.yml`. Scenario scripts in `tests/failure-recovery/` (`provider-outage.mjs`, `callback-replay.mjs`, `provider-timeout.mjs`) replay scenarios 1, 3 and 4 against a started `local` application on port 18080. Scenario 5 (a simulated crash), 6 and 7 are in-JVM only; scenario 9 is owned by the Spring Batch module. (The index table in the embedded design document lists a bulk script as a counterpart of scenario 9; only the three scripts named above exist in `tests/failure-recovery/`.)

> **Not claimed**
>
> RabbitMQ, Redis and PostgreSQL failures are simulated at the port level (a failing `DataSource`, a failing coordination stub). A real broker partition or a real PostgreSQL failover was **not** exercised; those belong to the `-Pcontainers` tests and to game days in a real environment.

22.4 Operator playbooks

Every alert links to a runbook in `docs/operations/runbooks/`; Appendix D reproduces them. The mapping from scenario to runbook:

Scenario	Runbooks
1 Provider outage	`provider-outage.md`, `provider-rate-limit.md`
3, 4, 5 Callbacks and ambiguity	`callback-signature-failures.md`, `consumer-crash-ambiguous-outcomes.md`
6 Database outage	`database-outage.md`
7 Redis outage	`redis-outage.md`
8 Poison and dead letters	`dlq-growth.md`
9 Bulk	`bulk-job-stuck.md`, `bulk-job-restart.md`
10 Backlog and retry storms	`queue-backlog.md`, `retry-storm.md`, `high-latency-and-resource-saturation.md`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Reference material

The full scenario analysis follows exactly as written in the repository, including the detailed proof for each test.

Reliability scenarios (full analysis)

Source document: `docs/architecture/reliability-scenarios.md`

Ten failure scenarios, each with detection, customer impact, recovery, data consistency, prevention and proving test. The test names below are the **required names** for the test-writing agent: class `FailureRecoveryIT` (`backend/src/test/java/com/example/messaging/...`, extends `AbstractIntegrationTest`, profile `test`, simulators driven by `FaultInjector`). Whether a test exists/passes is stated only in `docs/validation/VALIDATION_REPORT.md`, not here.

Test index

#	Scenario	Test (method in `FailureRecoveryIT`)	Script/e2e counterpart
1	Provider outage then recovery	`scenario01_providerOutage_retriesWithBackoff_thenDeadLetters_thenReplay`	`tests/failure-recovery` provider-outage script
2	Duplicate API submission (client retry)	`scenario02_duplicateSubmission_sameKey_returnsOriginal_noSecondMessage`	`tests/api-http` idempotency requests
3	Duplicate and out-of-order callbacks	`scenario03_duplicateAndOutOfOrderCallbacks_applyMonotonically`	`tests/failure-recovery` callback replay script
4	Provider timeout / ambiguous result	`scenario04_providerTimeout_movesToDeliveryUnknown_reconcilesWithoutDuplicateSend`	`tests/failure-recovery`
5	Consumer crash after provider acceptance	`scenario05_consumerCrashAfterProviderAccept_sweeperMarksDeliveryUnknown`	in-JVM only (simulated crash)
6	Database outage during submission / state update	`scenario06_databaseOutage_submissionFailsClosed_thenRecovers`	in-JVM (datasource fault wrapper)
7	Redis outage	`scenario07_redisOutage_fallsBackToDb_noDuplicates`	in-JVM (failing coordination stub)
8	Poison message	`scenario08_poisonMessage_goesToDlq_noInfiniteLoop`	in-JVM
9	Bulk job interrupted and restarted	`scenario09_bulkJobRestart_resumesWithoutDuplicates` (**owned by the Spring Batch module**, not present in the messaging-core `FailureRecoveryIT`)	`tests/failure-recovery` bulk script
10	Provider rate limiting, backpressure and queue backlog	`scenario10_providerRateLimitAndBackpressure_noAttemptConsumed_apiReturns503`	`performance/jvm-harness` (optional)

Supporting tests: `RetryPolicyTest`, `MessageStateMachineTest`, `SignatureVerifierTest`, `ReplayGuardTest`, `StuckMessageSweeperTest`, `InMemoryMessageBusTest`, `GracefulShutdownTest`, `ResilientProviderInvokerTest`, `EventNormalizerTest`, `RabbitBusSupportTest` / `RabbitTopologyTest` (pure parts of the Rabbit bus; the broker itself

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

is covered only by `RabbitMessageBusContainerIT`, a Testcontainers test that needs Docker and is **not** part of the default build), and the cross-cutting ITs `ChannelDispatchIT`, `ProviderOutcomeIT`, `CallbackApiIT`, `IdempotentConsumerIT`, `ConcurrencyIT`, `CancelRaceIT`, `BackpressureIT`, `ReconciliationIT`, `ReplayServiceIT`, `LoggingPrivacyIT`.

How the scenarios are driven (all in `FailureRecoveryIT`, which runs the real asynchronous pipeline): simulators with `FaultInjector` modes (`messaging.simulator.*`), an adjustable `Clock` (system time plus an offset, so lock TTLs, receipt timeouts and `reconcile-after` can be crossed without sleeping), a programmable `DispatchHook` (worker "death" after the provider call, injected processing failures), a `DataSource` wrapper that fails `getConnection` (scenario 6) and a switchable coordination backend wrapped by the production `ResilientCoordination` (scenario 7). Backoff is shortened to 40 ms base / 200 ms cap and `max-attempts` is 4 in that context; the production defaults are in `application.yml`.

---

1. Provider outage then recovery

Aspect	Detail
Detection	`messaging.provider.calls{outcome="TRANSIENT_FAILURE"}` rate up; `messaging.provider.circuit.state` = 2; `messaging.retry.scheduled` rising; alert `ProviderSubmissionSuccessLow`, `CircuitOpen`
Customer impact	Delay for non-critical; CRITICAL may use fallback channel if the request allows and consent permits
Recovery	Breaker half-opens, retries succeed; messages beyond `max-attempts` are `DEAD_LETTERED(RETRIES_EXHAUSTED)`; after recovery operators replay (`POST /dead-letters/{id}/replay`)
Data consistency	One `message_attempt` per try; no status skips (state machine); attempt_count matches attempts
Prevention	Breaker + backoff + jitter (ADR-0006), provider failover via `ProviderRegistry`, runbook `provider-outage.md`
Proof	`FailureRecoveryIT#scenario01_...`: fault mode TRANSIENT, assert backoff windows (the gap before attempt n+1 is at least half of the exponential ceiling), DLQ after N attempts (4 in the test context), `messaging.retry.scheduled` +3 and `messaging.deadletter.total{RETRIES_EXHAUSTED}` +1, legal transitions only; then fault NONE and `POST /dead-letters/{id}/replay` (policy pre-check audited, fresh attempt budget) reaches `SENT_TO_PROVIDER` and, after a callback, `DELIVERED`

2. Duplicate API submission

Detection	`messaging.idempotency.hits` counter; HTTP `Idempotent-Replay: true`
Customer impact	None (single message)
Recovery	Automatic
Data consistency	Exactly one `message_request` and one `idempotency_record` per `(tenant, key)`; parallel duplicates yield 200/409 `IDEMPOTENCY_IN_PROGRESS`, never two rows
Prevention	ADR-0004, UNIQUE(tenant_id, idempotency_key)
Proof	`scenario02_...`: sequential and concurrent (N threads) identical POSTs; different body same key gives 422 `IDEMPOTENCY_KEY_REUSED`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Duplicate and out-of-order callbacks

Detection	`messaging.callbacks{result="duplicate"|"out_of_order"|"stale"}`
Customer impact	None
Recovery	Automatic: duplicates ignored, lower-rank events recorded `applied=false`
Data consistency	Status never regresses; DELIVERED then BOUNCED keeps DELIVERED + audit `DELIVERY_EVENT_CONFLICT`; `delivery_event` unique key prevents duplicate rows
Prevention	CONTRACT 3.4 rank rule, `ReplayGuard`, unique constraint
Proof	`scenario03_...`: DELIVERED arrives before SENT; the late SENT is `out_of_order`; the same `X-Event-Id` again is answered `200 replayed` with no new row; a second DELIVERED under a new id is `stale`; BOUNCED after DELIVERED is a `conflict` with audit `DELIVERY_EVENT_CONFLICT`; the same payload event id behind two transport ids is a `duplicate` (unique key). `scenario03b_...`: a receipt that overtakes the consumer (message still PROCESSING) is applied and the redundant envelope is skipped. Final status and `applied` / `ignored_reason` / `messaging.callbacks{result}` are asserted

4. Provider timeout / ambiguous result

Detection	`messaging.transitions{to="DELIVERY_UNKNOWN"}`, `messaging.provider.calls{outcome="TIMEOUT"}`; `GET /failed-messages`
Customer impact	Possible delay; possible (not certain) delivery
Recovery	Late callback or `ReconciliationService.lookup`; "not received" triggers safe retry with same `idempotencyRef`
Data consistency	No blind resend: provider call count asserted == 1 until lookup says not received
Prevention	ADR-0010, provider idempotency ref
Proof	`scenario04_...`: (a) TIMEOUT: DELIVERY_UNKNOWN, provider call count stays 1, `reconcileOne` finds NOT_RECEIVED and exactly one more call is made; (b) AMBIGUOUS: lookup ACCEPTED keeps it unknown, a late DELIVERED callback wins, no second call; (c) lookup DELIVERED is applied with a `recon:` evidence event; (d) lookup FAILED is applied; (e) PUSH is retried and the provider deduplicates on the idempotency reference (2 calls, 1 accepted message)

5. Consumer crash after provider acceptance

Detection	**PROCESSING older than `lock-ttl` (sweeper), `messaging.transitions{to="DELIVERY_UNKNOWN"}` after restart**
Customer impact	Likely delivered; platform status lags
Recovery	`StuckMessageSweeper` to DELIVERY_UNKNOWN, then callback/lookup
Data consistency	`message_attempt` row `STARTED` exists with null outcome; not retried blindly
Prevention	Attempt row before call, lock TTL, ADR-0010

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Detection	PROCESSING older than `lock-ttl` (sweeper), `messaging.transitions{to="DELIVERY_UNKNOWN"}` after restart
Proof	`scenario05_...`: the simulator accepts, then a `DispatchHook` kills the worker before the outcome is persisted; the message stays PROCESSING with attempt row `STARTED` (outcome null) and the redelivery is skipped by the consumer dedupe; before the lock expires the sweeper does nothing; after `Clock` +3 min (lock TTL 2 min) the sweeper moves it to DELIVERY_UNKNOWN (`STUCK_SWEEP` audit) and the provider call count stays 1; reconciliation then finds ACCEPTED and the late receipt resolves it. `scenario05b_...`: SENT_TO_PROVIDER without a receipt past `receipt-timeout` (15 min) becomes DELIVERY_UNKNOWN

6. Database outage

Detection	`hikaricp_connections_pending`, `hikaricp_connections_timeout_total`, `/actuator/health` readiness DOWN, HTTP 503 rate
Customer impact	New submissions fail with 503 (clients retry with same key); in-flight dispatch pauses
Recovery	Connections recover; outbox publisher resumes; consumers' nacked messages redelivered
Data consistency	Fail closed: no 202 without commit (outbox in same tx); no ghost publishes; consumer cannot record outcome so it nacks/requeues; if provider call already made, becomes DELIVERY_UNKNOWN via sweeper
Prevention	Short Hikari `connectionTimeout`, readiness gating, retry with same idempotency key, HA database
Proof	`scenario06_...`: a wrapper `DataSource` throws on `getConnection` while a submit arrives: assert 503 `SERVICE_UNAVAILABLE` with `Retry-After`, no `message_request` and no `idempotency_record` row, schedulers keep running without crashing; after the restore the same key yields exactly one message (202) and a further attempt is the idempotent replay (200)

7. Redis outage

Detection	`messaging.redis.fallback` counter, Lettuce errors
Customer impact	None to slight latency (DB fallback); rate limiting degrades to in-memory per instance
Recovery	`ResilientCoordination` switches back when healthy
Data consistency	Idempotency still enforced by DB unique key; token bucket may over-admit slightly across instances (documented)
Prevention	Redis is fast-path only (ADR-0004)
Proof	`scenario07_...`: the coordination backend throws (wrapped by the production `ResilientCoordination`); duplicate submissions are still deduplicated by the database (202 then 200 replay, one row), dispatch still passes the provider rate limiter through the local fallback, `messaging.redis.fallback` increases; after recovery and the cool-down the counter stops moving

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8. Poison message

Detection	`messaging.deadletter.total{reason="POISON_MESSAGE"}`, `messaging.queue.dlq.depth`
Customer impact	None for others; that message needs operator attention
Recovery	Operator inspects, fixes cause, replays or discards
Data consistency	Status `DEAD_LETTERED`, audit with reason; bounded redeliveries (<= `poison-threshold`)
Prevention	Poison threshold; schema-tolerant envelope
Proof	`scenario08_...`: (a) an envelope for a message that does not exist and an incomplete envelope are dead-lettered on their first delivery; (b) a real message whose processing keeps throwing is delivered exactly `poison-threshold` (3) times, then lands in the bus DLQ and is mirrored in the database as `DEAD_LETTERED(POISON_MESSAGE)` through legal transitions; the consumer keeps serving other messages. (An unreadable Rabbit body goes straight to the DLQ: unit-tested for the pure part, broker part only in the container IT.)

9. Bulk job interrupted and restarted

Detection	`messaging.bulk.progress{jobId}` flat, job status RUNNING with stale step heartbeat
Customer impact	Campaign delay
Recovery	`POST /bulk-jobs/{id}/restart` resumes from last committed chunk
Data consistency	Deterministic keys `bulk-<jobId>-<externalRef>`: exactly one message per row after restart; counts reconcile (`processed = succeeded + failed + duplicate`)
Prevention	Chunk transactions, restartable job repository, skip limit
Proof	`scenario09_...`: generate synthetic CSV, fail writer mid-chunk, restart, count messages == valid unique rows

10. Provider rate limit, backpressure, backlog

Detection	`messaging.provider.calls{outcome="RATE_LIMITED"}`, `messaging.queue.depth`, oldest message age, `messaging.ratelimit.rejected`
Customer impact	Higher dispatch delay for LOW/NORMAL; CRITICAL prioritized; API returns 503 + `Retry-After` beyond backpressure threshold
Recovery	Token bucket paces; backlog drains; scale consumers only if provider limit allows
Data consistency	Rate-limit requeue consumes no attempt; no loss; priority order respected
Prevention	Token bucket, `max-queue-depth`, priority aging (spec 0001)
Proof	`scenario10_...`: (a) provider limit 2/s, six messages: `messaging.ratelimit.rejected{scope="provider"}` increases, every message ends with `attempt_count=1` and one attempt row, exactly six provider calls; (b) consumers paused and a backlog limit of 3: the API answers 503 `SERVICE_UNAVAILABLE` + `Retry-After` and persists nothing for the refused request, accepted messages are not lost, and the refused client's retry with the same key succeeds

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

How to run (once implemented)

`bash scripts/it.sh FailureRecoveryIT` (fast loop: compiles and runs only the named failsafe tests) or the full `bash scripts/mvn.sh clean verify`. Results are stated only in the validation report. The scripts in `tests/failure-recovery/` (`provider-outage`, `callback-replay`, `provider-timeout`) replay scenarios 1, 3 and 4 against a started application (profile `local`, port 18080).

Troubleshooting scenarios

Source document: `docs/leadership/troubleshooting-scenarios.md`

Six production-style problems with investigation steps. Fictional; derived from the design, not from real incidents.

TS-1: Customers report SMS arrived twice

Problem. Support sees duplicate SMS for 40 recipients in 20 minutes during a provider slowdown. **Investigation.** (1) Pick a message: `GET /messages/{id}/history`. (2) Count `message_attempt` rows: two attempts, first outcome TIMEOUT. (3) Was status DELIVERY_UNKNOWN in between? If not, retry-on-timeout was introduced (check recent deploy diff for outcome mapping). (4) If lookup said "not received" then the provider's lookup was inaccurate: check `idempotencyRef` echoed by provider in diagnostics. (5) Check whether the provider honors idempotency refs (vendor doc/sandbox). **Solution.** If code regression: roll back, restore mapping, add test. If provider lookup inaccuracy: increase `reconcile-after`, require callback absence for longer, disable auto-retry for that provider (config), escalate to vendor. **Trade-offs.** Longer unknown window vs duplicates. **Regression tests.** `FailureRecoveryIT#scenario04_...`; provider contract test for lookup semantics.

TS-2: Queue depth grows but consumers look idle

Problem. `messaging.queue.depth{NORMAL}` rising, CPU low, in-flight 0. **Investigation.** `GET /ops/queues`: consumers count; was `POST /ops/consumers/pause` used (audit)? Thread dump: workers `WAITING` on queue.take? Prefetch/ack: Rabbit unacked messages equal prefetch with a stuck worker (deadlock on a lock?). Check scheduler: if the outbox publisher thread died on an unchecked exception, depth would not grow, but retry poller could be dead instead. **Solution.** Resume consumers (paused by mistake) or fix swallowed exception: scheduler methods must catch and meter; restart instance; add liveness check on scheduler heartbeat. **Trade-offs.** More heartbeat metrics. **Tests.** Scheduler resilience test: exception in one tick does not stop subsequent ticks; pause/resume test.

TS-3: Delivered statuses stopped updating after key rotation

Problem. `invalid_signature` spike for one provider since 14:00; many messages SENT_TO_PROVIDER aging to DELIVERY_UNKNOWN. **Investigation.** Group failures by `keyId`: new keyId unknown (config missing). Confirm rotation time vs deploy. Check clock skew alternative (reason string). **Solution.** Add new key id secret to config (dual keys), redeploy; ask provider to replay webhooks; reconciliation covers the rest. **Trade-offs.** Dual-key window increases attack surface briefly. **Tests.** `SignatureVerifierTest` dual key; rotation test.

TS-4: POST /messages p99 latency doubled after release

Problem. p99 from 120 ms to 300 ms; DB CPU up. **Investigation.** `http_server_requests_seconds` by uri: only POST /messages. Hikari acquire time up. `pg_stat_statements`: a new `SELECT` per request on `recipient_preference` without index? Or N+1 in policy evaluation (loading all suppressions). `EXPLAIN` the query; check statement count in

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

integration test. **Solution.** Add `recipient_id` index/unique lookup, fetch preference with recipient via join, cache short-TTL; verify with statement-count test. **Trade-offs.** Caching staleness (consent changes) vs latency: keep TTL short and always re-check at dispatch. **Tests.** Statement-count test on submission; perf harness comparison.

TS-5: Bulk job finished "COMPLETED" but 3% fewer messages than rows

Problem. CSV 100,000 rows; `succeeded` 96,800; failures list only 400. **Investigation.** `processed = succeeded + failed + duplicate`? Duplicates count: file contains repeated `externalRef` (dedupe correct); message count by `bulk_job_id` vs succeeded; look for rows rejected at dispatch policy (OPTED_OUT/SUPPRESSED) which are REJECTED messages, not job failures: expected for marketing. Check `bulk_job.duplicate`. **Solution.** Explain via breakdown; improve job summary to show policy rejections (query by status) so counts reconcile; no data fix needed. **Trade-offs.** Extra aggregate query on job detail. **Tests.** Bulk IT with duplicates + opted-out recipients asserts breakdown arithmetic.

TS-6: Pods OOM-killed every few hours

Problem. Container restarts, `DELIVERY_UNKNOWN` count spikes after each restart. **Investigation.** Heap histogram before kill (`jcmd GC.class_histogram`), GC log shows old gen growing. Suspect: unbounded in-memory structure (e.g., failure map keyed by dispatchId for poison counting never cleaned; rate-limiter map per tenant). Check `-XX:MaxRAMPercentage`, off-heap (direct buffers from Netty/Lettuce). **Solution.** Bound/expire the structure (Caffeine with max size and TTL), fix leak, set memory request/limit with headroom, enable heap dump on OOM to secure volume. **Trade-offs.** Evicting poison counters can delay DLQ decision by one more delivery. **Tests.** Unit test of cache bounds; soak harness (executed locally if desired) watching heap.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

23. Architecture Mind Map

23.1 The whole platform on one page

The mind map is a memory aid: nine branches, each a concern, each with the mechanisms that answer it. If you can walk the map from memory and explain each leaf in one sentence, you can explain the platform.

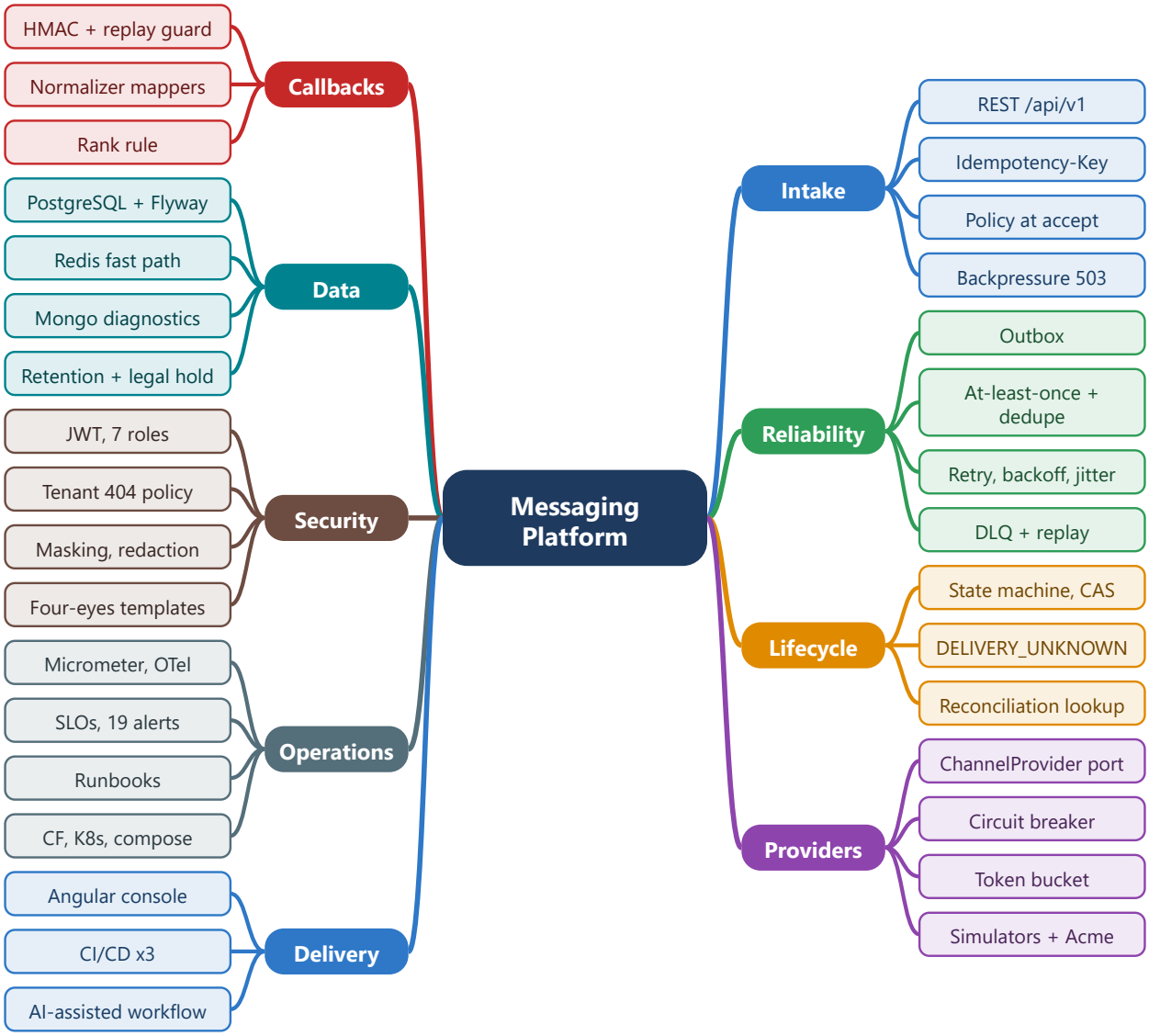


Figure 52. Architecture mind map: the major concerns of the platform and the mechanisms that address each. Implementation: docs/architecture/architecture-overview.md, docs/adr/

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

23.2 From concern to mechanism to evidence

Concern	Mechanism	Where it lives	ADR	Chapter	Proof
Never lose an accepted message	Transactional outbox, DB as source of truth	`scheduler/OutboxPublisher`, `messaging/OutboxWriter`	0009	7	Scenarios 1 and 6
No duplicates	Four-layer idempotency	`IdempotencyService`, `DispatchConsumer`, `ReplayGuard`, `ProviderRequest`	0004	5	Scenarios 2, 3
Honest status	12-state machine, CAS	`domain/MessageStateMachine`	0003	6	`MessageStateMachineTest`
Do not guess	`DELIVERY_UNKNOWN`, reconciliation	`StuckMessageSweeper`, `ReconciliationService`	0010	12	Scenarios 4, 5
Survive provider trouble	Breaker, backoff with jitter, DLQ, replay	`ResilientProviderInvoker`, `RetryPolicy`	0006	8, 13	Scenarios 1, 8, 10
Swap vendors	Ports and adapters	`ChannelProvider`, `ProviderRegistry`	0002	8	`NewProviderWithoutOrchestrationChangeTest`
Respect the customer	Ordered policy, twice	`PolicyService`	0012	11	`PolicyServiceTest`
Govern content	Versioned, four-eyes, pinned templates	`TemplateService`, `TemplateRenderer`	0007	10	`TemplateLifecycleIT`
Trust no one	JWT, tenant 404, HMAC callbacks	`SecurityConfig`, `SignatureVerifier`	0011	16	`SecurityIT`, `TenantIsolationIT`
Scale sends	Spring Batch chunks, token buckets	`batch/*`		14	Scenario 9
Operate it	Metrics, alerts, runbooks	`MessagingMetrics`, `observability/`		20	Static validation only
Run it anywhere	Profiles, CF and K8s assets	`cloud-foundry/`, `infrastructure/`	0008	17, 18	Parsed only, NOT executed

23.3 One message, end to end (the narrative to remember)

1. A **sender** posts with an `Idempotency-Key` and gets **202** after one transaction wrote the message, outbox row, audit event and idempotency response.
2. The **outbox publisher** claims the row, moves the message to `QUEUED`, and publishes an **envelope** (ids only).
3. A **consumer worker** checks cancel, expiry, policy, provider, breaker and token bucket; then **claims** the message with one transaction.
4. It calls the **provider** under a time limiter and breaker, outside any transaction, with an idempotency reference.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5. The outcome decides the next state: `SENT_TO_PROVIDER`, `RETRY_PENDING` (backoff), `FAILED` (maybe fallback), `DELIVERY_UNKNOWN` (do not guess), or `DEAD_LETTERED` after exhaustion.
6. A signed **callback** arrives later; HMAC, replay guard, normaliser and the rank rule move the message to `DELIVERED` or `FAILED` monotonically.
7. If no callback arrives, the **sweeper and reconciliation** resolve the unknown from provider evidence.
8. Everything is **audited** and visible in the console timeline; operators can cancel early, requeue or replay later.

23.4 Ten sentences to say in an interview

1. Accepted means persisted, not delivered.
2. The database is the source of truth; the broker is a delivery mechanism.
3. At-least-once plus idempotency gives effectively-once business processing.
4. Exactly-once across a provider boundary is impossible, so ambiguity is a state.
5. Every status change is a compare-and-set through one state machine.
6. Never hold a transaction across a provider call.
7. Retry only what is safe, with jitter, with a cap, with a budget.
8. Policy runs twice; fallback never bypasses consent.
9. Pin template versions; store variables, not rendered text.
10. State what was executed and what was not.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

24. API Examples

> **Where these examples come from**
>
> The request and response bodies below are shaped exactly by the committed OpenAPI specification (`docs/api/openapi.yaml`, identical to `backend/src/main/resources/openapi/messaging-api-v1.yaml`) and by the behaviour recorded in `docs/design/DEVIATIONS.md`. Where the specification carries an example (submit request, accepted response, error bodies, template create, preferences update, dev token) it is reproduced verbatim; other bodies are built from the schemas with illustrative values (ids, timestamps). They are **not captured from a run** of the service unless `docs/validation/e2e-run-output.txt` exists and says so; `OpenApiContractIT` is the test that keeps the served API and the specification in step.

Behaviours confirmed by the executed end-to-end run

The executed run in `docs/validation/e2e-run-output.txt` (Appendix E: 254 passed, 0 failed, 0 skipped, real HTTP against the packaged jar on profile `local`) confirms the status codes and error codes used in the examples below. Selected PASS lines:

Example	Confirming line in the run
24.4 Submit	`email accepted (202)`, `202 carries Location header`, `202 carries correlation id header`, `API-Version header present`
24.5 Duplicate	`same key + same body replays original (200 + Idempotent-Replay: true)`, `same key + different body -> 422 IDEMPOTENCY_KEY_REUSED`, `8 concurrent identical submissions yield exactly one message id`
24.6 Policy rejection	`opted-out recipient: marketing accepted as REJECTED (202)`, `rejection reason is OPTED_OUT and message is terminal`
24.7 Malformed	`malformed JSON -> 400 MALFORMED_REQUEST`, `error body does not leak stack/class names`
24.8 Unsupported channel	`unsupported channel -> 400 UNSUPPORTED_CHANNEL`
24.9 Unauthorized	`no token -> 401 UNAUTHENTICATED`, `tampered signature -> 401`, `AUDITOR cannot submit -> 403 FORBIDDEN`
24.10 Validation	`missing fields -> 400 VALIDATION_FAILED with field details`, `missing template variables -> 422 TEMPLATE_VARIABLE_MISSING`, `unknown recipient -> 422 RECIPIENT_NOT_FOUND`, `body > 256 KB -> 413 PAYLOAD_TOO_LARGE`
24.11 Cancel	`cancel a scheduled message -> 200 CANCELLED`, `cancel again -> 409 ILLEGAL_STATE_TRANSITION`
24.13 Templates	`author cannot approve own version -> 403`, `different approver publishes v1`, `v1 RETIRED and v2 PUBLISHED`, `the already accepted message keeps version 1`
24.14 Preferences	`stale preference write -> 409 CONFLICT_OPTIMISTIC_LOCK`
24.15 Callbacks	`same X-Event-Id again -> 200 replayed`, `wrong secret -> 401 INVALID_SIGNATURE`, `late SENT after DELIVERED is out_of_order and ignored`, `BOUNCED after DELIVERED is a conflict; first terminal state wins`, `event for an unknown message -> 202 unknown_message`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Example	Confirming line in the run
24.16 Bulk	`synthetic 500-row job accepted (202) with Location`, `processed = succeeded + failed + duplicate`, `restarted job completes all rows exactly once (succeeded = 20000, no duplicates)`
24.17 Rate limit	`burst of 400 parallel requests triggers 429 RATE_LIMITED`, `429 carries Retry-After`

The bodies printed in this chapter are still built from the OpenAPI schemas; the run output records pass or fail per check, not response bodies.

24.1 Conventions

Item	Value
Base path	`/api/v1` (local port 18080)
Auth	`Authorization: Bearer <JWT>`; callbacks use HMAC instead
Content type	`application/json`; errors share the `ApiError` shape
Correlation	`X-Correlation-Id` accepted or generated, echoed in headers and error bodies
Pagination	`page`, `size` (at most 200), `sort`; `PageResponse {content, page, size, totalElements, totalPages, sort}`
API version header	`API-Version: 1`
Local token	`POST /dev/token` with `{"subject":"demo-sender","tenant":"tenant-retail","roles":["SENDER"],"scopes":["messages:bulk"],"ttlSeconds":3600}`; roles are sent **without** the `ROLE_` prefix; only on profiles `local` and `test`

`ApiError` fields: `timestamp`, `status`, `error`, `code`, `message`, `path`, `correlationId`, `details[{field, message}]`.

24.2 Endpoint catalogue

Method and path	Roles	Success
`POST /messages`	SENDER, TENANT_ADMIN, PLATFORM_ADMIN	202 (+ `Location`); 200 on idempotent replay
`GET /messages`, `GET /messages/{id}`	any authenticated	200
`GET /messages/{id}/status`, `/history`	any authenticated	200
`POST /messages/{id}/cancel`	SENDER, TENANT_ADMIN, OPERATOR, PLATFORM_ADMIN	200 or 409
`POST /messages/{id}/requeue`	OPERATOR, PLATFORM_ADMIN	200
`GET /dead-letters`, `POST /dead-letters/{id}/replay`, `/discard`	OPERATOR (view also TENANT_ADMIN), PLATFORM_ADMIN	200
`GET /failed-messages`	OPERATOR, TENANT_ADMIN, PLATFORM_ADMIN	200

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Method and path	Roles	Success
`GET /delivery-events`	OPERATOR, AUDITOR, TENANT_ADMIN, PLATFORM_ADMIN	200
`GET /audit-events`	AUDITOR, TENANT_ADMIN, PLATFORM_ADMIN	200
`GET/POST /templates`, `GET /templates/{id}`, `/history`	any (create: TEMPLATE_AUTHOR, TENANT_ADMIN)	200, 201
`POST /templates/{id}/versions`, `PUT .../versions/{n}?locale=`	TEMPLATE_AUTHOR (create also TENANT_ADMIN)	201, 200
`POST .../versions/{n}/submit`, `/approve`, `/reject`, `/retire`	AUTHOR; APPROVER; APPROVER; APPROVER or TENANT_ADMIN	200
`POST /templates/preview`	TEMPLATE_AUTHOR, TENANT_ADMIN, OPERATOR	200
`GET /recipients`, `GET/PUT /recipients/{id}/preferences`	see Chapter 11	200
`POST /recipients/{id}/suppressions`, `DELETE .../{channel}`	TENANT_ADMIN, OPERATOR	201/200
`GET /providers`, `PUT /providers/{id}/config`, `PUT .../fault-mode`	OPERATOR+; PLATFORM_ADMIN; PLATFORM_ADMIN (local, test)	200
`GET /ops/queues`, `GET /ops/summary`, `POST /ops/consumers/pause`, `/resume`	OPERATOR, PLATFORM_ADMIN; any; PLATFORM_ADMIN	200
`POST /bulk-jobs`, `POST /bulk-jobs/generate-synthetic`, `GET /bulk-jobs[/{id}[/failures]]`, `POST .../stop`, `.../restart`	SENDER, TENANT_ADMIN, PLATFORM_ADMIN + scope `messages:bulk`	202, 200
`GET /diagnostics/messages/{id}`	OPERATOR, PLATFORM_ADMIN	200
`POST /callbacks/{providerId}`	HMAC	200 (replayed), 202
`GET /me`	any	200

24.3 Error codes

Code	HTTP	Meaning
`VALIDATION_FAILED`	400	Field-level validation; see `details`
`MALFORMED_REQUEST`	400	Unparseable body or callback
`UNSUPPORTED_CHANNEL`	400	Channel not EMAIL, SMS or PUSH
`UNAUTHENTICATED`	401	Missing, invalid or expired token
`INVALID_SIGNATURE`	401	Callback HMAC problem (uniform for all causes)
`FORBIDDEN`	403	Authenticated but not permitted
`NOT_FOUND`	404	Missing, or owned by another tenant
`IDEMPOTENCY_IN_PROGRESS`	409	First request with this key still in flight

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Code	HTTP	Meaning
`ILLEGAL_STATE_TRANSITION`	409	Action not allowed in the current status
`CONFLICT_OPTIMISTIC_LOCK`	409	Stale `version` on a preference or template write
`DUPLICATE_RESOURCE`	409	Unique conflict (additive code)
`REPLAYED_EVENT`	409	Defined; **unused** (replays answer 200)
`IDEMPOTENCY_KEY_REUSED`	422	Same key, different request body
`TEMPLATE_NOT_FOUND`, `TEMPLATE_VARIABLE_MISSING`, `TEMPLATE_RENDER_ERROR`	422	Template problems
`RECIPIENT_NOT_FOUND`	422	Unknown recipient in this tenant
`POLICY_SUPPRESSED`	422	Defined; **not returned by submission** (policy rejections answer 202 `REJECTED`)
`PAYLOAD_TOO_LARGE`	413	Body over 256 KB
`RATE_LIMITED`	429	Per-client token bucket empty; `Retry-After`, `X-RateLimit-*`
`INTERNAL_ERROR`	500	Unexpected
`SERVICE_UNAVAILABLE`	503	Backpressure or dependency outage; honour `Retry-After`

24.4 Submit a message

Request (`POST /api/v1/messages`, headers `Authorization`, `Idempotency-Key: order-8841-confirm-v1`). This is the example in the specification:

JSON

```
{
  "clientReference": "order-8841-confirm",
  "recipientId": "rcp-001",
  "channel": "EMAIL",
  "category": "TRANSACTIONAL",
  "priority": "NORMAL",
  "template": { "code": "TRADE_CONFIRMATION", "locale": "en-US", "version": null },
  "variables": { "accountLast4": "1234", "quantity": "10", "symbol": "ACME", "price": "250.00" },
  "fallbackChannels": ["SMS"],
  "schedule": { "sendAt": "2026-11-01T14:00:00Z", "expiresAt": "2026-11-02T14:00:00Z" }
}
```

Response `202 Accepted` with `Location: /api/v1/messages/6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11` (the specification's example; a future `sendAt` keeps the message `VALIDATED` until due):

JSON

```
{
  "id": "6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11",
  "status": "VALIDATED",
  "channel": "EMAIL",
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa",
  "scheduledAt": null,
  "statusReason": null,
  "links": { "self": "/api/v1/messages/6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11" }
}
```

24.5 Duplicate submission

The same request with the same `Idempotency-Key` returns **200** with header `Idempotent-Replay: true` and the **original body** (same `id`, no new rows):

JSON

```
{
  "id": "6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11",
  "status": "VALIDATED",
  "channel": "EMAIL",
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa",
  "scheduledAt": null,
  "statusReason": null,
  "links": { "self": "/api/v1/messages/6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11" }
}
```

The same key with a **different** body is `422`:

JSON

```
{
  "timestamp": "2026-11-01T14:00:03Z",
  "status": 422,
  "error": "Unprocessable Entity",
  "code": "IDEMPOTENCY_KEY_REUSED",
  "message": "Idempotency-Key was already used with a different request body",
  "path": "/api/v1/messages",
  "correlationId": "5d9c1d1e-86a1-4f0b-9d1f-2a33d3b0f1c4"
}
```

24.6 Policy rejection (implemented as 202, not 4xx)

A marketing message to a recipient who has not opted in is persisted and answered `202`:

JSON

```
{
  "id": "b2d1f0aa-3c4e-4f88-9a0b-6f7d2e1c8a55",
  "status": "REJECTED",
  "channel": "EMAIL",
  "correlationId": "7c1f3b22-0d1e-4a6e-8b59-9d2a1c4f6e10",
  "scheduledAt": null,
  "statusReason": "OPTED_OUT",
  "links": { "self": "/api/v1/messages/b2d1f0aa-3c4e-4f88-9a0b-6f7d2e1c8a55" }
}
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

24.7 Malformed request

A body that is not valid JSON:

JSON

```
{
  "timestamp": "2026-11-01T14:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "code": "MALFORMED_REQUEST",
  "message": "Request body is missing or is not valid JSON",
  "path": "/api/v1/messages",
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa"
}
```

24.8 Unsupported channel

This is the specification's example for the `BadRequest` response:

JSON

```
{
  "timestamp": "2026-11-01T14:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "code": "UNSUPPORTED_CHANNEL",
  "message": "Unsupported channel; supported channels are EMAIL, SMS, PUSH",
  "path": "/api/v1/messages",
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa",
  "details": [ { "field": "channel", "message": "must be one of EMAIL, SMS, PUSH" } ]
}
```

24.9 Unauthorized and forbidden

No token, an expired token, or a token that fails RS256 validation:

JSON

```
{
  "timestamp": "2026-11-01T14:00:00Z",
  "status": 401,
  "error": "Unauthorized",
  "code": "UNAUTHENTICATED",
  "message": "Authentication is required",
  "path": "/api/v1/messages",
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa"
}
```

A `SENDER` calling an operator endpoint (for example `GET /api/v1/ops/queues`) receives `403 FORBIDDEN` with the same shape. A message owned by another tenant is `404 NOT_FOUND`, not 403.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

24.10 Validation error

A request that fails field validation, such as an SMS with an invalid priority and no template code:

JSON

```
{
  "timestamp": "2026-11-01T14:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDATION_FAILED",
  "message": "Request validation failed",
  "path": "/api/v1/messages",
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa",
  "details": [
    { "field": "template.code", "message": "must not be blank" },
    { "field": "priority", "message": "must be one of CRITICAL, HIGH, NORMAL, LOW" }
  ]
}
```

A missing template variable is a `422 TEMPLATE_VARIABLE_MISSING` and nothing is persisted.

24.11 Cancel

`POST /api/v1/messages/{id}/cancel` while the message is still `VALIDATED`, `QUEUED` or `RETRY_PENDING` returns the lightweight status:

JSON

```
{
  "id": "6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11",
  "status": "CANCELLED",
  "statusReason": null,
  "channelUsed": null,
  "attemptCount": 0,
  "updatedAt": "2026-11-01T14:00:05Z",
  "terminal": true
}
```

After delivery the same call is `409` (the specification's example for `Conflict`):

JSON

```
{
  "timestamp": "2026-11-01T14:00:00Z",
  "status": 409,
  "error": "Conflict",
  "code": "ILLEGAL_STATE_TRANSITION",
  "message": "Action 'cancel' is not permitted while message is DELIVERED",
  "path": "/api/v1/messages/6f1c2f64/cancel",
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa"
}
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

24.12 Status and detail

`GET /api/v1/messages/{id}/status` (lightweight, built for polling):

JSON

```
{
  "id": "6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11",
  "status": "SENT_TO_PROVIDER",
  "statusReason": null,
  "channelUsed": "EMAIL",
  "attemptCount": 1,
  "updatedAt": "2026-11-01T14:00:02Z",
  "terminal": false
}
```

`GET /api/v1/messages/{id}` returns `MessageDetail`: masked address only, never the rendered body:

JSON

```
{
  "id": "6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11",
  "tenantId": "tenant-retail",
  "clientReference": "order-8841-confirm",
  "recipient": { "id": "rcp-001", "externalRef": "CUST-001", "maskedAddress": "r***@example.test" },
  "channel": "EMAIL",
  "channelUsed": "EMAIL",
  "category": "TRANSACTIONAL",
  "priority": "NORMAL",
  "status": "DELIVERED",
  "statusReason": null,
  "terminal": true,
  "template": { "code": "TRADE_CONFIRMATION", "versionNo": 3, "locale": "en-US" },
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa",
  "idempotencyKey": "order-8841-confirm-v1",
  "scheduledAt": null,
  "expiresAt": "2026-11-02T14:00:00Z",
  "attemptCount": 1,
  "maxAttempts": 5,
  "nextAttemptAt": null,
  "createdAt": "2026-11-01T14:00:00Z",
  "updatedAt": "2026-11-01T14:00:04Z",
  "attempts": [
    { "attemptNo": 1, "channel": "EMAIL", "providerId": "simulated-email", "outcome": "ACCEPTED",
      "providerMessageId": "em-77ab", "errorCode": null, "latencyMs": 42, "startedAt": "2026-11-01T14:00:01Z" }
  ],
  "deliveryEvents": [
    { "id": "e1", "messageId": "6f1c2f64-2f0e-4c7e-9a52-0a5d1a3b9c11", "providerId": "simulated-email",
      "providerEventId": "evt-1001", "type": "DELIVERED", "occurredAt": "2026-11-01T14:00:03Z",
      "receivedAt": "2026-11-01T14:00:04Z", "applied": true, "ignoredReason": null }
  ]
}
```

`GET /api/v1/messages/{id}/history` returns the merged timeline of `TimelineEntry {at, kind (AUDIT, ATTEMPT or DELIVERY_EVENT), type, summary, detail: object}`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

24.13 Templates

Create a template and its first `DRAFT` version (`POST /api/v1/templates`, the specification's example):

JSON

```
{
  "code": "PASSWORD_RESET",
  "name": "Password reset",
  "channel": "EMAIL",
  "category": "SECURITY_ALERT",
  "locale": "en",
  "subject": "Reset your password",
  "body": "<p>Use code {{code}} to reset your password. It expires in {{minutes|15}} minutes.</p>",
  "variables": ["code", "minutes"]
}
```

Preview without sending (`POST /api/v1/templates/preview`):

JSON

```
{ "channel": "EMAIL", "subject": "Reset your password",
  "body": "<p>Use code {{code}} to reset your password. It expires in {{minutes|15}} minutes.</p>",
  "variables": ["code", "minutes"], "sampleData": { "code": "482913" }, "locale": "en" }
```

JSON

```
{ "valid": true, "subject": "Reset your password",
  "body": "<p>Use code 482913 to reset your password. It expires in 15 minutes.</p>",
  "html": true, "sms": null, "errors": [] }
```

Approval is a state change on one version: `POST /api/v1/templates/{id}/versions/1/approve?locale=en` by a `TEMPLATE_APPROVER` who is not the author returns the `TemplateVersion` with `status: PUBLISHED`, `approvedBy` and `publishedAt` set; the same call by the author is refused.

24.14 Preferences

`PUT /api/v1/recipients/{id}/preferences` (the specification's example; `version` must match the stored version):

JSON

```
{
  "marketingOptIn": true,
  "emailEnabled": true,
  "smsEnabled": true,
  "pushEnabled": false,
  "quietStart": "22:00",
  "quietEnd": "07:00",
  "timeZone": "America/New_York",
  "fallbackOrder": ["SMS", "EMAIL"],
  "version": 3
}
```

A stale `version` yields `409 CONFLICT_OPTIMISTIC_LOCK`; the console shows a conflict dialog instead of overwriting.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

24.15 Callbacks

A signed provider callback (`POST /api/v1/callbacks/simulated-email`). Headers: `X-Signature: t=1767225600,v1=<hmac hex>`, `X-Provider-Key-Id: simulated-email-key`, `X-Event-Id: evt-1001`. The body is vendor specific:

JSON

```
{"event":"delivered","message_id":"em-77ab","event_id":"evt-1001","timestamp":"2026-10-18T10:15:30Z",
  "meta":{"idempotencyRef":"3f2c...:1"}}
```

Responses (`CallbackAck {result, eventId}`):

Case	Status	Body
Applied	202	`{"result":"accepted","eventId":"evt-1001"}`
Same `X-Event-Id` again	**200**	`{"result":"replayed","eventId":"evt-1001"}`
Same vendor event under another transport id	202	`{"result":"duplicate", ...}`
Receipt after the outcome is known	202	`{"result":"out_of_order", ...}`
Unknown message	202	`{"result":"unknown_message", ...}`
Bad signature, stale `t`, unknown provider, wrong key	401	`INVALID_SIGNATURE`
Unknown vendor status, missing event id	400	`MALFORMED_REQUEST`

24.16 Bulk jobs

Create from a CSV (`POST /api/v1/bulk-jobs`, multipart: `file`, `name`, `templateCode`, `channel`, `category`, `ratePerSecond`). The caller needs scope `messages:bulk`. Response `202` with a `Location` header:

JSON

```
{
  "id": "9a7e5c10-1b2d-4e3f-8a6b-0c9d8e7f6a51",
  "tenantId": "tenant-retail",
  "name": "october-newsletter",
  "status": "CREATED",
  "total": 0, "processed": 0, "succeeded": 0, "failed": 0, "duplicate": 0,
  "fileName": "9a7e5c10-1b2d-4e3f-8a6b-0c9d8e7f6a51.csv",
  "ratePerSecond": 50,
  "createdBy": "demo-sender",
  "startedAt": null,
  "finishedAt": null
}
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Synthetic job without an upload (`POST /api/v1/bulk-jobs/generate-synthetic`):

```json
{ "rows": 1000, "duplicateRate": 0.02, "invalidRate": 0.01,
  "templateCode": "WELCOME", "channel": "EMAIL", "category": "SERVICE" }
```

A finished job reconciles as `processed = succeeded + failed + duplicate`; `GET /api/v1/bulk-jobs/{id}/failures` lists `{lineNo, reason, rawLineHash}` and a `lineNo` of `0` is a job-level failure such as `SKIP_LIMIT_EXCEEDED`.

24.17 Backpressure and rate limiting

When the dispatch backlog is full the API answers 503 with a `Retry-After` header; the client retries with the **same** `Idempotency-Key`. When the per-client token bucket is empty it answers 429 with `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset`:

```json
{
  "timestamp": "2026-11-01T14:00:00Z",
  "status": 503,
  "error": "Service Unavailable",
  "code": "SERVICE_UNAVAILABLE",
  "message": "Dispatch backlog is full (5000 waiting); retry later",
  "path": "/api/v1/messages",
  "correlationId": "0b8f4a0e-6a77-4a5e-8d52-c3a1f7b2d9aa"
}
```

24.18 Reproducing these locally

```bash
bash scripts/run-local.sh --wait      # profile local, API on http://localhost:18080
bash scripts/seed-demo.sh             # synthetic demo data
bash scripts/smoke-test.sh            # liveness, readiness, 401 on /api/v1/me
node tests/failure-recovery/callback-replay.mjs   # scripted replay/duplicate callbacks
```

`tests/api-http` request collections, where present, and the OpenAPI spec at `docs/api/openapi.yaml` (also served at `/v3/api-docs` and `/swagger-ui.html` on the local profile) are the executable reference.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

25. Technical-Leadership Artifacts

25.1 What a technical lead produces besides code

Leading a platform like this is mostly **making decisions legible**: why RabbitMQ rather than Kafka, why the outbox, why 202 for a policy rejection. The artifacts in this chapter are the written form of that leadership: decision records, a delivery plan, a risk register, review practice, mentoring and incident learning. Each is a real file in the repository, and each links back to evidence in code and tests.

Artifact	Repository location	Purpose
Architecture decision records	`docs/adr/`	Record why, so decisions are not re-litigated or silently reversed
Build contract and deviations	`docs/design/CONTRACT.md`, `DEVIATIONS.md`	One source of truth; every divergence recorded, append only
PRD, feature spec, acceptance criteria	`docs/leadership/`	What and for whom, testable
HLD and LLD	`docs/leadership/hld.md`, `lld.md`	Structure and class level design
Sprint plan	`docs/leadership/sprint-task-breakdown.md`	Delivery order and dependencies
Risk register	`docs/leadership/risk-register.md`	Scored risks with owners and mitigations
Test strategy and PR checklist	`docs/leadership/test-strategy.md`, `pr-checklist.md`	Quality gates
Review scenarios and mentoring	`docs/leadership/code-review-scenarios.md`, `mentoring-examples.md`	Teaching by realistic case
Post-incident review template	`docs/operations/post-incident-review-template.md`	Blameless learning
Specs	`specs/`	Spec-driven change

25.2 ADR summary

All twelve ADRs are **Accepted**, dated 2026-10-18. Changing an accepted ADR requires a new ADR that supersedes it; a silent edit or a silent override by an agent is a process violation. The full text of each ADR appears in the chapter named in the last column.

ADR	Title	Decision	Main trade-off accepted	Chapter
0001	Asynchronous messaging	RabbitMQ behind a `MessageBus` port; Kafka documented for replay and stream workloads	Weaker replay and fan-out than Kafka, simpler operations and priorities	7
0002	Provider abstraction	Adapter per channel plus strategy routing through `ProviderRegistry`	An abstraction layer to maintain; lowest common denominator risk	8

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ADR	Title	Decision	Main trade-off accepted	Chapter
0003	Lifecycle state machine	Explicit transition table, CAS ownership, terminal states	Every new behaviour must fit the table	6
0004	Idempotency	Four layers: API key, consumer dedupe, callback event id, provider reference	More tables and code paths than a single check	5
0005	Database	PostgreSQL primary, Oracle-mappable portable SQL, Mongo only for diagnostics	No PostgreSQL-specific features (partial indexes, JSONB)	15
0006	Retry and dead letter	Exponential backoff with full jitter, caps, poison threshold, operator replay	Slower recovery of some messages for safety	13
0007	Template versioning	Pin version at accept time	Rendering twice (check at accept, render at dispatch)	10
0008	Cloud Foundry versus Kubernetes	Platform-neutral twelve-factor app; both manifests shipped	Two sets of deployment assets, neither executed here	17
0009	Outbox	Same-transaction outbox row, scheduled publisher with CAS claim	Publication latency up to the poll interval (200 ms)	7
0010	Ambiguous outcomes	`DELIVERY_UNKNOWN` plus reconciliation; no blind resend for EMAIL and SMS	Some messages wait for evidence	12
0011	Multi-tenancy and authz	JWT tenant claim, shared schema, 404 on foreign resource, role matrix	Shared schema needs row-level security for production	16
0012	Consent and suppression	Ordered policy; fallback never bypasses consent	Policy runs twice (accept and dispatch)	11

Template of an ADR

Context, Decision, Alternatives considered, Consequences (positive, negative, risks), Status, Validation (the code and tests that confirm it). The "Validation" section is what stops an ADR from being aspirational: each one points at classes and tests by name.

25.3 Decisions that implementation changed

Leadership includes updating the record honestly when reality teaches something. The deviations file shows decisions refined by implementation, each recorded rather than hidden:

Original expectation	Implemented	Why
Policy rejection might be a 4xx `POLICY_SUPPRESSED`	202 with `status=REJECTED`	Idempotent replay and audit stay consistent
Replayed callback is 409 `REPLAYED_EVENT`	200 `replayed`	The provider must stop retrying
Consumer claims then checks	Checks first, claim is one transaction	A rate-limit requeue must not consume an attempt or hit the dedupe table
Circuit permit taken at call time	Reserved before the claim, released if unused	A recovering provider must not burn attempt budgets

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Original expectation	Implemented	Why
Callback duplicate recorded as a `DUPLICATE` row	No second row	The unique `(provider_id, provider_event_id)` constraint forbids it
Gauges bound directly	Strong references kept	Micrometer holds gauge state weakly; gauges turned `NaN`

25.4 Sprint plan

The example plan is three two-week sprints for about five engineers plus QA. Estimates are relative story points, not measurements.

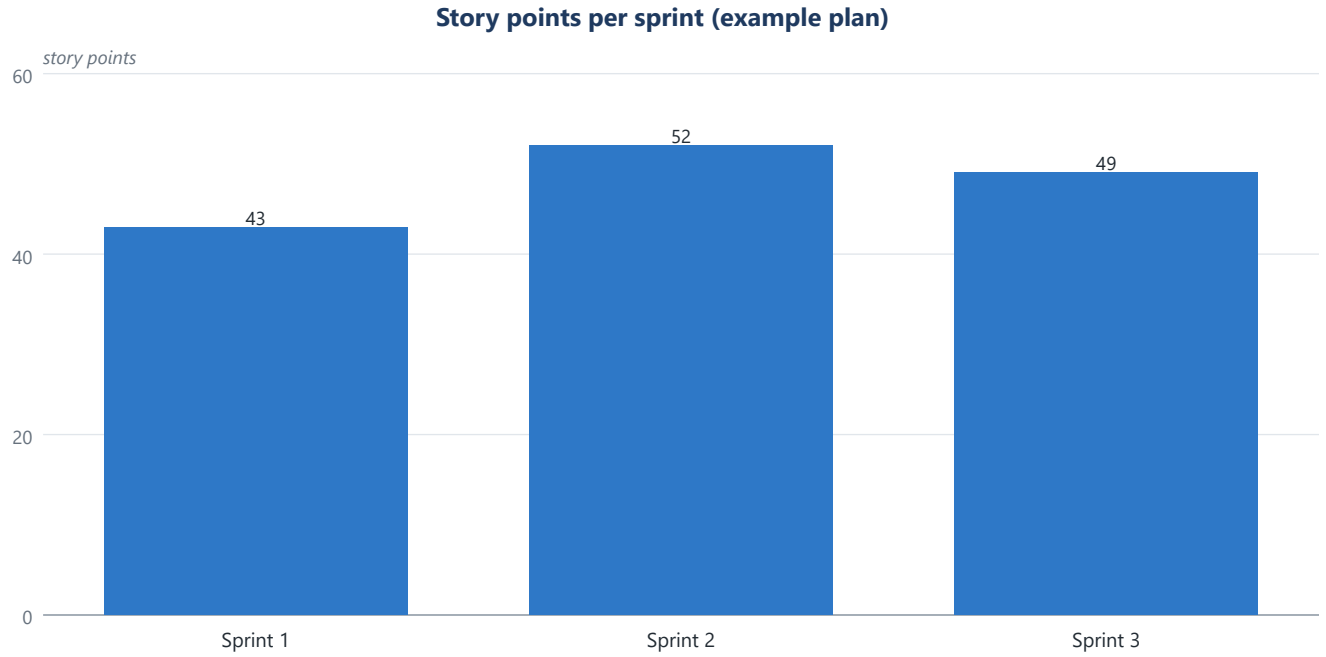


Figure 53. Planned story points per sprint from the example sprint breakdown (S1: 43, S2: 52, S3: 49). Relative estimates, not measurements. Implementation: docs/leadership/sprint-task-breakdown.md

Sprint	Goal	Demo
1 Walking skeleton and safe acceptance	Submit, persist, queue, dispatch to a simulator, see status	End-to-end happy path locally
2 Reliability	Failures are safe	Fault injection (timeout, outage, duplicate callbacks) with correct states
3 Scale, operability, UX	Bulk, console, observability, deployment assets, docs	Console walk-through; static validation of assets

Working agreements: trunk-based development with short-lived branches; pull requests under about 400 lines where possible; every PR uses `.github/pull_request_template.md`; security-sensitive PRs need an independent reviewer (`.github/CODEOWNERS`); an ADR for every cross-cutting decision.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

25.5 Risk register highlights

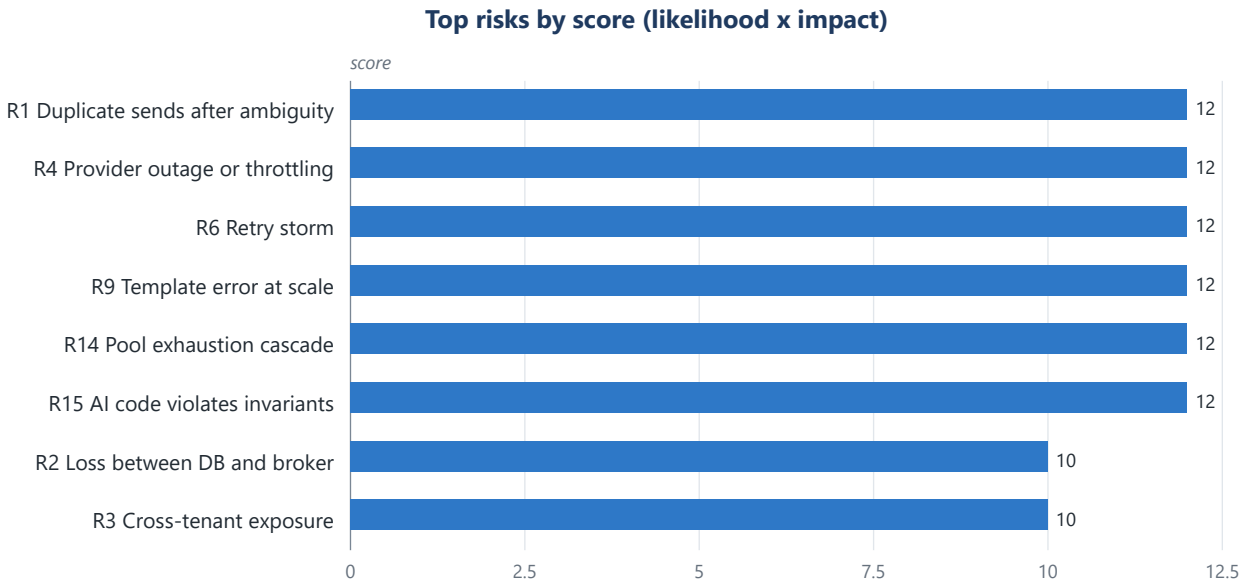


Figure 54. Highest-scoring risks from the risk register (R1, R4, R6, R9, R10, R12, R14, R15 all score 12). Implementation: docs/leadership/risk-register.md

Risks are scored likelihood times impact (1 to 5 each), with an owner role and a status. The register is reviewed at each sprint planning. Several risks are honest about what is **open** in this repository: R5 (H2 versus PostgreSQL divergence; container tests not executed here), R12 (runbooks untested), R13 (capacity assumptions; k6 not executed) and R16 (deployment assets never executed, accepted for a reference implementation).

25.6 Review examples

Review is where leadership scales. Two collections are included in full below. **Code review scenarios** (CR-1 to CR-6) each state the problem, the investigation, the solution, the trade-off and the regression test. The **AI code review** examples show ten defect classes that AI-generated code commonly introduces in this domain.

Defect class	Example	Invariant protected
Retry on timeout "to improve reliability"	CR-1, AI example 4	Never blind-retry an uncertain outcome
Status written without compare-and-set	CR-2, AI examples 1 and 3	CAS through the state machine
`findById` without tenant	CR-3, AI example 6	Tenant isolation
Unbounded executor	CR-4, AI example 7	Everything bounded
N+1 in a list endpoint	CR-5, AI example 5	Query count bounded
Missing idempotency	AI example 2	One effect per intent
Resource leak	AI example 8	Pool and stream hygiene
Insufficient tests	AI example 9	Tests prove the invariant
Sensitive data in logs	AI example 10	Data minimisation

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Reference material

ADR index and process

Source document: `docs/adr/README.md`

Binding companion: `docs/design/CONTRACT.md`. ADRs explain *why*; the contract states *what*. Changing an accepted ADR requires a new ADR that supersedes it (never a silent edit, never a silent override by an agent; see `ai-engineering/multi-agent-workflow.md`).

ADRs 0001-0012: status **Accepted** (ADR-0013 is **Proposed**), dated 2026-10-18, authored with Claude Code for this independent reference implementation. "Validation" sections point at code locations by package/class name from CONTRACT section 2; where behavior is not confirmed in this document the text says "see source".

ADR	Title	One-line decision
0001	Asynchronous messaging: RabbitMQ vs Kafka	RabbitMQ behind a `MessageBus` port; Kafka is the documented alternative for replay/stream workloads
0002	Provider abstraction	Adapter per channel + strategy-based routing via `ProviderRegistry`; new provider = new bean + config
0003	Lifecycle / state machine	Explicit transition table in `MessageStateMachine`, CAS ownership, terminal states
0004	Idempotency	Four layers: API key, consumer dedupe, callback event id, provider idempotency ref
0005	Database choice	PostgreSQL primary, Oracle-mappable SQL, Mongo only for provider diagnostics
0006	Retry and dead-letter	Exponential backoff + full jitter, caps, poison threshold, operator replay
0007	Template versioning	Pin version at accept time; accepted messages are immune to later template changes
0008	Cloud Foundry vs Kubernetes	Platform-neutral 12-factor app; both manifests shipped, neither executed here
0009	Outbox pattern	Same-transaction outbox row, scheduled publisher with CAS claim
0010	Ambiguous outcomes	`DELIVERY_UNKNOWN` + reconciliation lookup; no blind resend for EMAIL/SMS
0011	Multi-tenancy and authz	JWT tenant claim, shared schema, 404-on-foreign-resource, role matrix
0012	Consent and suppression	Ordered policy; fallback never bypasses consent
0013	Priority aging	**Proposed, not implemented**: bounded wait for LOW/NORMAL via effective-priority promotion (spec 0001 is a worked example)

Template used

Context, Decision, Alternatives considered, Consequences (positive/negative/risks), Status, Validation (code and tests).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Process rules

1. New cross-cutting decision: copy the structure above, number sequentially, add a row here.
2. Superseding: set the old ADR status to `Superseded by ADR-NNNN` and link both ways.
3. Deviations from CONTRACT go to `docs/design/DEVIATIONS.md` (append only) and must reference the ADR.

Sprint task breakdown

Source document: `docs/leadership/sprint-task-breakdown.md`

Illustrative plan for a team of ~5 engineers + QA, 2-week sprints. Estimates are story points (relative), not measurements. Each task has a definition of done per `ai-engineering/checklists/definition-of-done.md`.

Sprint 1: Walking skeleton and safe acceptance

Goal: submit, persist, queue, dispatch to a simulator, see status. Demo: end-to-end happy path locally.

ID	Task	SP	Depends	Acceptance
S1-1	Project skeleton, profiles, Flyway V1 schema, H2/PG	5		app boots on `local`; migrations apply
S1-2	Security: JWT resource server, dev token, tenant context	5	S1-1	401/403/404 matrix tests
S1-3	Domain: enums, `MessageStateMachine`, audit service	3	S1-1	full transition matrix test
S1-4	`POST /messages` + idempotency + validation	8	S1-2,3	AC-1..4 in `acceptance-criteria.md`
S1-5	Template service (create/publish/pin) + renderer	8	S1-1	pin, escape, variable validation tests
S1-6	Outbox + `InMemoryMessageBus` + `DispatchConsumer` happy path + simulators	8	S1-4	message reaches SENT_TO_PROVIDER
S1-7	Status/list/history endpoints	3	S1-4	pagination + masked output tests
S1-8	Metrics skeleton, correlation id, structured logs	3	S1-1	`/actuator/prometheus` lists core meters

Risks: schema portability H2/PG; mitigation: portable-SQL checklist.

Sprint 2: Reliability

Goal: failures are safe. Demo: fault injection (timeout, outage, duplicate callbacks) with correct states.

ID	Task	SP	Depends	Acceptance
S2-1	`RetryPolicy`, backoff/jitter, RETRY_PENDING, DLQ, poison	8	S1-6	scenario01, scenario08
S2-2	`ResilientProviderInvoker` (breaker, timeout)	5	S1-6	circuit metrics, outcomes
S2-3	Callback endpoint: HMAC, replay guard, normalization, ranking	8	S1-6	scenario03; forged callback 401
S2-4	Ambiguity: sweeper, `ReconciliationService`, lookup	8	S2-1	scenario04/05
S2-5	Rate limit (token bucket), Redis adapter + resilient fallback	5	S1-6	scenario07, scenario10

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ID	Task	SP	Depends	Acceptance
S2-6	Policy service (consent, suppression, quiet hours, fallback)	8	S1-4	table-driven tests
S2-7	Ops endpoints, replay/requeue/cancel	5	S2-1	scenario01 replay step
S2-8	`FailureRecoveryIT` consolidation	5	all	10 scenarios

Sprint 3: Scale, operability, UX

Goal: bulk, console, observability, deployment assets, docs.

ID	Task	SP	Depends	Acceptance
S3-1	Spring Batch bulk job + restart	8	S2-6	scenario09
S3-2	Angular console (messages, templates, queues, DLQ)	13	S2-7	unit tests, role-aware nav
S3-3	Mongo diagnostics + TTL	3	S2-3	best-effort test
S3-4	Dashboards/alerts, runbooks wiring	5	S2-*	alert rules lint (static)
S3-5	Docker/Compose/K8s/Helm/CF assets (static validation)	5		files lint; marked N/E
S3-6	Perf harness + k6 scripts	5	S2-8	harness executed locally; k6 N/E
S3-7	Security tests + threat model review	5		`tests/security`
S3-8	Docs, ADR review, release checklist	5		review gate

Cross-sprint working agreements

Trunk-based with short-lived branches; PRs < 400 lines where possible; every PR uses `.github/pull_request_template.md`; security-sensitive PRs need independent reviewer (CODEOWNERS); ADR for each cross-cutting decision.

Dependencies and risks

See `risk-register.md`.

Dependency and risk register

Source document: `docs/leadership/risk-register.md`

Likelihood (L) and Impact (I): 1 low to 5 high. Score = L x I. Owners are roles. Status values: Open, Mitigating, Accepted, Closed.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Dependencies

ID	Dependency	Type	Needed by	Fallback
D1	Enterprise IdP issuing JWT with `tenant`, `roles`, `scope` claims	external	Sprint 1 (prod), dev token locally	local dev issuer
D2	Provider vendor sandboxes, webhook specs, quotas	external	before real integration	simulators
D3	PostgreSQL/RabbitMQ/Redis/MongoDB environments	infra	staging	in-memory profiles
D4	Compliance/privacy review of consent policy	governance	before production	n/a
D5	Observability stack (Prometheus, Grafana, OTel collector)	infra	Sprint 3	local metrics endpoint
D6	Container/K8s/CF platform access	infra	Sprint 3	static validation

Risks

ID	Risk	L	I	Score	Mitigation	Owner	Status
R1	Duplicate sends after ambiguous outcomes	3	4	12	ADR-0010, provider idempotency refs, no blind retry, reconciliation tests	Backend lead	Mitigating
R2	Message loss between DB and broker	2	5	10	Outbox (ADR-0009), consumer dedupe	Backend lead	Mitigating
R3	Cross-tenant data exposure	2	5	10	Tenant-scoped repositories, 404 policy, matrix tests, independent review, RLS in prod	Security lead	Mitigating
R4	Provider outage/throttling	4	3	12	Breaker, backoff+jitter, multi-provider, runbooks	Ops lead	Mitigating
R5	H2 vs PostgreSQL behavior divergence	3	3	9	portable SQL rule, Testcontainers PG in CI	Backend lead	Open (container tests N/E here)
R6	Retry storm amplifies incident	3	4	12	full jitter, caps, shared token bucket, runbook	Backend lead	Mitigating
R7	Priority starvation of LOW traffic	3	2	6	priority aging spec 0001	Architect	Planned
R8	Forged/replayed callbacks	2	4	8	HMAC + replay guard + unique constraint	Security lead	Mitigating
R9	Template content error sent at scale	3	4	12	four-eyes, preview, cancel tooling, canary template rollout	Product owner	Mitigating
R10	PII leakage in logs/diagnostics	3	4	12	Redaction, minimization, log tests, review checklist	Security lead	Mitigating
R11	Consent logic misapplied (marketing as transactional)	2	5	10	category bound to template, audits, policy tests	Product owner	Mitigating
R12	Operational unreadiness (runbooks untested)	4	3	12	game days, PIR culture	Ops lead	Open

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ID	Risk	L	I	Score	Mitigation	Owner	Status
R13	Capacity assumptions wrong	3	3	9	load tests, metrics-based scaling, backpressure	Architect	Open (k6 N/E here)
R14	Connection pool exhaustion cascades	3	4	12	short tx, no tx over provider call, alerts	Backend lead	Mitigating
R15	AI-generated code violates invariants	3	4	12	invariants list, checklists, independent review, tests (`ai-engineering/`)	Tech lead	Mitigating
R16	Deploy assets never executed here	5	2	10	explicit N/E labelling; validate in real pipeline	DevOps lead	Accepted for reference
R17	Single datastore dependency (PostgreSQL)	2	5	10	HA, backups, DR design (not exercised)	DBA	Open

Review cadence

Reviewed at each sprint planning; scores updated with evidence; closed risks retained with reason.

Code review scenarios

Source document: `docs/leadership/code-review-scenarios.md`

Six scenarios drawn from the failure classes of this platform. Each: problem (what the PR does), investigation (what the reviewer checks), solution, trade-offs, regression tests. Code is illustrative; class names follow CONTRACT.

CR-1: Retry on timeout "to improve reliability"

Problem. PR changes `DispatchService` so `TIMEOUT` maps to `RETRY_PENDING`:

```java
case TIMEOUT -> transition(msg, RETRY_PENDING);   // "just retry, simpler"
```

Investigation. Does a timeout mean the provider did not accept? No. Read ADR-0010; check channel (SMS). Ask: what prevents two SMS? The provider idempotency ref only helps if the vendor honors it. Check `FaultInjector` AMBIGUOUS test: provider call count. **Solution.** Map to `DELIVERY_UNKNOWN`; reconciliation decides; PUSH may retry (collapse key):

```java
case TIMEOUT, AMBIGUOUS -> channel == PUSH ? retry(msg) : transition(msg, DELIVERY_UNKNOWN);
```

Trade-offs. Slower recovery for truly lost sends vs duplicate risk. Chosen: no duplicate; bounded delay via `reconcile-after`. **Regression tests.** `FailureRecoveryIT#scenario04_...` (provider call count == 1 until lookup says not received); `DispatchServiceTest` outcome table.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

CR-2: Status update without compare-and-set

Problem. Callback processor does `msg.setStatus(DELIVERED); repo.save(msg);` after reading the entity. **Investigation.** Concurrent actors: sweeper marks DELIVERY_UNKNOWN, operator cancels, consumer finishes. Read-modify-write lets the last writer win and bypasses `MessageStateMachine`; `@Version` would throw but the handler catches and ignores? Look for swallowed `ObjectOptimisticLockingFailureException`. **Solution.** Use CAS: `int updated = repo.casStatus(id, expectedFrom, DELIVERED, now)`; if 0, reload and re-evaluate by rank rule (not blind retry). **Trade-offs.** One more query on conflict; much stronger invariants. **Tests.** Concurrent callback vs sweeper race test; `MessageStateMachineTest` matrix; assert no direct `setStatus` usage via ArchUnit-style test.

CR-3: "Find by id" for convenience

Problem. New endpoint `GET /api/v1/delivery-events/{id}` uses `repo.findById(id)`. **Investigation.** Where does tenant come from? Not applied. Any tenant can enumerate. Check also response mapping for masked fields. **Solution.** `findByIdAndTenantId(id, ctx.tenantId())`; foreign => 404; PLATFORM_ADMIN scoped via header. **Trade-offs.** Slightly more repository methods; prevents BOLA. **Tests.** Cross-tenant matrix entry for the endpoint expecting 404; role test.

CR-4: Unbounded executor for provider calls

Problem. `Executors.newCachedThreadPool()` used by `ResilientProviderInvoker` "so calls never queue". **Investigation.** Under provider slowness each in-flight call parks a thread; cached pool grows without bound => OOM/thread exhaustion, and the breaker's time limiter cancels futures but threads blocked on sockets remain. **Solution.** Bounded `ThreadPoolExecutor` (core=max=concurrency+margin, small queue, `AbortPolicy` mapped to `RATE_LIMITED`/requeue), socket-level read timeouts in adapter, metric for rejections. **Trade-offs.** Rejections under overload (desired backpressure) instead of cascading failure. **Tests.** Load test with TIMEOUT fault: thread count stays bounded; `FailureRecoveryIT#scenario10_...`.

CR-5: N+1 in message list

Problem. `GET /messages` maps entities to DTOs; mapping touches `message.getRecipient().getEmail()` and `message.getAttempts().size()` lazily. **Investigation.** Enable statistics in a test: 1 + 2N queries for page size N. Also PII: mapper calls email getter. **Solution.** Projection/DTO query joining recipient (masked in SQL or mapper), `@EntityGraph` or batch size; counts via subquery or denormalized `attempt_count` (already a column). **Trade-offs.** Dedicated query vs entity reuse; faster, slightly more code. **Tests.** Statement-count assertion (<= 3) for page size 50; masking test.

CR-6: Logging the payload on failure

Problem. `log.error("Provider failed for {} with request {}", messageId, providerRequest)`. **Investigation.** `providerRequest` contains phone number and rendered body. **Solution.** Log ids and outcome codes; use `Redaction` for any structure; put sanitized snippet in diagnostics store (Mongo) with TTL. **Trade-offs.** Less detail in logs; more privacy; diagnostics endpoint for operators. **Tests.** Log capture test asserting absence of `+1555` / `@example.test` patterns during a failing send.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

CR-7 (bonus): Idempotency check after side effect

Problem. Service sends to provider first, then writes `idempotency_record`. **Solution.** Claim first (unique insert), act, complete; crash between leaves IN_PROGRESS which the TTL/sweeper resolves. **Tests.** `scenario02_...` plus crash-between test.

Mentoring examples

Source document: `docs/leadership/mentoring-examples.md`

Short dialogues between a senior engineer (S) and a developer (D), each with a code snippet. Goal: teach reasoning, not rules.

1. SOLID: open/closed with providers

D: I'll add `if (providerId.equals("acme-sms"))` in `DispatchService` for the Acme quirks. S: What happens when we add the fifth vendor? D: Another `if`. S: Then `DispatchService` changes for every vendor and every change risks the retry logic. Where does vendor-specific knowledge belong? D: In an adapter behind `SmsProvider`. S: And selection? D: `ProviderRegistry`, driven by `provider_config`.

JAVA

```
public interface SmsProvider { ProviderResult send(SmsRequest r); Optional<ProviderStatus> lookup(String
  idempotencyRef); String id(); }
@Component class AcmeSmsProvider implements SmsProvider { /* maps Acme errors to ProviderOutcome */ }
```

Takeaway: open for extension via new bean; closed for modification of orchestration (ADR-0002).

2. Patterns: why outbox, not "publish then save"

D: Can't I just call `bus.publish()` in the service method after `save()`? S: What if the publish throws? What if the app dies between the two lines? D: The client has a 202 but nothing is sent. S: And publishing inside the transaction? D: Rolled back DB, but message already on the bus. S: So we make the DB the single commit point.

JAVA

```
@Transactional
public MessageAcceptedResponse submit(...) {
  var msg = repo.save(MessageRequest.validated(...));
  outbox.save(OutboxEvent.dispatch(msg.getId()));   // same transaction, published later by OutboxPublisher
  return MessageAcceptedResponse.of(msg);
}
```

Takeaway: ADR-0009; at-least-once publish + idempotent consumer.

3. Testing: proving a race fix

D: I added a unique index; tests pass. S: Do they exercise concurrency? D: No, sequential. S: A sequential test passes even without the index if the code checks first. Make the race real.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```java
var start = new CountDownLatch(1);
var results = IntStream.range(0, 20).mapToObj(i -> pool.submit(() -> { start.await(); return post(key, body);
  })).toList();
start.countDown();
assertThat(results.stream().map(this::get).filter(r -> r.status()==202).count()).isEqualTo(1);
assertThat(messageRepo.countByTenantIdAndIdempotencyKey(tenant, key)).isEqualTo(1);
```

Takeaway: assert the invariant (one row), not just the status codes.

4. Security: tenant from claim, never from body

D: The request has `tenantId`, I'll use it so admins can send for any tenant. S: What stops a tenant A sender writing `tenant-b`? D: ...nothing. S: Where is trusted identity? D: The JWT claim in `TenantContext`.

```java
String tenant = tenantContext.requireTenant();      // from verified JWT
var recipient = recipientRepo.findByIdAndTenantId(req.recipientId(), tenant)
    .orElseThrow(() -> new NotFoundException("RECIPIENT_NOT_FOUND"));   // 404, not 403
```

Takeaway: ADR-0011; foreign resource looks nonexistent.

5. DB performance: the index that makes writes slower

D: The list is slow, I'll add indexes on every filter column. S: What does each index cost on `message_request`? D: Extra write per insert and per status change. S: And status changes about 5 times per message. Which query matters? D: `(tenant_id, status, created_at)` for the console list. S: Prove with a plan and a statement count; add one index per named query.

```sql
CREATE INDEX ix_msg_tenant_status_created ON message_request (tenant_id, status, created_at DESC);
```

Takeaway: `docs/database/query-plans-and-indexes.md`; every index has an owner query.

6. Production troubleshooting: queue depth rising

D: Queue depth is rising, I'll double the consumers. S: Before that: are consumers the bottleneck or the provider? D: How do I know? S: Compare in-flight with concurrency, look at provider latency and `RATE_LIMITED`. If workers are all waiting on a slow provider, more workers only add load.

```
GET /api/v1/ops/queues   -> inflight=8/8 (saturated)
messaging_provider_latency_seconds p95 = 9s (was 0.4s)  -> provider slow; follow provider-outage runbook
```

Takeaway: Little's law: throughput = concurrency / latency; fix the constraint, not the symptom.

7. Honest reporting

D: I'll mark the Helm chart "tested". S: Did you run `helm lint` or install it? D: I only read it. S: Then the doc says "not executed". Trust is the product. Takeaway: documentation honesty rules (CONTRACT section 12).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

AI code review: worked examples

Source document: `ai-engineering/ai-code-review/README.md`

Ten examples of defects that AI-generated (or hurried human) code commonly introduces in this domain. Each file has: **Flawed code/design**, **Why it is wrong**, **Corrected version**, **Rationale**, **Regression test**, and the invariant(s) from `../context-engineering.md` it protects. Snippets are illustrative Java in the style of the project; confirm real signatures in source.

#	File	Defect class
1	`01-incorrect-state-transition.md`	Illegal transition / bypassing the state machine
2	`02-missing-idempotency.md`	Duplicate effect on retry
3	`03-race-condition.md`	Read-modify-write status race
4	`04-unsafe-retry.md`	Blind retry after ambiguous outcome
5	`05-n-plus-one.md`	N+1 queries and DB performance
6	`06-incorrect-authorization-cross-tenant.md`	BOLA / missing tenant scope
7	`07-unbounded-concurrency.md`	Unbounded executor/queue
8	`08-resource-leak.md`	Connection/stream leak
9	`09-insufficient-tests.md`	Tautological or happy-path-only tests
10	`10-sensitive-data-in-logs.md`	PII/secret leakage

How to use in a review session: give the assistant the diff plus the invariants list; ask it to cite which invariant each finding violates; require a failing test before accepting a fix. These examples are teaching material authored for this repository; they are not records of defects found by an automated Copilot review.

AI review example 1: Incorrect state transition

Source document: `ai-engineering/ai-code-review/01-incorrect-state-transition.md`

Invariants: 1 (state machine + CAS), 5 (acceptance != delivery).

Flawed code

JAVA

```
// CallbackService
public void onReceipt(DeliveryEvent e) {
    MessageRequest m = repo.findById(e.messageId()).orElseThrow();
    if (e.type() == DeliveryEventType.SENT) {
        m.setStatus(MessageStatus.DELIVERED);      // "sent" treated as delivered
    }
    repo.save(m);
}
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Why it is wrong

- `SENT`/`PROVIDER_ACCEPTED` are rank 1: they mean provider acceptance, not delivery.
- Direct `setStatus` bypasses `MessageStateMachine` (a message in `CANCELLED` or already `FAILED` can become DELIVERED), no audit, no metric.
- No dedupe on `(provider_id, provider_event_id)`.

Corrected version

JAVA

```
@Transactional
public ApplyResult onReceipt(DeliveryEvent e) {
    if (!deliveryEvents.insertIfAbsent(e)) return ApplyResult.duplicate();
    MessageRequest m = repo.findByIdAndProvider(e.messageRef()).orElse(null);
    if (m == null) return deliveryEvents.markIgnored(e, "UNKNOWN_MESSAGE");
    Optional<MessageStatus> target = eventRules.targetStatus(m.getStatus(), e); // rank rule + table
    if (target.isEmpty()) return deliveryEvents.markIgnored(e, "OUT_OF_ORDER");
    int updated = repo.casStatus(m.getId(), m.getStatus(), target.get(), clock.instant());
    if (updated == 0) return reevaluate(e);   // lost a race: reload and re-apply rank rule once
    audit.transition(m, target.get(), e); metrics.transition(m.getStatus(), target.get());
    return ApplyResult.applied();
}
```

`eventRules.targetStatus` returns DELIVERED only for DELIVERED events from `SENT_TO_PROVIDER`, `PROCESSING` or `DELIVERY_UNKNOWN`; SENT/ACCEPTED events never produce DELIVERED.

Rationale

The transition table is the contract between actors; centralizing it makes illegal states unrepresentable and gives audit/metrics for free.

Regression tests

- `MessageStateMachineTest#allPairsMatchContractTable` (matrix).
- `DeliveryEventProcessorTest#sentEvent_doesNotMakeDelivered`, `#deliveredAfterCancelled_isIgnored`.
- `FailureRecoveryIT#scenario03_duplicateAndOutOfOrderCallbacks_applyMonotonically`.
- Structural: a test that fails if any class outside the transition service calls `setStatus`.

AI review example 2: Missing idempotency

Source document: `ai-engineering/ai-code-review/02-missing-idempotency.md`

Invariants: 2 (durable accept), 3 (idempotent consumer/callback).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Flawed code

JAVA

```
@PostMapping("/messages")
public ResponseEntity<MessageAcceptedResponse> submit(@RequestBody SubmitMessageRequest req) {
    MessageRequest m = service.create(req);          // always creates
    bus.publish(DispatchEnvelope.of(m));             // publish immediately, outside the tx
    return ResponseEntity.accepted().body(MessageAcceptedResponse.of(m));
}
```

JAVA

```
// consumer
public void onMessage(DispatchEnvelope env) { provider.send(load(env)); }   // no dedupe
```

Why it is wrong

- Client retry after timeout creates a second message; broker redelivery sends twice.
- Publish outside the transaction: crash after commit loses the message; publish before commit creates ghost messages.
- No `Idempotency-Key` handling; no consumer dedupe.

Corrected version

JAVA

```
@PostMapping("/messages")
public ResponseEntity<?> submit(@RequestHeader("Idempotency-Key") String key, @Valid @RequestBody
  SubmitMessageRequest req) {
    SubmitResult r = notificationService.submit(tenantContext.requireTenant(), key, req);
    return r.replay()
        ? ResponseEntity.ok().header("Idempotent-Replay", "true").body(r.body())
        : ResponseEntity.accepted().location(r.location()).body(r.body());
}
// service (one @Transactional): claim key -> persist message(VALIDATED) + outbox + audit -> complete key
// consumer
public void onMessage(DispatchEnvelope env) {
    if (!processed.tryInsert(env.dispatchId())) { ack(); return; }
    if (repo.casStatus(env.messageId(), QUEUED, PROCESSING) == 0) { ack(); return; }
    ...
}
```

Rationale

Duplicates are normal in distributed systems; every boundary needs its own dedupe (ADR-0004). Outbox removes dual write (ADR-0009).

Regression tests

- `FailureRecoveryIT#scenario02_duplicateSubmission_sameKey_returnsOriginal_noSecondMessage` (sequential + 20 concurrent).
- `DispatchConsumerTest#redeliveredEnvelope_callsProviderOnce`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- `NotificationServiceTest#sameKeyDifferentBody_returns422`.

AI review example 3: Race condition

Source document: `ai-engineering/ai-code-review/03-race-condition.md`

Invariants: 1, 3.

Flawed code

JAVA

```
// operator cancel API
MessageRequest m = repo.findByIdAndTenantId(id, tenant).orElseThrow(NotFoundException::new);
if (m.getStatus() == MessageStatus.QUEUED) {
    m.setStatus(MessageStatus.CANCELLED);
    repo.save(m);                       // meanwhile the consumer moved it to PROCESSING and called the
      provider
}
```

Why it is wrong

Check-then-act: between the read and the save the consumer can win; the message is both sent and marked CANCELLED. Even with `@Version`, a swallowed `ObjectOptimisticLockingFailureException` or a `merge` of a stale detached entity can overwrite.

Corrected version

JAVA

```
@Transactional
public MessageStatus cancel(String tenant, String id) {
    MessageRequest m = repo.findByIdAndTenantId(id, tenant).orElseThrow(NotFoundException::new);
    MessageStatus from = m.getStatus();
    stateMachine.require(from, MessageStatus.CANCELLED);          // 409 ILLEGAL_STATE_TRANSITION if not
      allowed
    if (repo.casStatus(id, from, MessageStatus.CANCELLED, clock.instant()) == 0) {
        throw new IllegalStateTransitionException(from, MessageStatus.CANCELLED); // lost race: 409, client
          re-reads
    }
    audit.transition(...); return MessageStatus.CANCELLED;
}
```

Consumer does `casStatus(QUEUED, PROCESSING)`: exactly one of cancel/consumer wins.

Rationale

The DB serializes the two `UPDATE ... WHERE status=:from`; the loser sees 0 rows and takes the safe path (consumer acks and skips; API returns 409).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Regression tests

- `CancelRaceIT#cancelVsConsume_exactlyOneWins_providerCalledAtMostOnceOrCancelled` (latch gate, 50 iterations): assert final status is CANCELLED with zero provider calls, or SENT_TO_PROVIDER with API 409, never both.
- `MessageStateMachineTest#cancelOnlyFromValidatedQueuedRetryPending`.

AI review example 4: Unsafe retry

Source document: `ai-engineering/ai-code-review/04-unsafe-retry.md`

Invariants: 5, 6, 4 (timeouts), idempotency of provider call.

Flawed code

JAVA

```
ProviderResult r;
int attempts = 0;
while (true) {                                  // retry loop inside the consumer thread
    try { r = smsProvider.send(req); break; }
    catch (Exception e) {                       // includes read timeout after the vendor accepted
        if (++attempts > 10) throw e;
        Thread.sleep(100);                      // fixed delay, holds the worker, no jitter
    }
}
```

Why it is wrong

- Retrying after a read timeout can send a duplicate SMS (the vendor may have accepted).
- Tight fixed-delay loop amplifies provider outage; blocks a worker thread (starvation); no attempt accounting, no state, lost on crash.
- Catches everything (permanent failures retried).

Corrected version

JAVA

```
ProviderResult r = invoker.send(provider, req, idempotencyRef);   // timeout + breaker, no loop
switch (r.outcome()) {
  case ACCEPTED -> transition(m, SENT_TO_PROVIDER);
  case TRANSIENT_FAILURE, RATE_LIMITED -> scheduleRetry(m, retryPolicy.nextDelay(attemptNo)); // RETRY_PENDING,
    jittered backoff
  case PERMANENT_FAILURE -> transition(m, FAILED);
  case TIMEOUT, AMBIGUOUS -> m.channel() == PUSH ? scheduleRetry(m, retryPolicy.nextDelay(attemptNo))
                                                 : transition(m, DELIVERY_UNKNOWN);   // reconcile, no blind
                                      resend
}
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Rationale

Classification decides retry safety; retry state is persisted (`next_attempt_at`) and spread by jitter; ambiguity is resolved by evidence (ADR-0006, ADR-0010).

Regression tests

- `FailureRecoveryIT#scenario04_providerTimeout_movesToDeliveryUnknown_reconcilesWithoutDuplicateSend` (provider call count == 1).
- `RetryPolicyTest#delayWithinJitterBounds_forEachAttempt`, `#capRespected`.
- `DispatchServiceTest#permanentFailure_isNotRetried`.

AI review example 5: N+1 queries

Source document: `ai-engineering/ai-code-review/05-n-plus-one.md`

Invariants: 9 (privacy in mapping), performance checklist.

Flawed code

JAVA

```
@GetMapping("/messages")
public PageResponse<MessageSummary> list(Pageable p) {
    Page<MessageRequest> page = repo.findAll(p);                        // also: no tenant filter
    return PageResponse.of(page.map(m -> new MessageSummary(
        m.getId(), m.getStatus(),
        m.getRecipient().getEmail(),                                     // lazy load per row + unmasked PII
        m.getAttempts().size())));                                       // lazy collection per row
}
```

Why it is wrong

1 + N (recipient) + N (attempts) queries per page; unfiltered `findAll` (cross-tenant!); email unmasked; sorting by non-indexed property.

Corrected version

JAVA

```
@Query("""
  select new com.example.messaging.web.dto.MessageSummary(m.id, m.status, m.channel, m.attemptCount,
         r.externalRef, m.createdAt)
  from MessageRequest m join m.recipient r
  where m.tenantId = :tenant and (:status is null or m.status = :status)
  order by m.createdAt desc""")
Page<MessageSummary> findSummaries(String tenant, MessageStatus status, Pageable p);
```

Masked address computed by `Redaction.maskEmail` only where needed (detail view); `attemptCount` is a column on the row; the query matches index `(tenant_id, status, created_at DESC)`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Rationale

Projection fetches exactly what the screen needs in one statement, uses the index, and removes both the leak and the loop.

Regression tests

- `MessageQueryIT#listPage_executesAtMostThreeStatements` (Hibernate `Statistics#getPrepareStatementCount`, page size 50, 200 rows seeded).
- `MessageControllerTest#list_neverReturnsFullAddress`, `#list_crossTenantRowsNotVisible`.
- Optional: plan review in `docs/database/query-plans-and-indexes.md`.

AI review example 6: Cross-tenant authorization

Source document: `ai-engineering/ai-code-review/06-incorrect-authorization-cross-tenant.md`

Invariants: 8 (tenant from JWT, scoped access, 404).

Flawed code

JAVA

```
@PostMapping("/messages/{id}/cancel")
@PreAuthorize("hasAnyRole('SENDER','TENANT_ADMIN','OPERATOR','PLATFORM_ADMIN')")
public MessageStatusResponse cancel(@PathVariable String id, @RequestParam(required = false) String tenantId) {
    MessageRequest m = repo.findById(id).orElseThrow(() -> new NotFoundException("NOT_FOUND"));
    String tenant = tenantId != null ? tenantId : m.getTenantId();    // tenant taken from the request
    return service.cancel(tenant, m);
}
```

Why it is wrong

- Role check only; no ownership check: tenant B can cancel tenant A's message by id (BOLA).
- `tenantId` request parameter is attacker-controlled.
- If the message belongs to another tenant, a 403 would leak existence; here `findById` has no tenant guard.

Corrected version

JAVA

```
@PostMapping("/messages/{id}/cancel")
@PreAuthorize("hasAnyRole('SENDER','TENANT_ADMIN','OPERATOR','PLATFORM_ADMIN')")
public MessageStatusResponse cancel(@PathVariable String id) {
    String tenant = tenantContext.requireTenantOrScoped();            // JWT claim; admin may scope via
      X-Tenant-Id only if tenant=*
    return service.cancel(tenant, id);                                // repo.findByIdAndTenantId ->
      NotFoundException (404) if foreign
}
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Rationale

Authorization is role AND resource ownership; foreign resources are indistinguishable from missing ones (ADR-0011).

Regression tests

- `tests/security`: tenant B token on tenant A message id for GET, status, history, cancel, requeue, diagnostics: all 404.
- `SecurityMatrixTest#everyIdEndpoint_foreignTenant_returns404` (parameterized over the controller mappings so new endpoints cannot be forgotten).
- `TenantContextTest#adminScopingRequiresWildcardTenant`.

AI review example 7: Unbounded concurrency

Source document: `ai-engineering/ai-code-review/07-unbounded-concurrency.md`

Invariants: 4 (bounded pools and queues, timeouts).

Flawed code

JAVA

```
private final ExecutorService executor = Executors.newCachedThreadPool();   // unbounded threads

public void onMessage(DispatchEnvelope env) {
    executor.submit(() -> dispatchService.process(env));    // consumer acks immediately, work is unbounded
    ack();
}
// bulk
rows.parallelStream().forEach(r -> notificationService.submitInternal(r));   // common pool, unbounded DB
  pressure
```

Why it is wrong

- Acking before processing loses messages on crash; unbounded threads under a slow provider: memory/thread exhaustion; DB pool starvation; no backpressure to the broker (prefetch is meaningless).
- `parallelStream` uses the shared ForkJoin pool and ignores rate limits and chunk transactions.

Corrected version

JAVA

```
ThreadPoolExecutor workers = new ThreadPoolExecutor(
    cfg.concurrency(), cfg.concurrency(), 0L, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(cfg.prefetch()),
    namedThreads("dispatch-worker-"),
    new ThreadPoolExecutor.AbortPolicy());                       // rejection => nack/requeue with delay, never
      drop

public void onMessage(DispatchEnvelope env, Ack ack) {
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
    try { workers.execute(() -> { process(env); ack.ack(); }); }  // ack after the outcome is persisted
    catch (RejectedExecutionException e) { ack.nackRequeueDelayed(env); metrics.rejected(); }
}
```

Bulk uses the Spring Batch chunk step with `bulk.concurrency` TaskExecutor and the writer-side `RateLimiter`.

Rationale

Bounded resources convert overload into backpressure (queue depth, 503 at API) instead of cascading failure.

Regression tests

- `DispatchConsumerTest#slowProvider_threadCountStaysBounded` (assert `Thread.getAllStackTraces` for `dispatch-worker-*` <= concurrency).
- `FailureRecoveryIT#scenario10_providerRateLimitAndBackpressure_noAttemptConsumed_apiReturns503`.
- `DispatchConsumerTest#crashBeforeAck_messageRedelivered`.

AI review example 8: Resource leak

Source document: `ai-engineering/ai-code-review/08-resource-leak.md`

Invariants: 4; performance checklist (pool, memory).

Flawed code

JAVA

```
public BulkResult importCsv(Path file) throws IOException {
    BufferedReader reader = Files.newBufferedReader(file);
    List<String> lines = reader.lines().toList();                // loads whole file; reader never closed on
      exception
    Connection c = dataSource.getConnection();                   // manual connection never closed
    for (String line : lines) { insert(c, parse(line)); }        // one transaction for everything
    return BulkResult.ok(lines.size());
}
```

Why it is wrong

- Reader and Connection leak on exceptions (pool exhaustion under repeated failures: `hikaricp_connections_active` climbs to max, API 503).
- Whole file in memory (OOM for large CSV); no chunking so a failure at row N loses all work and holds locks.
- Bypasses Spring Batch restartability.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Corrected version

JAVA

```
// Spring Batch step: FlatFileItemReader streams; chunk(500) transaction per chunk; framework manages resources
@Bean Step readValidateStep(JobRepository jr, PlatformTransactionManager tx, FlatFileItemReader<Row> reader,
                            RecipientRowProcessor p, BulkMessageWriter w) {
    return new StepBuilder("readValidateStep", jr).<Row, MessageDraft>chunk(500, tx)
        .reader(reader).processor(p).writer(w)
        .faultTolerant().skipLimit(cfg.skipLimit()).skip(InvalidRowException.class)
        .retryLimit(3).retry(TransientDataAccessException.class).build();
}
// any manual resource use:
try (var in = Files.newBufferedReader(file)) { ... }
```

Rationale

Frameworks and try-with-resources guarantee release; streaming and chunking bound memory and make restart cheap.

Regression tests

- `BulkJobIT#largeFile_streamed_heapBounded` (generate N rows with `SyntheticDatasetGenerator`, assert completion with a small heap setting or assert reader is not materialized).
- `BulkJobIT#writerFailure_releasesConnections` (Hikari active count returns to 0 via `HikariPoolMXBean`).
- `FailureRecoveryIT#scenario09_bulkJobRestart_resumesWithoutDuplicates`.

AI review example 9: Insufficient tests

Source document: `ai-engineering/ai-code-review/09-insufficient-tests.md`

Invariants: all (tests are how they are enforced).

Flawed tests

JAVA

```
@Test void retryWorks() {
    when(provider.send(any())).thenReturn(ProviderResult.transientFailure());
    dispatchService.process(envelope);
    verify(provider).send(any());                       // asserts only that a call happened
}

@Test void idempotent() {
    var r1 = service.submit(tenant, "k", req);
    var r2 = service.submit(tenant, "k", req);
    assertNotNull(r2);                                  // would pass even if two messages were created
}
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Why it is wrong

- Mocks the unit's own collaborators so the behavior (state, attempts, delay bounds, audit) is never asserted.
- `assertNotNull` proves nothing; no concurrency; no negative cases; no failure path; wall-clock dependent in places.
- A mutation (remove the unique check, change backoff to constant) would not fail these tests.

Corrected tests

JAVA

```
@Test void transientFailure_schedulesBackoffWithinJitterBounds() {
    clock.set(T0); random.seed(42);
    given(faults.mode(TRANSIENT));
    var id = submitAndDispatch();
    var m = repo.findById(id).orElseThrow();
    assertThat(m.getStatus()).isEqualTo(RETRY_PENDING);
    long delay = Duration.between(T0, m.getNextAttemptAt()).toMillis();
    assertThat(delay).isBetween(base / 2, base);                        // attempt 1: [0.5, 1.0] * base
    assertThat(attempts.findByMessageId(id)).extracting(Attempt::outcome).containsExactly(TRANSIENT_FAILURE);
    assertThat(metrics.counter("messaging.retry.scheduled", "channel", "SMS")).isEqualTo(1);
}

@Test void sameKey_twentyConcurrentRequests_createOneMessage() throws Exception {
    var results = runConcurrently(20, () -> api.post(token, "k-1", body));
    assertThat(results).filteredOn(r -> r.status() == 202).hasSize(1);
    assertThat(messages.countByTenantIdAndIdempotencyKey(TENANT, "k-1")).isEqualTo(1);
}
```

Rationale

Tests must fail when the invariant breaks. Technique: temporarily mutate the production code (remove the guard) and confirm the test fails ("test the test").

Regression guardrails

- Checklist `ai-engineering/checklists/test-generation.md` mandatory for generated tests.
- Review rule: reject tests with only `verify(...)`/`assertNotNull`/`isNotNull` on the behavior under test.
- Optional PIT mutation run on `domain` and `RetryPolicy` (not executed here).

AI review example 10: Sensitive data in logs

Source document: `ai-engineering/ai-code-review/10-sensitive-data-in-logs.md`

Invariants: 9 (no body/address/secret in logs, API, bus, Mongo), 11.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Flawed code

JAVA

```
log.info("Sending SMS to {} body={} provider request={}", recipient.getPhoneE164(), rendered.body(),
  providerRequest);
...
catch (SignatureException e) {
    log.warn("Invalid callback: body={} signature={} secret={}", rawBody, signatureHeader, secretFor(keyId));
}
```

Why it is wrong

Phone numbers, message content (may contain account details), signatures (replayable within tolerance) and an HMAC secret are logged: logs are widely accessible, retained, and shipped to third parties. Violates data minimization and secrets rules.

Corrected version

JAVA

```
log.info("Dispatching messageId={} channel={} attemptNo={} provider={}", m.getId(), m.getChannel(), attemptNo,
  providerId);
...
catch (InvalidSignatureException e) {
    metrics.callbacks("invalid_signature");
    log.warn("Callback rejected providerId={} keyId={} reason={}", providerId, keyId, e.reason());   // reason
      is an enum, never the secret/body
}
// when structured detail is genuinely needed: diagnostics.store(redaction.redact(providerRequest)) -> Mongo
  with TTL
```

MDC supplies `correlationId`, `tenant`, `messageId`. Masked addresses (`+1555***0101`) only where an address is useful.

Rationale

Logs carry identifiers and decisions; content goes nowhere by default; operators needing payload detail use the access-controlled, expiring diagnostics store with redaction.

Regression tests

- `LoggingPrivacyIT#sendAndFail_logsContainNoPhoneEmailBodyOrSecret`: attach a Logback `ListAppender`, run submit + dispatch + forged callback, assert no line matches `\+1555\d{4,}`, `@example\.test`, the rendered body text, `local-dev-secret-not-for-production`, or `v1=` signature values.
- `RedactionTest#masksKnownKeys` (`secret|password|token|authorization|api[-_]?key|signature`).
- Review checklist item in `checklists/security-review.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Post-incident review template

Source document: `docs/operations/post-incident-review-template.md`

Copy to `docs/operations/pir/YYYY-MM-DD-short-title.md` (create the folder when needed). Blameless: describe system and process conditions, not individuals.

1. Summary

- Title, date, severity (SEV1-4), duration, author, reviewers.
- One paragraph: what happened, who/what was affected, how it ended.

2. Impact

Dimension	Value
Messages affected (by status, channel, priority, tenant)	
Delayed by (p50/p95/max)	
Lost / duplicated / unknown-state messages	
SLO error budget consumed	
Customer or business impact (qualitative)	

3. Timeline (UTC)

Time	Event	Source (alert, log, metric, human)
	first symptom	
	detection	
	response started	
	mitigation applied	
	resolved	

Time to detect / acknowledge / mitigate / resolve.

4. Detection and response assessment

Did alerts fire? Right threshold? Was a runbook used? What was missing from the runbook?

5. Root cause and contributing factors

Use "5 whys" and list: trigger, latent conditions, missing safeguards. Reference ADRs/invariants violated or confirmed (e.g., ADR-0010 held / not held).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Data consistency review (platform-specific)

- Any messages in `DELIVERY_UNKNOWN` created? Resolved how?
- Any duplicates sent (compare attempts with same `idempotencyRef`)?
- Any accepted-but-never-queued messages (outbox NEW age)?
- DLQ contents and replay decisions (audit event ids).

7. What went well / what went poorly / where we got lucky

8. Action items

#	Action	Type (prevent/detect/mitigate)	Owner	Due	Tracking link

9. Lessons for AI-assisted development (if relevant)

Did generated code or a skipped checklist contribute? Which guardrail (checklist, test, ADR) should change?

10. Appendices

Graphs (names of dashboards/panels), log queries, config diffs, correlation ids.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

26. Limitations and Future Enhancements

This chapter is deliberately blunt. A reference implementation earns trust by being exact about its boundaries.

26.1 What is simulated

Real-world element	What stands in for it here	Consequence
Email, SMS and push vendors	`SimulatedEmailProvider`, `SimulatedSmsProvider`, `SimulatedPushProvider`, `AcmeSmsProvider` (all extend `SimulatedProviderBase`)	Vendor behaviour is a model: a compliant vendor that deduplicates on the idempotency reference and offers `lookup`. Real vendors may not, which is exactly the residual risk named in ADR-0010
Vendor webhooks	`CallbackEmitter` posts signed callbacks to the platform itself (`POST /dev/simulator/callbacks/emit`)	Realistic in format and signature, not in timing or failure distribution
Provider faults	`FaultInjector` modes with probabilities	Deterministic and random faults, not real network behaviour
Identity provider	`POST /dev/token` mints RS256 JWTs from an in-memory key (profiles `local`, `test` only)	No real IdP, no token revocation, no key rotation exercise
RabbitMQ	`InMemoryMessageBus` (bounded priority queue) in `local` and `test`	Same semantics, no network partitions, no persistence, no real Rabbit priority behaviour
Redis	In-memory coordination	The Lua token bucket and Lettuce adapter exist but did not run against a real Redis
MongoDB	`de.bwaldvogel:mongo-java-server` in-memory	Diagnostics store logic is tested; real Mongo operational behaviour is not
PostgreSQL	H2 2.x in `MODE=PostgreSQL`	Portable SQL runs; query plans, lock behaviour, vacuum, replicas and failover are not exercised
Customers and addresses	Synthetic: `recipient001@example.test`, `+1555010xxxx`, `SYN-0000001`	No real data anywhere

26.2 What was not executed

The honest list, repeated from the evidence legend. Every item below exists as a file in the repository and was reviewed or parsed, **not run**.

Asset	What was done	What was not
Docker images and Compose stack	Dockerfiles and compose parsed by `scripts/validate-infra.sh`	`docker build`, `docker compose config`, `docker compose up`
Kubernetes manifests and Helm chart	YAML parsed and structurally checked	`kubectl apply`, `kubeconform`, `helm lint`, `helm install`, any cluster
Cloud Foundry manifest and scripts	`validate-manifest.mjs`, `bash -n` on scripts	`cf push`, bindings, routes, rolling deploy, blue-green

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Asset	What was done	What was not
CI pipelines (GitHub Actions, Jenkins, Bamboo)	YAML parsed locally; Jenkinsfile not linted	Any pipeline run, Trivy, SBOM, OWASP dependency-check, npm audit in CI
k6 scripts (8)	Authored	Execution (k6 not installed); no k6 number exists
Prometheus rules and Grafana dashboards	Parsed for syntax and structure; names cross-checked against the meter catalogue	`promtool` evaluation; dashboards never rendered; nothing scraped from a running backend by these files
RabbitMQ adapter	Pure parts and topology versus `definitions.json`	`RabbitMessageBusContainerIT` (needs Docker), any broker run
Testcontainers tests (`-Pcontainers`)	Compile	Run
Real PostgreSQL, Redis, MongoDB	Not available	Any run
Oracle reference DDL	Written (`backend/src/main/resources/db/oracle/`)	Executed on Oracle; V200 additions were not mirrored into it
Angular console in a real browser against a real deployment	Unit tests and build per the validation report; `tests/end-to-end/ui-smoke.md` is a manual script	Automated browser end-to-end

What **was** executed on the authoring machine is recorded with exact commands in `docs/validation/`. As of this build, the executed end-to-end HTTP run (`docs/validation/e2e-run-output.txt`, reproduced in Appendix E) shows **254 checks passed, 0 failed, 0 skipped in 267.9 s** against the packaged jar on profile `local` (H2, in-memory bus, in-memory Mongo, simulators, Windows 11, JDK 21); the consolidated `docs/validation/VALIDATION_REPORT.md` is generated at the end of the project from real outputs. If a report is absent or silent on an item, treat the item as not executed. The end-to-end run, like every `local` run, says nothing about PostgreSQL, RabbitMQ, Redis or MongoDB behaviour.

26.3 Known gaps and technical debt (recorded in the repository)

Area	Gap	Source
Frontend	The history page's `HistoryEntry` model expects a flat shape; the API returns `TimelineEntry` with an object `detail`. The mapping change was recorded as still needed	DEVIATIONS, OpenAPI and frontend reconciliation
Frontend	`GET /api/v1/me` is not called (roles and tenant are read from the dev JWT for UX gating); diagnostics are not surfaced in the UI because raw payloads could contain content	DEVIATIONS
Retention	The `replay_guard` table is not purged by the callback module; the retention job deletes rows older than `replay-window`	DEVIATIONS item 8
Bulk	Jobs run on the node that accepted the request (single-node assumption for stop and orphan recovery); rows rejected inside a rolled-back chunk are not individually retained; duplicate-reference winner is non-deterministic with several workers	DEVIATIONS, bulk section

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Area	Gap	Source
Schema	`V200` (row ledger, legal hold) is not in the Oracle reference DDL	DEVIATIONS
Specs	`callback-flow.md` and `specs/0002-new-provider-adapter.md` still describe the earlier plan; `callbacks-normalization.md` is current	DEVIATIONS item 7
Rabbit	Only the total ready count is available for `/ops/queues` (reported under NORMAL); delays are rounded to TTL tiers	DEVIATIONS item 10
Cloud Foundry	The backend POM must include java-cfenv for automatic `VCAP_SERVICES` binding in the `cloud` profile	`docs/operations/cloud-foundry-guide.md`
CI	Profiles `static-analysis`, `security-scan`, `containers` must exist in the POM; no Checkstyle or SpotBugs configuration files; third-party actions tag-pinned, not SHA-pinned; jar name must match `finalName`	`docs/operations/ci-cd.md`
Observability	Alert thresholds and SLOs are illustrative; whether `messaging.bulk.progress` is a fraction or a count must be confirmed against `BulkJobStalled`	`docs/operations/observability.md`
Redis outage	Across several instances the token bucket may over-admit slightly while falling back to per-instance in-memory buckets	Reliability scenario 7
Ordering	No global or per-recipient ordering guarantee; callbacks may be out of order	Delivery semantics
Duplicates	Still possible: provider ignores the idempotency reference; operator replays a delivered message; client reuses a different key; key expiry then reuse	Delivery semantics

26.4 Not production-ready: what production would add

`docs/security/prod-vs-reference.md` is the authoritative list. In summary: a real identity provider and token lifecycle, row-level security or schema-per-tenant, mutual TLS between services, a WAF and edge rate limiting, a secret manager with rotation, hardened and scanned images with signed provenance, backup and disaster-recovery design that has been exercised, penetration testing, a compliance review of the consent policy, and real vendor contracts with their quotas, bounce semantics and webhook specifications.

26.5 Future enhancements

Enhancement	What it adds	Where it hooks in
Priority aging	Starvation protection for LOW traffic	`specs/0001-priority-aging.md`; the bus and `ScheduledDispatchPoller`
Kafka event stream sink	Analytics and replay of the event history without changing the transactional path	CDC or outbox consumer beside RabbitMQ (ADR-0001, HLD section 9)
CDC-based outbox	Removes the polling interval	Debezium reading `outbox_event`
Split consumers from API	Independent scaling and failure domains	Profiles controlling consumers and schedulers; separate Kubernetes Deployments

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Enhancement	What it adds	Where it hooks in
Autoscaling on queue depth	Scale on what actually saturates	KEDA or a custom metrics adapter
Row-level security	Defence in depth for tenant isolation	PostgreSQL RLS plus the tenant claim
Per-tenant quotas and fair queuing	Prevent one tenant from crowding others	`RateLimitFilter`, bus topology
Multi-region active/passive	Disaster recovery	Replicated database, regional brokers; idempotency keys already make client retry across regions safe
Real provider adapters	Production delivery	New `ChannelProvider` beans following the worked Acme example
Template canary rollout	Reduce blast radius of a bad template	Percentage-based version selection at pin time
Delivery analytics	Open and click funnels, deliverability dashboards	Engagement events already stored in `delivery_event`
Chaos and game days	Prove runbooks against real incidents	`tests/failure-recovery/`, runbooks

26.6 How to keep this document honest

When anything changes: update `DEVIATIONS.md` first, regenerate the validation report from real outputs, rebuild this PDF with `npm run pdf1` from `tools/pdf`, and re-run `node verify-pdf.mjs`. The rule from the build contract applies to every document: **never claim something ran unless it ran**.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix A. Traceability

The repository's traceability matrix is `docs/traceability/job-description-traceability-matrix.md` (with the machine-readable copy `job-description-traceability-matrix.csv`): for each stated job requirement it lists the implemented feature, the source location, the test or validation evidence, an interview talking point and the remaining limitation. See `docs/validation/` for which checks were actually executed and their results. This appendix is the chapter-level map inside this PDF: for each chapter, the primary code and the tests that prove it.

Chapter	Primary code	Proving tests
5 Submission	`web/MessageController`, `service/NotificationService`, `service/IdempotencyService`	`NotificationApiIT`, `IdempotencyServiceTest`, `BackpressureIT`, `OpenApiContractIT`, `FailureRecoveryIT#scenario02`
6 Lifecycle	`domain/MessageStateMachine`, `domain/MessageStatus`	`MessageStateMachineTest`, `CancelRaceIT`, `ConcurrencyIT`, `MessageLifecycleIT`
7 Dispatch	`scheduler/OutboxPublisher`, `messaging/DispatchConsumer`, `messaging/InMemoryMessageBus`, `messaging/RabbitMessageBus`	`DispatchFlowIT`, `IdempotentConsumerIT`, `InMemoryMessageBusTest`, `GracefulShutdownTest`, `RabbitTopologyTest`
8 Providers	`provider/ChannelProvider`, `ProviderRegistry`, `ResilientProviderInvoker`, `provider/simulator/*`	`ProviderRegistryTest`, `ResilientProviderInvokerTest`, `SimulatedProviderTest`, `FaultInjectorTest`, `NewProviderWithoutOrchestrationChangeTest`, `ProviderOutcomeIT`
9 Channels	`template/TemplateRenderer`, `callback/*PayloadMapper`	`TemplateRendererTest`, `ChannelDispatchIT`, `EventNormalizerTest`
10 Templates	`service/TemplateService`, `web/TemplateController`	`TemplateRendererTest`, `TemplateLifecycleIT`
11 Preferences	`service/PolicyService`, `service/PreferenceService`, `service/SystemSuppressionService`	`PolicyServiceTest`, `PreferenceApiIT`, `TenantIsolationIT`
12 Callbacks, reconciliation	`callback/*`, `scheduler/StuckMessageSweeper`, `service/ReconciliationService`	`SignatureVerifierTest`, `ReplayGuardTest`, `EventNormalizerTest`, `CallbackApiIT`, `ReconciliationIT`, `StuckMessageSweeperTest`
13 Retry, DLQ	`service/RetryPolicy`, `service/ReplayService`, `service/DefaultDeadLetterOperations`	`RetryPolicyTest`, `ReplayServiceIT`, `FailureRecoveryIT#scenario01`, `#scenario08`
14 Bulk	`batch/*`, `web/BulkJobController`	`BulkJobBatchIT`, `BulkJobApiIT`, `BulkJobRestartConcurrentIT`, `FailureRecoveryBulkIT`, `CsvRulesTest`, `RowProcessorTest`
15 Database	`backend/src/main/resources/db/migration/*`, `repository/*`, `scheduler/RetentionJob`	`RepositoryQueryCountIT`, `RetentionJobTest`, `OpenApiContractIT`
16 Security, audit	`config/SecurityConfig`, `security/*`, `common/Redaction`, `service/AuditService`	`SecurityIT`, `SecurityMatrixTest`, `TenantIsolationIT`, `JwtAuthConverterTest`, `DevTokenDisabledIT`, `LoggingPrivacyIT`, `RedactionTest`, `RateLimitIT`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Chapter	Primary code	Proving tests
17 to 19 Deployment, CI/CD	`cloud-foundry/*`, `infrastructure/*`, `.github/workflows/*`, `Jenkinsfile`, `bamboo-specs/*`	`scripts/validate-infra.sh`, `cloud-foundry/validate-manifest.mjs` (static checks only; deployments NOT executed)
20 Observability, performance	`metrics/MessagingMetrics`, `observability/*`, `performance/*`	`scripts/lib/static-checks.mjs` (static); k6 NOT executed
22 Failure scenarios	`pipeline/FailureRecoveryIT`, `batch/FailureRecoveryBulkIT`, `tests/failure-recovery/*.mjs`	Ten named scenario methods (Chapter 22)

Acceptance criteria ids (`AC-1` and so on, Chapter 2) are intended to appear in test display names or comments; the matrix in `docs/traceability/` and the test sources are the places to look up a specific id.

Test classes are in `backend/src/test/java/com/example/messaging/` under the packages `batch`, `callback`, `common`, `coordination`, `diagnostics`, `domain`, `messaging`, `pipeline`, `provider`, `retention`, `scheduler`, `security`, `service`, `template` and `web`. Frontend tests are the `*.spec.ts` files in `frontend/messaging-console/src/app/`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix B. Implemented Deviations from the Contract

The build contract (`docs/design/CONTRACT.md`) was the single source of truth during construction. Wherever the implementation had to differ, the difference was appended to `docs/design/DEVIATIONS.md` instead of silently changing shared decisions. The file is **append only** and is the authoritative record of implemented behaviour. It is reproduced here exactly as written in the repository, including its terse style, because the main chapters of this PDF rely on it.

Deviations record (docs/design/DEVIATIONS.md)

Source document: `docs/design/DEVIATIONS.md`

Frontend (messaging-console) deviations / assumptions

- Payloads for endpoints CONTRACT section 5 lists but does not detail (ops summary/queues, providers, bulk jobs, template detail/versions, audit/delivery events, history entries, dead letters, failed messages, preferences, template preview) use the shapes in `frontend/messaging-console/src/app/core/models/api.models.ts`; reconcile with the OpenAPI spec when the backend lands.
- `POST /dev/token` is called with roles WITHOUT the ROLE_ prefix (matches backend `Roles` constants), the sender persona requests scope `messages:bulk`, and the UI accepts both `access_token/expires_in` and Jackson-default `accessToken/expiresIn` response names; it also accepts ROLE_-prefixed roles when decoding the JWT.
- `GET /api/v1/me` is not called (roles/tenant are read from the dev JWT for UX gating only); `GET /api/v1/diagnostics/messages/{id}` is not surfaced in the UI because raw provider payloads could contain content.
- Fonts/icons are not loaded from a CDN (system font stack, text-only controls) so builds work offline.
- `frontend/mock-api/server.mjs` is an opt-in UI-development stub, not an application backend.

Backend foundation (stage 1) deviations / additions

- Additive error code `DUPLICATE_RESOURCE` (HTTP 409) for unique-constraint style conflicts (duplicate template code+channel, duplicate template locale version, data-integrity fallback). CONTRACT 5 lists no code for these; `CONFLICT_OPTIMISTIC_LOCK` would be misleading. It is part of the committed OpenAPI `ErrorCode` enum.
- Policy rejections at submission (opt-out, suppression, deleted recipient, no address, expiry before a deferred send) are persisted as `REJECTED` messages (ACCEPTED -> REJECTED, audited) and answered `202` with `status=REJECTED` and `statusReason`, rather than a 4xx. This keeps the idempotent replay and audit trail consistent. `POLICY_SUPPRESSED` is therefore not returned by stage 1; stage 2 may use it for dispatch-time rejections surfaced via other paths.
- `ProviderOutcome` lives in package `provider` (as requested for stage 1), not `domain` as CONTRACT 3.2 groups it; `MessageAttempt` imports it from there.
- New package `coordination` (CoordinationService, InMemoryCoordination, RedisCoordination, ResilientCoordination, RateLimitDecision) for the Redis ports required by CONTRACT 1; CONTRACT 2 does not list it.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Additional message_request columns beyond the CONTRACT 4 list (CONTRACT says "..."): `client_reference`, `options_json`, `metadata_json`, `fallback_channels`, `fallback_index`, `locked_until`, `dlq_reason`, `discarded`, `status_changed_at`. Additional columns: `provider_config.version`, `message_attempt.state` (STARTED/FINISHED), `outbox_event.available_at` (lets scheduled/deferred messages sit in the outbox until due; the publisher selects `status=NEW AND available_at <= now`), `bulk_job.template_code/channel/category`.
- Requeue/replay is implemented as: CAS status FAILED|DEAD_LETTERED -> QUEUED plus an outbox event (attemptNo = attemptCount + 1, available immediately). The outbox publisher must therefore publish regardless of whether the message is already QUEUED, and must only CAS VALIDATED -> QUEUED (never fail when it is already QUEUED). Cancelled messages are skipped by the publisher (status != VALIDATED/QUEUED).
- A template version's `created_by` is overwritten with the editor on edit (author of record), so four-eyes approval cannot be bypassed by having a second person edit a draft.
- `application-*.yml` has no default profile on purpose (secure by default: a forgotten profile must not enable the dev token issuer). Run with `--spring.profiles.active=local`.
- The committed OpenAPI contract test treats the stage-2 prefixes (`/api/v1/providers`, `/ops`, `/bulk-jobs`, `/callbacks`, `/diagnostics`) and `/dev/**` as "documented, not yet served"; stage 2 must delete `STAGE_2_PREFIXES` from `OpenApiContractIT` so that the full committed spec is enforced.

Backend stage 2 (messaging core) deviations / additions

Append-only record for the dispatch pipeline, callbacks, providers and ops APIs. Spring Batch (`batch/`, bulk-jobs controller, retention job) is built separately and is not covered here.

Interface and shared-code changes other code must know about

- `DispatchHandler` gained a default method `onPoison(envelope, deliveryCount, cause)`: the bus calls it after it dead-lettered an envelope on its own (poison threshold or unreadable body) so `DispatchConsumer` can mirror that in the database. Existing lambdas keep compiling.
- `MessagingMetrics`: new gauge `messaging.queue.oldest.age` (seconds, bound in `bindQueueStats`), `publishPercentiles(0.5, 0.95, 0.99)` on `messaging.provider.latency` and `messaging.dispatch.delay` (used for the p95 values of `/providers` and `/ops/summary`), and strong references to the suppliers handed to `bindQueueStats` / `bindCircuitState` (Micrometer holds gauge state weakly; without this the gauges turned into NaN after a GC).
- `NotificationService.requeueInternal` (operator requeue and replay) now sets `max_attempts = attempt_count + messaging.dispatch.max-attempts`: a replayed message gets a fresh attempt budget, otherwise it would be exhausted again by its first failure.
- `MessageDtos.MessageSummary` gained `channelUsed`, `templateCode`, `nextAttemptAt` (additive; the Angular console already reads them).
- Repositories gained queries: `MessageRequestRepository.markClaimed/setNextAttemptAt/applyFallback/setMaxAttempts/countByChannelSince*`, `MessageAttemptRepository.findFirstBy...`, `OutboxEventRepository.countByStatusAndAvailableAtLessThanEqual/existsByAggregateIdAndStatus`, `AuditEventRepository.countBy.../findFirstBy...` (reconciliation bookkeeping). New migration `V50__dispatch_indexes.sql` adds the indexes these queries need (also appended to `db/oracle/oracle_core_schema.sql`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- `GlobalExceptionHandler`: database connection failures (`DataAccessResourceFailureException`, `CannotCreateTransactionException`, `TransientDataAccessException`, JDBC connection exceptions in the cause chain) answer `503 SERVICE_UNAVAILABLE` + `Retry-After: 5` instead of 500 (fail closed, clients retry with the same idempotency key). `MessageController.submit` calls `BackpressureGuard.check()` first (503 + `Retry-After` when bus depth + due outbox rows reach `messaging.backpressure.max-queue-depth`).
- Configuration: the new keys live in two extra records instead of `MessagingProperties` (to avoid touching a file the batch module also edits): `messaging.pipeline.*` (`PipelineProperties`: auto-start, shutdown-timeout, requeue delays, replay-window, `scheduler.*`, `resilience.*`) and `messaging.simulator.*` (`SimulatorProperties`). `application-test.yml` sets `messaging.pipeline.auto-start=false` and `scheduler.enabled=false` so stage-1 tests that drive the lifecycle by hand are not disturbed; `AbstractPipelineIT` switches both on. `application.yml` defines the profile group `prod-like -> rabbit` (the Rabbit bus and topology are `@Profile("rabbit")`). Percentile histograms are also enabled for `messaging.dispatch.delay` and `messaging.provider.latency`.
- `backend/pom.xml`: `**/*ContainerIT.java` is excluded from test compilation in the default build (the Testcontainers dependencies exist only in profile `containers`); the profile re-includes it. `scripts/it.sh` is a fast loop for single failsafe tests.
- `ProviderAdminService` lives in package `provider` (not `service`) because it knows the simulator classes; `service`, `messaging`, `scheduler`, `callback` and `domain` never name a concrete provider (checked by `NewProviderWithoutOrchestrationChangeTest`).

Behaviour that differs from CONTRACT section 6 / the flow documents (and why)

1. Consumer order: cancel/status, expiry, policy, provider selection, circuit state and the provider token bucket are checked **before** the claim; the claim is then ONE transaction (unique `processed_message` row + CAS QUEUED to PROCESSING + `locked_until` + `attempt_count` + attempt row `STARTED`). Reason: a requeue because of the rate limiter reuses the same `dispatchId` and must not be blocked by the dedupe table, and must not consume an attempt. Expiry and dispatch-time REJECTED claim first and then move PROCESSING to FAILED(`EXPIRED`) / REJECTED because the state table has no QUEUED to REJECTED/FAILED.
2. Quiet hours met at dispatch time defer through the outbox (new outbox row with `available_at` = end of the window, message stays QUEUED); there is no QUEUED to VALIDATED transition.
3. The outbox publisher claims the row (`markSent` CAS), moves VALIDATED to QUEUED and publishes inside one transaction: a failed publish (backpressure, broker down) rolls everything back, giving at-least-once delivery. An envelope can overtake the commit, so the consumer requeues a message that is still VALIDATED for 150 ms (at most 40 deliveries, then dead-letter).
4. Multi-step outcomes use consecutive, individually legal transitions in one transaction: fallback after PERMANENT_FAILURE is PROCESSING to FAILED to QUEUED; fallback or dead letter after exhausted retries is PROCESSING to RETRY_PENDING to QUEUED / DEAD_LETTERED; reconciliation "not received" is DELIVERY_UNKNOWN to FAILED to QUEUED; mirroring a poison message is QUEUED to PROCESSING to FAILED to DEAD_LETTERED. The state table (and `MessageStateMachineTest`) are unchanged. Fallback gives the new channel a fresh attempt budget and uses a new idempotency reference group (`messageId:2`, ...).
5. The circuit-breaker permit is **reserved before the claim** (`ResilientProviderInvoker.acquire`, released again if the message is not claimed, rate-limited or fails to render): an open circuit or exhausted half-open probes requeue the delivery (0.1 to 5 s, no attempt consumed, as in ADR-0006), so a recovering provider never burns the attempt budget of messages that merely met a refusing breaker. `invoke(provider, request)` without a reservation still returns TRANSIENT `CIRCUIT_OPEN` without touching the provider (used by callers that do not reserve). The breaker leaves OPEN by itself (`automaticTransitionFromOpenToHalfOpenEnabled`), otherwise the

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

state pre-check would keep it open forever.

6. PUSH timeouts/ambiguous answers are retried like transient failures (the provider deduplicates on the idempotency reference); EMAIL/SMS go to DELIVERY_UNKNOWN. The provider token bucket uses `burst = rate`.
7. Callbacks: a replayed `X-Event-Id` is answered `200 {"result":"replayed"}` (not 409 `REPLAYED_EVENT`, which stays unused); an unknown provider, a key not bound to the provider (`provider_config.hmac_key_ref`, or `<ref>-next`) and every signature problem give the same `401 INVALID_SIGNATURE`; an event for an unknown message is `202 unknown_message` and is neither stored (`delivery_event.message_id` is a mandatory foreign key) nor remembered by the replay guard; `delivery_event` has a unique `(provider_id, provider_event_id)`, so a duplicate creates no second row (the callback-flow text about recording a DUPLICATE row cannot be done with that constraint); extra result tags `conflict`, `unknown_message`, `malformed`. `callback-flow.md` and `specs/0002-new-provider-adapter.md` still describe the earlier plan; `callbacks-normalization.md` is current. Acme's payload uses `id` for the event id (spec 0002 says `eid`) and the key id is `acme-sms-key` (seed `hmac_key_ref`; spec says `acme-key-1`).
8. Free-text provider reasons are never copied into audit or log lines (only code-like reasons); raw payloads go to Mongo redacted. The `replay_guard` table is not purged by this module: the retention job should delete rows older than `messaging.pipeline.replay-window`.
9. Reconciliation bookkeeping needs no new column: spacing and the `reconcile-max-checks` bound are derived from `RECONCILIATION_CHECK` audit events.
10. RabbitMQ: only the total ready count of the dispatch queue is available, so `GET /ops/queues` reports it under `NORMAL` (other priorities 0) when the Rabbit bus is active; delays are rounded up to the next TTL tier (5 s, 30 s, 2 m, 10 m) and delays above 10 m are finished with an `x-not-before` header. Rabbit code is NOT executed here (no broker): only `RabbitBusSupportTest` / `RabbitTopologyTest` (pure parts, topology vs `definitions.json`) run; `RabbitMessageBusContainerIT` compiles under `-Pcontainers` but was not run.
11. Simulators: PERMANENT mode answers `SIM_REJECTED` (no suppression); the address pattern `...+hardbounce@...` simulates a hard bounce (`HARD_BOUNCE`, adds a suppression). The simulator remembers the last request per idempotency reference in memory for tests (`lastRequest`). `POST /dev/simulator/callbacks/emit` answers 202 with `{"scheduled":[...]}`.

OpenAPI / frontend reconciliation

- Backend serves the committed OpenAPI. Additive fields keep the Angular console working unchanged: `ProviderHealth.rateLimitPerSec` (= `ratePerSec`) and `faultProbability`; `ProviderConfigRequest` accepts `rateLimitPerSec` as an alias of `ratePerSec`; `QueueStats.oldestMessageAgeSeconds` and `consumersPaused`; `OpsSummary.total`; `MessageSummary.channelUsed/templateCode/nextAttemptAt`. The spec was extended for the callback responses (200 replay, 400, `result` values), the emitter response and descriptions; `docs/api/openapi.yaml` is the identical copy.
- **Frontend change still needed** (not edited here): `HistoryEntry` in `api.models.ts` expects `{at, kind, title, detail: string, status, actor}` but `GET /messages/{id}/history` returns the specified `TimelineEntry {at, kind, type, summary, detail: object}` (the actor is `detail.actor`, the target status `detail.to`). Map `summary` to the title, render `detail` as an object and drop `status/actor`.
- `OpenApiContractIT` no longer exempts providers, ops, callbacks and diagnostics; `/api/v1/bulk-jobs` stays exempt only because the bulk controller lives in the separately built batch module and is absent here.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Test-support notes

- Pipeline tests (`AbstractPipelineIT`) run the real asynchronous pipeline with simulators, an adjustable clock (system time plus offset), a programmable `DispatchHook`, a failing-`DataSource` wrapper and a switchable coordination backend; every case retires leftovers of earlier cases, uses fresh recipients and zeroes the simulators. Raw JDBC writes into TIMESTAMP columns must use UTC `LocalDateTime` (see `utc(...)`) and TIME columns must be written through JPA, because Hibernate runs with `hibernate.jdbc.time_zone=UTC` while the JVM zone is not UTC.

Bulk jobs and retention (batch module) deviations / additions

- Extra configuration, all with safe defaults (no change to the shared `application*.yml` needed): `messaging.bulk.{max-upload-bytes,max-line-length,max-columns,default-rate-per-second,max-rate-per-second,retry-limit,max-synthetic-rows,max-parallel-jobs,launch-queue-capacity,stale-after,sweep-interval,stop-wait,sweeper-enabled,delete-file-on-completion}` (record `batch/BulkProperties`) and `messaging.retention.{enabled,dry-run,batch-size,pause-ms,max-batches-per-table,processed-message-days,outbox-sent-days,replay-guard-hours,purge-bulk-files,cron}` (record `scheduler/RetentionProperties`). The core keys of CONTRACT 9 stay in `MessagingProperties`.
- Migration `V200__bulk_ledger_legal_hold.sql`: `bulk_job_row` (the `(job_id, external_ref)` ledger of CONTRACT 7, plus the line number), a unique index on `bulk_job_failure(job_id, line_no)`, a `legal_hold` table honoured by `RetentionJob` (scope TENANT or MESSAGE, active while `released_at IS NULL`) and age indexes for the purge queries. The Oracle reference DDL was not extended.
- OpenAPI additions (additive, not yet in `messaging-api-v1.yaml`): optional query parameter `createRecipients` (default true) on `POST /bulk-jobs/generate-synthetic`, optional `sort` on the two list endpoints, `Location` header on the 202 responses, and `bulk_job_failure.line_no = 0` meaning "job level failure" (e.g. `SKIP_LIMIT_EXCEEDED`, `STEP_FAILED: <exception type>`).
- Authorization: create, generate-synthetic, stop and restart need role SENDER, TENANT_ADMIN or PLATFORM_ADMIN **and** scope `messages:bulk` for every caller, PLATFORM_ADMIN included; a refusal is audited as `BULK_JOB_DENIED` and answered 403. Reads (list/get/failures) need one of SENDER, TENANT_ADMIN, PLATFORM_ADMIN, OPERATOR, AUDITOR and are tenant scoped (404 across tenants).
- Counting: `processed = succeeded + failed + duplicate`. Rows rejected by policy at submission (opt-out, suppression, ...) are persisted as `REJECTED` messages but counted as `failed` with reason `POLICY_REJECTED: <reason>`. Duplicates are filtered (not skipped), recorded in `bulk_job_failure` as `DUPLICATE_EXTERNAL_REF: first seen on line N`, and do not count against `skip-limit`. Status is `COMPLETED_WITH_ERRORS` iff `failed > 0`.
- Restart: with `messaging.bulk.concurrency = 1` the reader checkpoints per chunk and a restart resumes from the last committed chunk; with more workers no exact position exists, so the restart re-reads the file from the top and the ledger plus the idempotency key `bulk-<jobId>-<externalRef>` make already processed rows no-ops. With several workers which occurrence of a duplicated reference wins is not deterministic (exactly one is sent either way).
- Details of rows rejected inside a chunk that is rolled back (job failed) are not kept; the job-level failure row explains why it failed. A rolled back chunk leaves no counter and no message, by design.
- `generate-synthetic` creates the recipients it needs (`SYN-0000001...`, fictitious `example.test` addresses) for the caller's tenant and generates exactly the variable columns of the chosen template; `createRecipients=false` skips that and every row then fails with `RECIPIENT_NOT_FOUND`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Upload files live in `<messaging.bulk.upload-dir>/<tenant>/<jobId>.csv` (server generated name, normalised to UTF-8/LF, BOM and trailing blank lines removed), are deleted when the job completes (`delete-file-on-completion`) and otherwise by `RetentionJob` after `message-days`. Jobs run on the node that accepted the request (single-node assumption for stop/orphan recovery): `stop` on a RUNNING job whose batch execution is not alive in this JVM closes the orphaned execution as STOPPED so it can be restarted; the sweeper marks jobs without progress for `stale-after` as ABANDONED (terminal for the API).
- Extra meters (additive): `messaging.bulk.writers.active`, `messaging.retention.deleted{table}`, `messaging.retention.run{dryRun}`. Extra audit actions: `BULK_JOB_DENIED`, `BULK_JOB_ABANDONED`, `RETENTION_PURGE`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix C. Glossary

One-line definitions in alphabetical order. Project terms follow `docs/design/CONTRACT.md`; other terms are general industry definitions. The glossary is reproduced from `docs/interview/glossary.md`.

Glossary

Source document: `docs/interview/glossary.md`

One-line definitions, alphabetical. Project terms follow `docs/design/CONTRACT.md`. Other terms are general industry definitions. This is study material for a reference implementation, not a description of any employer's systems.

- **ACID** - Atomicity, Consistency, Isolation, Durability: the guarantees of a database transaction.
- **Ack (acknowledgment)** - A consumer's signal to the broker that a message was processed and can be removed from the queue.
- **ADR (Architecture Decision Record)** - A short document recording one design decision, its context, and its consequences.
- **Aggregate** - A cluster of related entities treated as one unit for consistency, with a single root.
- **API gateway** - A front door that handles routing, authentication, rate limiting, and aggregation for backend APIs.
- **At-least-once delivery** - A guarantee that a message is delivered one or more times, so consumers must tolerate duplicates.
- **At-most-once delivery** - A guarantee that a message is delivered zero or one times, so loss is possible.
- **Audit event** - An append-only record of who did what to which entity and when.
- **Backoff** - Waiting longer between successive retries to give a failing dependency time to recover.
- **Backpressure** - Slowing or rejecting producers when consumers or queues cannot keep up (the platform returns 503 with `Retry-After`).
- **Bearer token** - A credential sent in the `Authorization` header that grants access to whoever holds it.
- **Blue-green deployment** - Running two identical environments and switching traffic from the old to the new in one step.
- **Bounce** - A provider report that a message could not be delivered, hard (permanent) or soft (temporary).
- **Buildpack** - A tool that turns application source into a runnable image or droplet without a Dockerfile.
- **Bulk job** - A batch send of one template to many recipients from a file, run with Spring Batch.
- **Bulkhead** - Isolating resources (thread pools, queues) so one failing part cannot sink the rest.
- **Callback (webhook)** - An HTTP request a provider sends to report delivery events.
- **Canary release** - Sending a small share of traffic to a new version before full rollout.
- **CAS (compare-and-set)** - An update that succeeds only if the current value matches what you expect, such as `UPDATE ... WHERE status=...`.
- **Category** - A message class: TRANSACTIONAL, SECURITY_ALERT, SERVICE, or MARKETING, which drives consent rules.
- **Channel** - The delivery medium: EMAIL, SMS, or PUSH.
- **Chunk** - A group of items read, processed, and committed together in Spring Batch.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Circuit breaker** - A guard that stops calling a failing dependency after repeated failures, then probes before resuming.
- **Cloud Foundry** - An application platform where developers push code and the platform builds and runs it.
- **Collapse key** - A push-notification key that lets a newer message replace an older pending one, making retries idempotent.
- **Consent** - A recipient's permission to receive a class of messages on a channel.
- **Consumer group / competing consumers** - Several workers reading one queue so each message is handled by one of them.
- **Correlation ID** - An identifier carried through all calls and logs of one request to trace it end to end.
- **CQRS** - Command Query Responsibility Segregation: separate models for writes and reads.
- **CSRF** - Cross-site request forgery: tricking a browser into sending an unwanted authenticated request.
- **Dead letter queue (DLQ)** - A queue holding messages that could not be processed after retries or were unprocessable.
- **DELIVERY_UNKNOWN** - Status meaning the provider outcome is not known, so the system waits and reconciles instead of resending.
- **Dependency injection** - Supplying an object's collaborators from outside instead of creating them inside.
- **Dirty read** - Reading data another transaction has not yet committed.
- **Dispatch** - The consumer-side processing of a queued message: policy check, render, provider call, outcome mapping.
- **DispatchEnvelope** - The small queue payload (ids, channel, priority, attempt) that carries no content or PII.
- **DLX (dead letter exchange)** - A RabbitMQ exchange that receives rejected or expired messages.
- **DR (disaster recovery)** - The plan and capability to restore service after losing a site or region.
- **Eventual consistency** - Replicas or views converge to the same state given time without new updates.
- **Exactly-once** - Each message affects state exactly once; achievable only inside a controlled boundary, never across an external provider.
- **Exponential backoff** - Retry delays that double each attempt up to a cap.
- **FAILED** - Terminal-unless-operator status after a permanent failure.
- **Fallback channel** - An alternative channel tried after permanent failure, only if consent allows.
- **Fan-out** - Delivering one input to many consumers or recipients.
- **Flyway** - A database migration tool that applies versioned SQL scripts in order.
- **Four-eyes principle** - Two different people must author and approve a change (template author and approver differ).
- **Full jitter** - Choosing the retry delay randomly between zero and the exponential ceiling to avoid synchronized retries.
- **G1 GC** - The JVM's default region-based collector that targets pause-time goals.
- **Garbage collection (GC)** - Automatic reclaiming of unreachable memory by the JVM.
- **Graceful shutdown** - Stopping intake, finishing in-flight work, then exiting.
- **Health probe** - A check (liveness, readiness, startup) used by the platform to decide whether to restart or route to an instance.
- **HikariCP** - The default JDBC connection pool in Spring Boot.
- **HMAC** - A keyed hash that proves a message came from a holder of the shared secret and was not altered.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **HPA (Horizontal Pod Autoscaler)** - A Kubernetes controller that scales pod count by metrics.
- **Idempotency** - Repeating an operation has the same effect as doing it once.
- **Idempotency key** - A client-supplied unique key that lets the server recognize and replay a duplicate request.
- **Idempotency reference** - The `messageId:attemptGroup` value sent to providers so they can dedupe.
- **In-flight** - Messages taken from the queue but not yet acknowledged.
- **Isolation level** - The degree to which concurrent transactions are shielded from each other's effects.
- **Jitter** - Random variation added to delays so many clients do not act at the same instant.
- **JPA** - Java Persistence API, the standard object-relational mapping specification.
- **JWKS** - A JSON Web Key Set: a published set of public keys used to verify JWTs.
- **JWT** - JSON Web Token: a signed, compact token carrying claims.
- **Kafka** - A distributed log platform for high-throughput streaming with replay.
- **Lazy loading** - Fetching related data only when first accessed.
- **Lifecycle (message)** - The state progression from ACCEPTED through terminal states such as DELIVERED.
- **Little's law** - In-flight work equals arrival rate times time in system, used for concurrency sizing.
- **Liveness probe** - A check that decides if a container is stuck and needs a restart.
- **Lost update** - Two concurrent writers where one overwrites the other's change unnoticed.
- **MDC** - Mapped Diagnostic Context: per-thread key-value data added to every log line.
- **Micrometer** - The metrics facade used by Spring Boot to publish to systems like Prometheus.
- **MongoDB** - A document database, used here only for best-effort provider diagnostics.
- **Monotonic rank rule** - A lower-ranked delivery event never overwrites a higher-ranked status.
- **Multi-tenancy** - One deployment serving several customers with strict data isolation.
- **N+1 query problem** - One query for a list plus one query per item, instead of a single joined fetch.
- **Non-repeatable read** - Re-reading a row inside a transaction returns a changed value.
- **OAuth2** - An authorization framework for delegated access using tokens.
- **OIDC (OpenID Connect)** - An identity layer on top of OAuth2 that adds an ID token.
- **OpenAPI** - A machine-readable specification format for REST APIs.
- **Optimistic locking** - Detecting conflicting writes with a version column and failing the stale one.
- **Outbox pattern** - Writing an event row in the same transaction as the data change, then publishing it separately.
- **OWASP** - An organization that publishes web security guidance such as the Top 10.
- **Partition (data)** - Splitting a large table or log into smaller pieces by key or time.
- **PCF** - Pivotal Cloud Foundry, a commercial distribution of Cloud Foundry.
- **Pessimistic locking** - Locking rows up front (`SELECT ... FOR UPDATE`) to prevent concurrent modification.
- **Phantom read** - A repeated query returns a different set of rows because of another transaction's insert or delete.
- **PII** - Personally identifiable information, minimized and masked in logs and APIs.
- **PKCE** - Proof Key for Code Exchange: protects the authorization code flow for public clients.
- **Poison message** - A message that always fails processing and would loop forever without a limit.
- **Policy service** - The component that applies recipient existence, suppression, consent, and quiet-hours rules.
- **Prefetch** - How many unacknowledged messages a consumer may hold at once.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Priority queue** - A queue that delivers higher-priority messages first (RabbitMQ `x-max-priority=9`).
- **Provider** - An external service that actually sends email, SMS, or push (simulated in the reference project).
- **Quiet hours** - A recipient-local time window in which non-urgent messages are deferred.
- **RabbitMQ** - A message broker with exchanges, queues, priorities, TTL, and dead lettering.
- **Rate limiting** - Capping the number of requests or sends per unit of time.
- **Readiness probe** - A check that decides whether an instance should receive traffic.
- **Reconciliation** - Comparing and correcting local state against the provider's truth, such as resolving DELIVERY_UNKNOWN.
- **Redis** - An in-memory data store used here for idempotency fast-path and token buckets.
- **Replay attack** - Re-sending a captured valid request to repeat its effect.
- **Replay guard** - A cache of seen event ids that rejects repeated callbacks.
- **Resilience4j** - A Java library providing circuit breaker, retry, rate limiter, and time limiter.
- **Resource server** - An API that validates access tokens and serves protected resources.
- **Retry storm** - Many clients retrying together and overloading a recovering service.
- **RETRY_PENDING** - Status waiting for the backoff time to elapse before re-queueing.
- **RPO (recovery point objective)** - The maximum acceptable amount of data loss, measured in time.
- **RTO (recovery time objective)** - The maximum acceptable time to restore service.
- **Saga** - A sequence of local transactions with compensating actions instead of one distributed transaction.
- **Serializable** - The strictest isolation level, behaving as if transactions ran one at a time.
- **Signature (callback)** - The `X-Signature: t=...,v1=...` header proving a callback's authenticity.
- **SLA / SLO / SLI** - Contractual promise, internal target, and measured indicator of service quality.
- **Spring Batch** - A framework for restartable, chunk-oriented bulk processing.
- **Stuck message sweeper** - A scheduled job that moves long-stalled messages to DELIVERY_UNKNOWN or re-queues them.
- **Suppression list** - Addresses blocked from receiving messages (hard bounce, complaint, regulatory).
- **Template version pinning** - Fixing a message to a specific template version at accept time (ADR-0007).
- **Tenant** - A customer organization whose data is isolated from others.
- **Testcontainers** - A library that runs real dependencies in Docker containers for tests (not executed in the reference environment).
- **Throughput** - Work completed per unit of time.
- **Token bucket** - A rate limiter that adds tokens at a fixed rate and spends one per request, allowing bursts up to capacity.
- **Transactional outbox** - See outbox pattern.
- **TTL (time to live)** - How long a message or cache entry remains valid.
- **Two-phase commit** - A protocol coordinating commit across several resources, avoided in favor of outbox and idempotency.
- **Virtual threads** - Lightweight JVM threads (Java 21) that make blocking-style code scale.
- **Webhook** - See callback.
- **Write skew** - Two transactions each read overlapping data, write different rows, and together violate a rule.
- **XSS** - Cross-site scripting: injecting script into pages viewed by other users.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Zero trust** - A security stance of verifying every request regardless of network location.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix D. Operational Runbooks

Every alert in `observability/prometheus/alerts.yml` links to a runbook under `docs/operations/runbooks/`. The runbooks for dead-letter growth, retry storms, bulk jobs, queue backlog and high latency appear in Chapters 13, 14 and 20; the others follow here. Runbooks are design-level procedures written against the metrics and endpoints of the platform. They have **not** been exercised in a real incident or a game day (risk R12 in the register).

Runbook index

Source document: `docs/operations/runbooks/README.md`

Runbooks are written from the design (not from real incidents). Commands use placeholders: set `BASE` to the API base URL (local default `http://localhost:18080`) and `TOKEN` to a bearer token for an OPERATOR or PLATFORM_ADMIN principal from your environment (locally via `POST /dev/token`). Do not paste real tokens into tickets or chats.

Runbook	Trigger
provider-outage.md	Provider circuit open, submission success low
queue-backlog.md	Queue depth or oldest-message age growing
dlq-growth.md	DLQ depth growing
retry-storm.md	Retry rate high, load amplification
callback-signature-failures.md	`invalid_signature` spike
database-outage.md	DB unreachable or pool exhausted
redis-outage.md	`messaging.redis.fallback` incrementing
bulk-job-stuck.md	Bulk job not progressing
bulk-job-restart.md	`BulkJobStalled` alert; stop/restart a FAILED/STOPPED/orphaned bulk job
high-api-latency.md	API error-budget burn, p95 latency alert, backend down (entry point to the next row)
consumer-crash-ambiguous-outcomes.md	Consumers died; DELIVERY_UNKNOWN growing
provider-rate-limit.md	Provider returns RATE_LIMITED
high-latency-and-resource-saturation.md	High API latency, GC, thread starvation, CPU saturation

Common first steps (all incidents):

1. Declare severity, open a channel, note start time (UTC).
2. `GET $BASE/api/v1/ops/queues` and `GET $BASE/api/v1/ops/summary` for the current picture.
3. `GET $BASE/api/v1/providers` for circuit and fault state.
4. Capture a `correlationId` from any failing request for trace/log search.
5. After resolution: fill `../post-incident-review-template.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Standard structure per runbook: Symptoms and alerts, Impact, Diagnose, Mitigate, Recover, Verify, Escalate, Prevent.

Runbook: Provider outage

Source document: `docs/operations/runbooks/provider-outage.md`

Symptoms and alerts

`ProviderCircuitOpen`, `ProviderSubmissionSuccessLow`; `messaging.provider.calls{outcome=TRANSIENT_FAILURE|TIMEOUT}` up; `messaging.retry.scheduled` rising; `GET /api/v1/providers` shows `circuitState=OPEN`.

Impact

Messages on that channel are delayed; CRITICAL may fall back to another channel only if the request listed `fallbackChannels` and consent allows. No data loss: messages sit `RETRY_PENDING`.

Diagnose

1. `curl -s -H "Authorization: Bearer $TOKEN" $BASE/api/v1/providers` : which provider, `lastError`, `successRate`, `p95LatencyMs`.
2. Provider status page / vendor support channel (real environments).
3. Distinguish: our network (timeouts everywhere) vs vendor (5xx) vs auth (permanent failures, 401/403 mapped to PERMANENT_FAILURE: this would be FAILED, not retry; check `errorCode`).
4. `GET $BASE/api/v1/failed-messages?...` sample reasons.
5. Check credentials expiry/rotation recently done.

Mitigate

- If another provider exists for the channel: raise its `priority_order` or disable the failing one: `PUT /api/v1/providers/{providerId}/config` (PLATFORM_ADMIN) `{ "enabled": false }`. In-flight retries pick the next provider via `ProviderRegistry`.
- Do not raise concurrency or lower backoff: that worsens the outage (see `retry-storm.md`).
- If outage is long and messages are time-sensitive (OTP): consider operational messaging to stakeholders; expired messages are not sent (expiry).
- Pause bulk jobs (`POST /api/v1/bulk-jobs/{id}/stop`) to preserve capacity for critical traffic.

Recover

1. When the provider is healthy the breaker half-opens and closes automatically; confirm `circuit.state` 0.
2. Messages in `RETRY_PENDING` continue. Messages that exhausted retries are `DEAD_LETTERED`: list `GET /api/v1/dead-letters`, filter reason `RETRIES_EXHAUSTED`, replay in batches with rate awareness: `POST /api/v1/dead-letters/{messageId}/replay` (consent and expiry re-evaluated; see `data-retention-and-replay-policy.md`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Check `DELIVERY_UNKNOWN` created during the outage; let reconciliation work (`consumer-crash-ambiguous-outcomes.md`).

Verify

Success rate >= SLO; backlog draining; no growth in DLQ; sample messages reach DELIVERED.

Escalate

Provider vendor support; platform on-call lead if > 30 min for CRITICAL traffic.

Prevent

Multi-provider per channel, tested failover, synthetic probes, SLO alerts, vendor SLAs (real project).

Proof of behavior: `FailureRecoveryIT#scenario01_providerOutage_retriesWithBackoff_thenDeadLetters_thenReplay`.

Runbook: Provider rate limit

Source document: `docs/operations/runbooks/provider-rate-limit.md`

Symptoms and alerts

`messaging.provider.calls{outcome="RATE_LIMITED"}` rising (provider returned 429/throttle); `messaging.ratelimit.rejected{scope}` for our own limiter; backlog growing for that channel; `Retry-After` hints from provider.

Impact

Throughput limited; retries with backoff; LOW/NORMAL delayed; CRITICAL prioritized by queue priority but shares the same provider quota.

Two layers of limiting

Layer	Config	Behavior
Our token bucket (per provider)	`messaging.providers.<id>.rate-per-sec`	empty bucket => requeue with short delay, no attempt consumed
Provider 429	outcome RATE_LIMITED	`RETRY_PENDING` with backoff (uses an attempt), honors Retry-After as minimum

If we see many provider 429s, our configured rate is above the vendor's real limit (or other consumers share the quota).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Diagnose

1. `GET /api/v1/providers` : `rateLimit`, `successRate`.
2. Compare `rate(messaging_provider_calls_total)` with configured rate x instance count (with Redis down, buckets are per instance: `redis-outage.md`).
3. Bulk job contribution (`GET /api/v1/bulk-jobs`).
4. Vendor quota changes or burst limit vs sustained limit.

Mitigate

1. Lower `rate-per-sec` to below the observed accepted rate (config change via `PUT /api/v1/providers/{providerId}/config`, PLATFORM_ADMIN).
2. Throttle/stop bulk jobs so CRITICAL/HIGH keep the quota.
3. Add a second provider/route if available (`provider-outage.md` mitigation).
4. Request a quota increase from the vendor (real environments).
5. Do not add consumers.

Recover

Backlog drains at the sustainable rate. Compute ETA: `depth / rate`.

Verify

`RATE_LIMITED` outcomes near zero; queue age falling.

Prevent

Configure from vendor contracts, per-priority reservation (aging/priority specs), load tests with `RATE_LIMIT` fault mode (local/test only).

Proof: `FailureRecoveryIT#scenario10_providerRateLimitAndBackpressure_noAttemptConsumed_apiReturns503`.

Runbook: Consumer crash and ambiguous outcomes

Source document: `docs/operations/runbooks/consumer-crash-ambiguous-outcomes.md`

Symptoms and alerts

Instances restarted/OOM-killed; growth in `messaging.transitions{to="DELIVERY_UNKNOWN"}`; `GET /api/v1/failed-messages` lists DELIVERY_UNKNOWN; `DeliveryUnknownAging`.

Impact

Some messages may have been sent but state is unknown. Risk assessment: duplicates if retried blindly (we do not), delays if left unresolved.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Diagnose

1. How many and since when: `GET $BASE/api/v1/failed-messages?status=DELIVERY_UNKNOWN` (filters: see OpenAPI).
2. Causes: crash (stale `locked_until`, attempt `STARTED` with no outcome), provider timeouts (attempt outcome TIMEOUT/AMBIGUOUS), missing receipts (SENT_TO_PROVIDER aged out).
3. For a sample: `GET /api/v1/messages/{id}/history` and `GET /api/v1/diagnostics/messages/{id}` (best-effort).
4. Instance cause: `kubectl logs --previous` / `cf logs --recent` (not executed here); OOM? (see `high-latency-and-resource-saturation.md`).

Mitigate

- Fix the crash cause first (memory limits, poison, bug).
- Ensure the sweeper and reconciliation are running (`StuckMessageSweeper`, `ReconciliationService` logs, schedule enabled).
- Do NOT bulk-requeue DELIVERY_UNKNOWN as FAILED. The `requeue` endpoint applies to FAILED/DEAD_LETTERED only by design.

Recover

1. Wait for callbacks (late receipts move to DELIVERED/FAILED).
2. After `reconcile-after`, reconciliation calls `lookup(idempotencyRef)`: | Result | Outcome | |---|---| | delivered / failed | state updated | | not received | automatic safe retry (single new attempt) | | unavailable | re-check with backoff |
3. Items still unknown after max checks: operator decides per business value (security alert: consider resend via a fresh message with explicit audit, accepting a duplicate risk; marketing: leave).
4. Record decisions in audit (`OPERATOR` actions).

Verify

DELIVERY_UNKNOWN count falls; no spike in duplicate provider calls (compare `message_attempt` rows per message with same `idempotencyRef`); SLI 8 within target.

Prevent

Attempt row before provider call, lock TTL tuned, graceful shutdown (30 s), memory limits and heap sizing, provider idempotency refs, runbook drills.

Proof: `FailureRecoveryIT#scenario04_...`, `#scenario05_consumerCrashAfterProviderAccept_sweeperMarksDeliveryUnknown`, ADR-0010.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Runbook: Callback signature failures

Source document: `docs/operations/runbooks/callback-signature-failures.md`

Symptoms and alerts

`CallbackSignatureFailures`; `messaging.callbacks{result="invalid_signature"}` spike; delivered statuses stop updating, messages drift to `DELIVERY_UNKNOWN`.

Two very different causes

Cause	Signal	Urgency
Attack / probing	many failures from unknown sources, mixed provider ids, no legit traffic loss	investigate, rate limit at edge
Misconfiguration / key rotation	failures for one provider and one `X-Provider-Key-Id` exactly after a change; legitimate receipts stop	high: delivery tracking degraded

Diagnose

1. Which provider and key id? Logs (structured, search `invalid_signature` + `providerId`, `keyId`). Raw bodies are not logged by design.
2. Timestamp tolerance: failure reason `timestamp outside tolerance` means clock skew on our hosts or provider; check NTP. Tolerance is 5 min (`messaging.callback.tolerance`).
3. Key rotation: was the new `keyId` added to `messaging.callback.secrets.<keyId>` before the provider switched? (`secrets-management.md`)
4. Body mutation: a proxy or gateway altering the raw body (re-serialization, charset, compression) invalidates HMAC. Verify the HMAC input is raw bytes.
5. Replay vs signature: `replayed` result is a different counter.

Mitigate

- Misconfiguration: deploy the missing secret/key id; do NOT disable verification or loosen tolerance "temporarily" (never turn off signature checks).
- Clock skew: fix NTP; tolerance changes require security approval.
- Attack: edge rate limits/IP allow-list for provider ranges; keep verifying.

Recover

After fix, providers typically retry webhooks (provider-specific). For lost receipts, reconciliation resolves `DELIVERY_UNKNOWN` through `lookup` (`consumer-crash-ambiguous-outcomes.md`).

Verify

`invalid_signature` rate back to baseline; `accepted` rate resumes; sample message reaches DELIVERED.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Prevent

Dual-key rotation procedure, rotation tests, alert before cutover, synthetic signed callback check using the simulator (`POST /dev/simulator/callbacks/emit` in local/test).

Proof: `SignatureVerifierTest`, `tests/security` forged-callback script.

Runbook: Database outage

Source document: `docs/operations/runbooks/database-outage.md`

Symptoms and alerts

`HikariPoolSaturated`, readiness DOWN, 503s on POST /messages, `hikaricp_connections_timeout_total` rising, outbox publisher and consumer errors in logs.

Impact

New submissions fail closed with 503 (no 202 without commit). Dispatch stalls. Provider calls already in progress may complete without a recorded outcome (becomes `DELIVERY_UNKNOWN` after recovery via sweeper). Callbacks fail (providers retry).

Diagnose

1. Is the DB down (connection refused) or slow (pool exhausted, locks)? Check `hikaricp_connections_active/pending/acquire_seconds`.
2. Pool exhaustion cause: long transactions (one holding a connection across a call), lock waits, slow query, connection leak (`leak-detection-threshold` logs).
3. PostgreSQL: `SELECT pid, state, wait_event_type, now()-xact_start AS age, left(query,80) FROM pg_stat_activity ORDER BY xact_start NULLS LAST;` and blocking locks via `pg_locks`. (Not executed here.)
4. Thread dump of app: `jcmd <pid> Thread.print` to see threads waiting in `HikariPool.getConnection`.
5. Recent deploy or migration? (long DDL lock) `schema-evolution.md`.

Mitigate

- Down: fail over/restore DB (platform DBA procedure); the app reconnects (Hikari `connection-test`/ `max-lifetime`).
- Locks: terminate the blocking session (`SELECT pg_terminate_backend(pid)`), after confirming it is not a legitimate migration.
- Leak/slow query: roll back release, kill offending query, add missing index (`query-plans-and-indexes.md`).
- Do not raise pool size blindly (shifts overload to DB; check `max_connections`).
- Clients: should retry with the same `Idempotency-Key`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Recover

1. Outbox publisher resumes automatically (pending `NEW` rows).
2. `StuckMessageSweeper` moves stale `PROCESSING` to `DELIVERY_UNKNOWN`; reconciliation proceeds.
3. Check outbox backlog age and queue depth, `provider-outage.md` style verification.

Verify

Readiness UP, pending connections 0, submission success restored, oldest outbox `NEW` age small, no duplicate sends (compare provider calls to attempts for the window).

Prevent

HA database, connection timeout short (fail fast), alerts on pending connections, statement timeouts, load tests, separate pool for batch.

Proof: `FailureRecoveryIT#scenario06_databaseOutage_submissionFailsClosed_thenRecovers`.

Runbook: Redis outage

Source document: `docs/operations/runbooks/redis-outage.md`

Symptoms and alerts

`RedisFallbackActive` (`messaging.redis.fallback` incrementing), Lettuce connection errors in logs, slightly higher latency on POST /messages.

Impact

By design low: Redis is a fast path only. `ResilientCoordination` falls back: idempotency to the DB unique constraints, rate limiting to per-instance in-memory buckets. Degradations to understand:

- API and provider rate limits are per instance, so cluster-wide limits can be exceeded by up to `instances x limit` (provider may rate-limit us: watch `RATE_LIMITED`).
- Idempotency lookups hit the DB (more load).
- Replay guard for callbacks relies on the `replay_guard` table.

Diagnose

1. `redis-cli -h <host> ping` (not executed here), cluster/sentinel status, memory/eviction (`INFO memory`), network.
2. Confirm fallback active: metric increments, health indicator state (see source).
3. Check DB load and `hikaricp_connections_pending` because DB absorbs more work.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Mitigate

- Restore Redis or fail over replica.
- If provider limits are at risk: temporarily lower `messaging.providers.<id>.rate-per-sec` (divide by instance count) or reduce replicas of consumers.
- Do not disable idempotency.

Recover

Automatic return when Redis is healthy; token buckets start full or empty depending on implementation (see source); watch for burst on recovery.

Verify

`messaging.redis.fallback` stops increasing; DB load normal; no duplicate messages created (query `message_request` duplicates by `(tenant_id, idempotency_key)` cannot exist: unique).

Prevent

Redis HA, memory policy `noeviction` for rate-limit keys or isolate them, alerts, DB capacity headroom for fallback.

Proof: `FailureRecoveryIT#scenario07_redisOutage_fallsBackToDb_noDuplicates`.

Runbook: High API latency

Source document: `docs/operations/runbooks/high-api-latency.md`

Entry point for the alerts `ApiErrorBudgetFastBurn`, `ApiErrorBudgetSlowBurn`, `ApiLatencyP95High` and `BackendDown` (`observability/prometheus/alerts.yml` references this file name). The detailed triage tree lives in high-latency-and-resource-saturation.md; this page only adds the first minutes per alert.

Alert	First checks	Then
`ApiErrorBudgetFastBurn` / `SlowBurn` (5xx ratio)	`GET $BASE/api/v1/ops/summary`; look at `http_server_requests_seconds_count{status=~"5.."}` by `uri`; check for 503 `SERVICE_UNAVAILABLE` (database unreachable or backpressure: the API fails closed and sends `Retry-After`)	`database-outage.md` if the database is the cause; `queue-backlog.md` if backpressure (`messaging.backpressure.max-queue-depth`) is the cause; otherwise the triage tree
`ApiLatencyP95High`	compare p95 by `uri`; `hikaricp_connections_pending`; GC pauses; provider latency	triage tree in `high-latency-and-resource-saturation.md`
`BackendDown` (target not scraped)	is the process/pod running and ready (`/actuator/health/readiness`)? recent deployment? scrape path/network?	restart one instance at a time; if a release caused it, `../rollback-plan.md`

Clients can safely retry 5xx/503 on `POST /api/v1/messages` with the same `Idempotency-Key` (ADR-0004). Fill `../post-incident-review-template.md` afterwards. This runbook is written from the design, not from a real incident.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Data retention and replay policy

Source document: `docs/operations/data-retention-and-replay-policy.md`

Retention

Numbers and mechanism: `docs/database/retention-and-archival.md`. Summary: operational rows 90 days, audit 400 days, idempotency/outbox/dedupe/callback guards days or hours, Mongo diagnostics 30 days TTL (all illustrative defaults).

Replay policy

Replay = operator-initiated re-injection of a `FAILED` or `DEAD_LETTERED` message into the dispatch path (`POST /api/v1/dead-letters/{id}/replay`, `POST /api/v1/messages/{id}/requeue`). It is the only way a soft-terminal message leaves its state.

Authorization

`OPERATOR` or `PLATFORM_ADMIN` only; tenant-scoped for operators; audited with actor, role, reason, correlation id.

Preconditions (checked by `ReplayService`)

Check	Behavior if fails
Status is `FAILED` or `DEAD_LETTERED`	409 `ILLEGAL_STATE_TRANSITION`
Not past `expires_at`	rejected as expired (time-sensitive content must not go out late)
Policy re-evaluation (consent, suppression, quiet hours)	`REJECTED` with reason; replay never bypasses consent
Template pinned version still renderable	proceeds even if RETIRED (ADR-0007)
Last known outcome not ambiguous	if attempts include TIMEOUT/AMBIGUOUS/unknown, run reconciliation lookup first or require explicit confirmation flag (see source) to accept duplicate risk
Rate awareness	replay goes through the normal queue priority and provider token bucket

Rules of engagement

1. Find and fix root cause before replay (poison and bug cases).
2. Replay in batches; verify the first batch.
3. Replay creates a new `dispatchId` and attempt group; `attempt_count` continues (cap rules per source).
4. Prefer replay at LOW or original priority; never raise priority to jump the queue without reason.
5. Maximum replay age: default 7 days for operational messages (illustrative); older requires PLATFORM_ADMIN and a recorded business reason.
6. Discard: `POST /api/v1/dead-letters/{id}/discard` records reason; message stays `DEAD_LETTERED` with discarded flag; counts excluded from DLQ depth alerts after discard (see source).
7. Bulk replay (many ids) must be scripted with throttling (`scripts/` helper, see source); no unbounded loops.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

What is NOT replayable

`DELIVERED`, `REJECTED`, `CANCELLED` (hard terminals). To resend content intentionally, submit a new message with a new idempotency key.

Replay of events (callbacks)

Provider callbacks are idempotent by `(provider_id, provider_event_id)`; re-posting a stored event is a no-op by design. There is no "replay from broker offset" capability (RabbitMQ; see ADR-0001 for the Kafka trade-off).

Evidence

Audit actions for replay/discard/requeue; metrics `messaging.deadletter.total`, `messaging.transitions{from="DEAD_LETTERED",to="QUEUED"}`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix E. End-to-End Run Output (Executed)

This appendix reproduces `docs/validation/e2e-run-output.txt` verbatim: the output of a real HTTP end-to-end run against the packaged backend jar on the `local` profile, produced by `node tests/end-to-end/e2e.mjs`. It is the strongest executed evidence in the repository and also shows what it is not: it ran on H2, the in-memory bus and in-memory Mongo with simulated providers, so it proves behaviour and contracts, not PostgreSQL, RabbitMQ or Redis capacity or operations. Docker, Kubernetes, Cloud Foundry, k6 and CI were not part of it.

```
# End-to-end run output (real HTTP, executed)
# Command: SERVER_PORT=18280 bash scripts/run-local.sh --messaging.dispatch.backoff-base-ms=300
  --messaging.dispatch.backoff-cap-ms=2000 #            --messaging.dispatch.reconcile-after=3s
  --messaging.pipeline.scheduler.reconcile-interval-ms=2000 --messaging.bulk.chunk-size=100
#          BASE_URL=http://localhost:18280 BACKEND_LOG=<backend console log> node tests/end-to-end/e2e.mjs
# Target : packaged jar backend/target/messaging-platform.jar, profile local (H2, in-memory bus, in-memory
  Mongo, simulators), Windows 11, JDK 21
# Note   : an earlier run of the same script found 9 failing checks, all test-side problems (a seeded
  suppressed recipient, a wrong audit action name,
#          upload counter expectations, a 503 from API backpressure while a 20,000-row bulk backlog was still
  draining, and a 400-way parallel burst
#          of which ~3% of TCP connects timed out (ETIMEDOUT, reproduced with an unauthenticated health probe:
  Windows loopback / Tomcat accept
#          backlog, not application logic); the rate-limit check now uses 40 parallel clients); the test was
  corrected, not weakened, and re-run on a freshly started backend. This file is that second run.
#
E2E run mvdzyc2r against http://localhost:18280 at 2026-10-18T15:45:11.331Z

== 1. Personas, authentication and authorization
  PASS  dev tokens minted for all personas
  PASS  health endpoint is UP
  PASS  liveness probe UP
  PASS  GET /me returns roles and tenant
  PASS  no token -> 401 UNAUTHENTICATED
  PASS  garbage token -> 401
  PASS  tampered signature -> 401
  PASS  AUDITOR cannot submit -> 403 FORBIDDEN
  PASS  OPERATOR cannot create templates -> 403
  PASS  SENDER cannot set fault mode -> 403
  PASS  OPERATOR cannot pause consumers -> 403
  PASS  SENDER cannot read audit events -> 403

== 2. Submit email, sms and push; delivery receipts
  PASS  email accepted (202)
  PASS  202 carries Location header
  PASS  202 carries correlation id header
  PASS  API-Version header present
  PASS  email reaches SENT_TO_PROVIDER (provider acceptance != delivery)
  PASS  detail has one ACCEPTED attempt via simulated-email
  PASS  detail masks the address and never returns body
  PASS  detail never contains rendered content or raw address
  PASS  detail pins template version
  PASS  simulator emits signed DELIVERED callback (202)
  PASS  email becomes DELIVERED
  PASS  status endpoint: terminal=true
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  PASS  sms accepted (202)
  PASS  sms reaches SENT_TO_PROVIDER
  PASS  sms attempt used simulated-sms
  PASS  push accepted (202)
  PASS  push reaches SENT_TO_PROVIDER
  PASS  push attempt used simulated-push
  PASS  message list filters by channel+status with PageResponse envelope
  PASS  list items carry channelUsed/templateCode
  PASS  list filters by correlationId
  PASS  page size is capped/validated (<=200)

== 3. Idempotency
  PASS  first submit 202
  PASS  same key + same body replays original (200 + Idempotent-Replay: true)
  PASS  same key + different body -> 422 IDEMPOTENCY_KEY_REUSED
  PASS  same key in another tenant is independent (202)
  PASS  missing Idempotency-Key -> 400
  PASS  8 concurrent identical submissions yield exactly one message id
  PASS  concurrent duplicates answered 202/200/409(in progress) only

== 4. Malformed input, validation and error contract
  PASS  malformed JSON -> 400 MALFORMED_REQUEST
  PASS  error body has ApiError fields
  PASS  error body does not leak stack/class names
  PASS  unsupported channel -> 400 UNSUPPORTED_CHANNEL
  PASS  missing fields -> 400 VALIDATION_FAILED with field details
  PASS  unknown priority enum -> 400
  PASS  unknown template -> 422 TEMPLATE_NOT_FOUND
  PASS  missing template variables -> 422 TEMPLATE_VARIABLE_MISSING
  PASS  unknown recipient -> 422 RECIPIENT_NOT_FOUND
  PASS  body > 256 KB -> 413 PAYLOAD_TOO_LARGE
  PASS  unknown message id -> 404 NOT_FOUND
  PASS  HTML/template injection in variables is accepted as data (202), not evaluated
  PASS  unsupported method -> 405 with ApiError
  PASS  unknown path -> 404
  PASS  X-Correlation-Id is echoed

== 5. Tenant isolation and platform-admin scoping
  PASS  other tenant gets 404 (not 403) for a message
  PASS  other tenant cannot cancel -> 404
  PASS  other tenant list never contains retail messages
  PASS  other tenant cannot read history -> 404
  PASS  sending to another tenant's recipient -> 422 RECIPIENT_NOT_FOUND
  PASS  X-Tenant-Id from a non-admin is ignored (still 404)
  PASS  platform admin reads across tenants
  PASS  platform admin write without X-Tenant-Id -> 400
  PASS  platform admin write with X-Tenant-Id -> 202
  PASS  platform admin with unknown tenant -> 400

== 6. Consent, suppression and quiet hours
  PASS  opted-out recipient: marketing accepted as REJECTED (202)
  PASS  rejection reason is OPTED_OUT and message is terminal
  PASS  no provider attempt for the rejected message
  PASS  transactional to the same recipient ignores marketing opt-out
  PASS  recipient without push token: REJECTED (no address)
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  PASS  operator adds a suppression
  PASS  suppressed address blocks even transactional email (REJECTED)
  PASS  operator removes the suppression
  PASS  after removal the same recipient is deliverable
  PASS  preferences readable
  PASS  quiet-hours window set around now
  PASS  stale preference write -> 409 CONFLICT_OPTIMISTIC_LOCK
  PASS  marketing inside quiet hours is deferred (VALIDATED with future scheduledAt)
  PASS  deferred message is not dispatched yet (no attempts)
  PASS  security alert ignores quiet hours (dispatched)
  PASS  deferred message can be cancelled
  PASS  quiet hours restored

== 7. Template lifecycle: create -> submit -> approve (four-eyes) -> publish -> use -> new version
  PASS  author creates template (201/200) with a DRAFT version
  PASS  duplicate code+channel -> 409
  PASS  a DRAFT template cannot be used for sending (422 TEMPLATE_NOT_FOUND)
  PASS  preview renders and HTML-escapes sample data
  PASS  preview reports undeclared variables as invalid
  PASS  author submits for approval
  PASS  author cannot approve own version -> 403
  PASS  TENANT_ADMIN is not an approver -> 403
  PASS  PENDING_APPROVAL version cannot be edited -> 409
  PASS  different approver publishes v1
  PASS  v1 is PUBLISHED
  PASS  published template can be used
  PASS  scheduled message accepted (not yet dispatched)
  PASS  message pinned to version 1 at accept time
  PASS  author creates a new DRAFT version (v2)
  PASS  v2 submit
  PASS  approver rejects with comment (back to DRAFT)
  PASS  v2 resubmit
  PASS  approver publishes v2
  PASS  v1 RETIRED and v2 PUBLISHED
  PASS  the already accepted message keeps version 1
  PASS  the scheduled (not yet sent) message keeps version 1
  PASS  a new message pins version 2
  PASS  cancel the scheduled message
  PASS  template change history is available
  PASS  template list filter by code
  PASS  other tenant cannot see the template -> 404

== 8. Simulator fault modes: retry, permanent failure, timeout, ambiguous, rate limit, outage -> DLQ -> replay
  PASS  transient failure -> RETRY_PENDING
  PASS  failed attempt recorded with TRANSIENT_FAILURE and nextAttemptAt set
  PASS  after the fault clears the retry is sent (SENT_TO_PROVIDER)
  PASS  retried message becomes DELIVERED
  PASS  attempt count > 1 and a later attempt ACCEPTED
  PASS  permanent failure -> FAILED
  PASS  exactly one attempt for a permanent failure (no retry)
  PASS  FAILED message appears in the failed-messages investigation list
  PASS  operator requeue of a FAILED message
  PASS  requeued message reaches the provider
  PASS  SENDER cannot requeue -> 403
  PASS  provider timeout -> DELIVERY_UNKNOWN (not blindly retried)
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  PASS  exactly one attempt while unknown
  PASS  reconciliation finds NOT_RECEIVED and safely re-sends
  PASS  RECONCILIATION_CHECK is audited
  PASS  ambiguous provider answer -> DELIVERY_UNKNOWN
  PASS  reconciliation sees the provider accepted it: no duplicate send, still DELIVERY_UNKNOWN
  PASS  late DELIVERED callback accepted
  PASS  DELIVERY_UNKNOWN -> DELIVERED by the late callback
  PASS  provider rate limit (429) -> RETRY_PENDING
  PASS  attempt recorded as RATE_LIMITED
  PASS  rate-limited message eventually sent
  PASS  platform admin lowers simulated-sms rate limit
  PASS  6 SMS through a 2/s limiter all dispatched (none lost)
  PASS  outage: retries exhausted -> DEAD_LETTERED
  PASS  dead-lettered after max attempts (5)
  PASS  dead-letter list shows RETRIES_EXHAUSTED
  PASS  provider health reflects the outage (failures, lastError)
  PASS  SENDER cannot replay dead letters -> 403
  PASS  operator replays the dead letter
  PASS  replayed message is sent after the outage ends (circuit recovers)
  PASS  replaying a message that is no longer dead-lettered -> 409

== 9. Signed provider callbacks: valid, bad signature, replay, duplicate, out-of-order
  PASS  callback target reaches SENT_TO_PROVIDER
  PASS  valid SENT callback accepted (202 accepted)
  PASS  wrong secret -> 401 INVALID_SIGNATURE
  PASS  timestamp outside the 5 minute tolerance -> 401
  PASS  key id not bound to the provider -> 401
  PASS  unknown provider -> 401 (same answer as a bad signature)
  PASS  callback without signature headers -> 401
  PASS  rejected callbacks changed nothing (still SENT_TO_PROVIDER)
  PASS  same X-Event-Id again -> 200 replayed
  PASS  same provider event id under another header id -> duplicate (no second row)
  PASS  signed but malformed payload -> 400
  PASS  event for an unknown message -> 202 unknown_message
  PASS  DELIVERED callback accepted
  PASS  message DELIVERED
  PASS  late SENT after DELIVERED is out_of_order and ignored
  PASS  BOUNCED after DELIVERED is a conflict; first terminal state wins
  PASS  status is still DELIVERED
  PASS  delivery events: applied DELIVERED and ignored OUT_OF_ORDER SENT recorded
  PASS  delivery events are tenant scoped (other tenant sees none)
  PASS  provider diagnostics available to operator (Mongo archive)
  PASS  diagnostics forbidden for SENDER

== 10. Cancel, history timeline, audit
  PASS  scheduled message accepted
  PASS  cancel a scheduled message -> 200 CANCELLED
  PASS  cancel again -> 409 ILLEGAL_STATE_TRANSITION
  PASS  a CANCELLED message cannot be requeued -> 409
  PASS  cancel after delivery -> 409
  PASS  history is a time-ordered merge of AUDIT, ATTEMPT and DELIVERY_EVENT entries
  PASS  history entries follow TimelineEntry {at,kind,type,summary,detail}
  PASS  audit entries carry the actor in detail.actor
  PASS  history never leaks content or raw addresses
  PASS  auditor reads audit events of the message
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  PASS  audit contains MESSAGE_ACCEPTED
  PASS  audit is tenant scoped (other tenant: 403 or empty)
  PASS  audit filter by action (template approval audited)
  PASS  audit trail is append-only through the API (no POST)

== 11. Bulk jobs: generate-synthetic, progress, failures, upload, authorization, stop/restart
  PASS  bulk without scope messages:bulk -> 403
  PASS  AUDITOR cannot start bulk -> 403
  PASS  rows=0 -> 400
  PASS  synthetic 500-row job accepted (202) with Location
  PASS  job finished COMPLETED_WITH_ERRORS (invalid + duplicate rows)
  PASS  total counts the rows
  PASS  processed = succeeded + failed + duplicate
  PASS  some rows failed and some were duplicates
  PASS  most rows succeeded
  PASS  failures endpoint lists rejected rows with line numbers and hashes only
  PASS  failure reasons include DUPLICATE_EXTERNAL_REF and a validation/recipient reason
  PASS  job list readable by tenant admin, newest first
  PASS  other tenant cannot see the job -> 404
  PASS  bulk messages flow through the same pipeline (list works)
  PASS  stop on a finished job -> 409
  PASS  CSV upload accepted (202)
  PASS  upload: counters add up (43 rows = succeeded + failed + 1 duplicate), bad rows rejected
  PASS  upload failures name RECIPIENT_NOT_FOUND, a malformed row and the duplicate
  PASS  upload without file -> 400
  PASS  upload with invalid CSV header -> 400
  PASS  large synthetic job accepted
  PASS  running job stopped between chunks (partial progress kept)
  PASS  restart accepted
  PASS  restarted job completes all rows exactly once (succeeded = 20000, no duplicates)
  PASS  stop without scope messages:bulk -> 403

== 11b. Let the bulk backlog drain (API backpressure would otherwise answer 503 to new submissions)
  PASS  pipeline drained (no VALIDATED/QUEUED/PROCESSING/RETRY_PENDING left)

== 12. Ops, providers, metrics, log hygiene
  PASS  queues: depth per priority, in-flight, consumers, DLQ, retry-pending, oldest age, paused flags
  PASS  SENDER cannot read queue stats -> 403
  PASS  summary: byStatus, byChannel, total, successRate, p95 dispatch delay
  PASS  summary total equals sum(byStatus)
  PASS  summary is tenant scoped (wealth total < global total)
  PASS  platform admin pauses consumers
  PASS  submission while consumers are paused is still accepted (202)
  PASS  while paused a new message is queued but not processed
  PASS  queue stats report consumersPaused=true and a positive depth
  PASS  platform admin resumes consumers
  PASS  queued message is processed after resume
  PASS  provider health lists the four providers with circuit state, rate limit, fault mode
  PASS  TENANT_ADMIN cannot change provider config -> 403
  PASS  invalid fault mode -> 400
  PASS  fault mode on unknown provider -> 404
  PASS  prometheus endpoint served
  PASS  meter present: messaging_messages_accepted_total
  PASS  meter present: messaging_transitions_total
  PASS  meter present: messaging_provider_calls_total
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  PASS  meter present: messaging_provider_latency_seconds
  PASS  meter present: messaging_dispatch_delay_seconds
  PASS  meter present: messaging_queue_depth
  PASS  meter present: messaging_queue_inflight
  PASS  meter present: messaging_queue_dlq_depth
  PASS  meter present: messaging_retry_scheduled_total
  PASS  meter present: messaging_deadletter_total
  PASS  meter present: messaging_callbacks_total
  PASS  meter present: messaging_idempotency_hits_total
  PASS  meter present: messaging_provider_circuit_state
  PASS  meter present: messaging_bulk_progress
  PASS  meter present: http_server_requests_seconds
  PASS  meter present: hikaricp_connections
  PASS  meter present: jvm_memory_used_bytes
  PASS  callback results counted by result tag
  PASS  no PII/tenant data in metric labels (no addresses or message ids)
  PASS  actuator exposure limited (env/heap dump not exposed)
  PASS  OpenAPI served (local profile) and lists the bulk-jobs and callbacks paths

== 13. API rate limit (token bucket per client)
  PASS  no connection errors under 40 parallel clients
  PASS  burst of 400 parallel requests triggers 429 RATE_LIMITED
  PASS  429 carries Retry-After
  PASS  requests before the limit were served (200)
  PASS  another client is unaffected

== 14. Backend log redaction spot check
  PASS  log is non-empty (dispatch activity logged)
  PASS  no raw email addresses in the log
  PASS  no phone numbers in the log
  PASS  no push tokens in the log
  PASS  no JWT / bearer tokens in the log
  PASS  no template variable values or rendered content in the log
  PASS  callback HMAC secret never logged
  PASS  no stack traces for handled client errors (no "at com.example" frames from 4xx)
  PASS  correlation id MDC present in log lines

==== RESULT: 254 passed, 0 failed, 0 skipped in 267.9 s
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix F. Validation Report and Measurements

The documents below are the repository's validation report and the executed local performance measurements. They are inlined from `docs/validation/` so this guide and the repository tell the same story. Anything listed there as not executed was not executed.

Validation report

Source document: `docs/validation/VALIDATION_REPORT.md`

Date of final verification: 2026-10-18. Machine: Windows 11, 8 logical CPUs, JDK 21 (GraalVM 21.0.7), Node 24, Chrome 155 headless. Every number below was produced by a command that was actually run on this machine; commands are given from the repository root. This is an independent reference implementation; results do not describe any employer's systems.

1. Executed and passed

Area	Command	Result (observed)
Backend compile + all tests	`bash scripts/mvn.sh clean verify`	BUILD SUCCESS in 6 min 15 s. Surefire (unit): 281 run, 0 failures, 0 errors, 0 skipped. Failsafe (integration): 226 run, 0 failures, 0 errors, 1 skipped (`BulkThroughputIT`, opt-in via `-Dbulk.measure=true`). Re-run by the lead after all agents finished.
Backend coverage (JaCoCo, from that run)	`backend/target/site/jacoco/jacoco.csv`	90.2 % instruction, 77.5 % branch (no coverage gate is enforced).
Backend static analysis	`bash scripts/mvn.sh verify -Pstatic-analysis`	Reported green by the integration agent after fixing 8 SpotBugs findings in code (0 SpotBugs bugs, 0 Checkstyle violations); two SpotBugs patterns are excluded with written reasons in `backend/config/spotbugs-exclude.xml`. Not re-run in the final lead pass.
OpenAPI contract	`OpenApiContractIT` (part of verify)	Passed: springdoc output contains every committed path; `docs/api/openapi.yaml` identical to the committed spec.
Angular lint / build / unit tests	`npx ng lint`, `npx ng build --configuration production`, `npx ng test --watch=false --browsers=ChromeHeadlessNoSandbox` (in `frontend/messaging-console`)	Lint clean; production build OK; 212 of 212 tests passed. Re-run by the lead.
End-to-end over real HTTP	`node tests/end-to-end/e2e.mjs` against `backend/target/messaging-platform.jar`, profile `local`	254 passed, 0 failed, 0 skipped (267.9 s). Output: `docs/validation/e2e-run-output.txt`. An earlier run had 9 failing checks, all test-side defects, corrected and re-run on a fresh backend (see the header of that file).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Area	Command	Result (observed)
Browser UI check	`node tests/end-to-end/ui-check.mjs` (headless Chrome via DevTools protocol)	16 pages and 4 interactive flows, 0 console errors, 0 failed API calls; 23 screenshots in `docs/validation/screenshots/`.
Smoke test on the packaged jar	`SERVER_PORT=18380 RUN_PID_FILE=1 bash scripts/run-local.sh` then `BASE_URL=http://localhost:18380 bash scripts/smoke-test.sh`	SMOKE OK (liveness, readiness, 401, tokens, submit 202, idempotent replay 200, missing key 400, message progressed to SENT_TO_PROVIDER, ops summary, Prometheus meters). `scripts/seed-demo.sh` submitted 24 messages (24 x 202) and a synthetic bulk job (202). Both scripts had stale request shapes (token field names, template variables, channel/category) that the lead found and fixed; the k6 helper `performance/k6/lib/common.js` received the same fix but k6 itself was not run.
Failure-recovery scenarios	`FailureRecoveryIT` (scenario01-08, 10) and `FailureRecoveryBulkIT#scenario09_bulkJobRestart_resumesWithoutDuplicates` (part of verify); `tests/failure-recovery/*.mjs`	ITs passed in verify. The three Node scripts (provider outage, callback replay, provider timeout) were run by the messaging agent against the packaged jar; all checks passed.
Static infra validation	`bash scripts/validate-infra.sh`	PASS=6, FAIL=0, SKIPPED=14 (bash -n on scripts, YAML/JSON parse with a home-grown syntax checker, Cloud Foundry manifest validator, alert/metric/runbook consistency, secret-pattern scan, workflow permission checks, `make-zip.py` parse).
Performance (local, single machine)	`node performance/node-load/load-driver.mjs ...`	See `docs/validation/performance-results.md`. Example: submit with 8 clients 178.9 req/s, p95 134.8 ms; with 32 clients 262 accepted req/s with 1,069 backpressure 503s. Load generator and backend shared the machine; H2 and the in-memory bus, not PostgreSQL/RabbitMQ.
Query plans	`QueryPlanIT` (part of verify)	EXPLAIN ANALYZE of 17 hot queries on 100,000 messages, **H2 plans only**: `docs/database/query-plans-executed.md`.
Bulk throughput	`BulkThroughputIT` with `-Dbulk.measure=true` (batch agent)	46-79 rows/s on this machine, bounded by the per-message submission path; see `docs/operations/bulk-test-plan.md`.
PDFs	`node tools/pdf/verify-pdf.mjs <pdf>`	See section 4.

2. NOT executed (no tool, service or credentials available)

Docker image builds; `docker compose config/up`; Trivy; hadolint; kubectl/Kustomize/kubeconform; Helm lint/template/install; `cf push` or any Cloud Foundry foundation (manifests are only parsed and checked by `cloud-foundry/validate-manifest.mjs`); GitHub Actions, Jenkins and Bamboo runs (YAML syntax only); k6 scripts (`performance/k6/*`); Testcontainers tests (`-Pcontainers`, including `RabbitMessageBusContainerIT`); real PostgreSQL, Oracle, RabbitMQ, Redis (`RedisCoordination` Lua token bucket compiled but never run) and MongoDB (tests use H2 in PostgreSQL mode, the in-memory bus, in-memory coordination and `mongo-java-server`); `-Psecurity-scan` (OWASP dependency-check; no NVD key, so **no dependency-vulnerability findings exist or are claimed**); promtool, Grafana rendering of the dashboards; OpenTelemetry export to a collector; Playwright; axe/Lighthouse/screen-reader testing; cross-browser testing; GitHub Copilot or any agent session other than Claude Code (the AI-workspace files are authored artifacts).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Known limitations and open issues (honest list)

- RabbitMQ and Redis adapters are unproven in execution; their pure logic is unit-tested (routing, topology vs `definitions.json`, fallback decorator with a stub).
- H2 behaviour is not PostgreSQL behaviour: list/audit queries sort by `(timestamp DESC, id)` and H2 reads the whole tenant/status partition; PostgreSQL plans are unverified.
- The API accepts faster than the pipeline dispatches on this machine (170-300 req/s vs about 120-200 msg/s) and applies backpressure (503) at 5,000 pending, by design.
- Bulk restart is deterministic only with one worker (resume from the last committed chunk); with more workers the ledger and idempotency keys prevent duplicates but re-read the file.
- The in-memory bus loses queued envelopes on restart; the stale-QUEUED poller (`pollStaleQueued`) re-publishes them, and the idempotent consumer makes duplicates harmless.
- Handled idempotency-key races are logged at ERROR by Hibernate although the request ends 200/409 correctly.
- Priority aging (spec 0001, ADR-0013) is a worked specification example and is **not implemented**.
- `scripts/stop-local.sh` needs `RUN_PID_FILE=1` when starting with `run-local.sh`.
- Provider p95 and dispatch-delay p95 gauges read 0 when idle (rolling ~2 minute window).

4. PDF verification

Run `node tools/pdf/verify-pdf.mjs docs/pdf/<file>.pdf --min-pages N`; results of the final build are appended below by `scripts/` output of the lead (see section 5).

5. Final artifacts

Both PDFs live in `docs/pdf/`; the ZIP `Schwab_Digital_Messaging_Platform.zip` is at the repository root (built by `python -I scripts/make-zip.py`). The final PDF page counts and ZIP file count are recorded in `docs/validation/final-artifacts.txt`.

Performance measurements (executed, local)

Source document: `docs/validation/performance-results.md`

Only numbers produced by the runs below are recorded. Raw console output: `docs/validation/performance-run-output.txt`; per-run JSON: `performance/node-load/results/*.json`.

Machine and configuration caveats (read before quoting any number)

- Laptop, Windows 11, 8 logical CPUs (Intel i5-1135G7 @ 2.40 GHz), 16 GiB RAM. **The load generator (Node 24), the backend (GraalVM JDK 21.0.7) and everything else ran on the same machine**, so client and server compete for CPU. Numbers are for relative comparison and sanity, not capacity planning.
- Backend: packaged jar `backend/target/messaging-platform.jar`, profile `local`, **default configuration** (H2 in-memory in PostgreSQL mode, in-memory bus with 8 consumer threads, in-memory coordination, in-memory Mongo, simulators with fault mode NONE, API rate limit 50 req/s burst 100 **per client subject**, backpressure threshold 5,000). Port 18280 (`SERVER_PORT=18280 bash scripts/run-local.sh`), fresh JVM for the whole run sequence. Not PostgreSQL, not RabbitMQ.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- The driver spreads load over 32 sender subjects (`--clients 32`, default) because one subject only measures the rate limiter. LOW priority is not used (quiet-hours deferral of SERVICE/LOW traffic would leave deferred rows behind; deferred rows stay VALIDATED by design).
- Single run per scenario, no repetition, so no confidence intervals. JIT warm-up of 5 s was discarded.
- k6 scripts (`performance/k6/*`) were **NOT executed**: k6 is not installed here.

Commands (repository root; `B=http://localhost:18280`)

```
node performance/node-load/load-driver.mjs --base-url $B --scenario health --concurrency 8 --duration 10
node performance/node-load/load-driver.mjs --base-url $B --scenario submit --concurrency 8  --duration 30
  --warmup 5
node performance/node-load/wait-drain.mjs  --base-url $B
node performance/node-load/load-driver.mjs --base-url $B --scenario submit --concurrency 32 --duration 30
  --warmup 5
node performance/node-load/wait-drain.mjs  --base-url $B
node performance/node-load/load-driver.mjs --base-url $B --scenario read   --concurrency 16 --duration 20
  --warmup 3
node performance/node-load/load-driver.mjs --base-url $B --scenario idempotent --requests 400 --concurrency 8
  --warmup 0
node performance/node-load/load-driver.mjs --base-url $B --scenario submit --concurrency 8 --duration 20
  --warmup 2 --fault TRANSIENT --fault-probability 1 --settle 60
```

(`.work/perf-run.sh` is the exact sequence that was executed.) Simulators were in NONE mode except in the last run.

Results

Scenario	Throughput	p50	p95	p99	Status counts
Health (`/actuator/health/readiness`), 8 clients, 10 s	862 req/s	5.6 ms	28.0 ms	66.8 ms	8,661 x 200
Submit `POST /messages` (mixed EMAIL/SMS/PUSH), 8 clients, 30 s	178.9 req/s	24.5 ms	134.8 ms	200.4 ms	5,375 x 202
Submit, 32 clients, 30 s	297.2 req/s offered; 262 req/s accepted	77.7 ms	309.6 ms	487.4 ms	7,859 x 202, 1,069 x 503 (API backpressure)
Dashboard/read mix (list 50 / ops summary / ops queues), 16 clients, 20 s	262 req/s	37.3 ms	184.1 ms	266.4 ms	5,248 x 200
- `GET /messages?size=50`	99 req/s	11.4 ms	119.0 ms	174.6 ms	
- `GET /ops/summary`	123 req/s	81.5 ms	219.8 ms	312.3 ms	
- `GET /ops/queues`	40 req/s	8.7 ms	112.2 ms	181.0 ms	
Idempotent replay (same key), 8 clients, 400 requests	478 req/s	13.7 ms	28.7 ms	87.3 ms	1 x 202, 140 x 200 (replay), 259 x 429 (single subject hit the 50 req/s API limit: expected)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Asynchronous pipeline (accept -> outbox -> bus -> consumer -> simulated provider -> SENT_TO_PROVIDER):

- After the 8-client run, 3,952 messages were still waiting; they drained at about **121 msg/s** end-to-end (31.5 s). After the 32-client run, 4,847 drained at about **201 msg/s** (22.2 s). So on this machine the API accepts faster (170-300 req/s) than the pipeline dispatches (about 120-200 msg/s), a backlog builds and API backpressure (HTTP 503 + `Retry-After`) engages at 5,000 pending, which is the designed protection.
- 147 / 377 / 584 messages stayed `VALIDATED` after the plateau: these are SERVICE-category pushes (price alert template) for recipients currently inside their quiet hours (recipient time zones such as Asia/Tokyo at the time of the run), deferred by policy, not stuck. `wait-drain.mjs` treats a 15 s plateau as drained.

Retry-storm style run (simulated-email TRANSIENT, probability 1, for the 20 s submit phase)

- 7,731 submissions accepted (202) at about 408 req/s, 456 rejected with 503 backpressure; accept p50/p95/p99 = 11.8 / 59.9 / 140.6 ms (note: lower than the healthy run because emails were failing fast and the machine was busy elsewhere; do not compare across runs).
- At the end of the storm: simulated-email circuit breaker `HALF_OPEN`, 807 deliveries delayed in the bus (requeued without consuming attempts while the breaker refused calls), 0 RETRY_PENDING snapshot at that instant, DLQ depth 0.
- Fault reset to NONE, 60 s later: circuit `CLOSED`, `VALIDATED` 584 (quiet-hours deferrals only), `SENT_TO_PROVIDER` 19,834 total, DLQ 0: the platform recovered without operator action and without dead letters in that window.

What these numbers do not show

No PostgreSQL/RabbitMQ/Redis cost, no network latency, no multi-node behaviour, no sustained (hours) load, no k6 ramping profiles, no GC/heap analysis. `/ops/summary` p50 of 81 ms with about 20k rows is the slowest read: it aggregates over the 24 h window (see `docs/database/query-plans-executed.md`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.